Providing Integrity for Authenticated Encryption in the Presence of Joint Faults and Leakage

Francesco Berti, Itamar Levi

Bar-Ilan University, Ramat-Gan, Israel
francesco.berti@biu.ac.il,itamar.levi@biu.ac.il

Abstract. Passive (leakage sensing) and active (fault injection) physical attacks pose a significant threat to cryptographic schemes. Although leakage-resistant cryptography has been well studied, little work has been done on mode-level security in the presence of adversaries that exploit both faults and leakage. In this paper, we focus on mode-level integrity for authenticated encryption (AE). First, we identify an inherent attack in the fault resilience model presented at ToSC 2023. This highlights how fragile the freshness condition of a forgery becomes when faults are injected into either the tag-generation or the encryption algorithm. Second, we provide new integrity definitions for AE in the presence of leakage and faults, following the *atomic* model, in which the scheme is divided into atoms (or components, e.g. a call to a block cipher) and the adversary is allowed to inject a fault only into the inputs of an atom. We envision this model as a first step toward *leveled implementations* in the faults scenario, the granularity of atoms can be made finer or coarser (for example, instead of considering a call to a block cipher, we could consider atoms to be rounds of the block cipher). We believe that protecting smaller blocks is easier than securing an entire scheme. The proposed model is highly flexible and allows us to understand where to apply countermeasures for faults. As we discuss, in some very interesting cases this model can reduce faults inside atoms to faults on their outputs. The proposed model, in addition to addressing the previous attack, models an adversary who can inject faults throughout the entire security game. This contrasts with the fault-resilience model, where the adversary first executes a phase in which faults can be injected into the real scheme, and then, in the second phase, no further faults are allowed and the adversary must distinguish the real scheme from an ideal one that outputs random ciphertexts. Third, we show that CONCRETE (presented at Africacrypt 2019), an AE-scheme using a single call to a highly leakage-protected (and thus very expensive) component, maintains integrity in the presence of leakage in both encryption and decryption, and faults only in decryption. On the other hand, a single fault in encryption is enough to forge. Therefore, we first introduce a weaker definition (which restricts the meaning of freshness), *weak integrity*, which CONCRETE achieves even if the adversary can introduce faults in the encryption queries (subject to certain limitations on the number and type of faults). Finally, we propose a variant, CONCRETE2, which is only slightly more computationally expensive, but still uses a single call to a strongly protected component, and which provides full integrity in the presence of leakage and faults (also in encryption).

Keywords: AE · Fault Injection · Fault-resistance · Integrity · Leakage-resistance · Model-Level Security · Side Channels · SCA

1 Introduction

Classically, in the standard-model an adversary can attack a cryptographic scheme by choosing its inputs and seeing its answer, the so-called *black-box* model [35]. However,

considering an adversary who has physical access to the device, far more powerful attacks are possible, both passive and active. On the one hand, passively, she can measure physical quantities involved in the cryptographic computation (side channels), such as time, electromagnetic radiation, power consumption, etc. [36, 37, 47]; and on the other hand, she can actively inject faults into the computation [18, 17, 2], obtaining critical information from the device's answers or fault-associated side channels, either to retrieve secret-material (e.g., the key) [28, 33, 56] or enough information to forge it (e.g. by replacing the nonce [55]), see [54] for more details. Finally, combined attacks manifest concrete threat [53, 52].

With respect to leakage-resistant symmetric cryptography, there is a large body of work that provides achievable security definitions for privacy [46], integrity [13, 14], and for AE [32, 3], and various constructions.

A particularly interesting line of research for construction is *leveled implementation*: as any mode-of-operation (mode-level) symmetric scheme is based on primitives such as block ciphers (BC) and hash functions (H), instead of protecting all primitives uniformly, one can design a scheme using weakly protected implementation for all primitives and a very well protected for just one or a few [46, 27, 5]. Such an approach gained large interest and acceptance in the community and was found to be particularly efficient because strong countermeasures against side-channel attacks are expensive, e.g. higher-order masking can be thousands of times slower (or more energy hungry) [20, 31, 34, 57]. There are many examples, both based on (tweakable [1]) block ciphers [13, 14, 9, 40] or sponges [25, 4]. In all these constructions, the bulk of the construction is carried by the weakly protected implementations that use ephemeral keys (i.e. keys generated in that particular query and used only a few times), while the strongly protected implementations are reserved for a few calls (usually one or two) where the master key of the scheme is used. To study the integrity of a MAC (Message Authentication Code) or a AE (Authenticated Encryption) scheme, one possibility is to utilize the *unbounded leakage model* [13], that works well with leveled implementations: it assumes that all inputs (even the key) and outputs of the weakly implemented components are leaked, while the strongly protected components leaks only their inputs and their outputs, but not the secret-inputs (e.g., key). These strongly protected components are modeled either as *leak-free* [13], i.e. except for the inputs (not the key) or the outputs of a block cipher (BC), no other information is obtained from the leakage, or *strongly unpredictable* (SUP–L2) [8], i.e. assumes that given a leakage of a blockcipher $\mathsf{F}_k(\cdot)$ and its inverse $\mathsf{F}^{-1}(\cdot)$, it is hard to find an input/output pair (x, y) that is both *fresh* (never received as an answer before) and *valid* ($y = \mathsf{F}_k(x)$). In the unbounded leakage model, it is possible to construct a MAC that provides authenticity when the tag-generation algorithm leaks, for example the well-known Hash-then-BC, i.e. $\tau = \mathsf{F}_k(\mathsf{H}(m))$, assuming a single call to a leak-free block cipher [46, 13]. On the other hand, decryption leakage is much more tricky, since the correct tag is often recomputed and thus can be leaked, i.e. $\mathsf{Vrfy}_k(m, \tau)$ accepts if $\tau \stackrel{?}{=} \mathsf{Mac}_k(m)$. To avoid this leakage exploitation path, it has been proposed to use the inverse in the verification, by doing a check on a different internal value in the protocol [14]: instead of comparing τ with the correct tag, $\mathsf{Vrfy}_k(m, \tau)$ invert the tag and checks if $\tilde{h} = \mathsf{F}_k^{-1}$ is equal to the digest $h = \mathsf{H}(m)$. The idea is that $\tilde{h}$ cannot be used for forgery because it is random as the output of a leak-free block cipher, and the adversary has to find a pre-image for this value (and we can assume that the hash function is range-oriented pre-image resistant). In the random oracle model, the previous MAC provides integrity in the presence of leakage in both tag-generation and verification, even if we assume that F is SUP–L2 [8]. To avoid using the random oracle model, which is undesirable especially in the presence of leakage [11], at Asiacrypt 2021 a MAC, LR–MAC1 was proposed that uses a *strongly unpredictable with*

[1] A *tweakable block cipher* (TBC) is a block cipher which has an additional input, the *tweak*, for flexibility [39]: $\mathsf{F}_k^{tw}(x)$ where tw is the tweak.

leakage tweakable-BC (TBC): the idea is to use a fixed input, 0^n, as the input and the message hash, $h = \mathsf{H}(m)$, as the tweak, i.e. $\tau = \mathsf{F}_k^h(0^n)$. This way, if $\mathsf{F}_k^{-1,h}(\tau) \neq 0^n$ during verification, the adversary cannot use the output of F^{-1} to forge [11].

Unsurprisingly, the inverse in decryption has also been used to build AE schemes, that provide authenticity with leakage in both encryption and decryption in the unbounded leakage model [14, 9, 4, 40]. Most schemes use 2 calls to the strongly protected implementation of a (T)BC, one to authenticate either the message or the ciphertext, and another to generate a first ephemeral key which is used to encrypt the message either with a rekeying-based encryption scheme (e.g. the one proposed at CCS 2015 [46]) or with a sponge. At Africacrypt 2019, a leakage-resistant AE scheme CONCRETE (see Fig. 2 and Alg. 2) was proposed using a single call to a strongly protected TBC. CONCRETE provides integrity in the unbounded leakage model even when both encryption and decryption leak [16]. It picks a random ephemeral key k_0, then computes a commitment c_0 on that key, then uses that first ephemeral key to encrypt using PSV (a leakage-resistant encryption scheme based on rekeying [46]) to obtain a ciphertext c, then it encrypts the ephemeral key k_0 with a TBC, where the tweak is the digest of the ciphertext, $h = \mathsf{H}(c)$, $c_{l+1} = \mathsf{F}_k^h(k_0)$. The adversary in the decryption first retrieves k_0, checks the correctness of c_0, and then retrieves m correctly from k_0.

Similarly to the leakage-resistant schemes, it is possible to withstand fault attacks with specific countermeasures not in the *mode level*(see [6] for example). On the other hand, for leakage-resistant schemes, it is possible to design schemes that are inherently more secure, when working at the mode-level. However, regarding fault-resistance, the literature is far more scarce and models always hold some assumptions or restrictions on the adversary: as an overview we discuss three models, faulting values in memory [30, 1, 10], bounding the information that an adversary can obtain via faults with the *accumulated interference* [29], defining fault-resilience [54].[2] We now explore these approaches in a bit more detail. In the *Faults in Memory* model, the adversary can inject a fault on a value kept in memory alone when it is used by the scheme, either setting it to a certain value or with a differential fault (moreover, this fault can be permanent or transient) [30]. The authors have also provided a fault-resilient signature scheme and AE scheme which withstands faults in encryption. With a similar model [1], the security of hedged Fiat-Shamir signatures has been assessed against faults. In another model, equipped with fault injection and leakage collection capabilities, the adversary accumulates information, the so-called *accumulated interference* [29]. In [54] Saha et al. introduce the concept of *fault-resilient* PRF: in this 2 stages game, first the adversary can fault a real PRF, then, she must distinguish the real PRF from an ideal primitive (in this second phase, no faults can be injected). From this notion, they define authenticity in the presence of faults for both MAC and AE. In the fault resilience model [54], the adversary performs all the faulted queries before "before" What - clearly define -connects to previous comments making any query to distinguish queries to the real scheme (that is, either to Mac and Vrfy, or to Enc and Dec in the AE case) from an ideal scheme (that is, either to an oracle which outputs a random value and another which always outputs reject). Moreover, the adversary can only inject faults in the direct queries (that is, to Mac or to Enc). They also provide a probabilistic MAC and a probabilistic AE scheme achieving the fault-resilience as defined. Finally, starting from the faults in memory model [30, 1], the authors of [10] extended it to the so-called *atomic model* where for example it is possible to divide a MAC into atoms (e.g. the hash, a call to BC) and assume that the adversary can only inject faults in the input of the atoms. Surprisingly, protecting against faults in verification is easier than in tag-generation: LR–MAC1 achieves this security without any modification (even if the adversary obtains

[2] In the fault-resilience model, the adversary first executes a phase in which faults can be injected into the real scheme, and then, no further faults are allowed and the adversary must distinguish the real scheme from an ideal one.

leakage in both tag-generation and verification in the unbounded leakage model), even if the adversary can fault every input of every atom (except the master key used by the leak-free TBC). However, if the adversary can inject faults into the tag-generation, the adversary can easily forge because she can choose the inputs of the leak-free TBC F_k and thus has oracle access to it.

To prevent such attacks, in [10], the authors discussed two choices: either 1) abandon the unbounded leakage model with two calls to a leak-free TBC, assuming that the output of the first leak-free TBC is not leaked; or 2) they use a randomized version (there is an additional random input to Mac, making internal values random), but this option only withstands differential faults.

1.1 Our Contribution

This paper provides both a theoretical contribution, pointing out a subtle problem in a previous model, and providing a security framework for integrity in the presence of faults and leakage based on the atomic model [10], and a practical one proving the integrity in the presence of leakage and faults (faults only in decryption) of an existing scheme, CONCRETE [16], and finally proposing a new scheme, CONCRETE2, which also withstands faults in encryption.

First, on the theoretical side, we point out a subtle problem in the fault-resilient framework [54]: a trivial fault attack breaks integrity for any scheme. The adversary can fault the return function of any algorithm, trivially forging: for example, when computing $\tau \leftarrow \mathsf{Mac}_k(m)$, she can insert a differential fault to switch the first bit of τ, then she outputs (m, τ) as forgery. This couple is considered fresh (since the output of Mac was not τ, but $\tau \oplus 10\ldots0$). This attack is clearly not significant since we can view (m, τ) as not fresh, which is indeed the case. However, it illustrates that the model should be studied carefully and precisely, and that there are semantics which if not treated may be dangerous. In our perspective, this is an inherent security consideration that highlights how difficult and delicate is modeling security against joint faults and leakage attacks.

Fortunately, in the atomic model, the previous attack cannot occur because the adversary can only fault the inputs of the atoms, and the outputs only when they are also intermediate computations. For example, in LR–MAC1 the output τ is the output of the F_k atom, while in CONCRETE, the adversary can inject a fault in c when it is used as input to H, but as ciphertext, but not as output since c is the output of the encryption part. The atomic model is immune to this theoretical problem. But more importantly it does not need, contrary to the fault-resilient framework, that all faulted queries must be performed before **all** any non-faulty queries. Therefore, we extend the atomic model to the AE case: we provide the integrity definition in the presence of faults and leakage. Moreover, we define a weaker version by which we disallow the adversary from forging using any ciphertext values that may result from faults during encryption, denoted *weak integrity*.

Second, we study the integrity of CONCRETE in the unbounded leakage model in the presence of faults. We choose CONCRETE because it is a probabilistic scheme (as many constructions against faults in direct queries [54, 10]) and uses only one leak-free call (and is thus very efficient). Similar to LR–MAC1, an adversary cannot forge with any fault in the decryption (except on the master key). This means that the whole integrity in the presence of leakage and faults (faults only in decryption) relies on the security with faults and leakage of a single call to a TBC (plus some black-box properties as collision-resistance and range-oriented with a random prefix pre-image-resistance for the hash function, and assuming that the block cipher is an ideal cipher, as detailed in Sec. 4.3). On the other hand, a single fault (even a differential one) **in encryption** is enough to forge. However, if the adversary is restricted to inject at most a set fault (and any differential fault she

chooses) per encryption query, CONCRETE provides *weak integrity*, as we prove. This is consistent with [10], which suggests that for authenticity, the direct queries (either Mac or Enc) must be randomized, and the adversary should not be able to derandomize them with faults (i.e. choosing all inputs for some critical computations, thus having oracle access to those atoms). We stress that we do not have this requirement for the inverse queries (either Vrfy or Dec), where by using the inverse of a (T)BC, F_k^{-1}, we can still prevent the adversary from forging even if she has oracle access to F_k^{-1}.

Third, we modify CONCRETE to withstand faults in encryption as well. We identify that 1) CONCRETE authenticates any ciphertext produced by the encryption part (independent of the message being encrypted), 2) since $c_i = \mathsf{E}_{k_i}(p_B) \oplus m_i$ (k_i is the i^{th} ephemeral key and p_B is a constant), an adversary can predict how modifying the message would modify the corresponding ciphertext (i.e, $c'_i = c_i \oplus \Delta$ encrypts $m'_i = m_i \oplus \Delta$). Therefore, in the newly proposed scheme we modify the encryption $c_i = \mathsf{E}_{k_i}(m_i)$ and additionally encrypt a MAC of the message m, getting CONCRETE2 (Fig. 4). We will give full proofs and intuition. With these changes, CONCRETE2 can provide integrity even in the unbounded leakage model, where the adversary can inject any fault in decryption, and one set fault and any differential fault per encryption query. If the adversary could inject two set faults per encryption query, she would obtain oracle access to $\mathsf{F}_k^{\cdot}(\cdot)$. By setting both inputs of F_k, she could choose k_0, compute the commitment, the MAC, and the encryption of m from k_0, compute the digest h, and then force the computation of $\mathsf{F}_k^h(k_0)$ in another query, thus forging successfully.

Note: all faults discussed here are variable faults (e.g., up to n-bit faults where n is the TBC-block-size), while previous constructions usually withstand a single fault per query (except [10]).

CONCRETE2 is a 2-pass scheme using a single call to a leak-free TBC, where these 2 passes cannot be performed in parallel. However, notably CONCRETE2 can still be very efficient in terms of performance in encryption, as compared to a 1-pass scheme, because the second pass may commence with a tiny delay (shift) from the start of the first pass (quasi 1-pass). For example, the second pass, the one that computes h, digesting the ciphertext blocks, may begin as soon as at least one of their blocks is produced (consider a round- based or mode-of-operation based construction). Compared with [54], their fault-resilient AE-scheme, MEM is a 3-pass scheme (in encryption quasi 2-pass), and can withstand a single fault per encryption query, and no fault in decryption. Moreover, CONCRETE2 (and CONCRETE only for faults in decryption) can withstand any number of full faults (that is, faults which alter completely a value, that in our scheme is at lesast n-bit long) in decryption and any number of differential faults in encryption (and one set fault), while MEM withstands a single fault. On the other hand, MEM provides privacy even if a single fault is injected in an encryption query, while, here we do not focus on this problem (however, we do suspect that CONCRETE2 also provides privacy in the presence of faults). As a side-result, we put forward a variant of CONCRETE2 using the popular sponge construction, as a conceptual idea. We leave a formal analysis of this variant to future works.

For the first time, we show that it is possible to construct a scheme in the unbounded leakage model that provides integrity even in the presence of faults, with modified and meaningful definitions adapted to the faults setting. We point out that all the faults our adversary can inject are n-bit (where n is the block-size of the block cipher). We have provided a gradual understanding of security, where we first analyze faults in decryption (in addition to leakage), and then also in encryption. Thus, we are gradually building a guided understanding of which elements should be protected and against what.

Organization of the Paper The rest of the paper is organized as follows. In Sec. 2, we recall the necessary preliminaries. The expert reader may skip this section, except for

Sec. 2.7 (where we introduce CONCRETE), and Sec. 2.8-2.10 (where we introduce security in the presence of faults). In Sec. 3, we explain the trivial attack in the fault-resilient framework (Sec. 3.1) and present our security definitions in the presence of leakage and faults (Sec. 3.2). Sec. 4 is devoted to explaining why CONCRETE provides integrity and weak integrity in the presence of faults and leakage (faults only in decryption for integrity). Therefore, we introduce CONCRETE2 in Sec. 5, proving its integrity in the presence of leakage and faults. Finally, Sec. 6 concludes the paper and outlines directions for future research.

2 Background

Notations. We denote by $\{0,1\}^n$ the set of all n-bit strings and by $\{0,1\}^*$ the set of all finite strings, by $x\|y$ the concatenation of the two strings x and y, and by $|x|$ the length of the string x. We denote by $\pi_{n,l}(x)$ the n leftmost bits of the string x, and by $\pi_{n,r}(x)$ the n rightmost bits. By $x \xleftarrow{\$} \mathcal{S}$, we denote that x is picked uniformly at random from the set $\mathcal{S}$, and by 0^n we denote a sequence of n zeros. When we *parse* a string x into n-bit blocks, we divide x into $(x_1, \ldots, x_l)$ with $|x_1| = \ldots = |x_{l-1}| = n$ and $0 < |x_l| \leq n$, where $x = x_1\| \ldots \|x_l$. When we *special parse*(j) a string x into n-bit blocks, we divide x into $x_1, \ldots, x_l$ with $|x_1| = \ldots = |x_{j-1}| = |x_{j+1}| = \ldots = |x_l| = n$ and $|x_j| \leq n$, where $x = x_1\| \ldots \|x_l$. With $\$(\cdot)$ we denote an oracle that outputs a random value. We say that two functions (or two oracles) have the same *signature* if they have the same input and output spaces.

A $(q_1, \ldots, q_d, t)$-adversary A is a probabilistic algorithm that can make q_i queries to oracle $\mathcal{O}_i$ and runs in time bounded by t. By $y \leftarrow \mathsf{A}^{\mathcal{O}_1,\ldots,\mathcal{O}_d}(x)$, we denote that the adversary A, on input x, with access to oracles $\mathcal{O}_1, \ldots, \mathcal{O}_d$, outputs y. Let Alg be a probabilistic algorithm. By $y \leftarrow \mathsf{Alg}(x)$, we denote that running Alg on input x produces y. If we want to explicitly denote the randomness r used in the previous execution, we write $y \leftarrow \mathsf{Alg}(x; r)$.

2.1 Cryptographic Primitives - Keyed Hash Functions

In our schemes, we will use hash functions and tweakable block ciphers (TBC). We start by introducing hash functions used to compress data.

2.1.1 Keyed Hash Functions and Collision Resistance

We use *keyed hash functions*, that is, our hash function H takes as input a key s, and a finite string x, and outputs an n-bit string. **The key s for hash functions is significantly different from the key of block ciphers for one significant reason [3]: the key of the hash function s is not kept secret [35]. In this paper the key of the hash is sampled uniformly at random.** In all security definitions involving hash functions, we assume that the adversary knows the key. **When the hash key s is clear from the context, we may omit it, since it is public [44, 42].**

It should be computationally infeasible for an adversary to find a *collision*, (2 different inputs with the same output):

Definition 1. A hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \{0,1\}^n$ is (t, ϵ)-*collision resistant* (CR) if $\forall$ t-adversaries A: $\Pr[\ (m_0, m_1) \leftarrow \mathsf{A}(s) \text{ s.t. } \mathsf{H}_s(m_0) = \mathsf{H}_s(m_1),\ m_0 \neq m_1 \mid s \xleftarrow{\$} \mathcal{HK}] \leq \epsilon$.

[3] It is possible that not all strings can be used as the key of a hash function. In this case, one can use a generation algorithm [35].

Unkeyed hash functions. If we consider a hash function $\mathsf{H} : \{0,1\}^* \to \{0,1\}^n$ that is unkeyed, we cannot hope for it to be collision resistant according to the previous definition [50, 49] because there exist many collisions due to the pigeonhole principle. So any algorithm that has hardcoded in itself a colliding pair for H can simply output this couple [35].

To avoid this problem, we define collision resistance for families[4] of hash functions. In this way, an efficient adversary cannot have hardcoded a collision for each hash key. This approach dates back from 1987 [22], and is very popular in literature and practical constructions [35, 23, 19, 50]. Noteworthy, there has been other attempts to define collision resistance not using families of hash functions, but based on *human ignorance*: "what makes a hash function collision resistant is humanity's inability to find a collision, not the computational complexity of printing one." [49].

2.1.2 Multi-input Collision Resistance

We will use a hash function that takes two inputs (in addition to the key), so we will define this object and collision resistance for it. We follow [26]:

Definition 2. A hash function $\mathsf{H} : \mathcal{HK} \times \mathcal{X}_1 \times \mathcal{X}_2 \to \{0,1\}^n$ is (t,ϵ)-*multi-input collision resistant* (CR) if $\forall$ t-adversaries A

$$\Pr[\ ((x_0,x_1),(x_0',x_1')) \leftarrow \mathsf{A}(s) \text{ s.t. } \mathsf{H}_s(x_0,x_1) = \mathsf{H}_s(x_0',x_1'),\ (x_0,x_1) \neq (x_0',x_1') \mid s \xleftarrow{\$} \mathcal{HK}] \leq \epsilon.$$

The need for a multi-input collision resistance. Looking ahead, the hash that we will use has two inputs. Given a collision resistant hash function H_s, we would like to build a multi-input collision resistance hash function H'_s that uses H_s (we omit for simplicity the public hash-key s to simplify the notation, as it is clear from the context). Clearly, we cannot follow the trivial idea $\mathsf{H}'(c,a) := \mathsf{H}(c\|a)$, because otherwise, it is easy to find two different couples (c,a) and (c',a') s.t. $\mathsf{H}'(c,a) = \mathsf{H}'(c',a')$: if $c = c_1\|c_2, c' = c_1, a' = c_2\|a$, then $\mathsf{H}'(c,a) = \mathsf{H}(c_1\|c_2\|a) = \mathsf{H}'(c',a')$.

One such function has been suggested with the original description of CONCRETE [16]:

$$\mathsf{H}'(c,a) := (c_1\|0\|c_2\|0\|...\|c_l\|0\|a_1\|1\|a_2\|1...\|a_{l_a}\|1\|a_{l_a}\|1),$$

where $a_1,...,a_{l_a}$ is a parsing of a and $c_1,...,c_l$ one of c in n-bit blocks, padding a and c if necessary (the *padding* of a string x consists in the appending to x of 10^α, where α is the smallest integer s.t. $|x\|10^\alpha|$ is a multiple of n) [5].

2.2 Cryptographic Primitives - (Tweakable) Block ciphers

We use (tweakable) block ciphers to produce pseudorandom values.

Definition 3 ([39])**.** A *tweakable block cipher* (TBC) $\mathsf{F} : \mathcal{K} \times \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$ is a family of permutations, where $\mathsf{F}(k,tw,\cdot) : \{0,1\}^n \to \{0,1\}^n$ is a permutation $\forall (k,tw) \in \mathcal{K} \times \mathcal{TW}$. For simplicity, we often write $\mathsf{F}_k^{tw}(x)$ instead of $\mathsf{F}(k,tw,x)$. With $\mathsf{F}_k^{-1,tw}$, we denote the inverse of F_k^{tw}.
A *block cipher* (BC) is a TBC without any tweak, that is $|\mathcal{TW}| = 1$.

[4] We use the key to index the family of hash functions that we use. In [19], the authors use a system parameter to index them.

[5] It is necessary to be sure that all blocks of a and c are full, otherwise it is possible to mount an attack. E.g., with $n = 2$, $c = 000$, $a = 0111$, $c' = 0000$, $a' = 111$ (we thank the ToSC reviewer for having spotted this).

Definition 4. A TBC $\mathsf{F} : \mathcal{K} \times \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$ is a $(q, t, \epsilon_{\mathsf{sTPRP}})$-sTPRP (*strong Tweakable Pseudo Random Permutation*) if for any (q_E, q_I, t)-adversary A

$$|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{F}_k(\cdot),\mathsf{F}_k^{-1,\cdot}(\cdot)}] - \Pr[1 \leftarrow \mathsf{A}^{\mathsf{f}^{\cdot}(\cdot),\mathsf{f}^{-1,\cdot}(\cdot)}] \leq \epsilon$$

where $q_E + q_I \leq q$, $k \xleftarrow{\$} \mathcal{K}$, and $\mathsf{f} \xleftarrow{\$} \mathcal{TPERM}$, where $\mathcal{TPERM}$ is the set of the functions with the same signature as F_k, that is the functions $\mathsf{f} : \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$, s.t. $\forall tw \in \mathcal{TW}$ $\mathsf{f}^{tw}(\cdot) : \{0,1\}^n \to \{0,1\}^n$ is a permutation. If $q_I = 0$, F is a TPRP (*Tweakable Pseudo Random Permutation*).

That is, the difference between a TPRP and a strong TPRP is that in the latter case the adversary has also access to the inverse. Moreover, if in the previous definition $q_I = 0$ and we pick f from all the functions $\mathsf{f} : \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$, we have a *Pseudo Random Function* PRF (see below).

In some cases, we need to model the block cipher as an ideal cipher

Definition 5 ([21])**.** A block cipher $\mathsf{E} : \mathcal{K} \times \{0,1\}^n$ is an *ideal cipher* if it is chosen uniformly at random among all block ciphers with the same signature.

That is, an ideal cipher is a family of $|\mathcal{K}|$ independent permutations. When an ideal cipher E is used, even only by the oracles, the adversary is always allowed to query it directly, choosing both the key and the input for each call she makes.

2.2.1 (Strong) PRP and PRF

We can define strong PRP, and PRP for block ciphers by simply adapting the previous definition to their syntax.

Definition 6. A BC $\mathsf{F} : \mathcal{K} \times \{0,1\}^n \to \{0,1\}^n$ is a $(q, t, \epsilon_{\mathsf{sPRP}})$-sPRP (*strong Pseudo Random Permutation*) if for any (q_E, q_I, t)-adversary A

$$|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{F}_k(\cdot),\mathsf{F}_k^{-1}(\cdot)}] - \Pr[1 \leftarrow \mathsf{A}^{\mathsf{f}(\cdot),\mathsf{f}^{-1}(\cdot)}] \leq \epsilon$$

where $q_E + q_I \leq q$, $k \xleftarrow{\$} \mathcal{K}$, and $\mathsf{f} \xleftarrow{\$} \mathcal{PERM}$, where $\mathcal{PERM}$ is the set of the permutations with the same signature as F_k, that is the permutations $\mathsf{f} : \{0,1\}^n \to \{0,1\}^n$. If $q_I = 0$, F is a PRP.

Sometimes, the adversary has access only to F_k, and we are not interested in the fact that F_k is a permutation, but only that its outputs look random. If this is the case we say that F is a PRF (Pseudo Random Function).

Definition 7. A BC $\mathsf{F} : \mathcal{K} \times \mathcal{X} \to \{0,1\}^n$ is a $(q, t, \epsilon_{\mathsf{PRF}})$-PRF (*Pseudo Random Function*) if for any (q_E, q_I, t)-adversary A

$$|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{F}_k(\cdot)}] - \Pr[1 \leftarrow \mathsf{A}^{\mathsf{f}(\cdot)}] \leq \epsilon$$

where $k \xleftarrow{\$} \mathcal{K}$, and $\mathsf{f} \xleftarrow{\$} \mathcal{FUNC}$, where $\mathcal{FUNC}$ is the set of the functions with the same signature as F_k, that is, the functions $\mathsf{f} : \mathcal{X} \to \{0,1\}^n$.

Note that there is no difference between the PRF definition for a TBC or a BC, simply in the first case, $\mathcal{X} = \mathcal{TW} \times \{0,1\}^n$, while for BC, $\mathcal{X} = \{0,1\}^n$.

A (q, t, ϵ)-PRP is a $(q, t, \epsilon + q^2 2^{-n})$-PRF (with n the input size of F) due to the well-known switching lemma (see, for example [35]).

2.3 MACs, and (Authenticated) Encryption Schemes

In the previous sections, we have formally defined hash functions, and (tweakable) block ciphers (Sec. 2.1, and Sec. 2.2). These are the building blocks of the MAC, encryption and AE primitives, which we define here.

2.3.1 MACs (Message Authentication Codes)

We use a Message Authentication Code, MAC, to authenticate a message.

Here, we give the syntax definition for the cryptographic construction to authenticate: Message Authentication Codes (MAC)

Definition 8. A *Message Authentication Code (*MAC*)* is a triple of algorithms $\Pi = (\mathsf{Gen}, \mathsf{Mac}, \mathsf{Vrfy})$ where

- The key-generation algorithm Gen generates a key from the set of keys, $\mathcal{K}$.
- The tag-generation algorithm Mac is a deterministic algorithm which takes as input a key $k \in \mathcal{K}$, and a message $m \in \mathcal{M}$, and outputs a tag τ. We denote this with $\tau \leftarrow \mathsf{Mac}_k(m)$.
- The verification algorithm Vrfy is a deterministic algorithm which takes as input a key $k \in \mathcal{K}$, a message $m \in \mathcal{M}$, and a tag τ, and outputs $\top$ ("valid") or $\bot$ ("invalid"). We denote this with $\top / \bot = \mathsf{Vrfy}_k(m, \tau)$.

We require *correctness*, that is $\forall (k, m) \in \mathcal{K} \times \mathcal{M}$, $\top = \mathsf{Vrfy}_k(m, \mathsf{Mac}_k(m))$.

Its security definition can be found easily, [35] (it can also be obtained from Def. 16 removing faults and leakage).

2.3.2 ivE - iv-Based Encryption Schemes

For encryption, we use iv-based encryption schemes:

Definition 9. An iv*-based encryption scheme (*ivE*)* is a triple of algorithms $\Pi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ where

- The key-generation algorithm Gen generates a key from the sets of keys, $\mathcal{K}$.
- The encryption algorithm Enc is a deterministic algorithm which takes as input a key $k \in \mathcal{K}$, an initialization vector $\mathsf{iv} \in \mathcal{IV}$, and a message $m \in \mathcal{M}$, and outputs a ciphertext $c \in \mathcal{C}$. We denote this with $c \leftarrow \mathsf{Enc}_k(\mathsf{iv}, m)$.
- The decryption algorithm Dec is a deterministic algorithm which takes as input a key $k \in \mathcal{K}$, an $\mathsf{iv} \in \mathcal{IV}$, and a ciphertext $c \in \mathcal{C}$, and outputs a message $m \in \mathcal{M}$ or $\bot$ ("invalid"). We denote this with $\bot / m = \mathsf{Dec}_k(\mathsf{iv}, c)$.

In the security definitions, we assume that the iv is picked uniformly at random from $\mathcal{IV}$. Thus, we denote with $\mathsf{Enc}^{\$}_k(m)$, the fact that we pick $\mathsf{iv} \xleftarrow{\$} \mathcal{IV}$, and, then, we compute $\mathsf{Enc}_k(\mathsf{iv}, m)$:

Definition 10. A (q, t, ϵ)*-secure* ivE*-scheme (*ivE*)* $\Pi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ if $\forall$ (q, t)-adversary A

$$|\Pr[\mathsf{A}^{\mathsf{Enc}^{\$}_k} \Rightarrow 1] - \Pr[\mathsf{A}^{\$} \Rightarrow 1]| \leq \epsilon,$$

where $\mathsf{Enc}^{\$}_k(m)$ on input m, simply picks $\mathsf{iv} \in \mathcal{IV}$ and outputs (c, iv) with $c \leftarrow \mathsf{Enc}_k(\mathsf{iv}, m)$; while $\$(m)$ simply picks $\mathsf{iv} \xleftarrow{\$} \mathcal{IV}$ and outputs (iv, c), with $c \xleftarrow{\$} \{0,1\}^{|\mathsf{Enc}(\mathsf{iv}, m)|}$.

2.3.3 Authenticated Encryption (AE)

The cryptographic primitive that provides privacy and authenticity is *authenticated encryption* (AE). Following [16, 54] we use a probabilistic encryption scheme. We start giving its syntax definition:

Definition 11. An *authenticated encryption* (AE) scheme is a triple of algorithms $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ where

- The key-generation algorithm Gen generates a key from the sets of keys, $\mathcal{K}$.
- The encryption algorithm AEnc is a probabilistic algorithm which takes as input a key $k \in \mathcal{K}$, associated data $a \in \mathcal{AD}$ and a message $m \in \mathcal{M}$, and outputs a ciphertext $c \in \mathcal{C}$. We denote this with $c \leftarrow \mathsf{AEnc}_k(a, m)$.
- The decryption algorithm ADec is a deterministic algorithm which takes as input a key $k \in \mathcal{K}$, an associated data $a \in \mathcal{AD}$ and a ciphertext $c \in \mathcal{C}$, and outputs a message $m \in \mathcal{M}$ or $\perp$ ("invalid"). We denote this with $\perp /m = \mathsf{ADec}_k(a, c)$.

We require *correctness*, that is $\forall (k, a, m) \in \mathcal{K} \times \mathcal{AD} \times \mathcal{M}$, $m = \mathsf{ADec}_k(a, c)$ $\forall c \leftarrow \mathsf{AEnc}_k(a, m)$. A *nonce-based* AE *(*nAE*)* is an AE with an additional input called the nonce.

Note that, differently from the standard nAE definition, see [48, 51], we allow AEnc to be probabilistic as in [54].

In the black-box model (that is when the adversary can choose the inputs and only sees the outputs of their oracles), the standard security notion [45] for AE is the following:

Definition 12. An AE-scheme $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ is (q_E, q_D, t, ϵ)-AE-*secure* if for any (q_E, q_D, t)-adversary A, the following advantage

$$|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{AEnc}_k(\cdot,\cdot),\mathsf{ADec}_k(\cdot,\cdot)}] - \Pr[1 \leftarrow \mathsf{A}^{\$(\cdot,\cdot),\perp(\cdot,\cdot)}] \leq \epsilon, \tag{1}$$

where $\$(\cdot,\cdot)$ is an oracle which on input (a, m) outputs $c \xleftarrow{\$} \{0,1\}^{|\mathsf{AEnc}_k(a,m)|}$ and $\perp$ is an oracle which always output $\perp$ ("invalid"). The adversary is not allowed to query his second oracle on input (a, c), if she has received c as an answer from the first oracle on input (a, m) for any $m \in \mathcal{M}$. For nAE-*security*, we adapt the previous definition to its syntax, and we add the requirement that the nonce is not repeated between different AEnc_k/\$-queries.

This security definition gives both privacy and authenticity because: if the adversary can distinguish the queries to the second oracle, then, she can *forge*, that is find a couple (a, c) that is *fresh* (that is, never the output from a query to the first oracle) and *valid* (that is $\mathsf{ADec}_k(a, c) \neq \perp$). Moreover, if she can distinguish the queries to the first oracle, then she can distinguish $\mathsf{AEnc}_k(\cdot,\cdot)$ from $\$(\cdot,\cdot)$.

2.4 Leakage

The previous definitions are *black-box*, because the adversary chooses the inputs of their oracles and sees only their outputs. On the other hand, cryptographic algorithms are usually implemented on electronic devices. Thus, an adversary can measure physical quantities involved during the computations, such as instantaneous power consumption, time, or electromagnetic/radio-frequency (RF) radiation, etc. [36, 37, 47, 24]. All these are collectively denoted by *leakage*, and they can provide powerful additional information to the adversary.

Notations. We denote that the computation performed by an oracle O leaks, adding the suffix L, that is OL. The leakage of the implementation of the oracle $\mathsf{O}_k(x)$ on input (x) is given by the *leakage function* $\mathsf{L}_\mathsf{O}(x; k)$. The leakage function represents the information that the adversary gets via leakage from the computation of oracle on input x. When $\mathsf{O}_k(x)$ is a probabilistic oracle, we assume that $\mathsf{O}_k(x)$ has access to a random tape carrying the value r, then $\mathsf{L}_\mathsf{O}(x, r; k)$ takes also r as input. Similar definitions and notations may be found in [12].

2.5 *Integrity* in the Presence of Leakage

When the adversary does not only choose the inputs of the oracles and sees their outputs, but also has access to the leakage when they perform their computations, we do not provide a comprehensive security definition for both privacy and authenticity, for two reasons. First, if we naively modify Eq. 1 for AE-*security* (Def. 12), adding the leakage of all the oracles, we face a problem. We would need to compute a leakage for the ideal primitives ($ and $\perp$), that is indistinguishable from the leakage of the real primitives (AEnc and ADec). This is quite cumbersome and complex as discussed in [41]. Instead, it is possible to give different definitions for authenticity and privacy in the presence of leakage [13]. Second, we note that these two definitions and security goals are inherently different. For authenticity, we require that producing a *complete* fresh and valid ciphertext is difficult. For privacy, instead, the ciphertext must give *no information* about its plaintext (except for the message length). Thus, conceptually one can consider different leakage models for these two security definitions (or base security on different assumptions regarding the leakage) as the leakage of a single bit can be enough to break privacy, in contrast to the need to guess *completely* a valid ciphertext for integrity. Berti et al. [14] introduced a security definition for integrity in the presence of leakage. We give the definition modified for the case of probabilistic encryption [16]:

Definition 13. An AE-scheme $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ provides (q_E, q_D, t, ϵ)-*ciphertext integrity with leakage in encryption and decryption*, CIL2, if for any (q_E, q_D, t)-adversary A

$$\Pr[(a, c) \leftarrow \mathsf{A}^{\mathsf{AEncL}_k(\cdot,\cdot),\mathsf{ADecL}_k(\cdot,\cdot)} \mid \text{s.t. } (a, c) \text{ is fresh and valid}] \leq \epsilon. \tag{2}$$

With *fresh* we denote that c has never been obtained as an answer from a $\mathsf{AEncL}_k(a, m)$ query for any m, and with *valid* that $\mathsf{ADec}_k(a, c) \neq \perp$. (CIL means that only AEnc leaks [16]).

Modeling leakage - the unbounded leakage model. The previous definition (Def. 13) assumes that when the adversary queries either the AEnc_k or the ADec_k oracle, she receives the output and the leakage of its computation. As in all definitions involving leakage, the leakage function must be realistic, to have a security definition that is relevant in practice, but not too generous (for example assuming that the key leaks in full with one leakage is both very generous, and realistically it would reflect an impossible game to win), otherwise it is impossible to provide security.

For integrity, we use the *unbounded leakage model* [13] in which we assume that the scheme uses a *leveled implementation*, the primitives (hash functions, TBC, BC) are divided into two types: *strongly protected primitives* which are modeled as *leak-free*, that is, they leak nothing about their internals, and *weakly protected primitives*. When a strongly protected primitive is used, $\mathsf{L_O}$ gives its inputs and output, while when a weakly protected primitive is used, its inputs, output *and* its secret key. This can be schematically illustrated: We use green color for the inputs and outputs of the scheme, orange for the internal value computed and the key of the weakly protected primitives, and red for the keys of the strongly protected primitives (see Fig. 2). In the unbounded leakage model, $\mathsf{L_O}$ gives all the orange values.

Note that strongly protected components are usually much slower than unprotected (or slightly protected) components [31, 34, 57, 20], thus, when we design a scheme, we aim to reduce their usage.

2.6 *Privacy* in the Presence of Leakage.

We now define privacy in the presence of leakage. As already discussed, modifying the standard real-vs-ideal game, (that is, distinguishing AEnc_k from $), by adding leakage to

both sides has no clear or *trivial* solution. There are two possible approaches:
Pereira et al. [46] started from the standard CPA-security (Chosen-Plaintext Attack) [35], adding leakage to the game. We modify this definition to make it coherent with the AE-syntax [6]:

Definition 14. An AE-scheme $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ is $(q_{\mathsf{L}}, q_E, t, \epsilon)$-CPAL-*secure (*CPA *with leakage)*, if for any (q_L, q_E, t)-adversary $\mathsf{A} = (\mathsf{A}_1, \mathsf{A}_2)$

$$\Big| \Pr[b = b' \mid b' \leftarrow \mathsf{A}_2^{\mathsf{L}(\cdot,\cdot;\cdot),\mathsf{AEncL}_k(\cdot,\cdot)}(c^*, \ell^*, st),\ (c^*, \ell^*) = \mathsf{AEncL}_k(a, m_b),$$
$$(st, m_0, m_1) \leftarrow \mathsf{A}_1^{\mathsf{L}(\cdot,\cdot;\cdot),\mathsf{AEncL}_k(\cdot,\cdot)} \text{ s.t. } |m_0| = |m_1|, b \xleftarrow{\$} \{0,1\}] - \frac{1}{2} \Big| \leq \epsilon$$

where $\mathsf{L}(\cdot,\cdot;\cdot)$ is an oracle used to model the leakage, A_1 is a $(q_{1,L}, q_{1,E}, t_1)$-adversary, A_2 is a $(q_{2,L}, q_{2,E}, t_2)$-adversary, with $q_{1,L} + q_{2,L} \leq q_L$, $q_{1,E} + q_{2,E} \leq q_E$ and $t_1 + t_2 \leq t$. A scheme is CPAL2 if in the previous definition the adversary receives both ℓ^* and also ℓ^*_D which is the leakage of $\mathsf{ADec}_k(a, c^*)$.

Note that $\mathsf{L}(\cdot,\cdot;\cdot)$ is an oracle used by A to understand and model the leakage of AEnc. In these queries, A can choose both the inputs and the key, as suggested in [46].

Alternative approach proposed by Barwell et al. [3] started from the real-vs-ideal game and gave the adversary the ability to access leaking queries in both worlds

Definition 15. An AE-scheme $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$-scheme has $(q_E, q_{EL}, t, \epsilon)$-*encryption security with leakage* (IND−CLPA) if for any (q_E, q_{EL}, t)-adversary A

$$|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{AEnc}_k(\cdot,\cdot),\mathsf{AEncL}_k(\cdot,\cdot)}] - \Pr[1 \leftarrow \mathsf{A}^{\$(\cdot,\cdot),\mathsf{AEncL}_k(\cdot,\cdot)}]| \leq \epsilon,$$

where $\forall(a, m)$ $\$(a, m)$ picks $c \xleftarrow{\$} \{0,1\}^{|\mathsf{AEnc}_k(|a,m)|}$. [7]

Differences between these definitions. The IND−CLPA notion does not guarantee message security in the presence of leakage. Indeed, a scheme can still be IND−CLPA-secure even if the leakage function reveals the full message, i.e., $\mathsf{L}_{\mathsf{AEnc}}(a, m; k) = m$. IND−CLPA provides privacy only for encryption when there is no leakage (thus, it provides at most *leakage-resilience*).

On the other hand, CPAL provides *leakage-resistance*, because it provides privacy for encryption when there is leakage. However, to have a secure scheme that is CPAL-compliant, actual leakage functions are further restricted (or must hold more underlying assumptions), since they cannot leak any part of the message. Noteworthy CPAL provides a meaningful definition which can be tested and falsified in practice.

2.7 CONCRETE

Pereira et al. [46] proposed a scheme which is CPAL-secure, PSV (see Alg. 1, and Fig. 1). It relies on a leveled implementation. They use the master key k to create the first ephemeral key k_1 from a random value iv via a strongly protected BC F: $k_1 = \mathsf{F}_k(\mathsf{iv})$. This key k_1 is then used with a weakly protected BC E in two ways: (i) to refresh itself generating $k_2 = \mathsf{E}_{k_1}(p_A)$, and (ii) to create a pseudo-random value $\mathsf{E}_{k_1}(p_B)$, which is XORed with the first message block m_1. This gives the first ciphertext block $c_1 = \mathsf{E}_{k_1}(p_B) \oplus m_1$ (p_A and p_B

[6] The original definition [46] for encryption scheme, is obtained from ours replacing the AE scheme with an encryption scheme.

[7] The original definition [3] is not quantitative and it is for nonce-based AE, and can be obtained from ours, simply adding the nonce as input to AEnc_k. Moreover, the adversary is not allowed to repeat queries between her oracles, because they are deterministic, and she cannot repeat the nonce.

are public constants). Then, we iterate, creating new ephemeral keys, and encrypting new blocks of the message until we have encrypted the last message block m_l. The decryption algorithm takes as input $(\mathsf{iv}, c_1\|...\|c_l)$, recomputes the first ephemeral key k_1 from iv, and, then, it correctly decrypts.

Algorithm 1 The leakage-resistant encryption scheme PSV [46].

It uses a leak-free BC $\mathsf{F} : \mathcal{K} \times \{0,1\}^n \to \{0,1\}^n$, and a weakly protected BC $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$.

Gen:

- $k \xleftarrow{\$} \mathcal{K}$
- $p_A, p_B \xleftarrow{\$} \{0,1\}^n$

$\mathsf{Enc}_k(m)$:

- Parse m in $m_1, ..., m_l$
- $\mathsf{IV} \xleftarrow{\$} \{0,1\}^n$
- $k_1 = \mathsf{F}_k(\mathsf{iv})$
- For $i = 1, ..., l$
  - $y_i = \mathsf{E}_{k_i}(p_B)$
  - $c_i = \pi_{|m_i|}(y_i) \oplus m_i$
  - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
- $c = (c_1, \ldots, c_l)$
- Return (iv, c)

$\mathsf{Dec}_k(\mathsf{iv}, c)$:

- Parse c in $c_1, ..., c_l$
- $k_1 = \mathsf{F}_k(\mathsf{iv})$
- For $i = 1, ..., l$
  - $y_i = \mathsf{E}_{k_i}(p_B)$
  - $m_i = \pi_{|c_i|}(y_i) \oplus c_i$
  - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
- Return $m = (m_1, \ldots, m_l)$

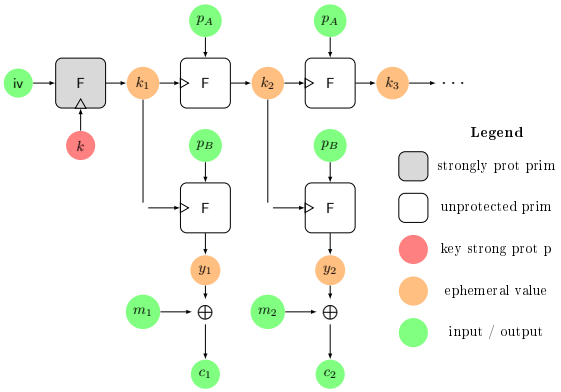

Figure 1: PSV [46]

Berti et al. [14], based on PSV, proposed a scheme, EDT (Encrypt-then-Tag), which is CIL2-secure in the unbounded leakage model [8]. They extended PSV to authenticate the ciphertext using the classic hash-then-MAC paradigm. First, the ciphertext is hashed, then the master key is reused with a strongly protected TBC to compute the tag $\tau = \mathsf{F}^1_k(\mathsf{H}(c_1|...|c_l))$. In decryption, however, they avoid recomputing $\tilde{\tau}$ (to prevent its leakage). Instead, they invert the TBC, compute $\tilde{h} = \mathsf{F}_k^{-1,1}(\tau)$, and check if $\tilde{h} = h$, where $h =$

[8] We give a simplified version. For the full details, see the original paper.

$\mathsf{H}(c_1|...|c_l)$. The core idea is that, in order to use $\tilde{h}$ for a forgery, an adversary would need to find a pre-image of it; however, this is hard since $\tilde{h}$ behaves as a random value (requiring a range-oriented pre-image resistant hash function H [14]). This implies that to check the authenticity of a message we actually compute something ($\tilde{h}$) that, if leaked, does not compromises security.

To reduce the number of times the strongly protected TBC is used Berti et al. [16, 15] proposed a new AE-construction in 2019: $\mathsf{CONCRETE}$ (Commit-Encrypt-Send-the-Key) (see Fig. 5, and Fig. 2, and Alg. 2). Instead of deriving the first ephemeral key k_0, they sample it at random. They then commit to the key, with $c_0 = \mathsf{E}_{k_0}(p_B)$, and refresh it with $k_1 = \mathsf{E}_{k_0}(p_A)$. Then, to encrypt the message, they use PSV with k_1 as the first ephemeral key. Finally, to send the first ephemeral key k_0, they used the strongly protected TBC and the master key k, producing $c_{l+1} = \mathsf{F}^h k(k_0)$. Here $h = \mathsf{H}_s(c_0|c_1|...|c_l, a)$ is a multi-input collision resistant hash function (see Def. 2, Sec. 2.1.2 where we also give a possible hash function). In decryption, given the ciphertext $c = (c_0, c_1, ..., c_l, cl + 1)$ and associated data a, the algorithm first retrieves $k_0 = \mathsf{F}^{-1,h} k(cl + 1)$, then checks the correctness of the commitment c_0, and finally decrypts. $\mathsf{CONCRETE}$ has been proved to be AE secure in the black-box model, $\mathsf{CIL2}$ in the unbounded leakage model assuming that F is leak-free and $\mathsf{CPAL2}$ secure [16]. We now iterate the main results:

AE-security. For simplicity, in all the theorem statements we omit the quantitative bounds for time (which can be found in [15]).

Theorem 1. *Let* F *be a* $(q_{\mathsf{F}}, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$ *with* n*-bit blocks, let* E *be a* $(2, \epsilon_{\mathsf{PRF}})$-$\mathsf{PRF}$*, and let* H *be a multi-input* (ϵ_{CR})*-collision resistant hash function, then, the mode* $\mathsf{CONCRETE}$*, which encrypts at most* L*-block long messages is* (q_E, q_D, ϵ)-AE*-secure with*

$$\begin{aligned}\epsilon \leq & \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + \frac{(q_D+1)(L+1)(q+q_E)}{2^{n+1}} + \frac{q_D}{2^n} \\ & + (q_E(L+1) + q_D)\epsilon_{\mathsf{PRF}} + \frac{q_E(L+1)[q_E(L+1)-1]}{2^{n+1}} + \frac{(q_D+q_E)(q_D+q_E-1)}{2^{n+1}}.\end{aligned}$$

We briefly explain the terms of the bound:

- $\epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + \frac{(q_D+1)(L+1)(q+q_E)}{2^{n+1}} + \frac{q_D}{2^n} + (q_D+1)\epsilon_{\mathsf{PRF}}$ is needed for authenticity. In particular,
  - $\epsilon_{\mathsf{sTPRP}} + \frac{(q_D+1)(q_E)}{2^{n+1}}$ because the outputs of F and F^{-1} are random;
  - $q_D\epsilon_{\mathsf{PRF}}$ because we are checking c_0 and not k_0. We assume c_0 is random because E is a PRF;
  - $\frac{(q_D+1)(L+1)(q+q_E)}{2^{n+1}}$ because in every decryption query, the k_0 we have obtained has never been used before (to use the PRF-security of E).
- $(q_E(L+1) + q_D)\epsilon_{\mathsf{PRF}} + \frac{q_E(L+1)[q_E(L+1)-1]}{2^{n+1}} + \frac{(q_D+q_E)(q_D+q_E-1)}{2^{n+1}}$ is for confidentiality for PSV. In particular,
  - $\frac{q_E(L+1)[q_E(L+1)-1]}{2^{n+1}}$ to ensure that all keys used in the PSV part have never been used before;
  - $q_E(L+1)\epsilon_{\mathsf{PRF}}$ is due to the fact that we replace all calls to E with calls to a random function in the encryption queries.

The idea of the proof is first to prove the integrity (see the following theorem for more details), then, for an encryption query if k_0 is random, all ciphertext blocks $c_0, \ldots, c_l$ are random due to the PRF-security of E (except if there are two ephemeral keys which collide in the whole story of the game). The last ciphertext block, c_l is random due to the sTPRP security of F and the collision resistance of H.

***Integrity* in the presence of leakage - CIL2** Now, we move to authenticity in the presence of leakage (denoted $\mathsf{CIML2}$, a variant of $\mathsf{CIL2}$ where the adversary controls the randomness).

Algorithm 2 The CONCRETE algorithm [16, 15].

It uses a TBC $\mathsf{F} : \mathcal{K} \times \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$ with a strongly protected implementation, a BC $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ with a weakly protected implementation and a hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \mathcal{TW}$. $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \times \{0,1\}^* \to \mathcal{TW}$ is a multi-input collision resistant hash function based on H.

Gen:

- $k \xleftarrow{\$} \mathcal{K}$, $s \xleftarrow{\$} \mathcal{HK}$
- $p_A, p_B \xleftarrow{\$} \{0,1\}^n$

$\mathsf{AEnc}_k(a, m)$:

- Parse m in $m_1, ..., m_l$
- $k_0 \xleftarrow{\$} \{0,1\}^n$
- $c_0 = \mathsf{E}_{k_0}(p_B)$
- $k_1 = \mathsf{E}_{k_0}(p_A)$
- For $i = 1, ..., l-1$
  - $y_i = \mathsf{E}_{k_i}(p_B)$
  - $c_i = y_i \oplus m_i$
  - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
- $y_l = \mathsf{E}_{k_l}(p_B)$
- $c_l = \pi_{|m_l|}(y_l) \oplus m_l$
- $h = \mathsf{H}_s(c_0 \| ... \| c_l, a)$
- $c_{l+1} = \mathsf{F}_k^h(k_0)$
- Return $c = c_0 \| ... \| c_l \| c_{l+1}$

$\mathsf{ADec}_k(a, c)$:

- Special Parse (l) c in $c_0, ..., c_l, c_{l+1}$
- $h = \mathsf{H}_s(c_0 \| ... \| c_l, a)$
- $k_0 = \mathsf{F}_k^{-1,h}(c_{l+1})$
- $\tilde{c_0} = \mathsf{E}_{k_0}(p_B)$
- If $c_0 \neq \tilde{c_0}$
  - Return $\perp$
- Else $k_1 = \mathsf{E}_{k_0}(p_A)$
- For $i = 1, ..., l-1$
  - $y_i = \mathsf{E}_{k_i}(p_B)$
  - $m_i = y_i \oplus c_i$
  - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
- $y_l = \mathsf{E}_{k_l}(p_B)$
- $m_l = \pi_{|c_l|}(y_l) \oplus c_l$
- Return $m = m_1 \| ... \| m_l$

$\mathsf{H}'_s(c, a)$

- Parse a and c
- $a' = a_1 \| 1 \| a_2 \| 1 \| ... \| 1 \| a_{l_a} \| 1^{n-l_a-1}$
- $c' = c_1 \| 0 \| c_2 \| 0 \| ... \| 0 \| c_{l_c} \| 0^{n-l_c-1}$
- $h = \mathsf{H}_s(c' \| a')$

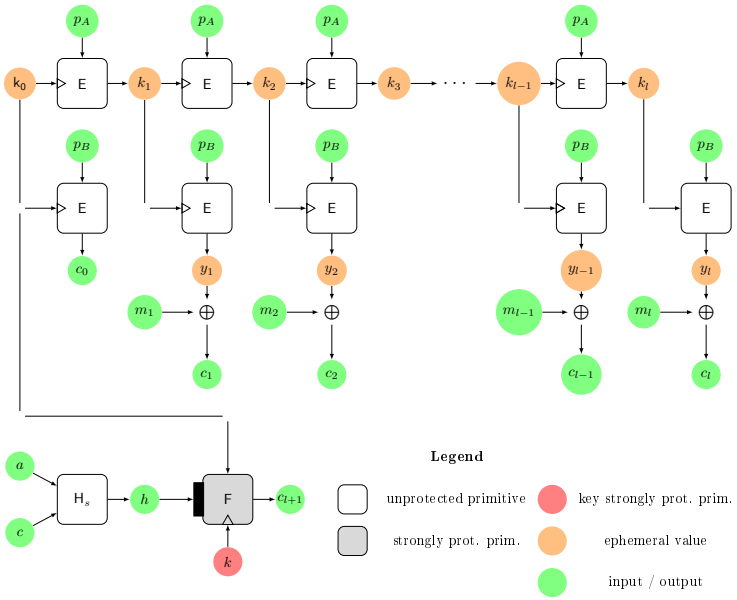

Figure 2: CONCRETE [16]. H is a multi-input hash function. $c = c_0 \| ... \| c_l$.

Thus, in this model the adversary can choose k_0 for encryption queries. Assuming F is leak-free, the leakage functions of CONCRETE behave as follows:

- For encryption, $\mathsf{L}_{\mathsf{AEnc}}(a, m, k_0; k)$ returns nothing, since with k_0 the adversary can already compute all ephemeral values (except $\mathsf{F}_k^h(k_0)$, which corresponds to c_{l+1}).
- For decryption, $\mathsf{L}_{\mathsf{AEnc}}(a, c; k) := k_0$, because from k_0 the adversary can recompute all ephemeral values and the output.

Theorem 2. *Let* F *be a leak-free* $(q_{\mathsf{F}}, \epsilon_{\mathsf{sTPRP}})$-sTPRP *with* n*-bit blocks, let* E *be a* $(2, \epsilon_{\mathsf{PRF}})$-PRF*, and let* H *be a* (ϵ_{CR})*-collision resistant hash function, then, in the unbounded leakage model, the mode* CONCRETE*, which encrypts at most* L*-block long messages is* (q_E, q_D, ϵ)-CIML2*-secure with*

$$\epsilon \leq \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + \frac{(q_D+1)(L+1)(q+q_E)}{2^{n+1}} + \frac{q_D+1}{2^n} + (q_D+1)\epsilon_{\mathsf{PRF}}.$$

The idea of the proof is that if the ciphertext is fresh, then, the key obtained by $\mathsf{F}_k^{-1,h}$ is fresh (if there are no collisions for the hash), thus the probability that c_0 is a correct commitment for the key is $\leq \epsilon_{\mathsf{PRF}} + 2^{-n}$ because E is a good BC.

***Privacy* in the presence of leakage - CPAL2** When we move to privacy in the presence of leakage, we start observing that we cannot allow the adversary to choose k_0 and/or to leak without bounds[9] on the output values of E. Thus, we state the privacy for CONCRETE (the proof and the hypothesis for the leakage can be found in [16]):

Theorem 3. *Let* F *be a* $(q_E+1, \epsilon_{\mathsf{sTPRP}})$-sTPRP *whose implementation is leak-free, let* E *be a* $(2, \epsilon_{\mathsf{PRF}})$-PRF *whose implementation has some leakage property, let* PSVs^I *be* $(q_{\mathsf{L}}, \epsilon_{\mathsf{EavL2}})$-EavL2*-secure, then, the mode* CONCRETE*, if it encrypts at most* L*-blocks messages, is* (q_E, ϵ)-CPAL2*-secure with*

$$\epsilon \leq \epsilon_{\mathsf{sTPRP}} + \frac{q_E}{2^n} + \epsilon_{2\text{-}\mathsf{sim}'} + (L+1)\epsilon_{\mathsf{PRF}} + L(\epsilon_{2\text{-}\mathsf{sim}} + \epsilon_{\mathsf{EavL2}}).$$

[9] Otherwise it would be hard to achieve any security, and such restrictions or assumptions on the actual leakage function would not be meaningful

Here PSVs^I denotes an idealized, single-message version of PSV, where both the random value y and the new ephemeral key are chosen uniformly at random. EavL2 is an eavesdropper game in which the adversary must distinguish the encryption (with leakage) of two chosen plaintexts. As in CPAL2, she also receives the leakage of the decryption of the resulting ciphertext. Intuitively, EavL2 is the CPAL2-game with $q_E = 0$, i.e., no encryption queries are allowed. The definition of $\epsilon_{\mathsf{2\text{-}sim}}$, $\epsilon_{\mathsf{2\text{-}sim}'}$ and the leakage property of the implementation of E are left to the original paper [15].

In summary, the CPAL2-security of the full scheme is reduced to the EavL2-security of a simplified scheme that only encrypts n-bit messages.

2.8 *Integrity* in the Presence of Faults and Leakage

In this subsection we recall the two approaches proposed for defining integrity (with leakage) in the presence of faults: [10] and [52]. In both definitions, the key-generation algorithm is assumed to be leakage-resistant (since it is done only once and/or can be protected [10]).

Notations. We denote a faulted oracle by adding the suffix Flt to it. The fault is seen as an additional input of the oracle. Finally, with $\mathsf{A}^{\mathsf{L},\mathsf{Flt}}$ we denote an adversary which holds a leaky and faulty model.

2.8.1 Atomic Model

Berti et al. [10] start from the standard black-box authenticity definition of a MAC and add leakage and faults. They assume that the adversary must choose all the faults *before* the actual computation starts. Thus, if the adversary wants to see the output (and the leakage) of a computation of the implementation of algorithm Algo where she injects some faults, she chooses the inputs (the leakage) and the faults $\mathsf{Flt}_{\mathsf{Algo}}$ at the same time [11].

To represent the fault Flt that the adversary wants to inject they introduce the *dependency matrix*. If an algorithm Algo computes outputs y from inputs $(x_1, x_2, ..., x_n)$, we can reduce its implementation to a sequence of steps, called *atoms*: $f_1(x_1, x_2, ..., x_n) = z_1,\ f_2(x_1, x_2, ..., x_n, z_1) = z_2,\ \ldots,\ f_N(x_1, x_2, ..., x_n, z_1, ..., z_{N_1}) = z_N$. where f_i is any deterministic function or the random picking[12]. Note that this description is not unique. As an example, we describe CONCRETE with these functions in Sec. 4.1, and of LR–MACr in the following of this section. The output y is obtained from the outputs of the f_is choosing some of them (and eventually rearranging) via the function Select. This function depends only on the length of the inputs. That is, $y = \mathsf{Select}(y_1, ..., y_N)$, with $y = y_{i_1}| \ldots |y_{i_L}$ and $L = g(|x_1|, \ldots, |x_n|)$ for some function $g : \mathbb{N}^n \to \mathbb{N}$. The indices $i_1, \ldots, i_L$ are in $[N]$. Equivalently, we can see them as the output of a function $g_{out} : \mathbb{N}^n \to \mathcal{P}([N])$, which depends only on the input lengths.

But, usually, we do not effectively use all the inputs for the f_is. For example, in CONCRETE we compute $k_{i+1} = \mathsf{E}_{k_i}(p_A)$, thus, in this computation all inputs and all ephemeral values are not used except for k_i. The *dependency matrix* is built from the

[10] Since our adversary is very strong, some assumptions and security mechanisms will be required, otherwise security will either be impossible with the proposed tools used or simply too expensive; we believe these are standard scenarios and questions asked by security architects in the different implementation levels. For example, taking the master key, we may assume that we implement the (T)BC with an implementation where the key is either: (1) hard-coded, (2) partially hard coded (3) uses a secured memory or derived from a secure primitive (perhaps with a TPM) (4) may be generated aided a PUF mechanism.

[11] They assume that the adversary cannot see the leakage of part of the computation, and then chooses the faults accordingly. This is a strong requirement. At a lower level, Berndt et al. [6] did not make this assumption (restriction).

[12] Note that this approach is coherent with the first works in leakage-resilient cryptography. We can see atoms as the Virtual Turing Machines of [43].

inputs of the f_is, by writing *only* the effective inputs[13] and putting the string ϵ for all others. As an example, we give the dependency matrix of LR–MACr below

They assume that the adversary can only set faults between the atoms. Considering *leveled implementations* in the faults context, the granularity of atoms can be made finer or coarser as discussed below. We believe that it would be easier to protect smaller blocks than a full scheme. This model is quite flexible and allows us to understand where fault countermeasures are strictly needed. Moreover, the adversary is not allowed to input any fault in the selection function Select. That is, she can only fault the inputs of the atoms. Thus, we can represent the fault $\mathsf{Flt}_{\mathsf{Algo}}$ with the *fault matrix* which shows how each input is modified. That is, if in f_i, the input x_j is replaced with a stuck-at fault with x'_j (resp. with the differential fault Δ), the fault matrix represents this by setting the i, j-th element to x'_j (resp.$\cdot \oplus \Delta$), or if there is no fault, we set it to $\cdot$.

Clearly, we cannot achieve security if the adversary can fault all inputs and outputs. For example, she could set one bit of the key to 0 and observe whether it affects the output. Therefore, some inputs of certain atoms must be assumed protected and cannot be faulted. When we assume that the x_i input of the jth atom is protected and cannot be faulted, we denote this by setting the (j, i)th element of the fault matrix to $\perp$; if it can only be faulted with differential faults, we use $\perp /\oplus$. The set of all these protected inputs is denoted with $\mathcal{I}$. Additionally, we can bound the number and/or the type of faults the adversary can do. The set of all admissible faults is denoted with $\mathcal{F}$.

Finally, we do not allow the adversary to *always* replace an input x_i (or the randomness [14]), when it is effectively used, with another input, x'_i (that is, if with the fault Flt we have turned the computation of $\mathsf{Mac}_k(m)$ in the computation of $\mathsf{Mac}_k(m')$ by replacing m with m' every time m is used, then, the output $\tau \leftarrow \mathsf{Mac}_k(m', \mathsf{Flt})$ is clearly s.t. $\mathsf{Vrfy}_k(m', \tau) = \top$, but we consider (m', τ) an invalid forgery). Note that two different implementations of the same algorithm can be divided into different atoms, thus there are two different fault matrices (capturing the dependency of faults from the implementations) [10].

Having presented the model, we can give Berti et al.'s definition of security in the presence of leakage and faults:

Definition 16. A $(\mathcal{I}_{\mathsf{Mac}}, \mathcal{I}_{\mathsf{Vrfy}})$-protected implementation of MAC, with leaking function pair $\mathsf{L} = (\mathsf{L}_{\mathsf{Mac}}, \mathsf{L}_{\mathsf{Vrfy}})$ and fault admissible sets $\mathcal{F} = (\mathcal{F}_{\mathsf{Mac}}, \mathcal{F}_{\mathsf{Vrfy}})$, is $(q_{FL}, q_M, q_V, t, \epsilon)$ *strongly-existentially unforgeable against $\mathcal{F}$-fault-then-leakage attacks in tag-generation and verification (*SUF–FL2*)* if for all (q_{FL}, q_M, q_V, t)-adversaries $\mathsf{A}^{\mathsf{L},\mathsf{Flt}}$ we have

$$\Pr[\mathsf{SUF\text{–}FL2}_{\mathsf{MAC},\mathcal{F},\mathsf{L},\mathsf{A}} \Rightarrow 1] \leq \epsilon$$

where the $\mathsf{SUF\text{–}FL2}_{\mathsf{MAC},\mathcal{F},\mathsf{L},\mathsf{A}}$-experiment is defined in Tab. 1.

Note that in [46], the adversary can model the leakage. Thus, we gave her access to an oracle $\mathsf{L}(\cdot, \cdot; \cdot)$ where she chooses *all* the inputs, even the key, and then she sees the leakage. Here we proceed similarly, and we give the adversary access to an oracle $\mathsf{L} - \mathsf{Flt}(\cdot, \cdot, \cdot; \cdot)$, where the adversary chooses *all* the inputs and the faults, even the key, and then she sees the leakage. In this way, she can (try to) model the leakage in the presence of faults.

Observations. First, we observe that the atoms could be finer or coarser (we can consider atoms, for example, a call of a block cipher, or its rounds or even the gates). Moreover, this model covers both *transient* (faults which modify a value for a single computation) and *persistent* (faults which modify a value for all subsequent computations) faults. In addition, when all the inputs of a component are known before the start of the computation any

[13] We say that the input y is *ineffective* for the function $f(x, y)$, if $\forall x, y, y' \ f(x, y) = f(x, y')$. All the other inputs are effective.

[14] Otherwise, the scheme would be derandomized.

Table 1: The SUF–FL2 experiment. To simplify the description of the experiment, we have omitted the checks that $\mathsf{Flt} \in \mathcal{F}_M$ (resp. $\mathcal{F}_V$) during tag-generation (resp. verification) queries and $\mathsf{Flt} \in \mathcal{F}_F$ for the output.

The $\mathsf{SUF\text{–}FL2}_{\mathsf{MAC},\mathcal{F},\mathsf{L},\mathcal{F},\mathsf{A}^{\mathsf{LFlt}}}$ experiment	
Initialization: $k \leftarrow \mathsf{Gen}$ $\mathcal{S} \leftarrow \emptyset$ Finalization: $(m,\tau) \leftarrow \mathsf{A}^{\mathsf{L-Flt},\mathsf{MacLFlt}_k,\mathsf{VrfyLFlt}_k}$ If $(m,\tau) \in \mathcal{S}$ or $\mathsf{Vrfy}_k(m,\tau) = \perp$ Return 0 Return 1	Oracle $\mathsf{MacLFlt}_k(m,\mathsf{Flt})$: $\tau = \mathsf{Mac}_k(m,\mathsf{Flt})$ $\mathcal{S} \leftarrow \mathcal{S} \cup (m,\tau)$ Return $(\tau, \mathsf{L}_M(\mathsf{Mac}_k(m,\mathsf{Flt})))$ Oracle $\mathsf{VrfyLFlt}_k(m,\tau,\mathsf{Flt})$: $\mathsf{ans} = \mathsf{Vrfy}_k(m,\tau,\mathsf{Flt})$ $\ell = \mathsf{L}_V(\mathsf{Vrfy}_k(m,\tau,\mathsf{Flt}))$ Return (ans,ℓ)

(deterministic) fault inside them can be seen as a fault on its output (e.g., considering the well-known MAC Hash-then BC if we compute $\mathsf{Mac}_k(m) = \mathsf{F}_k(\mathsf{H}(m))$, with F a BC and H a hash function, if we consider H and F the atoms of this computation, since the adversary chooses m, any faults in the computation of $\mathsf{H}(m)$ can be replaced by a correspondent fault on $h = \mathsf{H}(m)$ in the computation of $\tau = \mathsf{F}_k(m)$). *Finally, when the forgery of the adversary is checked, no faults are allowed, because if the adversary can force the* Vrfy *algorithm to output always* $\top$ *("accept"), no cryptographic countermeasure can be useful.*

2.9 Example of Atomic Model - LR–MACr - Proving Security in the Presence of Faults (and Leakage)

To clarify this better, we apply the atomic model to LR–MACr (Alg. 3 and Fig. 3), a MAC proposed by Berti et al. [10]. This MAC is probabilistic. It takes as input a random value which is used *both* in the hash and as input of the TBC. Therefore, the adversary cannot predict h.

Now, we move to its leakage functions in unbounded leakage model, and then, to the divisions into atoms of the tag-generation and verification algorithms.

Leakage functions. Regarding the leakage functions, we have $\mathsf{L}_{\mathsf{Mac}} = \mathsf{L}_F$ (the leakage of the computation of F), while $\mathsf{L}_{\mathsf{Vrfy}} = (\tilde{x}, \mathsf{L}_{\mathsf{F}^{-1}})$.

Division into atoms of Mac. Since there is a random input r, we assume an atom, denoted with f_0, is dedicated to this picking. From now on, we consider the randomness r as an additional input, thus, it cannot always be replaced with the same value. As the key is protected against faults, we use as atoms the picking $r \xleftarrow{\$} \{0,1\}^n$, the hash function $\mathsf{H}_s(\cdot)$, and the TBC $\mathsf{F}_k^{\cdot}(\cdot)$ [15] . Therefore, we do not need to protect any input of our atoms, thus, $\mathcal{I}_{\mathsf{Mac}} = \emptyset$.

For Mac, we have as inputs $x_1 = m$, then f_0 takes no input and outputs $x_2 = r$ picked uniformly at random, $y_1 = h = \mathsf{H}_s(r\|m) = \mathsf{H}_s(x_2\|x_1)$, $\tau = y_2 = \mathsf{F}_k^h(r) = \mathsf{F}_k^{y_1}(x_2)$. We assume that the adversary can only insert differential faults. Thus, we can define the dependency matrix for Mac and characterize the fault matrices:

$$\begin{pmatrix} \epsilon & \epsilon & \epsilon \\ x_1 & x_2 & \epsilon \\ \epsilon & x_2 & y_1 \end{pmatrix} \quad \text{and} \quad \mathsf{Flt}_{\mathsf{Mac}} = \begin{pmatrix} \epsilon & \epsilon & \epsilon \\ \cdot & \oplus z_2 & \epsilon \\ \epsilon & \oplus z_3 & \oplus z_4 \end{pmatrix}.$$

[15] Since the atom is F_k the protection of k is implicit, while if we use F as an atom we would have to protect the k input for F.

Algorithm 3 The LR–MACr algorithm [10].

It uses a strongly protected TBC $\mathsf{F} : \mathcal{K} \times \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$ and a hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^n \to \mathcal{TW}$.

- Gen
  - $k \xleftarrow{\$} \mathcal{K}$
  - $s \xleftarrow{\$} \mathcal{HK}$
- $\mathsf{Mac}_k(m)$:
  - $r \xleftarrow{\$} \{0,1\}^n$
  - $h = \mathsf{H}_s(r\|m)$
  - $\tau = \mathsf{F}_k^h(r)$
  - Return (r, τ)
- $\mathsf{Vrfy}_k(m, r, \tau)$:
  - $h = \mathsf{H}_s(r\|m)$
  - $\tilde{x} = \mathsf{F}_k^{h,-1}(\tau)$
  - If $\tilde{x} \stackrel{?}{=} r$ Return $\top$
  - Else Return $\bot$

Note that since x_1 is used only once, then we cannot fault it (otherwise, we are computing the Mac of $m \oplus z_1$). Finally, $z_2 \neq z_3$ (if both are not equal to 0^n, that is, not injecting a fault), because otherwise, we are always replacing the random input with another.

Division into atoms of Vrfy. For Vrfy_k, the inputs are $(x_1, x_2, x_3) = (m, r, \tau)$, the atoms are $\mathsf{H}_s(\cdot)$ and $\mathsf{F}_k^{-1,\cdot}(\cdot)$, and the comparison: $h = y_1 = \mathsf{H}_s(r\|m) = \mathsf{H}_s(x_2\|x_1)$, $\tilde{x} = y_2 = \mathsf{F}_k^{-1,h}(\tau) = \mathsf{F}_k^{-1,y_1}(x_3)$ and the output is the result of the comparison $\tilde{x} \stackrel{?}{=} r$, that is, $y_2 \stackrel{?}{=} x_2$. As before, $\mathcal{I}_{\mathsf{Vrfy}} = \emptyset$. Thus, we can define the dependency matrix for Vrfy and characterize the faults, the adversary can inject (as before only differential faults):
$\begin{pmatrix} x_1 & x_2 & \epsilon & \epsilon & \epsilon \\ \epsilon & \epsilon & x_3 & y_1 & \epsilon \\ \epsilon & x_2 & \epsilon & \epsilon & y_2 \end{pmatrix}$ and $\mathsf{Flt}_{\mathsf{Vrfy}} = \begin{pmatrix} \cdot & \oplus z_2 & \epsilon & \epsilon & \epsilon \\ \epsilon & \epsilon & \cdot & \cdot & \epsilon \\ \epsilon & \oplus z_5 & \epsilon & \epsilon & \oplus z_6 \end{pmatrix}$. Note that x_1 and x_3 are used only twice, moreover $z_2 \neq z_5$ (if there is a fault).

SUF–FL2-security of LR–MACr. Finally, we move to the proof of the SUF–FL2-security of LR–MACr.

To prove the SUF–FL2-security of LR–MACr, we model the TBC as strongly unpredictable in the presence of leakage[16]. We assume that the key of the TBC is protected against faults. Now, we can state the SUF–FL-security of LR–MACr [17] in the unbounded leakage model (also with leakage for the TBC):

[16] *Strong unpredictability in the presence of leakage* (SUP–L2), introduced in [8], models the security of strongly protected (T)BC. It implies that finding a fresh and valid couple (input/output) for the BC even with leakage is difficult. It is a falsifiable notion by an evaluation lab. It is less demanding than asking that a BC is leak-free.

[17] For simplicity, we do not state the time bounds. They can be found in [10].

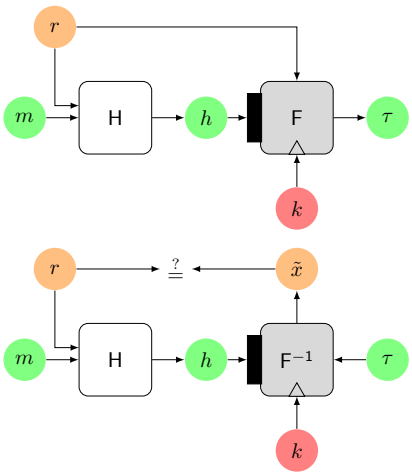

Figure 3: LR–MACr [10]

Theorem 4. *Let hash function* H *be* $(t_1, \epsilon_{\mathsf{CR}})$*-collision resistant and* $(t_2, \epsilon_{\mathsf{PR}})$*-preimage resistant. Let* F *be a* $(q_{FL}, q_M, q_V, t_3), \epsilon_{\mathsf{SUP\text{-}L2}})$-SUP–L2 TBC *with fault immune long-term key. Then, for any* (q_{FL}, q_M, q_V, t)*-adversary* $\mathsf{A}^{\mathsf{L},\mathsf{Flt}}$ *with leaking function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{Mac}}, \mathsf{L}_{\mathsf{Vrfy}})$ *and fault-injection pair* $\mathsf{Flt} = (\mathsf{Flt}_{\mathsf{Mac}}, \mathsf{Flt}_{\mathsf{Vrfy}})$. LR–MACr *is* $(q_{FL}, q_M, q_V, t, \epsilon)$*-strongly existentially unforgeable against unbounded differential fault-then-leak attacks in tag-generation and verification in the hash oracle model, with*

$$\epsilon \leq \epsilon_{\mathsf{CR}} + (q_V + 1)\epsilon_{\mathsf{SUP\text{-}L2}} + \epsilon_{\mathsf{PR}} + \frac{q_M^2}{2^{n+1}} + \frac{q_M}{2^n}.$$

For technical reasons, the SUF–FL2-security of LR–MACr has been proved in the *hash oracle model*[18].

2.10 Fault-Resilient PRFs (frPRFs), MACs, Encryption Schemes and frAEs

Saha et al. [54] introduced the notion of fault-resilient PRF (frPRF). Let $n^i_{\mathsf{pre}} \leq 1$ be the number of correct couples (input/output) that are leaked by the ith query. First, they require that $n^i_{\mathsf{pre}} \leq 1$; second, after having done *all* faulted queries, for any fresh query, they require that the outputs from F_k are indistinguishable from random. They formalize this as follows:

Definition 17. Let F be a PRF. It is a $(q_{\mathsf{Flt}}, q_{\mathsf{F}}, t, \epsilon)$-fault resilient PRF (frPRF) if $\forall$ $(q_{\mathsf{Flt}}, q_F, t)$-adversary A

$$\Pr[\exists i \in [q_{\mathsf{Flt}}] \text{ s.t. } n^i_{\mathsf{pre}}(\mathsf{A}) > 1] + |\Pr[\mathsf{A}^{\mathsf{FFlt}_k,\mathsf{F}_k} \Rightarrow 1] - \Pr[\mathsf{A}^{\mathsf{FFlt}_k,\$} \Rightarrow 1]| \leq \epsilon,$$

where $k \xleftarrow{\$} \mathcal{K}$. The adversary runs in time t and it is allowed q_{Flt} query to $\mathsf{FFlt}_k(\cdot)$, and *after having done all* the queries to the $\mathsf{FFlt}_k(\cdot)$ oracle, she can do q_{F} queries to her second oracle (either $\mathsf{F}_k(\cdot)$ or $\$(\cdot)$.

[18] In this model, the adversary may choose any target value for the hash H_s (in order to attempt a pre-image attack), provided that this value has not appeared as the output of a previous evaluation of H.

Note that the adversary can ask queries to the FFlt_k oracle, with no faults. Similarly, they define a fault-resilient random oracle (which gives random outputs for fresh inputs even if the adversary has faulted previous queries).

When we move from a PRF to a MAC, we have to consider that an adversary has to forge (that is, distinguishing if the last oracle is implemented with Vrfy_k or $\perp$). Although strictly speaking for a secure MAC it is not necessary that Mac_k outputs (pseudo)random tags, they start from the frPRF definition, getting:

Definition 18. A $(q_{\mathsf{Flt}}, q_M, q_V, t, \epsilon)$-*fault resilient* MAC *(*frMAC*)* is a MAC s.t. $\forall$ $(q_{\mathsf{Flt}}, q_M, q_V, t)$-adversary A

$$|\Pr[\mathsf{A}^{\mathsf{MacFlt}_k,\mathsf{Mac}_k,\mathsf{Vrfy}_k} \Rightarrow 1] - \Pr[\mathsf{A}^{\mathsf{MacFlt}_k,\$,\perp} \Rightarrow 1]| \leq \epsilon,$$

where $k \xleftarrow{\$} \mathcal{K}$. The adversary is not allowed to forward queries between different oracles (that is, there is a list $\mathcal{S}$ that contains all the couples (m, τ), with m the queries asked to Mac_k or \$, and τ is the oracle's answer, and a list $\mathcal{S}_{\mathsf{Flt}}$ of all queries to MacFlt_k which for every query on input m, contains the couple message (m', τ), with m' any value that m takes via fault, such that $\mathsf{Vrfy}_k(m', \tau) \neq \perp$; the adversary cannot ask to Vrfy_k any query in $\mathcal{S} \cup \mathcal{S}_{\mathsf{Flt}}$ and to the Mac_k/\$ oracle a query on input m, if there is a couple (m, τ) in $\mathcal{S}_{\mathsf{Flt}}$ for any τ). As in the frPRF definition, the adversary has to do all queries to the $\mathsf{MacFlt}_k(\cdot)$ oracle before doing any queries to her second and third oracles (respectively either Mac_k or \$, and Vrfy_k or $\perp$).

When we pass from a PRF to an ivE scheme, we have to consider that ivE schemes takes a random input for encryption queries. Thus, we have the frivE-security notion:

Definition 19. A $(q_{\mathsf{Flt}}, q_E, q_D, t, \epsilon)$-*fault resistant* ivE-*scheme (*frivE*)* $\Pi = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ if $\forall$ $(q_{\mathsf{Flt}}, q_E, q_D, t)$-adversary A

$$|\Pr[\mathsf{A}^{\mathsf{Enc}^{\$}\mathsf{Flt}_k,\mathsf{Enc}_k^{\$}} \Rightarrow 1] - \Pr[\mathsf{A}^{\mathsf{Enc}^{\$}\mathsf{Flt}_k,\$} \Rightarrow 1]| \leq \epsilon,$$

where $k \xleftarrow{\$} \mathcal{K}$. As in the frPRF definition, the adversary has to do all queries to $\mathsf{Enc}^{\$}\mathsf{Flt}_k(\cdot)$ before doing any queries to her second (either $\mathsf{Enc}_k^{\$}$ or \$).

Combining together the frivE and frMAC security definitions, we have the security notion for AE-schemes, frAE:

Definition 20. A $(q_{\mathsf{Flt}}, q_E, q_D, t, \epsilon)$-*fault resilient* AE-*scheme (*frAE*)* $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ if $\forall$ $(q_{\mathsf{Flt}}, q_E, q_D, t)$-adversary A

$$|\Pr[\mathsf{A}^{\mathsf{AEncFlt}_k,\mathsf{AEnc}_k,\mathsf{ADec}_k} \Rightarrow 1] - \Pr[\mathsf{A}^{\mathsf{AEncFlt}_k,\$,\perp} \Rightarrow 1]| \leq \epsilon,$$

where $k \xleftarrow{\$} \mathcal{K}$. The adversary is not allowed to forward queries between different oracles. That is, there is a list $\mathcal{S}$ which for any query with input (a, m) contains the couple (a, c), with c the answer of the AEnc_k or \$ oracle, and a list $\mathcal{S}_{Flt}$ of all queries to $\mathsf{AEncFlt}_k$ which for every query on input (a, m), contains the couple message (a', c), with a' any value that a takes via fault, such that $\mathsf{ADec}_k(a', c) \neq \perp$; the adversary cannot ask to ADec_k any query in $\mathcal{S} \cup \mathcal{S}_{Flt}$. As in the frPRF definition, the adversary has to do all queries to the $\mathsf{AEncFlt}_k(\cdot)$ oracle before doing any queries to her second and third oracles (respectively either AEnc_k or \$, and ADec_k or $\perp$). We can give the same definition for nAE-scheme, simply modifying the definition for the nAE-syntax. In this latter case, the adversary cannot force to repeat nonces via faults.

Again, the adversary can ask queries with no faults to the $\mathsf{AEncFlt}_k$ oracle.

2.10.1 A Secure frMAC: Hash-then-frPRF

They start from the well-known *Hash-then*-MAC. As for LR–MACr, they use a random input r. They hash the message and r, to obtain $h = \mathsf{H}(r\|m)$, and, they compute $\tau = \mathsf{F}_k(h)$, where F is a frPRF. We depict it in Alg. 4.

For security, they assume that the H is a fault-resilient random oracle (frRO), which means that if the input is fresh and random, the internal states and outputs are unpredictable [54].

frMAC-security. They prove the frMAC-security of frMAC [54]:

Theorem 5. *If* $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \{0,1\}^n$ *is a* frRO*, and* F_k *is a* $(q_{\mathsf{Flt}}, q_M + q_V, t, \epsilon_{\mathsf{frPRF}})$*-secure-*frPRF*, then,* frMAC *is a* $(q_{\mathsf{Flt}}, q_M, q_V, q_H, t, \epsilon)$*-secure-*frMAC *against adversaries that performs differential faults, with*

$$\epsilon \leq \epsilon_{\mathsf{frPRF}} + \frac{\binom{q_E}{2} + q_E q_H}{2^{|r|}} + \frac{2(q_H + q_E + q_V)^2 + 2(q_E + q_V + 1) + 2q_{\mathsf{Flt}} q_V}{2^{|h|}} + \frac{q_V}{2^{|\tau|}},$$

with $q_E = 2q_{\mathsf{Flt}} + q_M$ *and* q_H *is the number of queries made to the random oracle* H*.*

Algorithm 4 The frMAC algorithm.

It uses a frPRF $\mathsf{F} : \mathcal{K} \times \{0,1\}^n \to \{0,1\}^n$ and a hash function
$\mathsf{H} : \mathcal{HK} \times \{0,1\}^n \to \mathcal{TW}$.

- Gen:
  - $k \xleftarrow{\$} \mathcal{K}$
- $\mathsf{Mac}_k(m)$:
  - $r \xleftarrow{\$} \{0,1\}^n$
  - $h = \mathsf{H}(r\|m)$
  - $\tau = \mathsf{F}_k(h)$
  - Return (r, τ)
- $\mathsf{Vrfy}_k(m, r, \tau)$:
  - $h = \mathsf{H}(r\|m)$
  - If $\tau \stackrel{?}{=} \mathsf{F}_k(h)$ Return $\top$
  - Else Return $\bot$

2.10.2 A Secure frAE: MAC-then-Encrypt-then-MAC (MEM)

Saha et al. [54] observe that using a composition of an encryption scheme and a frMAC (e.g, Enc-then-MAC) is insecure because the adversary can fault the input of frMAC and enable the *decoupling attack*. Let us suppose that Enc outputs a ciphertext c which is the input of frMAC. We inject a fault in c before frMAC processes it, thus, modifying it to c'. Then (c', τ) is a valid forgery, with τ the output of frMAC for the previous call.

To prevent such an attack, they propose an AE scheme MEM (MAC-then-Encrypt-then-MAC) [54]. The idea is to first authenticate the message m (and the AD, a) along with a random input r, with frMAC, obtaining the first tag τ_1. Then, we use PSV (Sec. 2.7) to

encrypt both r and m, obtaining c, using τ_1 as the iv. Finally, we authenticate τ_1, c with frMAC, obtaining τ_2.
The intuition is that this construction prevents the decoupling attack. If the adversary modifies the input of the second MAC, she must change either τ_1 or c. In the first case, τ_1' is no longer valid. In the second case, when c is changed into c', the tag τ_1 will not be correct for the decryption of c', which yields (r', m'). We depict MEM in Alg. 5.

frAE-security. The security relies on the security in the presence of faults and leakage of MAC and ivE [54].

Theorem 6. *If* MAC_1 *and* MAC_3 *are two independent* $(q_{\mathsf{Flt}}, q_M, q_V, q_p, t, \epsilon_{\mathsf{frMAC}})$*-fault resilient* MAC*s, and* $\Pi_2 = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ *be a* $(q_{\mathsf{Flt}}, q_E, t, \epsilon_{\mathsf{frivE}})$*-fault-resilient* ivE*-scheme, which is built using a secure* frPRF F*, a pseudorandom key generator* KG *s.t.* $\mathsf{Enc}_{k_2} = \mathsf{KG}(\mathsf{F}_{k_2}(\mathsf{iv})) \oplus m$*, then, randomized* MAC*-then-*MAC *(*MEM*) is a randomized* $(q_{\mathsf{Flt}}, q_M, q_V, t, \epsilon)$*-*$\mathsf{frAE}$*-secure scheme against adversaries which perform differential faults, with*

$$\epsilon \leq 2\epsilon_{\mathsf{frMAC}} + \epsilon_{\mathsf{frivE}} + \frac{q_E q_V}{2^{|r|}} + \frac{\binom{q_E}{2}}{2^{|\tau_1|}} + \frac{\binom{q_E}{2}}{2^{|r|}},$$

with $q_E = 2q_{\mathsf{Flt}} + q_M$ *and* q_p *is the maximum number of queries made to either* H_1 *and* H_2*.*

3 *Integrity* in the Presence of Faults (and Leakage)

This section our integrity framework and definition in the presence of faults and leakage. To motivate it, we start by exposing a trivial fault attack that is not covered by the frMAC and frAE definitions (Sec. 2.10).

3.1 A Trivial Attack on frMAC and frAE

In our opinion, the frMAC (and frAE) framework is not completely suited to provide security because it does not cover a trivial attack (which depends solely on the framework, and not on the inherent security of a scheme). In the frMAC security definition, for a faulted query on input (m, Flt), we keep in memory the couples (m', τ) such that $\mathsf{Vrfy}_k(m', \tau) = \top$, with $\tau \leftarrow \mathsf{Mac}_k(m, \mathsf{Flt})$ and m' any value that m assumes via fault (Sec. 2.10). However, we *do not* keep in memory the couples (m', τ') such that $\mathsf{Vrfy}_k(m', \tau') = \top$, where both m' and τ' are values modified by faults.

This can easily be exploited: Let us query Mac_k on input (m, Flt), where Flt is the flipping of the first bit of the tag τ when it is output (that is, we inject a fault in the Return part of the algorithm). The faulted computation outputs $\tau' = \tau \oplus 1|0^{|\tau|-1}$ instead of the correct tag τ. We can now forge with (m, τ): indeed, $\mathsf{Vrfy}_k(m, \tau) = \top$, so the forgery is valid. It is also fresh, because non-fresh queries are only those of the form (m', τ') with $\mathsf{Vrfy}(m', \tau') \neq \perp$, where m' comes from faults. In our case, no such couple exists. Note that the previous attack does not use any information learned via leakage and can be done against any MAC scheme.

Crucial observations. The same attack can be done to a frAE-secure encryption scheme when the tag (or any block of the ciphertext) is computed. The previous attack may be deemed trivial, but nonetheless exposes a hole in the security model of [54]. In our opinion, the previous attack does not pose any security problem, and it is completely acceptable to consider the previous forgery not fresh, making the adversary unable to win with such a forgery. But the model should cover such a situation.

Algorithm 5 The MEM algorithm [54].

It uses a frPRF $\mathsf{F} : \mathcal{K} \times \{0,1\}^n \to \{0,1\}^n$ with a strongly protected implementation, a BC $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ with a weakly protected implementation and a hash function $\mathsf{H} : \{0,1\}^* \to \mathcal{TW}$ which is modelled as a random oracle.

Gen:

- $k_1, k_2, k_3 \xleftarrow{\$} \mathcal{K}$
- $p_A, p_B \xleftarrow{\$} \{0,1\}^n$
- Return $k = (k_1, k_2, k_3)$

$\mathsf{MEMEnc}_k(a, m)$:

- Parse m in $m_1, ..., m_l$
- Parse k in k_1, k_2, k_3
- $(\tau_1, r) \leftarrow \mathsf{Mac}_{k_1}(a \| m)$:
  - $r \xleftarrow{\$} \{0,1\}^n$
  - $h_1 = \mathsf{H}(r \| a \| m)$
  - $\tau_1 = \mathsf{F}_{k_1}(h_1)$
- $c = \mathsf{Enc}_{k_2}(r \| m, \tau_1)$:
  - $k_0 = \mathsf{F}_{k_2}(\tau_1)$
  - $c_0 = \mathsf{E}_{k_0}(p_B) \oplus r$
  - $k_1 = \mathsf{E}_{k_0}(p_A)$
  - For $i = 1, ..., l-1$
    - $y_i = \mathsf{E}_{k_i}(p_B)$
    - $c_i = y_i \oplus m_i$
    - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
  - $y_l = \mathsf{E}_{k_l}(p_B)$
  - $c_l = \pi_{|m_l|}(y_l) \oplus m_l$
  - $C = (c_0, c_1, ..., c_l)$
- $\tau_2 \leftarrow \mathsf{Mac}_{k_3}(\tau_1 \| C)$:
  - $h_2 = \mathsf{H}(\tau_1 \| C)$
  - $\tau_2 = \mathsf{F}_{k_3}(h_2)$
- Return $c = (\tau_1, C, \tau_2)$

$\mathsf{MEMDec}_k(a, c)$:

- Parse c in τ_1, C, τ_2
- Parse k in k_1, k_2, k_3
- $\mathsf{Vrfy}_{k_3}(\tau_1 \| C, \tau_2)$:
  - $h_2 = \mathsf{H}(\tau_1 \| C)$
  - $\tilde{\tau_2} = \mathsf{F}_{k_3}(h_2)$
- If $\tau_2 \stackrel{?}{=} \tilde{\tau_2}$
  - $r \| m = \mathsf{Dec}_{k_2}(C, \tau_1)$:
    - Parse C in $c_0, c_1, ..., c_l$
    - $k_0 = \mathsf{F}_{k_2}(\tau_1)$
    - $r = \mathsf{E}_{k_0}(p_B) \oplus c_0$
    - $k_1 = \mathsf{E}_{k_0}(p_A)$
    - For $i = 1, ..., l-1$
      - $y_i = \mathsf{E}_{k_i}(p_B)$
      - $m_i = y_i \oplus c_i$
      - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
    - $y_l = \mathsf{E}_{k_l}(p_B)$
    - $m_l = \pi_{|c_l|}(y_l) \oplus c_l$
    - $m = (m_1, ..., m_l)$
  - $\mathsf{Vrfy}_{k_3}(a \| m, \tau_1)$:
    - $h_1 = \mathsf{H}(r \| a \| m)$
    - $\tilde{\tau_1} = \mathsf{F}_{k_1}(h_1)$
  - If $\tau_1 \stackrel{?}{=} \tilde{\tau_1}$
    - Return m
- Return $\perp$

Table 2: The CIL2F2 experiment (Def. 21). To simplify its description we have omitted the checks that $\mathsf{Flt} \in \mathcal{F}_E$ (resp. $\mathcal{F}_D$) during encryption (resp. decryption) queries. The flag invalid can be set to 1 by Faulted.

The $\mathsf{CIL2F2}_{\Pi,\mathcal{F},\mathsf{L},\mathcal{F},\mathsf{A}^{\mathsf{LFlt}}}$ experiment	
Initialization: $k \leftarrow \mathsf{Gen}$ $\mathcal{S} \leftarrow \emptyset$ Finalization: $(a^*, c^*) \leftarrow \mathsf{A}^{\mathsf{L-Flt},\mathsf{AEncLFlt}_k,\mathsf{ADecLFlt}_k}$ If invalid $= 1$ Return 1 If $(a^*, c^*) \in \mathcal{S}$ or $\mathsf{ADec}_k(a^*, c^*) = \perp$ Return 0 Return 1	Oracle $\mathsf{AEncLFlt}_k(m, \mathsf{Flt})$: $c = \mathsf{AEnc}_k(a, m, \mathsf{Flt})$ $a' \leftarrow \mathsf{Faulted}(a, m, \mathsf{Flt})$ $\mathcal{S} \leftarrow \{(a', c)\} \cup \mathcal{S}$ Return $(c, \mathsf{L}_E(\mathsf{AEnc}_k(a, m, \mathsf{Flt})))$ Oracle $\mathsf{ADecLFlt}_k(a, c, \mathsf{Flt})$: $\mathsf{ans} = \mathsf{ADec}_k(a, c, \mathsf{Flt})$ $\ell = \mathsf{L}_D(\mathsf{ADec}_k(a, c, \mathsf{Flt}))$ Return (ans, ℓ)

This attack shows the complexity and subtility of modeling security against fault when the adversary can inject faults into the Mac *or* Enc *algorithm* [19].

We start by observing that the previous attack may not happen in the atomic model (Sec. 2.8.1), because we assume that the output of the scheme is formed by rearranging (and dropping) the outputs of the atoms via the Select function. In particular, for LR-MAC1, this cannot happen since the tag is the output of the block cipher F.

3.2 Security Definitions for Integrity in the Presence of Faults and Leakage

Here, we give our integrity definition for AE in the presence of faults and leakage. Our model assumes that the key generation does not leak and cannot be faulted, as in all leakage-resilient and fault-resilient models [46, 3, 13, 32, 30, 1, 10, 54].

In view of the attack explained in the previous section, **and the fact that we do not have to separate faults queries from normal queries** we follow the Berti et al. [10] framework (see Sec. 2.8). Adapting Def. 16 to the AE syntax we obtain:

Definition 21. A $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$-protected implementation of the AE-scheme $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ with leaking function pair $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ and fault admissible sets $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ is said to have $(q_{FL}, q_E, q_D, t, \epsilon)$-*ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption (*CIL2F2*)* if for all (q_{FL}, q_E, q_D, t)-adversaries $\mathsf{A}^{\mathsf{L-Flt}}$ we have: $\Pr[\mathsf{CIL2F2}_{\Pi,\mathcal{F},\mathsf{L},\mathsf{A}} \Rightarrow 1] \leq \epsilon$. The $\mathsf{CIL2F2}_{\Pi,\mathcal{F},\mathsf{L},\mathsf{A}}$-experiment is defined in Tab. 2, and Faulted is an algorithm that takes as input (a, m, Flt), looks through all the values that a assumes during the encryption and outputs a' s.t. $\mathsf{ADec}_k(a', c) \neq \perp$, with c. Moreover, if there is more than one valid couple Faulted blocks the execution, and returns the flag invalid which makes the adversary wins the experiment. We may add a suffix -de or -dd or d to denote that we allow only differential faults in encryption or in decryption or in both, respectively.

The adversary can win if she forces Faulted to output two valid couples, (a', c) and (a'', c), where a' and a'' are different values of a induced by faults. This idea is inspired by [30]. The decoupling attack [52] is covered by the previous definition.

[19] It is out of the paper's scope, but modeling such attacks is challenging in defining security notions as CCA with faults, when we want to prevent the adversary from forwarding queries between her oracles.

The q_{FL} queries are for the adversary to model the leakage in the presence of faults. Thus, as in [46], the adversary chooses all the inputs and the key. In the unbounded leakage model, where the strongly protected components are modeled as leak-free, there is nothing to model (because the adversary cannot obtain from leakage more than what we assume she can obtain: all inputs and outputs of every component except for the key of the leak-free primitives), so, we omit the q_{FL} queries.

Looking ahead, CONCRETE uses only once a, thus, due to the atomic model, the adversary cannot fault a, which means that Faulted returns a if $\mathsf{ADec}_k(a, c) = \top$, otherwise nothing. So, Faulted can never set the flag invalid to 1.

On the other hand, we may regard the previous security definition as too strict. In fact, an adversary could forge simply by taking some values that the ciphertext assumes. Let $c = \mathsf{Select}(y_1, ..., y_m)$. A possible forgery is (a', c'), where a' is one of the values that a assumes during an encryption query (say the ith). The forged ciphertext is $c' = \mathsf{Select}(y'_1, ..., y'_N)$, where each y'_j is one of the values that y_j takes in the same query. The decoupling attack is an example of such an attack. We can be satisfied with a scheme which is forgeable only with such forgeries, as long as an adversary is only able to find a single valid forgery *per encryption query*. Thus, we introduce the weak integrity in presence of faults and leakage.

Definition 22. A $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$-protected implementation of the AE-scheme $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$ with leaking function pair $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ and fault admissible sets $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ is $(q_{FL}, q_E, q_D, t, \epsilon)$ *weak ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption (*wCIL2F2*)* if for all (q_{FL}, q_E, q_D, t)-adversaries $\mathsf{A}^{\mathsf{L-Flt}}$ we have

$$\Pr[\mathsf{wCIL2F2}_{\mathcal{F},\mathsf{L},\mathsf{A}} \Rightarrow 1] \leq \epsilon$$

where the $\mathsf{wCIL2F2}_{\Pi,\mathcal{F},\mathsf{L},\mathsf{A}}$-experiment is defined in Tab. 3, and Faulted is an algorithm that takes as input all the values that a, and c assume during the encryption and outputs the couple (a', c') s.t. $\mathsf{ADec}_k(a', c') \neq \perp$.

Note that in order to force the scenario that per every encryption query the adversary can only produce a *single* valid forgery, we keep in memory all the possible combinations of the values that the associated data and ciphertext blocks assume during the computation (via faults). All these combinations are kept in memory along with the inputs of that query, that is, the message m, the associated data a, and the randomness used r. The adversary has two possibilities to win: 1) either by outputting a couple (a^*, c^*) that has never been one of these possible combinations, which is valid, or, 2) by outputting two couples (a^*, c^*), and $(a^\dagger, c^\dagger)$, that are two possible combinations coming *from the same encryption query*, which are valid.

With respect to similar definitions (for example [30, 52]) we do not ask that for each encryption query, there is only a valid decryption query that can be produced, but simply that the adversary cannot find two valid forgeries. *In other words, we move from an existential condition, namely, the existence of two valid couples (a', c') among all possible values during an encryption query, to a computational one: the adversary must find two such valid couples* [20]*. We believe that for security this is a sufficient condition, since the problem is not if two such forgeries exist, but if an adversary can find them.*

Practical implications of our definitions and security models (atomic model + unbounded leakage model). We now explain what an implementer should understand,

[20] For a message of L blocks, if the adversary can inject one fault per ciphertext block, there are 2^L possible ciphertexts c.

Table 3: The wCIL2F2 experiment. If AEnc is a probabilistic scheme, $c = \mathsf{AEnc}_k(a, m; r)$ denotes that the randomness r is used. $\mathsf{Faulted}'$ returns all possible values that a and c can take during the encryption query (according to the atomic model, Sec. 3.2). We denote with $(a^*, c^*) \notin \mathcal{S}$ that there exists no entry $(x, y, z, w, u) \in \mathcal{S}$ s.t. $(a^*, c^*) = (x, y)$. Note that Faulted (Tab. 2) and $\mathsf{Faulted}'$ are two different algorithms: the first looks among all possible values that a assumes via fault during the encryption and find the one s.t. $\mathsf{ADec}_k(a, c) \neq \perp$; instead $\mathsf{Faulted}'$ outputs all possible combinations of a, c obtained during an encryption query via fault without checking their validity. Note that we can do this in an equivalent way, keeping in memory all possible values of associated data and ciphertext blocks per encryption query, and then do the check based on this.

The $\mathsf{wCIL2F2}_{\Pi,\mathcal{F},\mathsf{L},\mathcal{F},\mathsf{A}^{\mathsf{LFlt}}}$ experiment	
Initialization: $k \leftarrow \mathsf{Gen}$ $\mathcal{S} \leftarrow \emptyset$ Finalization: $(a^*, c^*, a^\dagger, c^\dagger) \leftarrow \mathsf{A}^{\mathsf{L-Flt},\mathsf{AEncLFlt}_k,\mathsf{ADecLFlt}_k}$ If $\mathsf{ADec}_k(a^*, c^*) \neq \perp \wedge (a^*, c^*) \notin \mathcal{S}$ Return 1 If $\mathsf{ADec}_k(a^*, c^*) \neq \perp \wedge \mathsf{ADec}_k(a^\dagger, c^\dagger) \neq \perp$ If $\exists (a, m, r)$ s.t. $(a^*, c^*, a, m, r), (a^\dagger, c^\dagger, a, m, r) \in \mathcal{S}$ Return 1 Return 0 Return 0	Oracle $\mathsf{AEncLFlt}_k(m, \mathsf{Flt})$: $c = \mathsf{AEnc}_k(a, m; r)$ For $(a', c') \leftarrow \mathsf{Faulted}'(a, m, \mathsf{Flt}; r)$ $\mathcal{S} \leftarrow \{(a', c', a, m, r)\} \cup \mathcal{S}$ Return $(c, \mathsf{L}_E(\mathsf{AEnc}_k(a, m, \mathsf{Flt})))$ Oracle $\mathsf{ADecLFlt}_k(a, c, \mathsf{Flt})$: $\mathsf{ans} = \mathsf{ADec}_k(a, c, \mathsf{Flt})$ $\ell = \mathsf{L}_V(\mathsf{Vrfy}_k(m, \tau, \mathsf{Flt}))$ Return (ans, ℓ)

and how a scheme should be implemented, when it is (w)-CIL2F(2) secure under both the atomic model and the unbounded leakage model:

- *Strongly protected components against leakage:* these primitives should be protected against side-channel attacks and faults [21] (considering also combined attacks, as [53, 52]); moreover, the correctness of the computation should be enforced.
- *Not strongly protected against leakage components:* no protections against side-channel; the only protection against faults is *ensuring the correctness of the computation*, not in its secrecy.
- *Communication between the components:* first, we note that the keys of the strongly protected components should never appear as outputs of another component (otherwise, they should be assumed to be leaked to the adversary, making the protections against side-channel adversaries useless). We have two cases:
  - *Unbounded number of faults of any kind:* no need to protect communication between primitives.
  - *Bounding the number and/or the types of faults:* implementers should protect the communications such that at most that number (and/or type) of faults can be injected. No protection is needed to maintain secrecy.

This implies that we have reduced the surface for a combined fault and leakage attack, only to the strongly protected components (when an unbounded number of faults of any type are allowed in the communication between atoms) and fault detections. Looking ahead, we will observe that in practice our model can give even more information to implementers where protections can be removed, as we will discuss after each security proof below. Thus, our model explains where the protection effort against physical attacks should be concentrated to guarantee the integrity of an AE-scheme.

3.2.1 Comparison between the Atomic Model and the Fault Resilient Model

To conclude this section, we compare the fault-resilient model (see Sec. 2.10) and the atomic model used in our paper. Firstly, as previously mentioned, the attack presented in Sec. 3.1 does not work. Furthermore, in our model, there is no difference between queries in which the adversary injects faults (always to AEnc_k), and those in which the adversary interacts with the challenge oracle (either AEnc or \$). The adversary always interacts with the real scheme (AEnc_k). This implies that, unlike in the fault-resilient model, our model does not involve two separate phases: one where the adversary can inject faults and one where she cannot.

4 The Integrity of CONCRETE in the Presence of Leakage and Faults

This section is devoted to studying the integrity of CONCRETE in the presence of faults and leakage. First, we apply the atomic model to CONCRETE. Second, we prove that in the atomic model, if the adversary is not allowed to inject a fault into the key of the leak-free TBC F, CONCRETE ensures security in the presence of a leaking encryption, and a leaking and faulty decryption. Third, we prove that if the adversary can insert a single-bit fault in encryption, she can forge, and thus break integrity. Fourth, we prove that if the adversary is only allowed differential (or random) faults, CONCRETE provides weak-integrity with faults.

We emphasize that here we study CONCRETE only in the presence of faults during decryption. We do this for various reasons: first, to understand fault attacks and mode

[21] We have assumed in our model that no faults can be injected in an atom. This is equivalent to assume that the output of the atom is correct and no information is obtained when the adversary can inject faults inside it.

level countermeasures against them, it is better to start protecting only one algorithm either AEnc or ADec, than gradually both [22]. Since in [10], it was already shown that a verification algorithm can withstand faults during verification in every atom except for a single call to a strongly protected TBC, we start by considering this scenario. Furthermore, in some deployment scenarios it may be assumed that an adversary cannot fault the encryption algorithm.

4.1 CONCRETE and its Division in Atoms

We start by dividing both the encryption and the decryption algorithm of $\mathsf{CONCRETE}$ into atoms. To make the notation clearer, instead of indexing with a subset in $\mathbb{N}$, we prefer to use an indexation similar to the one used by subsections in LaTeX(e.g., $0.1, 0.2, \ldots, 0.n, 1.1, \ldots$).

4.2 Divisions into Atoms of CONCRETE

Encryption. First, we observe that the associated data, a, is used only once by $\mathsf{CONCRETE}$, thus, according to the atomic model (Sec. 2.8.1), it cannot be faulted.

For the query on input (a, m), we define $(x_1, \ldots, x_l) = (m_1, \ldots, m_l)$, the parsing of m in n-bit long blocks, $x_{l+1} = a$. Similar to what was done in [10], we assume that the master key is hard-coded into the implementation, thus, we consider $\mathsf{F}^{\cdot}_k(\cdot)$ as an atom; moreover, we consider the generation of randomness as an additional atom. So, as atoms, we consider $\mathsf{F}^{\cdot}_k(\cdot)$, $\mathsf{E}_{\cdot}(p_A)$, $\mathsf{E}_{\cdot}(p_B)$, $\mathsf{H}_s(\cdot,\cdot)$, the XOR $\cdot \oplus \cdot$, and the random picking $\cdot \xleftarrow{\$} \{0,1\}^n$.

Although this partitioning may seem very coarse, it should be understood as a first attempt to analyze security. Looking ahead, we will prove that $\mathsf{CONCRETE}$ withstands any faults in decryption, except for faults in the strongly protected TBC. Thus, we can reduce the integrity problem in the presence of faults to the requirement that a single call to a TBC be protected against faults. Building such a TBC is the subject of a different research project.

We define $f_{0.1} = k_0 \xleftarrow{\$} \{0,1\}^n$, $f_{0.2} = \mathsf{E}_{k_0}(p_B)$, then we iterate the following three steps for $i = 1, ..., l-1$: $f_{i.1}$ is the computation of k_i via $\mathsf{E}_{k_{i-1}}(p_A)$, $f_{i.2}$ is the computation of y_i via $\mathsf{E}_{k_i}(p_B)$, $f_{i.3}$ is the computation of c_i via $y_i \oplus m_i$; after that, $f_{l.1}$ is the computation of k_l via $\mathsf{E}_{k_{l-1}}(p_A)$, $f_{l.2}$ is the computation of y_l via $\mathsf{E}_{k_l}(p_B)$, $f_{l.3}$ is the computation of c_l via $\pi_{|m_l|}(y_l) \oplus m_l$, $f_{l+1.1}$ is the computation of h via $\mathsf{H}_s(c_0 \| \ldots \| c_l, a)$, and $f_{l+1.2}$ is the computation of c_{l+1} via $\mathsf{F}^h_k(k_0)$.

We observe that $f_{0.1}$ uses no input, while $f_{0.2}$ only $z_{0.1} = k_0$. For all $i = 1, ..., l$, $f_{i.1}$ uses only $z_{i-1.1} = k_{i-1}$, $f_{i.2}$ uses only $z_{i.1} = k_i$, and $f_{i.3}$ only $z_{i.2} = y_i$ and $x_i = m_i$. Finally, $f_{l+1.1}$ uses $x_{l+1} = a$, and $z_{0.2} = c_0$, $z_{1.3} = c_1, \ldots, z_{l.3} = c_l$, while $f_{l+1.2}$ uses only $z_{l+1.1}$ and $z_{0.1}$. The output is $(z_{0.2}, z_{1.3}, ..., z_{l.3}, z_{l+1.2}) = (c_0, c_1, ..., c_l, c_{l+1})$, that is, $\mathsf{Select}(z_{0.0}, \ldots, z_{l+1.2}) = (z_{0.2}, z_{1.3}, ..., z_{l.3}, z_{l+1.2})$.

We now turn to the dependency matrix, which can be directly obtained from these equations:

[22] A similar approach has been done for leakage attacks, where the first papers protect only the encryption algorithm against leakage attacks [46, 13].

$$\begin{pmatrix}
\epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon \\
\epsilon & \cdots & \epsilon & \cdots & \epsilon & z_{0.1} & \epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon \\
\vdots & & & & & \vdots & \ddots & \ddots & & \vdots & & \vdots & \vdots & & \vdots \\
\epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & z_{i-1.1} & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon \\
\epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \cdots & z_{i.1} & \epsilon & \epsilon & \cdots & \epsilon \\
\epsilon & \cdots & x_i & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \cdots & \epsilon & z_{i.2} & \epsilon & \cdots & \epsilon \\
\vdots & & & & \vdots & \vdots & \vdots & & \vdots & & \vdots & \ddots & \ddots & & \vdots \\
\epsilon & \cdots & \epsilon & \cdots & x_{l+1} & \epsilon & z_{0.2} & \cdots & \epsilon & \cdots & \epsilon & \epsilon & z_{i.3} & \cdots & \epsilon \\
\epsilon & \cdots & \epsilon & \cdots & \epsilon & z_{0.1} & \epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & z_{l+1.2}
\end{pmatrix} \tag{3}$$

We represent the rows corresponding to the computations 0.1,0.2,i.1,i.2,i.3,$l+1.1$, and $l+1.2$, together with the columns corresponding to the variables m_1, m_i, a, k_0, c_0, k_i, y_i, c_i, and h.

Decryption. We proceed in a similar way for decryption. For the query on input (a, c), we define $(x_0, x_1, \ldots, x_{l+1}) = (c_0, \ldots, c_{l+1})$, the parsing of c into n-bit blocks, $x_{l+2} = a$. Moreover, as atoms, we consider $\mathsf{F}_k^{-1,\cdot}(\cdot)$, $\mathsf{E}_\cdot(p_A)$, $\mathsf{E}_\cdot(p_B)$, $\mathsf{H}_s(\cdot,\cdot)$ and the XOR $\cdot \oplus \cdot$. Additionally, as in [10], we do not consider the comparison $c_0 \stackrel{?}{=} \tilde{c}_0$ as an atom, because we assume that both values are known by the adversary (one via leakage, and the other is part of the input).

We define $f_{0.0}$ as the computation of h via $\mathsf{H}_s(c_0\|...\|c_l, a)$, $f_{0.1}$ as the computation of k_0 via $\mathsf{F}_k^{-1,h}(c_{l+1})$, $f_{0.2}$ as the computation of c_0 via $\mathsf{E}_{k_0}(p_B)$, then we iterate these three blocks for $i = 1, ..., l-1$: $f_{i.1}$, the computation of k_i via $\mathsf{E}_{k_{i-1}}(p_A)$, $f_{i.2}$, the computation of y_i via $\mathsf{E}_{k_i}(p_B)$, and $f_{i.3}$, the computation of m_i via $y_i \oplus c_i$.

We observe that $f_{0.0}$ uses $(x_0, \ldots, x_{l+1})$, and x_{l+2} as input (that is, $(c_0, \ldots, c_l)$, and a), while $f_{0.1}$ uses only $z_{0.0} = k_0$. For all $i = 1, ..., l$, $f_{i.1}$ uses only $z_{i-1.1} = k_{i-1}$, $f_{i.2}$ uses only $z_{i.1} = k_i$, and $f_{i.3}$ only $z_{i.2} = y_i$ and $x_i = c_i$. The output is $(z_{1.3}, ..., z_{l.3}) = (m_1, ..., m_l)$, for a valid query, $\perp$ otherwise. Thus, we can describe the dependency matrix:

$$\begin{pmatrix}
x_0 & \cdots & x_i & \cdots & \epsilon & x_{l+2} & \epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots \\
\epsilon & \cdots & \epsilon & \cdots & x_{l+1} & \epsilon & z_{0.0} & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots \\
\vdots & & & & \vdots & \ddots & \ddots & & \vdots & & \vdots & \vdots & \vdots & \\
\epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & z_{i-1.1} & \cdots & \epsilon & \epsilon & \epsilon & \cdots \\
\epsilon & \cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \cdots & z_{i.1} & \epsilon & \epsilon & \cdots \\
\epsilon & \cdots & x_i & \cdots & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \cdots & \epsilon & z_{i.2} & \epsilon & \cdots \\
\vdots & & & & \vdots & \vdots & \vdots & & \vdots & & \ddots & \ddots & \ddots & \\
\end{pmatrix} \tag{4}$$

We have represented the rows corresponding to the computations 0.0, 0.1, i.1, i.2, i.3, together with the columns corresponding to the variables c_0, c_i, c_{l+1}, a, k_0, k_i, y_i, m_i.

4.3 Integrity of CONCRETE under Faults in Decryption (and Leakage)

In this section, we prove that CONCRETE does not lose integrity if the adversary can inject any number or type of faults between the atoms in decryption. First, as proved in [10], it is easier to provide authenticity for a MAC, when the adversary can inject faults only in verification than only in encryption. Therefore, it makes sense to evaluate, as a first step, if CONCRETE withstands faults only in decryption. This is because the authentication part of CONCRETE has some similarities with the MACs of [10] (note that the hash of

the message is used as a tweak). This analysis, despite potentially being interesting for deployment in certain cases, is provided as the prologue of the analysis (similarly to what has been done for leakage-resilient encryption where first the adversary can only leak in encryption, for example, see [46, 32]).

Before going into the proof, we need to introduce a new security definition for hash functions. Then, we will prove the security of CONCRETE.

4.3.1 Pre-image resistance with chosen target and random n-prefix for hash functions (PR–corpi) and its relations with collision resistance

Pre-image resistance with chosen target and random n-prefix for hash functions. In the proof, we need the following security property for a hash function $\mathsf{H} : \{0,1\}^* \to \mathcal{TW}$: given $y \in \mathcal{TW}$ chosen by the adversary and $x' \xleftarrow{\$} \{0,1\}^n$, it should be computationally infeasible to find $x'' \in \{0,1\}^*$ s.t. $x = x' \| x''$ and $\mathsf{H}(x) = y$. Formally,

Definition 23. A hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \mathcal{TW}$ is (t, ϵ)-*pre-image resistant with chosen target and random n-bit prefix input (*n–PR–corpi*)* if for any t-adversary $\mathsf{A} = (\mathsf{A}_1, \mathsf{A}_2)$

$$\Pr[\mathsf{H}_s(x) = y \wedge \pi_n(x) = x' \mid x \leftarrow \mathsf{A}_2(\mathsf{st}, x'),\ x' \xleftarrow{\$} \{0,1\}^n,\ (\mathsf{st}, y) \leftarrow \mathsf{A}_1(s),\ s \xleftarrow{\$} \mathcal{HK}] \leq \epsilon.$$

Now, we discuss the relation between PR–corpi and the well-known definition of collision-resistance (Def. 1).

We can prove that if a hash function is collision-resistant (Def. 1) then, it is PR–corpi (Def. 23), with n-bit prefix, using the rewinding technique [38]: let $\mathsf{A} = (\mathsf{A}_1, \mathsf{A}_2)$ be a PR–corpi adversary, we build a collision-resistance adversary B as follows: B sends s, the hash key she has received to A_1. When A_1 outputs (st, y), B picks $x' \xleftarrow{\$} \{0,1\}^n$ and gives x' and st to A_2 who outputs x s.t. $\pi_n(x) = x'$ and $\mathsf{H}_s(x) = y$. Then, B rewinds A_2 and she picks $z' \xleftarrow{\$} \{0,1\}^n$ and she sends z', st to A_2 who outputs z s.t. $\pi_n(z) = z'$ and $\mathsf{H}_s(z) = y$. If $x' \neq z'$ then, A has found a collision for the hash function. B wins with probability less than $\epsilon^2_{\mathsf{PR–corpi}}$, thus

$$\epsilon^2_{\mathsf{PR–corpi}} \leq \epsilon_{\mathsf{CR}} \text{ and } \epsilon_{\mathsf{PR–corpi}} \leq \sqrt{\epsilon_{\mathsf{CR}}}$$

(for simplicity, we have omitted the probability that $x' = z'$).

We believe that for a good hash function the adversary can win with a much lower success rate (for example, it is easy to see that in the random oracle model, $\mathsf{PR–corpi} = q/\mathcal{Y}$ where $\mathcal{Y}$ is output space for H and q the number of queries.

4.3.2 Integrity of CONCRETE in the Presence of Faults in Decryption (and Leakage)

Here, we prove the CIL2F–d security (Def. 21) of CONCRETE. We remind that the suffix –d denotes that the adversary is only allowed to inject faults in decryption.

Leakage function and faulty matrix. For the leakage function, assuming that F is implemented in a strongly protected way that we model as leak-free, we adopt the standard leakage functions from the unbounded leakage model (see Sec. 2.7): for encryption, $\mathsf{L}_{\mathsf{AEnc}}(a, m, k_0; k) := k_0$ (unlike CIML2 [16], here, the adversary does not choose k_0, because we assume that the random source cannot be faulted, thus this is random value not known to the adversary. From k_0, all other ephemeral values can be computed); for decryption $\mathsf{L}_{\mathsf{ADec}}(a, c; k) := k_0$. Regarding faults, the adversary is allowed to set in decryption any fault she wants between the atoms without any restriction and none in encryption, that is all entries of $\mathcal{I}_{\mathsf{AEnc}}$ are set to $\perp$, while for $\mathcal{I}_{\mathsf{ADec}}$ we allow the adversary to choose any entry different from ϵ of the matrix defined in Eq. 4, Sec. 4.2, as long as they are compliant with Sec. 2.8.1.

Theorem 7. *Let* F *be a strongly protected* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$*, let* E *be an ideal block cipher, let* A *be allowed* q_I *queries to* E*, let* H *be both* $(t_2, \epsilon_{\mathsf{CR}})$*-collision-resistant and* $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$-$n$-$\mathsf{PR\text{-}corpi}$ *hash function. Then,* $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* $\mathsf{CONCRETE} = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encoding messages at most* Ln *bits long, with leakage function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and fault admissible sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ *has* $(q_I, q_E, q_D, t, \epsilon)$-ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption $(\mathsf{CIL2F\text{-}d})$*, with*

$$\epsilon \leq \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + (q_D + 1)\epsilon_{\mathsf{PR\text{-}corpi}} + O(q^2 + q_D q_I)2^{-n}.$$

Idea of the proof. First, we observe that any fault injected after the second atom does not give any advantage to the adversary. In fact, after k_0 is obtained, the adversary can already compute by herself whether c_0 is correct. Then, let (a^*, c^*) be a forgery and let $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_{l^*}^*, a^*)$ and $k_0^* = \mathsf{F}_k^{-1,h^*}(c_{l^*+1}^*)$. Suppose that the adversary has already forced a decryption query to compute $k_0^* = \mathsf{F}_k^{-1,h^*}(c_{l^*+1}^*)$, and suppose that it had been done for the first time in the i^{th} decryption query (the $\mathsf{CIML2}$ proof is enough to prove that if $c_{l^*+2} = \mathsf{F}_k^{h^*}(k_0^*)$ in an encryption query, then, as shown in [16], the adversary cannot use it). Now, there are two possibilities: 1) the adversary has already forced a correct computation of H outputting h^*. Thus, to forge the adversary either finds a collision for h^* (but H is collision resistant) or the input of this computation c^i, a^i is s.t. $c_0^i = \mathsf{E}_{k_0^i}(p_B)$ (but, k_0^i is random, thus, $\mathsf{E}_{k_0^i}(p_B)$ is random, since E is an ideal cipher). 2) the adversary has to find a pre-image of h^* whose first n-bits are $\mathsf{E}_{k_0}(p_B)$ (which are random), but H is $\mathsf{PR\text{-}corpi}$. Instead, if the adversary has not forced this computation before, then k_0 is random, thus $\mathsf{E}_{k_0}(p_B)$ is random, and therefore the probability that $c_0 = \tilde{c}_0$ is 2^{-n}.

We leave the proof to the next subsubsection, with the formal statement and time bounds.

On the need for the ideal block cipher model for E. In our security proof we need to show that given a random k_0, and its direct leakage, $c_0 = \mathsf{E}_{k_0}(p_B)$ remains indistinguishable from random. This seems straightforward to prove, but there is a problem: we need to give k_0 to the $\mathsf{CIL2F}$ adversary (as it is leaked). Therefore, we cannot use the standard PRF-game, where a PRF adversary A' against E_{k_0}, with k_0 picked uniformly at random, uses a $\mathsf{CIL2F}$ adversary A, because A' should need oracle access to E_{k_0} or a random function e, but she also needs to give to A the leakage which is a function of k_0. **But if A' knows k_0, she trivially wins the PRF game**. Moreover, if the adversary A has inserted a differential fault in k_0, or she has set to a fixed value some of its bits, A' should be able to compute the required queries correctly, and this is impossible without having access to k_0 [23].

The use of the ideal cipher model in the presence of leakage seems arguable at first glance. I.e., one may wonder how a block cipher could be ideal when it is instantiated and the adversary receives leakage of its computations. On the other hand, note that we are only using the ideal hypothesis to prove that given a random k_0, leakage and fault associated with $\mathsf{E}_{k_0}(p_B)$, $\mathsf{E}_{k_0}(p_B)$ remains random, and cannot be forecasted before. **We emphasize that the adversary receives only one such leakage instance under k_0.** We are confident that this assumption does not harm the concrete security of our scheme and the use of the ideal cipher model even if it is paired with that leakage.

[23] For the latter problem, we may have thought of a PRF-game where the adversary can ask key-related queries. Since this solution does not solve the first problem (the leakage of k_0), we have not pursued it, but we have used the ideal-cipher model.

Faults inside the atoms. In the previous proof, we have never used the fact that E is an ideal block cipher except, during decryption queries, namely for the computation of $\mathsf{E}_{k_0}(p_B)$ which serves as the correct commitment to the ephemeral key retrieved from F_k^{-1} (and which we compute outside the simulation of the decryption algorithm). Thus, if any fault is inserted in the computation of E during a decryption query, the adversary does not gain any advantage in forging. Moreover, injecting any fault into H (in decryption) does not pose a problem: similarly to [10], an adversary can set the output of H when it is used, which is easier and more powerful than faulting the internal computation of H. Thus, our construction will only require no faults occurring within F. *That is, in the implementation of* CONCRETE *only a single call to a single primitive must be protected against faults and leakage to provide integrity in the presence of leakage and faults (in decryption).* We formalize this intuition in the next subsubsection, giving the model, the theorem and the proof.

Take-away for implementers. The previous theorem states that, for decryption queries, we *only* need to ensure that the strongly protected TBC F is well secured against leakage[24]. In decryption queries, *we do not have to protect anything else against leakage and/or faults* (even if the output of F is not correct, there is nothing the adversary can do). **This significantly reduces the overhead of protecting such a scheme**.

An implementer can therefore come to the plausible conclusion that the only practical way to break strong unpredictability with leakage is recovering the key of the TBC for a good TBC [25]. Thus, it should be enough: to 1) to use a good hash function and a good BC for E (without considering any protection against leakages and faults for them, we only require that when they are properly executed they provide correct functionality), and 2) to use a good TBC for F whose implementation does not leak the key.

4.3.3 Details of the proof of Thm. 7

In this subsection we provide the details of Thm. 7. Moreover, we prove the stronger result with the adversary asserting faults inside any atom except for the strongly protected TBC F. The uninterested reader can skip to Sec. 4.4

Theorem 8. *Let* F *be a strongly protected* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-sTPRP*, let* E *be an ideal block cipher, let* A *be allowed* q_I *queries to* E*, let* H *be both a* $(t_2, \epsilon_{\mathsf{CR}})$*-collision-resistant and* $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$-$n$-PR–corpi *hash function. Then,* $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* $\mathsf{CONCRETE}\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encoding messages at most* Ln *bits long, with leaking function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and fault admissible sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ *has* $(q_I, q_E, q_D, t, \epsilon)$-ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in decryption and leakage (whitout faults) attacks in encryption (CIL2Fd)*, with*

$$\begin{aligned}\epsilon \leq & \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}} \\ & + \frac{(q+2)(q+1)}{2^{n+1}} \\ & + q_D[q_E(L+1)(q_D+1)/2 + q_I + (q_D+1)(L+2)/2]2^{-n},\end{aligned}$$

where $q = q_E + q_D + 1$*,* $t_1 = t + qt_{\mathsf{H}} + (q(2L+1)+1)t_{\mathsf{E}}$*,* $t_2 = t + q(t_{\mathsf{H}}) + (q_E+q_D)[(2L+3)t_{\mathsf{E}}]$ $t_3 = t + q(t_{\mathsf{H}}) + (q_E + q_D + 1)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] - t_{\mathsf{F}} - t_{\mathsf{E}}$.

We assume that picking a value uniformly at random does not take time.

[24] We leave the security requirement for F as an open problem, when the adversary injects faults in F^{-1}.

[25] Technically, it is not necessary, but we do not see any other way to produce a valid couple input/output except by recovering the key

Proof. We use a sequence of games for the proofs. Let E_i be the event that A wins Game i, that is, she outputs a fresh and valid forgery. For simplicity, we consider the decryption of the challenge associated data and ciphertext as the $q_D + 1$th decryption query.

Game 0. It is the standard $\mathsf{CIL2Fd}_{\mathcal{F},\mathsf{L},\mathsf{A}^{\mathsf{L},\mathsf{Flt}}}$ game. Let E_0 be the event that A wins this game.

Game 1. It is Game 0 except that we have replaced F with a random tweakable permutation f. Let E_1 be the event that A wins this game.

Transition between Game 0 and Game 1. Since Game 0 and Game 1 are the same except for the replacement of F with f, we can build an adversary B to bound the difference. She plays the sTPRP game, having to distinguish F_k from f, and simulates A's oracles using its own.

At the start of the game, B picks s in $\mathcal{HK}$ and gives it to A, and an ideal block cipher E. Moreover, he has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a, m), B simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_l$ as follows [26]: $c_0 = \mathsf{E}_{k_0}(p_B)$, and then, for all $i = 1, \ldots, l$, $k_i = \mathsf{E}_{k_{i-1}}(p_A)$, $y_i = \mathsf{E}_{k_i}(p_B)$ and $c_i = y_i \oplus m_i$ (for $i = l$ $c_l = \pi_{|m_l|}(y_l) \oplus m_l$. Then, B computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ and she queries her oracle on input (h, k_0) receiving as answer c_{l+1}. So, she can answer A $c = (c_0 \| \ldots \| c_{l+1})$ and the leakage k_0. Moreover, B adds (a, c) to the list $\mathcal{S}$.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (2l+1)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L+1)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c, Flt), B proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where a and $c_0, \ldots, c_l$ are modified according to the 0.0 row of Flt, 2) she queries her inverse oracle on input (h, c_{l+1}) which are modified according to 0.1 row of Flt, obtaining k_0 as answer, 3) she computes $\tilde{c}_0 = E_{k_0}(p_A)$ with k_0 modified according the 0.2 line of Flt, if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 4) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 5) finally, B answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, B does one oracle query and time $t_{\mathsf{H}} + (2l+1)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L+1)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), B proceeds as follows: 1) she computes $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_l^*, a^*)$, 2) she queries her inverse oracle on input (h^*, c_{l+1}^*), obtaining k_0^* as answer, 3) she computes $\tilde{c}_0^* = \mathsf{E}_{k_0^*}(p_A)$, if $\tilde{c}_0^* = c_0^*$ and $(a^*, c^*) \notin \mathcal{S}$, she outputs 1; otherwise 0.

For this, B does one oracle query and time $t_{\mathsf{H}} + t_{\mathsf{E}}$.

Thus, in total B does at most $q_E + q_D + 1 = q$ queries to her oracle and runs in time bounded by $t + q(t_{\mathsf{H}}) + (q(2L+1)+1)t_{\mathsf{E}} = t_1$.

If her oracle is implemented with F, B correctly simulates Game 0 for A, otherwise Game 1. Since F is a $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$,

$$|\Pr[E_1] - \Pr[E_0]| \leq \epsilon_{\mathsf{sTPRP}}.$$

Game 2. It is Game 1, where in decryption instead of using f^{-1}, where f is a random tweakable permutation, we pick its answers uniformly at random (and we keep a list of all queries already done to make the answers coherent). Let E_2 be the event that A wins this game.

Transition between Game 1 and Game 2. In Game 2, for decryption queries, we are using a random tweakable permutation, while in Game 3 a random function. Due to the

[26] Note that each of the steps, into which we divide this computation, corresponds to an atom (see Sec. 2.8).

well-known birthday bound, we have that

$$|\Pr[E_2] - \Pr[E_1]| \leq \frac{(q+2)(q+1)}{2^{n+1}}.$$

Game 3. It is Game 2, where during decryption queries (except for the challenge one), immediately after k_0 is computed (via f^{-1}) the decryption oracle computes $c_{l+2} := \mathsf{E}_{k_0}(p_B)$, without any faults inserted, and appends this to its normal answer (note that c_{l+2} is not part of the ciphertext; it is an auxiliary value introduced only for the security analysis). Let E_3 be the event that the adversary wins this game.

Transition between Game 2 and Game 3. We build an adversary C. C behaves as A for encryption queries. For decryption queries, C proceeds as follows: when A does a decryption query on input (a, c), C simply forwards this query to her oracle. After having received the answer and the leakage k_0, C simply takes k_0, calls the ideal cipher E on input (k_0, p_B), obtains c_{l+2}, and answers A with the oracle's answer appended with c_{l+2}. Thus, C needs time $t + q_D t_{\mathsf{E}}$ and does $q_I + q_D$ queries to the ideal cipher E, that is, this introduces an additional overhead of q_D queries to E (together with their computational cost). Thus,

$$\Pr[E_2] = \Pr[E_3].$$

Game 4. It is Game 3, except that we abort if the following event Bad_1 occurs: during a decryption query, when a new key k_0 is picked, there exists a query to the ideal cipher on input (k_0, p_B). Let E_4 be the event that the adversary wins this game.

Transition between Game 3 and Game 4. The two games are identical except if the following Bad_1 event happens: there exists a decryption query s.t. it requires to pick a new key k_0^i and there exists a previous query to the ideal cipher E on input (k_0^i, p_B). We observe that during an encryption query there are at most $L+1$ queries to $(\cdot, p_B)$ with at most $L+1$ different keys, while during a decryption query there are at most $L+2$ different queries to the ideal block cipher with input (k_i, p_B) (including k_0 and the subsequent k_i values). The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, during the jth decryption query there are at most $q_E(L+1) + q_I + (j-1)(L+2)$ different keys that, if picked, would trigger the Bad_1 event. Therefore, the probability of the Bad_1 event is bounded by

$$\begin{aligned}
\Pr[\mathsf{Bad}_1] \leq & \sum_{j=1}^{q_D} \frac{q_E(L+1) + q_I + (j-1)(L+2)}{2^n} \\
\leq & \frac{q_D[q_E(L+1) + q_I] + q_D(\frac{q_D+1}{2})(L+2)}{2^n} \\
= & q_D \left[q_E(L+1)\frac{q_D+1}{2} + q_I + \frac{(q_D+1)(L+2)}{2} \right] 2^{-n}.
\end{aligned}$$

Hence, $|\Pr[E_4] - \Pr[E_3]|$

$$\leq q_D \left[q_E(L+1)(q_D+1)/2 + q_I + (q_D+1)(L+2)/2 \right] 2^{-n}.$$

Game 5. It is Game 4, where the following event Bad_2 does not happen: the hash queries induced during encryption and decryption queries induce a collision for the hash function. Let E_5 be the event that the adversary wins this game.

Transition between Game 4 and Game 5. Since Game 4 and Game 5 are the same except if event Bad_2 happens, it is enough to bound $\Pr[\mathsf{Bad}_2]$ to bound the difference $|\Pr[E_4] - \Pr[E_5]|$. To bound $\Pr[\mathsf{Bad}_2]$, we build an adversary D who wants to find a collision for the hash function. At the start of the game, D receives a key for the hash

function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks a random tweakable permutation f. Finally, he has a list $\mathcal{H}$ which is empty.

When A does an encryption query on input (a, m), D simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_l$ as follows: $c_0 = \mathsf{E}_{k_0}(p_B)$, and then, for all $i = 1, \ldots, l$, $k_i = \mathsf{E}_{k_{i-1}}(p_A)$, $y_i = \mathsf{E}_{k_i}(p_B)$ and $c_i = y_i \oplus m_i$ (for $i = l$ $c_l = \pi_{|m_l|}(y_l) \oplus m_l$. Then, D computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_l)$, she adds $((a, c_0 \| \ldots \| c_l), h)$ to $\mathcal{H}$ and she computes $c_{l+1} = \mathsf{f}(h, k_0)$. So, she can answer A $c = (c_0 \| \ldots \| c_{l+1})$ and the leakage k_0.

For this, D needs time $t_{\mathsf{H}} + (2l+1)t_{\mathsf{E}} + t_{\mathsf{F}} \leq t_{\mathsf{F}} + t_{\mathsf{H}} + (2L+1)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c, Flt), D proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where a and $c_0, \ldots, c_l$ are modified according to the 0.0 row of Flt, and she adds $((c_0 \| \ldots \| c_l, a), h)$ to $\mathcal{H}$, 2) she computes $k_0 = \mathsf{f}^{-1}(h, c_{l+1})$ where the inputs are modified according to 0.1 row of Flt, obtaining k_0 as answer, 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 4) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 5) finally, D answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, D needs time $t_{\mathsf{F}} + t_{\mathsf{H}} + (2l+2)t_{\mathsf{E}} \leq t_{\mathsf{F}} + t_{\mathsf{H}} + (2L+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), D proceeds as follows: 1) she computes $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_l^*, a^*)$ and she adds $((c_0^* \| \ldots \| c_l^*, a^*), h)$ to $\mathcal{H}$, 2) she looks through $\mathcal{H}$ to see if there is a collision. If it is the case, she outputs it, otherwise $0^n, 1^n$. For this, D needs time t_{H}.

Thus, in total D runs in time bounded by $t + q(t_{\mathsf{H}}) + (q_E + q_D)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] = t_2$.

D correctly simulates Game 4 for A. Moreover, D finds a collision for the hash function H if event Bad_2 happens. Therefore, since H is a $(t_2, \epsilon_{\mathsf{CR}})$-collision-resistant hash function,

$$|\Pr[E_5] - \Pr[E_4]| = \Pr[\mathsf{Bad}_2] \leq \epsilon_{\mathsf{CR}}.$$

Game 6. It is Game 5, where the following event Bad_3 does not happen: for any tweak h for f used only in decryption queries, (that is, used only in f^{-1}) there exist two decryption queries, which we call the ith and the jth s.t.:

- $h' = \tilde{h}^i$,
- $c_0^j = c_{l+2}^i$ where:
  - $h' = \mathsf{H}_s(c_0^j \| \ldots \| c_l^j, a^j)$ (where a^j and $c_0^j, \ldots, c_l^j$ are modified according to the 0.0 line of Flt^j);
  - $c_{l+2}^i = \mathsf{E}_{k_0^i}(p_B)$, where $k_0 = \mathsf{f}^{-1}(h^i, \tau^i)$ is computed (modified according to the 0.1 row of Flt^i); and
  - $\tilde{h}^i$ is h^i (modified according to the 0.1 row of Flt^i).

Let E_6 be the event that the adversary wins this game.

Transition between Game 5 and Game 6. Since Game 5 and Game 6 are the same except if event Bad_3 happens, it is enough to bound $\Pr[\mathsf{Bad}_3]$ to bound the difference $|\Pr[E_5] - \Pr[E_6]|$. To bound $\Pr[\mathsf{Bad}_3]$, we divide it into $q_D + 1$ different events: $\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.q_D+1}$, where $\mathsf{Bad}_{3.\iota}$ is event Bad_3 where $\iota = i$. Clearly, $\mathsf{Bad}_3 = \bigcup_{\iota=1}^{q_D+1} \mathsf{Bad}_{3.\iota}$. To bound $\Pr[\mathsf{Bad}_{3.\iota}]$ we build an adversary EE^ι against the n–PR–corpi-security of the hash function H.

The n–PR–corpi-adversary EE^ι. At the start of the game, EE^ι receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks a random tweakable permutation f. Finally, she has two lists $\mathcal{H}$ and $\mathcal{CH}$ which are empty.

When A does an encryption query on input (a, m), EE^ι (we use EE^ι) to identify both EE_1^ι and EE_2^ι) simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_l$ as follows: $c_0 = \mathsf{E}_{k_0}(p_B)$, and then, for all $i = 1, \ldots, l$, $k_i = \mathsf{E}_{k_{i-1}}(p_A)$, $y_i = \mathsf{E}_{k_i}(p_B)$ and $c_i = y_i \oplus m_i$ (for $i = l$ $c_l = \pi_{|m_l|}(y_l) \oplus m_l$. Then, EE^ι computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$, she adds $((c_0, \ldots, c_l, a), h)$ to $\mathcal{H}$, she computes $c_{l+1} = \mathsf{f}(h, k_0)$, and she adds (h, c_{l+1}, c_0) to $\mathcal{CH}$. So, she can answer A $c = (c_0 \| \ldots \| c_{l+1})$ and the leakage k_0.

For this, EE^ι needs time $t_{\mathsf{H}} + (2l+1)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L+1)t_{\mathsf{E}}$.

When A does the i^{th} decryption query, $i < \iota$ on input (a, c, Flt), EE_1^ι proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where $c_0, \ldots, c_l$ and a are modified according to the 0.0 row of Flt, and she adds $((c_0, \ldots, c_l, a), h)$ to $\mathcal{H}$ (with $c_0, \ldots, c_l$ and a modified according to the 0.0 row of Flt), 2) she computes $k_0 = \mathsf{f}^{-1}(h, c_{l+1})$ where the inputs are modified according to 0.1 row of Flt, obtaining k_0, as answer; she sets $c_{l+1}^i := c_{l+1}$, $h^i = h$ (where c_{l+1} and h are modified according to the 0.1 row of Flt), 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, and she adds $(h^i, c_{l+1}^i, c_{l+2})$ to $\mathcal{CH}$, 4) she sees if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 6) finally, EE_1^ι answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, EE_1^ι needs time $t_{\mathsf{H}} + (2l+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L+2)t_{\mathsf{E}}$.

When A does the ι^{th} decryption query on input (a, c, Flt), EE_1^ι proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where a and $c_0, \ldots, c_l$ are modified according to the 0.0 row of Flt, and she adds $((c_0, \ldots, c_l, a), h)$ to $\mathcal{H}$ (with $c_0, \ldots, c_l$ and a modified according to the 0.0 row of Flt), 2) she sets h^ι and c_{l+1}^ι, which are h and c_{l+1} modified according to the 0.1 row of Flt and she checks if there is an entry in $\mathcal{CH}$ (h^ι, c_{l+1}^ι). If this is the case EE_1^ι aborts. Otherwise, EE_1^ι outputs h^ι as her challenge and $\mathsf{st} = (h^\iota, c_{l+1}^\iota, \mathcal{CH}, \mathcal{H}, s, \mathsf{f})$. Then, $\mathsf{EE}_2^\iota(\mathsf{st})$, parses st in $h^\iota, c_{l+1}^\iota, \mathcal{CH}, \mathcal{H}, s, \mathsf{f})$, and she computes $k_0 = \mathsf{f}^{-1}(h^\iota, c_{l+1}^\iota)$. 3) after that, EE_2^ι computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, and she adds $(h^i, c_{l+1}^i, c_{l+2})$ to $\mathcal{CH}$, 4) she sees if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 6) finally, EE_2^ι answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A. (If $\iota = q_D + 1$ the decryption query associated to A output, EE proceeds as as normal decryption query, simply assuming that there is no fault injected).

For this, EE_1^ι needs time t_{H}, while EE_2^ι needs time $(2l+2)t_{\mathsf{E}} \leq (2L+2)t_{\mathsf{E}}$. To be polynomial, we assume that f is simulated via lazy sampling and EE_1 puts in st the samples she has already done.

When A does the i^{th} decryption query, $i > \iota$ on input (a, c, Flt), EE_2^ι proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where $c_0, \ldots, c_l$ and a are modified according to the 0.0 row of Flt, and she adds $((c_0, \ldots, c_l, a), h)$ to $\mathcal{H}$ (with $c_0, \ldots, c_l$ and a modified according to the 0.0 row of Flt), 2) she computes $k_0 = \mathsf{f}^{-1}(h, c_{l+1})$ where the inputs are modified according to 0.1 row of Flt, obtaining k_0, as answer; she sets $c_{l+1}^i := c_{l+1}$, $h^i = h$ (where c_{l+1} and h are modified according to the 0.1 row of Flt), 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, 4) she sees if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 6) finally, EE_2^ι answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, EE_2^ι needs time $t_{\mathsf{H}} + (2l+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), EE_2^ι proceeds as follows: 1) she computes $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_l^*, a^*)$ and she adds $((c_0^* \| \ldots \| c_l^*, a^*), h)$ to $\mathcal{H}$, 2) she computes $k_0 = \mathsf{f}^{-1}(h, c_{l+1})$ 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, 4) she sees if $\tilde{c^*}_0 \neq c_0^*$, she answers $\perp$; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i^* = \mathsf{E}_{k_{i-1}^*}(p_A)$, b) computes $y_i^* = \mathsf{E}_{k_i^*}(p_B)$, c) computes $m_i^* = y_i^* \oplus c_i^*$ (if $i = l$ $\pi_{|c_l^*|}(y_l^*) \oplus c_l^*$, 6) finally, EE_2^ι observe that A has given a correct decryption query, and she looks through $\mathcal{H}$ to see if there is an entry $(c_0 \| \ldots \| c_l, a), h)$ s.t $h = h^\iota$ and $c_0 = c_{l+2}^\iota$. If it is the case, she outputs it, otherwise 0^n.

Thus, in total EE runs in time bounded by $t + q(t_{\mathsf{H}}) + (q_E + q_D + 1)(2L + 3)t_{\mathsf{E}} = t_3'$.

EE correctly simulates Game 5 for A.

Bounding $\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}]$**.** First, we observe that if EE_1^ι aborts, it means that there is a previous query for which $h^i = h^\iota$ and $c_{l+2}^i = c_{l+2}^\iota$, thus event $\mathsf{Bad}_{3.\iota}$ is event $\mathsf{Bad}_{3.i}$, thus,

$$\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1} \wedge \mathsf{EE}^\iota \text{ aborts}] = 0.$$

Second, we observe that k_0^ι is picked randomly because EE^ι has not aborted (and (k_0^ι, p_B) has never been queried before the ι decryption query to E). to pick the random prefix uniformly at random or to pick a k_0 uniformly at random and as a random prefix $\mathsf{E}(k_0, p_B)$ is indistinguishable for the n-PR-corpi game. We are doing the latter case for EE^ι. Thus, if event $\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}$ happens, then, EE^ι wins the n-PR-corpi game. Since EE is a t_3'-adversary and H is $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$-$n$-$\mathsf{PR}$-$\mathsf{corpi}$-secure (because we need to subtract the time needed to compute the random prefix), $t_3' \leq t_3$, then,

$$\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] \leq \epsilon_{\mathsf{PR\text{-}corpi}}.$$

Concluding the proof. First, we observe that

$$\begin{aligned}
\Pr[\mathsf{Bad}_3] &\leq \sum_{\iota=1}^{q_D+1} \Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] \\
&\leq \sum_{\iota=1}^{q_D+1} \epsilon_{\mathsf{PR\text{-}corpi}} \leq (q_D + 1)\epsilon_{\mathsf{PR\text{-}corpi}}. \\
\text{Thus } &|\Pr[E_5] - \Pr[E_6]| \leq (q_D + 1)\epsilon_{\mathsf{PR\text{-}corpi}}.
\end{aligned}$$

Second, we observe that $\Pr[E_6] = 0$. This happens because the adversary in the final verification query cannot have h^* equal to an h of an encryption query. Moreover, if $h^* = h$ of a decryption query, it cannot be the case that $\tilde{c}_0^* = c_0^*$ since EE^ι covers also the case of k_0^ι random and c_{l+2}^ι (which is c_0^ι correctly computed) equal to the c_0 that is computed to obtain h^ι). Thus,

$$\begin{aligned}
\Pr[E_0] \leq &\sum_{i=1}^{6} |\Pr[E_{i-1}] - \Pr[E_i]| \\
\leq &\epsilon_{\mathsf{sTPRP}} + \frac{(q+2)(q+1)}{2^{n+1}} + \\
&+ q_D[q_E(L+1)(q_D+1)/2 + q_I + (q_D+1)(L+2)/2]2^{-n} \\
&+ \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}} = \epsilon.
\end{aligned}$$

□

Now, we move to the stronger result. We start by formalizing faults inside the components.

Formalizing faults inside the components. Before stating the theorem, we need to formalize the injection of faults into the atoms. We do this with the *atom-faults vector* $\mathsf{Flt}^{\mathsf{atom}}$. First, we enumerate all the calls to the atoms (and we consider only those for which a fault is interesting, i.e., not the XOR). Let n_{atom} be the number of such calls. $\mathsf{Flt}^{\mathsf{atom}}$ is a vector of n_{atom} faults: $(\mathsf{Flt}_1, \ldots, \mathsf{Flt}_{n_{\mathsf{atom}}})$ where Flt_i describes the faults to be injected in the call of the atom corresponding to index i. For example, if the ith call corresponds to a block cipher Flt_i explain how, when and which wires of E are modified during that computation. If no fault is injected, we write $\mathsf{Flt}_i = \epsilon$. For the decryption of CONCRETE, the atoms whose faults are described by $\mathsf{Flt}^{\mathsf{atom}}$ are H, F, and E, where the index are:

- 1 for $\mathsf{H}_s(c_0 \| \ldots \| c_l, a)$,
- 2 for $\mathsf{F}_k^{-1,h}(c_{l+1})$,
- 3 for $\mathsf{E}_{k_0}(p_B)$,
- $2 + 2i$ $(i = 1, \ldots, l)$ for $\mathsf{E}_{k_{i-1}}(p_A)$,
- $2i + 3$ $(i = 1, \ldots, l)$ for $\mathsf{E}_{k_{i-1}}(p_B)$.

Similarly to what is done to define $\mathcal{I}$ (Sec. 2.8.1), we define $\mathcal{I}^{\mathsf{atom}}$ with the set of protected calls and denote by $\perp$ in $\mathsf{Flt}^{\mathsf{atom}}$ that the corresponding call cannot be faulted. Let $\mathcal{I}^c = (\mathcal{I}, \mathcal{I}^{\mathsf{atom}})$ be the set of protected inputs and atoms. In the next theorem, $\mathcal{I}^c_{\mathsf{AEnc}} = (\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}^{\mathsf{atom}}_{\mathsf{AEnc}})$ where all the entries of $\mathcal{I}_{\mathsf{AEnc}}$ are ϵ, while all the entries of $\mathcal{I}^{\mathsf{atom}}_{\mathsf{AEnc}}$ are $\perp$ (i.e., no fault is allowed); instead for decryption, we use $\mathcal{I}^c_{\mathsf{ADec}} = (\mathcal{I}_{\mathsf{ADec}}, \mathcal{I}^{\mathsf{atom}}_{\mathsf{ADec}})$ where all the entries of $\mathcal{I}_{\mathsf{ADec}}$ are free (except those forced to be equal to ϵ by the matrix defined in Eq. 4). The adversary can choose all the entries of $\mathcal{I}^{\mathsf{atom}}_{\mathsf{ADec}}$ except for the second one (corresponding to $k_0 = \mathsf{F}_k^{-1,h}(c_{l+1})$ which is $\perp$ (i.e., no fault is allowed). Note that we are using the same $\mathcal{I}_{\mathsf{AEnc}}$, and $\mathcal{I}_{\mathsf{ADec}}$ as those used for Thm. 7. We are now ready to formally state the stronger result:

Theorem 9. *Let* F *be a* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$*-strongly protected* sTPRP*, let* E *be an ideal block cipher, let* A *be allowed* q_I *queries to* E*, let* H *be a* $(t_2, \epsilon_{\mathsf{CR}})$*-collision resistant hash function, and also* $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$*-*$n$*-*PR–corpi*-secure. Then,* $(\mathcal{I}^c_{\mathsf{AEnc}}, \mathcal{I}^c_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* $\mathsf{CONCRETE}\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encoding messages of length at most* Ln *bits, with leaking function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and fault admissible sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ *achieves* $(q_I, q_{FL}, q_E, q_D, t, \epsilon)$-ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption (CIL2Fd)*, with*

$$\epsilon = \epsilon_{\mathsf{sTPRP}} + \frac{(q_D+2)(q_D+1)}{2^{n+1}} + \frac{(q_D+1)[q_E(L+1) + q_I + q_D(L+2)/2]}{2^n} + \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}}$$

with $q = q_E + q_D + 1$, $t_1 = t + qt_{\mathsf{H}} + (q(2L+1)+1)t_{\mathsf{E}}$, $t_2 = t + q(t_{\mathsf{H}}) + (q_E+q_D)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}]$, $t_3 = t + q(t_{\mathsf{H}}) + (q_E + q_D + 1)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] - t_{\mathsf{F}} - t_{\mathsf{E}}$.

(We assume that picking a value uniformly at random does not take time.)

The proof is the same as for Thm. 7, with the following modifications: Specifically, for every adversary used in that proof, we modify its description so that it applies the faults specified in $\mathsf{Flt}^{\mathsf{atom}}$ For example, in the description of B, for a decryption query on input (a, c, Flt), B computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_l, \mathsf{Flt}_1)$, where a and $c_0, \ldots, c_l$ are modified according the 0.0 row of Flt, Flt_1 is the first element of $\mathsf{Flt}^{\mathsf{atom}}$; and $\mathsf{H}_s(a, c, \mathsf{Flt}_1)$ denotes the computation of $\mathsf{H}_s(a, c)$ applying the fault described as Flt_1, etc. We can iterate this for every adversary and every primitive that can be faulted. For brevity, we omit the detailed description of all modified adversaries, as the procedure is analogous.

4.4 CONCRETE is not Secure in the Presence of Fault(s) in Encryption

So far, we have not considered faults occurring during encryption. In this section, we show that even a single fault in encryption suffices to break the CIL2F security of CONCRETE.

In particular, the standard *decoupling attack* [52] can be applied: it is enough to flip a bit in $c_1, \ldots, c_l$ at the moment when $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ is computed. Concretely, suppose the fault replaces c_1 with c_1'. Let $(c_0, c_1, \ldots, c_l, c_{l+1})$ be the output of AEnc_k on input $(a, (m_1, \ldots, m_l))$, but with the above fault injected. Then, $(a, c_0, c_1', c_2, \ldots, c_l, c_{l+1})$ is a valid ciphertext, which decrypts to $(m_1 \oplus c_1 \oplus c_1', m_2, \ldots, m_l)$. Hence, the adversary obtains a valid forgery. We conclude that CONCRETE does not provide ciphertext integrity if a fault is injected during encryption, even in the weakest form (e.g., setting or flipping a single bit).

4.5 Weak Integrity of CONCRETE in the Presence of Fault(s) in Encryption

In the attack described above, the value c_1 is replaced by c_1' only during the computation of h, and therefore the weak integrity of CONCRETE is not violated. We now analyze the wCIL2F2-security of CONCRETE.

First, suppose the adversary can set both k_0 and h in a single encryption query. In this case, she can trivially generate a forgery: she chooses a key k_0 of her choice, computes the corresponding $c_0, \ldots, c_l$ encrypting $(m_1, \ldots, m_l)$, and then computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ for an a of her choice. Finally, $c_{l+1} = \mathsf{F}_k(h, k_0)$, since she can set the two inputs of $\mathsf{F}_k(\cdot, \cdot)$, therefore $(a, c_0, \ldots, c_l, c_{l+1})$ forms a valid ciphertext. Hence, CONCRETE does not guarantee integrity if the adversary can control both values simultaneously within a single encryption.

Note that this attack crucially requires the ability to *set* both k_0 and h. A differential fault alone does not suffice: if k_0 is sampled at random, then $(c_0, \ldots, c_l)$ are pseudorandom even in the presence of faults, and consequently $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ is indistinguishable from random (under reasonable assumptions on H)[27]. Therefore, the adversary cannot predict in advance the offset needed to transform the actual pair (k_0, h) used in the computation into one of her choice.

We thus prove below that CONCRETE is wCIL2F2-secure against differential faults in encryption (combined with arbitrary faults in decryption). The proof follows the same structure as in Section 4.4, with the only difference that for each encryption query we store, in place of the original ciphertext c, the modified ciphertext

$$c' = (a, c_0', \ldots, c_l', c_{l+1}),$$

where each c_i' denotes the value of c_i as it is modified before entering the H atom (i.e., in computation $f_{l+1.1}$).

Take-away for implementers. Before presenting the proof, we briefly list its practical implications. For decryption queries, the considerations are the same as for Thm. 7. For encryption queries, implementers should focus on ensuring that the randomness source remains reliable and produces truly random outputs even in the presence of faults. Moreover, every primitive should be protected against faults, and it must be guaranteed that between atomic calls only differential faults can occur between the atoms (at most a single value can be set). With respect to leakage, only the single call to F requires protection: in particular, the adversary must not be able to extract the key of F through leakage.

4.5.1 wCIL2F2-Security of CONCRETE with only Differential Faults in Encryption

In this subsection, we present the details of the previous results and their proofs. The uninterested reader can skip to Sec. 5.

[27] Strictly speaking, H is not a random oracle. Nevertheless, for a reasonable hash function, if its inputs are pseudorandom then the output should be computationally indistinguishable from random.

Leakage function and faulty matrix. For the leakage functions, we adopt the same ones as for Thm. 7: $\mathsf{L}_{\mathsf{AEnc}}(a, m, k_0; k) := \emptyset$, and $\mathsf{L}_{\mathsf{ADec}}(a, c; k) := k_0$. Regarding faults, the adversary is allowed to set any fault she wants between the atoms in decryption, without any restriction, while in encryption, she is restricted to differential faults only. In other words, all entries of $\mathcal{I}_{\mathsf{AEnc}}$ are either ϵ (when the corresponding value in the dependency matrix is ϵ) or $\perp$ /$\oplus$; while $\mathcal{I}_{\mathsf{ADec}}$ is empty, that is, we allow the adversary to choose any entry different from ϵ of the matrix defined in Eq. 4 as long as they are compliant with Sec. 2.8.1. Accordingly, $\mathcal{F}_E$ and $\mathcal{F}_D$ are defined in the natural way.

Theorem 10. *Let* F *be a strongly protected* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$*, let* E *be an ideal block cipher, let* A *be allowed* q_I *queries to* E*, let* H *be a* $(t_2, \epsilon_{\mathsf{CR}})$*-collision resistant and* $(t_3, \epsilon_{\mathsf{PR-corpi}})$*-*$n$*-*$\mathsf{PR}$–$\mathsf{corpi}$ *hash function. Then,* $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* $\mathsf{CONCRETE}\ \Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encoding messages at most* Ln *bits long, with leaking function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and fault admissible sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ *has* $(q_I, q_{FL}, q_E, q_D, t, \epsilon)$ weakly ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption $(\mathsf{wCIL2F2} - (\mathsf{de}))$ *with*

$$\begin{aligned}\epsilon \leq & \epsilon_{\mathsf{sTPRP}} + \frac{(q_D+2)(q_D+1)}{2^{n+1}} + \frac{(q_D+1)[q_E(L+1)+q_I+q_D(L+2)/2]}{2^n} \\ & + \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR-corpi}}\end{aligned}$$

with $q = q_E + q_D + 1$, $t_1 = t + qt_{\mathsf{H}} + (q(2L+1)+1)t_{\mathsf{E}}$, $t_2 = t + q(t_{\mathsf{H}}) + (q_E+q_D)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}]$, $t_3 = t + q(t_{\mathsf{H}}) + (q_E+q_D+1)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] - t_{\mathsf{F}} - t_{\mathsf{E}}$.

(We assume that sampling a value uniformly at random takes negligible time.).

Proof. We use a sequence of games for the proofs. For simplicity, we consider the decryption of the challenge associated data and ciphertext as the $q_D + 1$th decryption query. The proof is very similar to the proof of Thm. 7. For simplicity, we omit most details when they are the same as previous proofs, and we leave the full proof with all the details to App. A.1.

Game 0. It is the standard $\mathsf{wCIL2F2}_{\mathsf{CONCRETE},\mathcal{F},\mathsf{L},\mathsf{A}^{\mathsf{L},\mathsf{Flt}}}$ game. Let E_0 be the event that A wins this game.

Game 1. It is Game 0, where we simply put (a', c', a, m, r) in $\mathcal{S}$ where a' is a modified according to the $l + 1.1$ row of Flt and $c' = (c'_0, \ldots, c'_l, c_{l+1})$ with c'_i $(i = 0, \ldots, l)$ is c_i modified according to the $l+1.1$ row of Flt, and c_{l+1} is left unchanged (That is, we take the actual input of H). Let E_1 be the event that A wins this game. Clearly, $\Pr[E_0] \leq \Pr[E_1]$.

Game 2. It is Game 1 except that we have replaced F with a random tweakable permutation f. Let E_2 be the event that A wins this game.

Transition between Game 1 and Game 2. Since Game 1 and Game 2 are the same except for the replacement of F with f, we can build an adversary B to bound the difference. She plays the sTPRP game, having to distinguish F_k from f, and simulates A's oracles using its own. B behaves as B in Thm. 7 with the following differences:

- B has $L + 1$ lists $\mathcal{S}_0, \mathcal{S}_1, \ldots, \mathcal{S}_L$ which are empty.
- In encryption, B behaves as B with the modifications due to the injection of the faults, and it adds the output of the computation $c_i = y_i \oplus m_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus m_l$) with y_i and c_i modified according to the $i.3$ line of Flt, In detail:
  When A does an encryption query on input (a, m), B simply picks k_0 uniformly at random from $\{0,1\}^n$. After having parsed m in $(m_1, \ldots, m_l)$, she proceeds as follows: 1) she computes $c_0 = \mathsf{E}_{k_0}(p_B)$, where k_0 is modified according to the 0.2 row of Flt, and she adds c_0 to $\mathcal{S}_0$; 2) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $c_i = y_i \oplus m_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus m_l$) with

y_i and c_i modified according to the $i.3$ line of Flt, and adds c_i to $\mathcal{S}_i$; 3) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where $c_0, \ldots, c_l$, and a are modified according to the $l+1.1$ row of Flt; moreover, she adds c'_i to $\mathcal{C}_i$, for $i = 0, \ldots, l$, where c'_i is c_i modified according to the element of the $l + 1.1$th row corresponding to c_i. 4) she queries her oracle on input (h, k_0) which are modified according to $l + 1.2$ row of Flt, obtaining c_{l+1} as answer; 5) finally, B answers $c = (c_0, \ldots, c_l)$ and the leakage k_0 to A. Moreover, if there is no fault inserted and in the 0.2 row and in the $l + 1.2$ row of fault (or the same fault for k_0 in both rows), she adds $c' = (c'_0, \ldots, c'_l, c_{l+1}, a')$ to $\mathcal{S}$ (where a' is a modified according to the $l + 1.1$th row of Flt), otherwise nothing.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (2l + 1)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L + 1)t_{\mathsf{E}}$.

- B behaves in the same way for decryption queries and to check A's output.

In total B does at most $q_E + q_D + 1 = q$ queries to her oracle and runs in time bounded by $t + q(t_{\mathsf{H}}) + (q(2L + 1) + 1)t_{\mathsf{E}} = t_1$.

If her oracle is implemented with F, B correctly simulates Game 1 for A, otherwise Game 2. Since F is a $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$,

$$|\Pr[E_2] - \Pr[E_1]| \leq \epsilon_{\mathsf{sTPRP}}.$$

Game 3. It is Game 2, where in decryption instead of using f^{-1}, where f is a random tweakable permutation, we pick its answers uniformly at random (we answer consistently using a table, i.e., if the same query repeats, we return the same answer). That is, we replace f^{-1} with a random function (when the inputs are fresh). Let E_3 be the event that A wins this game.

Transition between Game 2 and Game 3. In Game 3, for decryption queries, we are using a random tweakable permutation, while in Game 3 a random function. Due to the well-known birthday bound, we have that

$$|\Pr[E_3] - \Pr[E_2]| \leq \frac{(q_D + 2)(q_D + 1)}{2^{n+1}}.$$

Game 4. It is Game 3, where during decryption queries (not for the challenge one), immediately after k_0 is computed, the decryption algorithm computes $c_{l+2} := \mathsf{E}_{k_0}(p_B)$, without any faults inserted, and appends c_{l+2} to the decryption algorithm's normal answer.. Let E_4 be the event that the adversary wins this game.

Transition between Game 3 and Game 4. We build an adversary C. It is the same as C in the proof of Thm. 7.

Therefore, C needs time $t + q_D t_{\mathsf{E}}$ and does $q_I + q_D$ queries to the ideal cipher E, i.e.

$$\Pr[E_3] = \Pr[E_4].$$

Game 5. It is Game 4, with the condition that the following event does not happen: during a decryption query, when a new key k_0 is picked, there does not exist a query to the ideal cipher on input (k_0, p_B). Let E_5 be the event that the adversary wins this game.

Transition between Game 4 and Game 5. The two games are identical except if the following Bad_1 event happens: there exists a decryption query s.t. it requires to pick a new key k_0^i and there exists a previous query to the ideal cipher E on input (k_0^i, p_B).

It is the same as the transition between Game 3 and Game 4 in Thm. 7. Therefore,

$$\begin{aligned}
\Pr[\mathsf{Bad}_1] \leq & \sum_{j=1}^{q_D+1} \frac{q_E(L+1) + q_I + (j-1)(L+2)}{2^n} \\
\leq & \frac{(q_D+1)(q_E(L+1)) + (q_D+1)q_I + (L+2)(q_D+1)q_D/2}{2^n} \\
\leq & \frac{(q_D+1)[q_E(L+1) + q_I + q_D(L+2)/2]}{2^n}
\end{aligned}$$

Therefore,

$$|\Pr[E_5] - \Pr[E_4]| \leq (q_D + 1)[q_E(L+1) + q_I + q_D(L+2)/2]2^{-n}.$$

Game 6. It is Game 5, where the following event Bad_2 does not happen: the hash queries induced during encryption and decryption queries induce a collision for the hash function. Let E_6 be the event that the adversary wins this game.

Transition between Game 5 and Game 6. Since Game 5 and Game 6 are the same except if event Bad_2 happens, it is enough to bound $\Pr[\mathsf{Bad}_2]$ to bound the difference $|\Pr[E_5] - \Pr[E_6]|$. To bound $\Pr[\mathsf{Bad}_2]$, we build an adversary D who wants to find a collision for the hash function. D works as D in the proof of Thm. 7 with the same differences that we have previously detailed for B. In total, D runs in time bounded by $t + q(t_{\mathsf{H}}) + (q_E + q_D)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] = t_2$.

D correctly simulates Game 6 for A. D finds a collision for the hash function H if event Bad_2 happens. Thus, since H is a $(t_2, \epsilon_{\mathsf{CR}})$-collision resistant hash function,

$$|\Pr[E_6] - \Pr[E_5]| = \Pr[\mathsf{Bad}_2] \leq \epsilon_{\mathsf{CR}}.$$

Game 7. It is Game 6, except we rule out the following event Bad_3: for any tweak h for f used only in decryption queries, (that is, used only in f^{-1}) there exist two decryption queries, which we call the ith and the jth s.t.

- $h' = \tilde{h}^i$
- $c_0^j = c_{l+2}^i$, where
  - $h' = \mathsf{H}_s(c_0^j \| \ldots \| c_l^j, a^j)$ (where a^j and $c_0^j, \ldots, c_l^j$ are modified according to the 0.0 line of Flt^j),
  - c_{l+2}^i obtained in the ith decryption query, in which $\mathsf{f}^{-1}(h^i, \tau^i)$ is computed
  - $\tilde{h}^i$ is modified according to the 0.1 row of Flt^i.

Let E_7 be the event that the adversary wins this game.

Transition between Game 6 and Game 7. Since Game 6 and Game 7 are the same except if event Bad_3 happens, it is enough to bound $\Pr[\mathsf{Bad}_3]$ to bound the difference $|\Pr[E_6] - \Pr[E_7]|$. To bound $\Pr[\mathsf{Bad}_3]$, we divide it into $q_D + 1$ different events:$\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.q_D+1}$, where $\mathsf{Bad}_{3.\iota}$ is event Bad_3 happening with $\iota = i$. Clearly, $\mathsf{Bad}_3 = \bigcup_{\iota=1}^{q_D+1} \mathsf{Bad}_{3.\iota}$. To bound $\Pr[\mathsf{Bad}_{3.\iota}]$ we build an adversary EE^ι against the n–PR–corpi-security of the hash function H.

The n–PR–corpi-adversary EE^ι. EE^ι works as EE^ι in the proof of Thm. 7 with the same differences that we have previously detailed for B. In total EE runs in time bounded by $t + q(t_{\mathsf{H}}) + (q_E + q_D + 1)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] = t_3'$.

EE correctly simulates Game 5 for A, except for the computation of Flt.

Bounding $\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}]$. First, we observe that if EE_1^ι aborts, it means that there is a previous query for which $h^i = h^\iota$ and $c_{l+2}^i = c_{l+2}^\iota$, thus event $\mathsf{Bad}_{3.\iota}$ is event 3.i, thus,

$$\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] = 0.$$

Second, we observe that since k_0^ι is picked randomly (and (k_0^ι, p_B) has never been queried before the ι decryption query to E), to pick the random prefix uniformly at random or to pick a k_0 uniformly at random and as a random prefix $\mathsf{E}(k_0, p_B)$ is indistinguishable for the n–PR–corpi game. We are doing the latter case for EE^ι. Thus, if event $\mathsf{Bad}_{3.\iota} \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}$ happens, then, EE wins the n–PR–corpi game. Since EE is a t_3'-adversary and H is $(t_3, \epsilon_{\mathsf{PR}\text{–}\mathsf{corpi}})$-$n$–$\mathsf{PR}$–$\mathsf{corpi}$-secure (because we need to subtract the time needed to compute the random prefix), then,

$$\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] \leq \epsilon_{\mathsf{PR}\text{–}\mathsf{corpi}}.$$

Note that as it happened for D the fact that she does not compute correctly Flt it is not a problem, because the correct computation of Flt is only needed to compute the output of the game and the output of EE^{ι} is produced before the effect of Flt enters into play.

Concluding the proof First, we observe that

$$\Pr[\mathsf{Bad}_3] \leq \sum_{\iota=1}^{q_D+1} \Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] \leq \epsilon_{\mathsf{PR\text{-}corpi}} \leq (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}}.$$

Therefore, $|\Pr[E_5] - \Pr[E_6]| \leq (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}}$. Second, we observe that $\Pr[E_6] = 0$. Indeed, in the final verification query, the adversary cannot use an h^* used in an encryption query, since this would require a collision for H (ruled out by Bad_2). Similarly, she cannot is an h^* use in a decryption query, since otherwise Bad_3 would be triggered. Therefore, no adversary can win Game 6. Putting everything together, we obtain,

$$\begin{aligned}\Pr[E_0] \leq & \Pr[E_6] \leq \sum_{i=1}^{6} |\Pr[E_{i+1}] - \Pr[E_i]| \\ \leq & \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}} + \epsilon_{\mathsf{sTPRP}} \\ & + \frac{(q_D+2)(q_D+1)}{2^{n+1}} + \frac{(q_D+1)[q_E(L+1)+q_I+q_D(L+2)/2]}{2^n}.\end{aligned}$$

□

Note that in our definition of $\mathsf{wCIL2F}$ the adversary is required to output two valid pairs, hence the search for such pairs is delegated to the adversary. Had we instead required uniqueness for each encryption query, the proof would have been considerably more involved, since we would need to explicitly rule out all possible collisions of hashes and invalid decryptions.

CONCRETE is not wCIL2F2 if the adversary can set 2 values. It is clear that the decoupling attack does not affect the $\mathsf{wCIL2F2}$ security, thus, we may wonder if $\mathsf{CONCRETE}$ is $\mathsf{wCIL2F2}$ secure if the adversary can set values. Unfortunately, this is not the case. Consider the following attack: 1) the adversary chooses k_0, m, and a. Then, she computes $c_0, \ldots, c_l$ as follows: $c_0 = \mathsf{E}_{k_0}(p_B)$, and then, for all $i = 1, \ldots, l$, $k_i = \mathsf{E}_{k_{i-1}}(p_A)$, $y_i = \mathsf{E}_{k_i}(p_B)$ and $c_i = y_i \oplus m_i$ (for $i = l$ $c_l = \pi_{|m_l|}(y_l) \oplus m_l$). Finally, she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$. 2) The adversary does an encryption query on input (a', m'). In the computation she injects the following fault: in the computation of c_{l+1}, she replaces the k_0 and h obtained in the current encryption with the k_0 and h obtained in 1). She obtains a ciphertext c'. 3) The forgery is $(a, (c_0, \ldots, c_l, \ldots c'_{l+1}))$.

This shows that when the adversary can fully set two internal values, the strong protection of the tweakable block cipher can be bypassed, since faults allow her to enforce chosen inputs to F. *In contrast, the decryption algorithm is designed so that even unlimited access to* F^{-1} *does not enable forgery.* We conjecture that in the unbounded leakage model, no meaningful security can be achieved if the adversary is allowed to arbitrarily set all the inputs of a strongly protected component during encryption.

But if the adversary is only allowed to set one value (and put all the differential faults she wants), $\mathsf{CONCRETE}$ remains $\mathsf{wCIL2F2}$.

Theorem 11. *Let* F *be a strongly protected* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$, *let* E *be an ideal block- ipher, let* A *be allowed* q_I *queries to* E, *let* H *be a* $(t_2, \epsilon_{\mathsf{CR}})$*-collision resistant and* $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$-$n$-$\mathsf{PR\text{–}corpi}$ *hash function. Then,* $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* $\mathsf{CONCRETE}$ $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$, *encoding messages at most* Ln *bits long, with leaking function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and fault admissible sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$

achieves $(q_I, q_{FL}, q_E, q_D, t, \epsilon)$ weakly ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption (wCIL2F2)) as long the adversary sets a single fault per encryption query *with*

$$\begin{aligned}\epsilon \leq & \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}} + \epsilon_{\mathsf{sTPRP}} \\ & + \frac{(q_D+2)(q_D+1)}{2^{n+1}} + \frac{(q_D+1)[q_E(L+1)+q_I+(q_D(L+2)/2]}{2^n},\end{aligned}$$

with $q = q_E+q_D+1$, $t_1 = t+qt_{\mathsf{H}}+(q(2L+1)+1)t_{\mathsf{E}}$, $t_2 = t+q(t_{\mathsf{H}})+(q_E+q_D)[(2L+3)t_{\mathsf{E}}+t_{\mathsf{F}}]$, $t_3 = t+q(t_{\mathsf{H}})+(q_E+q_D+1)[(2L+3)t_{\mathsf{E}}+t_{\mathsf{F}}]-t_{\mathsf{F}}-t_{\mathsf{E}}$.

(We assume that picking a value uniformly at random does not take time).

We omit the proof since it is the same as the previous.

5 CONCRETE2: a CIL2F2-Secure Scheme

In this section, we present CONCRETE2 designed to achieve CIL2F2. We start by stating why CONCRETE does not achieve CIL2F2, then we describe the scheme and prove its CIL2F security.

5.1 Design of CONCRETE2

Why CONCRETE is not CIL2F2. There are two main reasons why CONCRETE fails to provide CIL2F2: First, every possible ciphertext block $c_1, \ldots, c_l$ is, in effect, "authenticated" [28], irrespective of whether this is the encryption of the required message; Second, given a ciphertext block c_i, encrypting m_i, an adversary can compute the ciphertext block corresponding to another message block m_i' (which is $c_i' = c_i \oplus m_i \oplus m_i'$).

Designing CONCRETE2. To solve the first problem, it suffices to additionally encrypt a tag of the message m. For example, we can hash the message, $h' = \mathsf{H}_s(c_0 \| m)$ (where c_0 should be random), and we compute a ciphertext block $c_{l+1} = \mathsf{E}_{k_{i+1}}(p_B) \oplus h'$, where $k_{i+1} = \mathsf{E}_{k_i}(p_A)$. In decryption, we *both* check if we have the correct commitment for the retrieved key k_0, and if we have encrypted the correct commitment of the message.

For the second problem, instead of XORing the message to a random value $y_i = \mathsf{F}_{k_i}(p_B)$, we use y_i as an ephemeral key, that is, $k_{i,E} = y_i = \mathsf{E}_{k_i}(p_B)$ and we compute $c_i = \mathsf{E}_{k_{i,E}}(m_i)$. In this way, even knowing c_i, the adversary cannot predict c_i', the encryption of another message block (moreover, although the ephemeral key $k_{i,E}$ potentially may be leaked, this cannot give any additional information than how to encrypt this particular block). We adopt a similar idea, following the MAC proposed at CCS'15 [46] (Alg. 6), to compute the commitment of m.

Thus, we have obtained CONCRETE2 which we describe in Fig. 5, and in Alg. 7, and Fig. 4. We explain below how we can modify CONCRETE2 to treat messages whose length is not a multiple of n.

We leave as an open question if it is possible to provide a sponge-based construction.

Compared to MEM [54], our scheme uses the strongly protected component significantly less (only once, compared to three times), and it provides authenticity even if the adversary can inject arbitrary faults during decryption and operate under leakage (for more details, see Sec. 5.6).

[28] Strictly speaking, CONCRETE does not authenticate these ciphertext blocks, but uses their hash to send k_0. Since there is a commitment on k_0, this indirectly provides authenticity.

Algorithm 6 The leakage-resistant PSV-MAC [46].

It uses a leak-free BC $\mathsf{F} : \mathcal{K} \times \{0,1\}^n \to \{0,1\}^n$, and a weakly protected BC $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$.

Gen:

- $k \xleftarrow{\$} \mathcal{K}$

$\mathsf{Mac}_k(m)$:

- Parse m in $m_1, ..., m_l$
- $\mathsf{iv} \xleftarrow{\$} \{0,1\}^n$
- $k_1 = \mathsf{F}_k(\mathsf{iv})$
- For $i = 1, ..., l$
  - $k_{i+1} = \mathsf{E}_{k_i}(m_i)$
- $\tau = k_{l+1}$
- Return (iv, τ)

$\mathsf{Vrfy}_k(m, (\mathsf{iv}, \tau))$:

- Parse m in $m_1, ..., m_l$
- $k_1 = \mathsf{F}_k(\mathsf{iv})$
- For $i = 1, ..., l$
  - $k_{i+1} = \mathsf{E}_{k_i}(m_i)$
- $\tilde{\tau} = k_{l+1}$
- If $\tilde{\tau} = \tau$
  - Return $\top$
- Return $\bot$

CONCRETE2 if the Message is not Full. We explain how we can modify CONCRETE2 if the last message block is not full. The idea is to pad the last block full with 1s until it reaches full length. Together with the ciphertext, we send the length of the last message block, which is used as input to the hash function. Formally, we proceed as for CONCRETE2 with these differences:

- instead of using m_l, we use $m'_l = m_l \| 1^{n-|m_l|}$
- $h = \mathsf{H}_s(c, a, \lambda)$ with $\lambda = |m_l|$
- The output is (c, λ)

Note that in most standard security definitions, it is already assumed that the adversary learns the plaintext- length [35, 51, 45].

5.1.1 Black-box-Security of CONCRETE2

We now prove the AE-security of CONCRETE2 in the black-box model (i.e., without leakage).

Theorem 12. *Let* F *be a* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-sTPRP*, let* E *be a* $(2, t_2, \epsilon_{\mathsf{PRF}})$-PRF*, and let* H *be a* $(t_1, \epsilon_{\mathsf{CR}})$*-collision resistant hash function; then* CONCRETE2 $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encrypting any message of at most* Ln *bits, is* (q_E, q_D, t, ϵ)-AE*-secure, with*

$$\epsilon \leq \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + 2(q + q_E(L+1))\epsilon_{\mathsf{PRF}} + \frac{O(q^2)}{2^n}.$$

Explanation of the bound. The final term arises from the probability of collisions among the ephemeral keys and from the switch between a random function and a random permutation. The temrs $\epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + 2q_D\epsilon_{\mathsf{PRF}}$ captures the integrity bound. Instead, for privacy we have to replace all calls to E with calls to a random function (thus the reason for $2q_E(L+1)\epsilon_{\mathsf{PRF}}$). Finally, we remark that although each key is used at most twice as input to E, there exist distinguishing attacks that exploit a very small number of queries [7].

Algorithm 7 The CONCRETE2 algorithm. The changes from CONCRETE (Alg. 2) are highlighted.

It uses a TBC $\mathsf{F} : \mathcal{K} \times \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$ with a strongly protected implementation, a BC $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ with a weakly protected implementation and a multi-input collision resistant hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \mathcal{TW}$. For simplicity, we assume that all message blocks are *full*, that is, $|m_l| = n$. If they are not full, see Sec. 5.1.

Gen:

- $k \xleftarrow{\$} \mathcal{K}$, $s \xleftarrow{\$} \mathcal{HK}$
- $p_A, p_B \xleftarrow{\$} \{0,1\}^n$

$\mathsf{AEnc}_k(a, m)$:

- Parse m in $m_1, ..., m_l$
- $k_0 \xleftarrow{\$} \{0,1\}^n$
- $k_{0,E} = \mathsf{E}_{k_0}(p_A)$
- $k_{0,A} = \mathsf{E}_{k_0}(p_B)$
- $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$
- $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$
- For $i = 1, ..., l$
  - $k_{i,E} = \mathsf{E}_{k_i}(p_B)$
  - $c_i = \mathsf{E}_{k_{i,E}}(m_i)$
  - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
  - $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$
- $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$
- $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$
- $h = \mathsf{H}_s(c_0 \| ... \| c_{l+1}, a)$
- $c_{l+2} = \mathsf{F}_k^h(k_0)$
- Return $c = c_0 \| ... \| c_l \| c_{l+1} \| c_{l+2}$

$\mathsf{ADec}_k(a, c)$:

- Parse c as $c_0, ..., c_l, c_{l+1}, c_{l+2}$
- $h = \mathsf{H}_s(c_0 \| ... \| c_l \| c_{l+1}, a)$
- $k_0 = \mathsf{F}_k^{-1,h}(c_{l+2})$
- $k_{0,E} = \mathsf{E}_{k_0}(p_A)$
- $k_{0,A} = \mathsf{E}_{k_0}(p_B)$
- $\tilde{c}_0 = \mathsf{E}_{k_{0,E}}(p_B)$
- If $c_0 \neq \tilde{c}_0$
  - Return $\perp$
- Else $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$
- For $i = 1, \ldots, l$
  - $k_{i,E} = \mathsf{E}_{k_i}(p_B)$
  - $m_i = \mathsf{E}^{-1}_{k_{i,E}}(c_i)$
  - $k_{i+1} = \mathsf{E}_{k_i}(p_A)$
  - $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$
- $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$
- $\tilde{k}_{l,A} = \mathsf{E}^{-1}_{k_{l+1,E}}(c_{l+1})$
- If $k_{l,A} \neq \tilde{k}_{l,A}$
  - Return $\perp$
- Return $m = m_1 \| ... \| m_l$

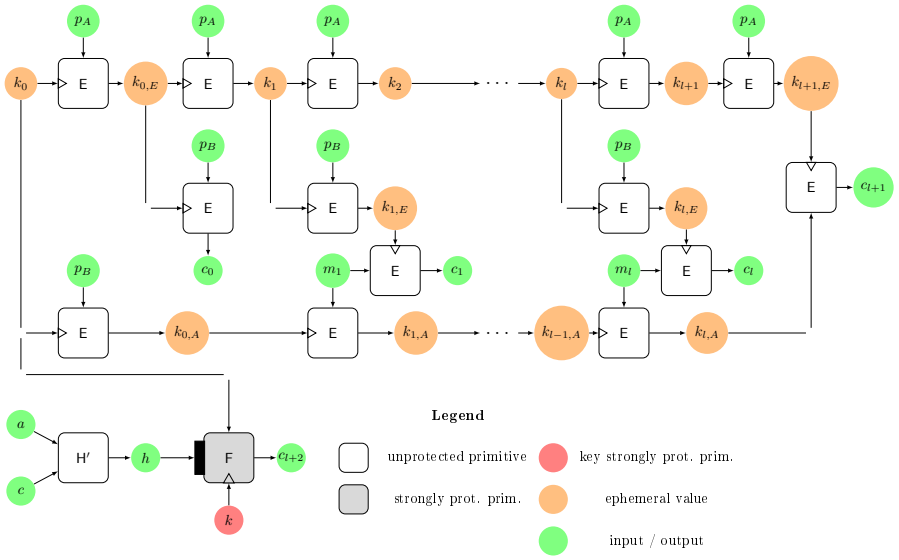

Figure 4: CONCRETE2.

Idea of the proof. We start by ruling out the possibility that an adversary can forge. If we replace F with a random function, then (as long as there are no collisions in the hash function) the recovered key k_0 is uniform. In this case, if k_0 has never been used before, the corresponding check value $\tilde{c}_0$ is distributed uniformly at random, since E behaves like a PRF. Hence, the probability that an adversary correctly guesses $c_0 = \tilde{c}_0$ is bounded by 2^{-n}. For privacy, note that once k_0 is random, all blocks $c_0, \ldots, c_{l+1}$ are also pseudorandom, provided that there are no collisions in the ephemeral keys generated during encryption. This follows from the PRF property of E. Finally, the last block c_{l+2} is pseudorandom by construction, since it is the output of a sTPRP. Altogether, this ensures that the ciphertext is indistinguishable from random and that forgery succeeds only with negligible probability.

5.1.2 Details of the black-box security of CONCRETE2.

In this section, we give the details ot the AE-security (Def. 12) of CONCRETE2 (Alg. 7) in the black-box case. Readers mainly interested in the high-level picture may safely skip ahead to Sec. 5.2.

Theorem 13. *Let* F *be a* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-sTPRP*, let* E *be a* $(2, t_2, \epsilon_{\mathsf{PRF}})$-PRF*, and let* H *be a* $(t_1, \epsilon_{\mathsf{CR}})$*-collision resistant hash function; then* CONCRETE2 $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encrypting messages at most* Ln *bits long, is* (q_E, q_D, t, ϵ)-AE*-secure, with*

$$\epsilon \leq \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + 2(q + q_E(L+1))\epsilon_{\mathsf{PRF}} + \frac{q_D[1 + (3L+4)(q_E + (q_D - 1)/2]}{2^n} + \frac{q_E(q_E - 1)(3L + 4 + (L+1)(4L+6)) + q(q-1)}{2^{n+1}},$$

with $q = q_E + q_D$*,* $t_1 = t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}]$*,* $t_2 = t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] - 2t_{\mathsf{E}}$*,* t_{E} *the time needed to compute once* E*,* t_{H} *to compute once* H *(we assume that picking a value uniformly at random does not take time).*

Proof. As usual, we use a sequence of Games. Let E_i be the event that the adversary wins Game i, that she outputs a fresh and valid forgery.

Game 0. It is the standard AE Game against CONCRETE2.

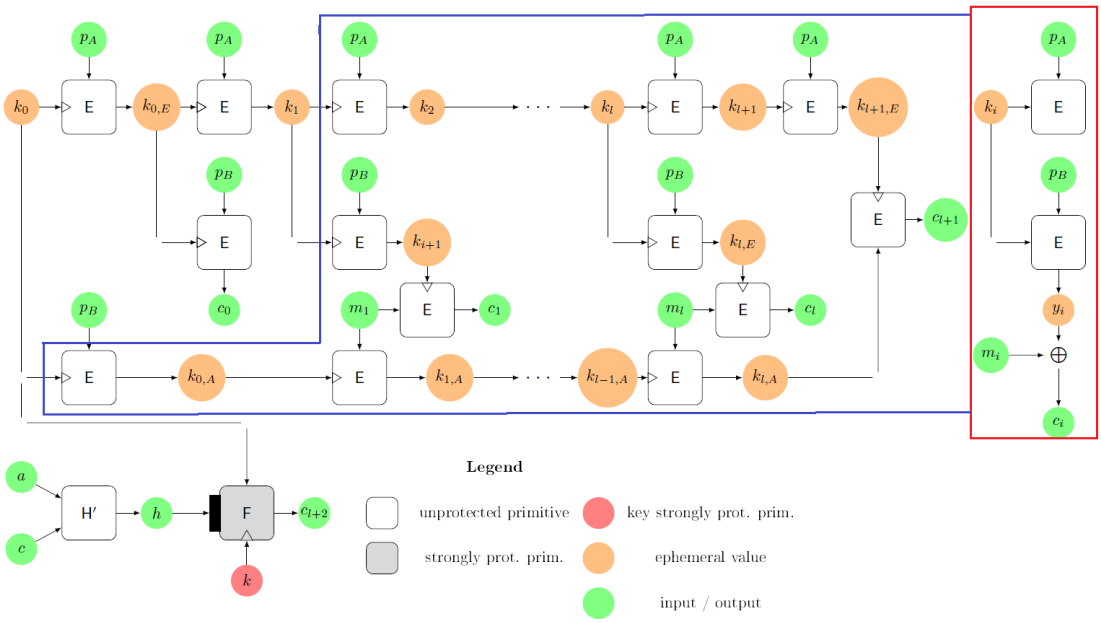

Figure 5: CONCRETE [16] and CONCRETE2. CONCRETE uses the red part, while CONCRETE2 the blue part.

Game 1. This is Game 0 where we replace F with a tweakable random permutation f.

Transition between Game 0 and Game 1. Since Game 0 and Game 1 are the same except for the replacement of F with f, we can build an adversary B to bound the difference.

She plays the sTPRP Game, having to distinguish F_k from f, and simulates A's oracles using its own.

At the start of the Game, B picks s in $\mathcal{HK}$ and gives it to A, and a block cipher E. Moreover, she has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a, m), B simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: $1i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $1ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$; $1iii)$ $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $1iv)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then, for all $i = 1, \ldots, l$, $2i)$ B computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ B computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ B computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $4ii)$ she queries her oracle on input (h, k_0), receiving as answer c_{l+2}. So, she can answer A $c = (c_0 \| \ldots \| c_{l+2})$. Moreover, B adds (a, c) to the list $\mathcal{S}$.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c, Flt), B parses $c = (c_0, \ldots, c_{l+2})$, and she proceeds as follows: $1i)$ B computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $1ii)$ she queries her inverse oracle on input (h, c_{l+2}). Then, $2i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $2ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$; $2iii)$ $\tilde{c}_0 = \mathsf{E}_{k_{0,E}}(p_B)$, and she checks if $c_0 \stackrel{?}{=} \tilde{c}_0$, if this is not the case she answers $\perp$ to A; otherwise $2iv)$ she computes $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. After that B computes for all $i = 1, \ldots, l$, $3i)$ B computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $3ii)$ $m_i = \mathsf{E}^{-1}_{k_{i,E}}(c_i)$; $3iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $3iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. Finally, $4i)$ B computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $4ii)$ $\tilde{c}_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. $4iii)$ if $\tilde{c}_{l+1}$ is equal to c_{l+1}, B answers $m = (m_1, \ldots, m_l)$ to A; otherwise $\perp$.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

B outputs whatever A does.

Thus, in total B makes at most $q_E + q_D = q$ queries to her oracle and runs in time bounded by $t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] = t_1$.

If her oracle is implemented with F, B correctly simulates Game 0 for A, otherwise Game 1. Since F is a $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$,

$$|\Pr[E_1] - \Pr[E_0]| \leq \epsilon_{\mathsf{sTPRP}}.$$

Game 2. This is Game 1, where instead of using f and f^{-1}, we use two tweakable random functions f and g, except we enforce consistency when g is queried on values previously defined via f (that is, if we have obtained $c_{l+2} = \mathsf{f}(h, k_0)$ we define $\mathsf{g}_k(h, c_{l+2}) := k_0$ [and vice-versa]).

Transition between Game 1 and Game 2. In Game 2, for decryption queries, we are using a sTPRP, while in Game 3 a rnadom function. Due to the well-known birthday bound, we have that

$$|\Pr[E_1] - \Pr[E_2]| \leq 2\frac{q(q-1)}{2^{n+1}},$$

because in addition to switching two PRPs with two PRFs (but since these functions are correlated we do a single switch), we have to consider the probability that we cannot do the simulation correctly, that is, when there exists a tweak h and two inputs x, x' s.t. $\mathsf{f}(h, x) = \mathsf{f}(h, x')$ (in this case, we cannot compute the inverse correctly) and vice-versa.

New sampling of f and g. From now on, f and g are sampled consistently from a table $\mathcal{L}$. Whenever $\mathsf{f}(h, x)$ (resp. $\mathsf{g}(h, z)$) is queried:

- If a triple (h, x, z) already appears in $\mathcal{L}$, return z (resp. return x).
- Otherwise, sample $z \xleftarrow{\$} \{0,1\}^n$ (resp. $x \xleftarrow{\$} \{0,1\}^n$), add (h, x, z) to $\mathcal{L}$, and return the sampled value.

We have already covered the case that we cannot correctly simulate the Game.

Game 3. This is Game 2 where we assume that during the execution of the Game, there are no collisions in the hash function.

Transition between Game 2 and Game 3. Since Game 2 and Game 3 are the same except if the following event Bad_1 happens: there is a collision for the hash function.

To bound $\Pr[\mathsf{Bad}_1]$ we build adversary C.

She plays the CR Game against the hash function H, using the adversary A.

At the start of the Game, C receives a key s in $\mathcal{HK}$ and gives it to A, and a block cipher E. Moreover, she has two lists $\mathcal{S}$ and $\mathcal{H}$ which are empty.

When A does an encryption query on input (a, m), C simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: $1i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $1ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$; $1iii)$ $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $1iv)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then, for all $i = 1, \ldots, l$, $2i)$ C computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ C computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ C computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h, k_0)$, keeping in memory (h, k_0, c_{l+2}) for f. So, she can answer A $c = (c_0 \| \ldots \| c_{l+2})$. Moreover, C adds (a, c) to the list $\mathcal{S}$.

For this, C needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c), C parses $c = (c_0, \ldots, c_{l+2})$, and she proceeds as follows: $1i)$ C computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_{l+1})$; and she computes $k_0 = \mathsf{g}(h, c_{l+2})$, keeping in memory (h, k_0, c_{l+2}). Then, $2i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $2ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$; $2iii)$ $\tilde{c}_0 = \mathsf{E}_{k_{0,E}}(p_B)$, and she checks if $c_0 \stackrel{?}{=} \tilde{c}_0$, if this is not the case she answers $\perp$ to A; otherwise $2iv)$ she computes $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. After that C computes for all $i = 1, \ldots, l$, $3i)$ C computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $3ii)$ $m_i = \mathsf{E}^{-1}_{k_{i,E}}(c_i)$; $3iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$;

and $3iv$) $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. Finally, $4i$) C computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $4ii$) $\tilde{c}_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$; $4iii$) if $\tilde{c}_{l+1}$ is equal to c_{l+1}, C answers $m = (m_1, \ldots, m_l)$ to A; otherwise $\perp$.

For this, C needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

After all encryption and decryption queries, C looks into her list $\mathcal{H}$ if she finds a collision: if it is the case, she outputs it, otherwise $(0^n, 1^n)$.

For this, C needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

Thus, in total C runs in time bounded by $t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] = t_1$.

C correctly simulates Game 2 for A. Since Game 2 and Game 3 are the same, except if event Bad_1 happens, and C wins her CR-Game if event Bad_1 happens, and C is a t_1-adversary and H is a $(t_1, \epsilon_{\mathsf{CR}})$-collision-resistant hash function,

$$|\Pr[E_2] - \Pr[E_3]| = \Pr[\mathsf{Bad}_1] \leq \epsilon_{\mathsf{CR}}.$$

Game 4. This is Game 3, with the condition that the following event does not happen: during a decryption query, when a new key k_0 is picked by g [29], k_0 has never been picked before.

Transition between Game 3 and Game 4. The two Games are the same except if the following Bad_2 event happens: there exists a decryption query s.t. it requires to pick a new key k_0^i and this key k_0^i has already been used before as a key for E. We observe that during an encryption or a decryption query, there are at most $3L+4$ different keys (3, k_i, $k_{i,E}$, and $k_{i,A}$, for each message blocks [for the last, we do not use $k_{l,A}$ as a key], which are at most L, 3, k_0, $k_{0,E}$, $k_{0,A}$ are for the start, and 2, k_{l+1}, and $k_{l+1,E}$ for the end). Thus, during the jth decryption query there are at most $q_E(3L+4) + (j-1)(3L+4)$ different keys that, if picked, would trigger the Bad_2 event. Thus,

$$\begin{aligned}
\Pr[\mathsf{Bad}_2] &\leq \sum_{j=1}^{q_D} \frac{q_E(3L+4) + (j-1)(3L+4)}{2^n} \\
&\leq \frac{q_D q_E(3L+4) + q_D(\frac{q_D-1}{2})(3L+4)}{2^n} \\
&= q_D[(3L+4)(q_E + \frac{q_D-1}{2})]2^{-n}.
\end{aligned}$$

$$\text{Thus, } |\Pr[E_3] - \Pr[E_4]| \leq \Pr[\mathsf{Bad}_2] = q_D[(3L+4)(q_E + \frac{q_D-1}{2})]2^{-n}.$$

Game 5. This is Game 4, where for every decryption query where a new k_0 is picked, instead of computing $k_{0,E}$ and $k_{0,A}$, we pick two random values.

Transition between Game 4 and Game 5. We use a sequence of Hybrids, Game 4^0, ..., Game 4^{q_D}. In Game 4^i, in the first i decryption queries, if k_0 is freshly picked, we pick $k_{0,E}$ and $k_{0,A}$ uniformly at random, in all the others $k_{0,E}$ and $k_{0,A}$ are correctly computed. Note Game 4 is Game 4^0, while Game 5 is Game 4^{q_D}.

Transition between Game 4^j and Game 4^{j+1}.

To bound $|\Pr[E_{4^j}] - \Pr[E_{4^{j+1}}]|$ we build a $(2, t_2)$-PRF adversary D^j.

She plays the PRF Game against the block cipher E, using the adversary A. That is, she has oracle access to an oracle which is either $\mathsf{E}_{k_0}(\cdot)$ or $\$(\cdot)$ for a random k_0, and she has to distinguish the two cases.

At the start of the Game, D^j receives a key s in $\mathcal{HK}$ and gives it to A, and a block cipher E. Moreover, she has a list $\mathcal{S}$ which is empty.

[29] that is, there is no entry in the list used for f and g containing (h, k_0, c_{l+2}).

When A does an encryption query on input (a, m), D^j simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: $1i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $1ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$; $1iii)$ $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $1iv)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then, for all $i = 1, \ldots, l$, $2i)$ D^j computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ D^j computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ D^j computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h, k_0)$, keeping in memory (h, k_0, c_{l+2}) for f. So, she can answer A $c = (c_0 \| \ldots \| c_{l+2})$. Moreover, D^j adds (a, c) to the list $\mathcal{S}$.

For this, D^j needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c), D^j parses $c = (c_0, \ldots, c_{l+2})$, and she proceeds as follows: $1i)$ D^j computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_{l+1})$; and $1ii)$, except for the j^{th} decryption, she computes $k_0 = \mathsf{g}(h, c_{l+2})$, keeping in memory (h, k_0, c_{l+2}). Then, a) if this is one of the first $j-1$ decryption queries and k_0 is freshly computed, then D^j $a2i)$ picks $k_{0,E}$, and $k_{0,A}$ uniformly at random (if not, D^j follows what has already been done for that k_0); b) if this is the j^{th} decryption query and k_0 should be freshly picked (and not already computed), $b2i)$ she queries her oracle on input p_A and p_B, obtaining $k_{0,E}$ and $k_{0,A}$ respectively (if k_0 should not be freshly picked, she follows what done with k_0 before); c) if this one of the last $q_D - j$ decryption query, if k_0 is freshly picked, D^j computes $c2i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $c2ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$ (otherwise, she follows what already done for that k_0). Moreover, for all decryption queries D^j computes $2iii)$ $\tilde{c}_0 = \mathsf{E}_{k_{0,E}}(p_B)$, and she checks if $c_0 \stackrel{?}{=} \tilde{c}_0$, if this is not the case she answers $\perp$ to A; otherwise $2iv)$ she computes $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. After that D^j computes for all $i = 1, \ldots, l$, $3i)$ D^j computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $3ii)$ $m_i = \mathsf{E}^{-1}_{k_{i,E}}(c_i)$; $3iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $3iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. Finally, $4i)$ D^j computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $4ii)$ $\tilde{c}_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$; $4iii)$ if $\tilde{c}_{l+1}$ is equal to c_{l+1}, D^j answers $m = (m_1, \ldots, m_l)$ to A; otherwise $\perp$.

For this, D^j needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$ except for the j^{th} decryption query where she needs time $t_{\mathsf{H}} + (4(l+1))t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1))t_{\mathsf{E}}$ and does two oracle queries.

After all encryption and decryption queries, D^j outputs whatever A does.

Thus, in total D^j runs in time bounded by $t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] - 2t_{\mathsf{E}} = t_2$ and does two oracle queries.

D^j correctly simulate Game 4^j for A if the oracle she has access is implemented with E_{k_0}; otherwise Game 4^{j+1}.

Thus,

$$|\Pr[E_{4^j}] - \Pr[E_{4^{j+1}}]| = |\Pr[1 \leftarrow \mathsf{D}^{j,\mathsf{E}_{k_0}(\cdot)}] - \Pr[1 \leftarrow \mathsf{D}^{j,\$(\cdot)}]| \leq \epsilon_{\mathsf{PRF}},$$

since D^j is a $(2, t_2)$-adversary and E is a $(2, t_2, \epsilon_{\mathsf{PRF}})$-$\mathsf{PRF}$.

Bounding $|\Pr[E_4] - \Pr[E_5]|$. Now, we can easily bound

$$|\Pr[E_4] - \Pr[E_5]| \leq \sum_{j=0}^{q_D-1} |\Pr[E_{4^j}] - \Pr[E_{4^{j+1}}]| \leq \sum_{j=0}^{q_D-1} \epsilon_{\mathsf{PRF}} = q_D \epsilon_{\mathsf{PRF}}.$$

Game 6. This is Game 5, where for every decryption query where a new k_0 is picked, instead of computing $\tilde{c}_0$, we pick it uniformly at random.

Transition between Game 5 and Game 6. We use a sequence of Hybrids, Game 5^0, ..., Game 5^{q_D}. In Game 5^i, in the first i decryption queries, if k_0 is freshly picked, we pick $\tilde{c}_0$ and k_1 uniformly at random, in all the others $\tilde{c}_0$ and k_1 are correctly computed. Note Game 5 is Game 5^0, while Game 6 is Game 5^{q_D}.

Transition between Game 5^j and Game 5^{j+1}.

To bound $|\Pr[E_{5^j}] - \Pr[E_{5^{j+1}}]|$ we build a $(2, t_2)$-PRF adversary EE^j.

She plays the PRF Game against the block cipher E, using the adversary A. That is, she has oracle access to an oracle which is either $\mathsf{E}_{k_{0,E}}(\cdot)$ or $\$(\cdot)$ for a random $k_{0,E}$, and she has to distinguish the two cases.

At the start of the Game, EE^j receives a key s in $\mathcal{HK}$ and gives it to A, and a block cipher E. Moreover, she has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a, m), EE^j simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: $1i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $1ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$; $1iii)$ $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $1iv)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then, for all $i = 1, \ldots, l$, $2i)$ EE^j computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ EE^j computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ EE^j computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h, k_0)$, keeping in memory (h, k_0, c_{l+2}) for f. So, she can answer A $c = (c_0 \| \ldots \| c_{l+2})$. Moreover, EE^j adds (a, c) to the list $\mathcal{S}$.

For this, EE^j needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c), EE^j parses $c = (c_0, \ldots, c_{l+2})$, and she proceeds as follows: $1i)$ EE^j computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_{l+1})$; and $1ii)$, except for the j^{th} decryption, she computes $k_0 = \mathsf{g}(h, c_{l+2})$, keeping in memory (h, k_0, c_{l+2}). Then, a) if this is one of the first $j-1$ decryption queries and k_0 is freshly computed, EE^j $a2i)$ picks $k_{0,E}$, and $k_{0,A}$ uniformly at random (if not, EE^j follows what has already been done for that k_0); $a2ii)$ EE^j picks $\tilde{c}_0$ and k_1 uniformly at random b) if this is the j^{th} decryption query and k_0 should be freshly picked (and not already computed), $b2i)$ she picks $k_{0,A}$ uniformly at random; $b2ii)$ she queries her oracle on input p_A and p_B, obtaining k_1 and $\tilde{c}_0$ respectively (if k_0 should not be freshly picked, she follows what done with k_0 and $k_{0,E}$ before); c) if this one of the last $q_D - j$ decryption query, if k_0 is freshly picked, $c2i)$ EE^j uniformly at random picks $k_{0,E}$ and $k_{0,A}$; and she computes $c2iii)$ $\tilde{c}_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $c2iv)$ she computes $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. (otherwise, she follows what already done for that k_0). Moreover, for all decryption queries EE^j checks if $c_0 \stackrel{?}{=} \tilde{c}_0$, if this is not the case she answers $\perp$ to A; otherwise she goes on in the computation. After that EE^j computes for all $i = 1, \ldots, l$, $3i)$ EE^j computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $3ii)$ $m_i = \mathsf{E}^{-1}_{k_{i,E}}(c_i)$; $3iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $3iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. Finally, $4i)$ EE^j computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $4ii)$ $\tilde{c}_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$; $4iii)$ if $\tilde{c}_{l+1}$ is equal to c_{l+1}, EE^j answers $m = (m_1, \ldots, m_l)$ to A; otherwise $\perp$.

For this, EE^j needs time $t_{\mathsf{H}} + (4(l+1))t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1))t_{\mathsf{E}}$ except for the j^{th} decryption query where she needs time $t_{\mathsf{H}} + (4(l+1)-2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)-2)t_{\mathsf{E}}$ and does two oracle queries.

After all encryption and decryption queries, EE^j outputs whatever A does.

Thus, in total EE^j runs in time bounded by $t + q[t_{\mathsf{H}} + (4(L+1))t_{\mathsf{E}}] - 2t_{\mathsf{E}} \leq t_2$ and does two oracle queries.

EE^j correctly simulate Game 5^j for A if the oracle she has access is implemented with $\mathsf{E}_{k_{0,E}}$; otherwise Game 5^{j+1}.

Thus,

$$|\Pr[E_{5^j}] - \Pr[E_{5^{j+1}}]| = |\Pr[1 \leftarrow \mathsf{EE}^{j,\mathsf{E}_{k_{0,E}}(\cdot)}] - \Pr[1 \leftarrow \mathsf{EE}^{j,\$(\cdot)}]| \leq \epsilon_{\mathsf{PRF}},$$

since EE^j is a $(2, t_2)$-adversary and E is a $(2, t_2, \epsilon_{\mathsf{PRF}})$-$\mathsf{PRF}$.

Bounding $|\Pr[E_5] - \Pr[E_6]|$. Now, we can easily bound

$$|\Pr[E_5] - \Pr[E_6]| \leq \sum_{j=0}^{q_D-1} |\Pr[E_{5^j}] - \Pr[E_{5^{j+1}}]| \leq \sum_{j=0}^{q_D-1} \epsilon_{\mathsf{PRF}} = q_D \epsilon_{\mathsf{PRF}}.$$

Game 7. This is Game 6, where for every fresh decryption query we answer $\perp$.

Transition between Game 6 and Game 7. We use a sequence of Hybrids, Game 6^0, ..., Game 6^{q_D}. In Game 6^i, in the first i decryption queries, if they are fresh, we answer $\perp$, in all the others we proceed as in the previous game. Note Game 6 is Game 6^0, while Game 7 is Game 6^{q_D}.

Transition between Game 6^j and Game 6^{j+1}.

To bound $|\Pr[E_{6^j}] - \Pr[E_{6^{j+1}}]|$ we observe that in the j^{th} decryption query if it is fresh, k_0 is picked uniformly at random (there are no other possibilities since k_0 is random due to event Bad_2 not happening, and there are no collisions in the hash function, due to event Bad_1 not happening), thus $\tilde{c}_0$ is picked uniformly at random. Therefore, the probability that $c_0 = \tilde{c}_0$ is 2^{-n}. Thus,

$$|\Pr[E_{5^j}] - \Pr[E_{5^{j+1}}]| = \Pr[\tilde{c}_0 = c_0 | \tilde{c}_0 \xleftarrow{\$} \{0,1\}^n] = 2^{-n}.$$

Bounding $|\Pr[E_6] - \Pr[E_7]|$

$$\leq \sum_{j=0}^{q_D-1} |\Pr[E_{6^j}] - \Pr[E_{6^{j+1}}]| \leq \sum_{j=0}^{q_D-1} 2^{-n} = q_D 2^{-n}.$$

Thus, from now on, we can omit to compute decryption queries when they are fresh, but we can simply answer $\perp$, invalid [30].

Game 8. This is Game 7, with the condition that the following event does not happen: during an encryption, when a new key k_0 is picked, k_0 has never been picked before.

Transition between Game 7 and Game 8. The two games are the same except if the following Bad_3 event happens: there exists an encryption query s.t. the freshly picked key k_0^i has already been used before as a key for E. We observe that during an encryption query, there are at most $3L + 4$ different keys. Thus, during the jth encryption query there are at most $(j-1)(3L+4)$ different keys that, if picked, would trigger the Bad_3 event. Thus,

$$\Pr[\mathsf{Bad}_3] \leq \sum_{j=1}^{q_E} \frac{(j-1)(3L+4)}{2^n} = \frac{(3L+4)}{2^n} \frac{q_E(q_E-1)}{2} = \frac{q_E(q_E-1)(3L+4)}{2^{n+1}}.$$

$$\text{Thus, } |\Pr[E_7] - \Pr[E_8]| \leq \Pr[\mathsf{Bad}_3] = \frac{q_E(q_E-1)(3L+4)}{2^{n+1}}.$$

Game 9. This is Game 8, where for every encryption query, instead of computing $k_{0,E}$ and $k_{0,A}$, we pick two random values.

Transition between Game 8 and Game 9. We use a sequence of Hybrids, Game 8^0, ..., Game 8^{q_E}. In Game 8^i, in the first i encryption queries, we pick $k_{0,E}$ and $k_{0,A}$ uniformly at random, in all the others $k_{0,E}$ and $k_{0,A}$ are correctly computed. Note Game 8 is Game 8^0, while Game 9 is Game 8^{q_E}.

Transition between Game 8^j and Game 8^{j+1}.

To bound $|\Pr[E_{8^j}] - \Pr[E_{8^{j+1}}]|$ we build a $(2, t_2)$-PRF adversary $\mathsf{FF}^{j,0}$.

She plays the PRF Game against the block cipher E, using the adversary A. That is, she has oracle access to an oracle which is either $\mathsf{E}_{k_0}(\cdot)$ or $\$(\cdot)$ for a random k_0, and she has to distinguish the two cases.

[30] Note that the second check on $\tilde{k}_l$ is not needed to provide integrity in the black-box scenario since it is with this first check alone. The second check only helps us against faults.

At the start of the Game, $\mathsf{FF}^{j,0}$ receives a key s in $\mathcal{HK}$ and gives it to A, and a block cipher E. Moreover, she has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a, m), $\mathsf{FF}^{j,0}$ simply picks k_0 uniformly at random from $\{0,1\}^n$ (except for the j^{th} encryption query) and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: a) for the first $j-1$ encryption queries, $\mathsf{FF}^{j,0}$ picks $k_{0,E}$ and $k_{0,A}$ uniformly at random b) for the j^{th} encryption query $\mathsf{FF}^{j,0}$ queries her oracle on input p_A and p_B, obtaining $k_{0,E}$ and $k_{0,A}$ respectively c) for the last $q_E - j$ encryption queries, $\mathsf{FF}^{j,0}$ computes $c1i)$ $k_{0,E} = \mathsf{E}_{k_0}(p_A)$; $c1ii)$ $k_{0,A} = \mathsf{E}_{k_0}(p_B)$. Moreover, for all encryption queries, $\mathsf{FF}^{j,0}$ computes $1iii)$ $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $1iv)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then, for all $i = 1, \ldots, l$, $2i)$ $\mathsf{FF}^{j,0}$ computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ $\mathsf{FF}^{j,0}$ computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ $\mathsf{FF}^{j,0}$ computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h, k_0)$, keeping in memory (h, k_0, c_{l+2}) for f. So, she can answer A $c = (c_0 \| \ldots \| c_{l+2})$. Moreover, $\mathsf{FF}^{j,0}$ adds (a, c) to the list $\mathcal{S}$.

For this, $\mathsf{FF}^{j,0}$ needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$ except for the j^{th} where the cost is time $t_{\mathsf{H}} + (4(l+1))t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1))t_{\mathsf{E}}$ and two oracle queries.

When A does a decryption query on input (a, c), $\mathsf{FF}^{j,0}$ simply answers $\perp$. This takes no time.

After all encryption and decryption queries, $\mathsf{FF}^{j,0}$ outputs whatever A does.

Thus, in total $\mathsf{FF}^{j,0}$ runs in time bounded by $t + q_E[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] - 2t_{\mathsf{E}} \leq t_2$ and does two oracle queries.

$\mathsf{FF}^{j,0}$ correctly simulates Game 8^j for A if the oracle she has access is implemented with E_{k_0}; otherwise Game 8^{j+1}.

Thus,

$$|\Pr[E_{8^j}] - \Pr[E_{8^{j+1}}]| = |\Pr[1 \leftarrow \mathsf{FF}^{j,0,\mathsf{E}_{k_0}(\cdot)}] - \Pr[1 \leftarrow \mathsf{FF}^{j,0,\$(\cdot)}]| \leq \epsilon_{\mathsf{PRF}},$$

since $\mathsf{FF}^{j,0}$ is a $(2, t_2)$-adversary and E is a $(2, t_2, \epsilon_{\mathsf{PRF}})$-$\mathsf{PRF}$.

Bounding $|\Pr[E_8] - \Pr[E_9]|$

$$\leq \sum_{j=0}^{q_E-1} |\Pr[E_{8^j}] - \Pr[E_{8^{j+1}}]| \leq \sum_{j=0}^{q_E-1} \epsilon_{\mathsf{PRF}} = q_E \epsilon_{\mathsf{PRF}}.$$

Game 10. This is Game 9, where for every encryption query, we pick uniformly at random c_0 and k_1.

Transition between Game 9 and Game 10. We use a sequence of Hybrids, Game 9^0, ..., Game 9^{q_E}. In Game 9^i, in the first i encryption queries, we pick c_0 and k_1 uniformly at random, in all the others c_0 and k_1 are correctly computed. Note Game 9 is Game 9^0, while Game 10 is Game 9^{q_E}.

Transition between Game 9^j and Game 9^{j+1}.

To bound $|\Pr[E_{9^j}] - \Pr[E_{9^{j+1}}]|$ we build a $(2, t_2)$-PRF adversary $\mathsf{GG}^{j,0}$.

She plays the PRF Game against the block cipher E, using the adversary A. That is, she has oracle access to an oracle which is either $\mathsf{E}_{k_{0,E}}(\cdot)$ or $\$(\cdot)$ for a random $k_{0,E}$, and she has to distinguish the two cases.

When A does an encryption query on input (a, m), $\mathsf{GG}^{j,0}$ simply picks $k_{0,E}$ uniformly at random from $\{0,1\}^n$ (except for the j^{th} encryption query and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: a) for the first $j-1$ encryption queries, $\mathsf{GG}^{j,0}$ picks c_0 and k_1 uniformly at random b) for the j^{th} encryption query $\mathsf{GG}^{j,0}$ queries

her oracle on input p_A and p_B, obtaining c_0 and k_1 respectively c) for the last $q_E - j$ encryption queries, $\mathsf{GG}^{j,0}$ computes $c1i)$ $c_{0,E} = \mathsf{E}_{k_{0,E}}(p_B)$; $c1ii)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Moreover, for all encryption queries, $\mathsf{GG}^{j,0}$ $1i)$ picks $k_{0,A}$ uniformly at random, and she computes $1iii)$ $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$; and $1iv)$ $k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then, for all $i = 1, \ldots, l$, $2i)$ $\mathsf{GG}^{j,0}$ computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ $\mathsf{GG}^{j,0}$ computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ $\mathsf{GG}^{j,0}$ computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_{l+1}, a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h, k_0)$, keeping in memory (h, k_0, c_{l+2}) for f. So, she can answer A $c = (c_0 \| \ldots \| c_{l+2})$. Moreover, $\mathsf{GG}^{j,0}$ adds (a, c) to the list $\mathcal{S}$.

For this, $\mathsf{GG}^{j,0}$ needs time $t_{\mathsf{H}} + (4(l+1))t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1))t_{\mathsf{E}}$ except for the j^{th} when she needs time $t_{\mathsf{H}} + (4(l+1)-2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)-2)t_{\mathsf{E}}$ and two oracle queries.

When A does a decryption query on input (a, c), $\mathsf{GG}^{j,0}$ simply answers $\perp$. This takes no time.

After all encryption and decryption queries, $\mathsf{GG}^{j,0}$ outputs whatever A does.

Thus, in total $\mathsf{GG}^{j,0}$ runs in time bounded by $t + q_E[t_{\mathsf{H}} + (4(L+1))t_{\mathsf{E}}] - 2t_{\mathsf{E}} \leq t_2$ and does two oracle queries.

GG^{j} correctly simulates Game 9^j for A if the oracle she has access is implemented with $\mathsf{E}_{k_{0,E}}$; otherwise Game 9^{j+1}.

Thus,

$$|\Pr[E_{9^j}] - \Pr[E_{9^{j+1}}]| = |\Pr[1 \leftarrow \mathsf{GG}^{j,0\mathsf{E}_{k_{0,E}}(\cdot)}] - \Pr[1 \leftarrow \mathsf{GG}^{j,0,\$(\cdot)}]| \leq \epsilon_{\mathsf{PRF}},$$

since $\mathsf{GG}^{j,0}$ is a $(2, t_2)$-adversary and E is a $(2, t_2, \epsilon_{\mathsf{PRF}})$-$\mathsf{PRF}$.

Bounding $|\Pr[E_9] - \Pr[E_{10}]|$

$$\leq \sum_{j=0}^{q_E-1} |\Pr[E_{9^j}] - \Pr[E_{9^{j+1}}]| \leq \sum_{j=0}^{q_E-1} \epsilon_{\mathsf{PRF}} = q_E \epsilon_{\mathsf{PRF}}.$$

Game 11. This is Game 10 where in all encryption queries we replace $c_1, \ldots, c_l$ and c_{l+2} with random values.

Transition between Game 10 and Game 11. We use a sequence of Hybrids, Game 10^0, ..., Game 10^{L+1}. In Game 10^i, we replace the first i ciphertext blocks among $c_1, \ldots, c_l, c_{l+1}$ with random values (note that c_{l+2} is already random since it is the output of a random function). Note Game 10 is Game 10^0, while Game 11 is Game 10^{L+1}.

Game $\tilde{10}^j$. To bound $|\Pr[E_{10^j}] - \Pr[E_{10^{j+1}}]|$ we introduce Game $\tilde{10}^j$, which is Game 10, with the condition that the following event does not happen: during an encryption, when a new key k_j is picked, k_j has never been picked before.

Transition between Game 10^j and Game $\tilde{10}^j$. The two Games are the same except if the following Bad_3^j event happens: there exists an encryption query s.t. the key picked k_j^i has already been used before as a key for E. We observe that during an encryption query there are at most $3L + 4$ different keys. Thus, during the jth encryption query there are at most $(j-1)(3L+4) + 2j$ different keys that, if picked, would trigger the Bad_3^j event. Thus,

$$\begin{aligned}\Pr[\mathsf{Bad}_3^j] \leq & \sum_{i=1}^{q_E} \frac{(i-1)(3L+4)+2j}{2^n} = \frac{(3L+4+2j)}{2^n} \frac{q_E(q_E-1)}{2} \\ = & \frac{q_E(q_E-1)(3L+4+2j)}{2^{n+1}}.\end{aligned}$$

$$\text{Thus, } |\Pr[E_{10^j}] - \Pr[E_{\tilde{10}^j}]| \leq \Pr[\mathsf{Bad}_3^j] = \frac{q_E(q_E-1)(3L+4+2j)}{2^{n+1}}.$$

Transition between Game $\tilde{1}\tilde{0}^j$ and Game 10^{j+1}. We use a sequence of Hybrids, Game $\tilde{1}\tilde{0}^{0,j}$, ..., Game $\tilde{1}\tilde{0}^{q_E,j}$, Game $\bar{1}\bar{0}^j$, ..., Game $\bar{1}\bar{0}^{j,q_E}$. In Game $\tilde{1}\tilde{0}^{i,j}$, in the first i encryption queries, we pick $k_{j,E}$ and k_{j+1} uniformly at random, in all the others $k_{j,E}$ and k_{j+1} are correctly computed. In Game $\bar{1}\bar{0}^{i,j}$, in all encryption queries, we pick $k_{j,E}$ and k_{j+1} uniformly at random, and in the first i encryption queries, we pick c_j uniformly at random, in all the others c_j is correctly computed. Note Game $\tilde{1}\tilde{0}^j$ is Game $\tilde{1}\tilde{0}^{0,j}$, while Game $\tilde{1}\tilde{0}^{q_E,j}$ is Game $\bar{1}\bar{0}^{0,j}$ and Game $\bar{1}\bar{0}^{q_E,j}$ is Game 10^{j+1}.

Transition between Game $\tilde{1}\tilde{0}^{j,j'}$ and Game $\tilde{1}\tilde{0}^{j,j'+1}$.

To bound $|\Pr[E_{\tilde{1}\tilde{0}^{j,j'}}] - \Pr[E_{\tilde{1}\tilde{0}^{j,j'+1}}]|$ we build a $(2,t_2)$-PRF adversary $\mathsf{FF}^{j',j}$.

She plays the PRF Game against the block cipher E, using the adversary A. That is, she has oracle access to an oracle which is either $\mathsf{E}_{k_j}(\cdot)$ or $\$(\cdot)$ for a random k_0, and she has to distinguish the two cases.

At the start of the Game, $\mathsf{FF}^{j',j}$ receives a key s in $\mathcal{HK}$ and gives it to A, and a block cipher E. Moreover, she has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a,m), $\mathsf{FF}^{j',j}$ simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1,\ldots,m_l)$, computes $c_0,\ldots,c_{l+1}$ as follows: $\mathsf{FF}^{j',j}$ picks $c_0,\ldots,c_{j-1}$, $k_0,\ldots,k_{j-1},k_{0,E},\ldots,k_{j-1,E}$ and $k_{0,A}$ uniformly at random (moreover, for the first $j-1$ encryption queries, she picks k_j uniformly at random); then, a) for the first $j'-1$ encryption queries, she picks $k_{j,E}$ and k_{j+1} uniformly at random; b) for the j'^{th} encryption query $\mathsf{FF}^{j',j}$ queries her oracle on input p_A and p_B, obtaining $k_{j,E}$ and $k_{j,A}$ respectively; c) for the last q_E-j encryption queries, $\mathsf{FF}^{j',j}$ picks uniformly at random k_j and computes $c1i)$ $k_{j,E}=\mathsf{E}_{k_j}(p_A)$; $c1ii)$ $k_{j+1}=\mathsf{E}_{k_j}(p_B)$. Moreover, for all encryption queries, $\mathsf{FF}^{j',j}$ computes $c_j=\mathsf{E}_{k_{j,E}}(m_j)$, and, for $i=1,\ldots,j$ $k_{i,A}=\mathsf{E}_{k_{i-1,A}}(m_i)$. Then, for all $i=j+1,\ldots,l$, $2i)$ $\mathsf{FF}^{j',j}$ computes $k_{i,E}=\mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i=\mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1}=\mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A}=\mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ $\mathsf{FF}^{j',j}$ computes $k_{l+1,E}=\mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1}=\mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ $\mathsf{FF}^{j',j}$ computes $h=\mathsf{H}_s(c_0\|\ldots\|c_{l+1},a)$; and $4ii)$ she computes $c_{l+2}=\mathsf{f}(h,k_0)$, keeping in memory (h,k_0,c_{l+2}) for f. So, she can answer A $c=(c_0\|\ldots\|c_{l+2})$. Moreover, $\mathsf{FF}^{j',j}$ adds (a,c) to the list $\mathcal{S}$.

For this, $\mathsf{FF}^{j',j}$ needs time $t_{\mathsf{H}}+(4(l-j)+2)t_{\mathsf{E}} \le t_{\mathsf{H}}+(4(L-j)+2)t_{\mathsf{E}}$ except for the j^{th} when she needs time $t_{\mathsf{H}}+(4(l-j))t_{\mathsf{E}} \le t_{\mathsf{H}}+(4(L-j))t_{\mathsf{E}}$ and two oracle queries.

When A does a decryption query on input (a,c), $\mathsf{FF}^{j',j}$ simply answers $\perp$. This takes no time.

After all encryption and decryption queries, $\mathsf{FF}^{j',j}$ outputs whatever A does.

Thus, in total $\mathsf{FF}^{j',j}$ runs in time bounded by $t+q_E[t_{\mathsf{H}}+(4(L-j)+2)t_{\mathsf{E}}]-2t_{\mathsf{E}} \le t_2$ and does two oracle queries.

$\mathsf{FF}^{j',j}$ correctly simulates Game $\tilde{1}\tilde{0}^{j',j}$ for A if the oracle she has access is implemented with E_{k_j}; otherwise Game $10^{j'+1,j}$.

Thus,

$$|\Pr[E_{\tilde{1}\tilde{0}^{j',j}}] - \Pr[E_{\tilde{1}\tilde{0}^{j'+1,j}}]| = |\Pr[1 \leftarrow \mathsf{FF}^{j',j,\mathsf{E}_{k_j}(\cdot)}] - \Pr[1 \leftarrow \mathsf{FF}^{j',j,\$(\cdot)}]| \le \epsilon_{\mathsf{PRF}},$$

since $\mathsf{FF}^{j',j}$ is a $(2,t_2)$-adversary and E is a $(2,t_2,\epsilon_{\mathsf{PRF}})$-PRF.

Bounding $|\Pr[E_{\tilde{1}\tilde{0}^j}] - \Pr[E_{\bar{1}\bar{0}^{0,q_E,j}}]|$

$$\le \sum_{j'=0}^{q_E-1} |\Pr[E_{\tilde{1}\tilde{0}^{j',j}}] - \Pr[E_{\tilde{1}\tilde{0}^{j'+1},j}]| \le \sum_{j'=0}^{q_E-1} \epsilon_{\mathsf{PRF}} = q_E\epsilon_{\mathsf{PRF}}.$$

Transition between Game $\bar{1}0^{j',j}$ and Game $\bar{1}0^{j'+1,j}$.

To bound $|\Pr[E_{\bar{1}0^{j',j}}] - \Pr[E_{\bar{1}0^{j'+1,j}}]|$ we build a $(2,t_2)$-PRF adversary $\mathsf{GG}^{j',j}$.

She plays the PRF Game against the block cipher E, using the adversary A. That is, she has oracle access to an oracle which is either $\mathsf{E}_{k_{j,E}}(\cdot)$ or $\$(\cdot)$ for a random $k_{j,E}$, and she has to distinguish the two cases.

When A does an encryption query on input (a,m), $\mathsf{GG}^{j',j}$ if $j \neq l+1$ simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1,\ldots,m_l)$, computes $c_0,\ldots,c_{l+1}$ as follows: $\mathsf{GG}^{j',j}$ picks $c_0,\ldots,c_{j-1}$, $k_0,\ldots,k_j,k_{0,E},\ldots,k_{j-1,E}$ and $k_{0,A}$ uniformly at random (moreover, for the first $j-1$ encryption queries, she picks $k_{j,E}$ uniformly at random); then, a) for the first $j'-1$ encryption queries, she picks c_j uniformly at random; b) for the j'^{th} encryption query $\mathsf{GG}^{j',j}$ queries her oracle on input m_j, obtaining c_j; c) for the last $q_E - j$ encryption queries, $\mathsf{GG}^{j',j}$ picks uniformly at random $k_{j,E}$ and computes $c_j = \mathsf{E}_{k_{j,E}}(m_j)$. Moreover, for all encryption queries, $\mathsf{GG}^{j',j}$ computes for $i = 1,\ldots,j$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. Then, for all $i = j+1,\ldots,l$, $2i)$ $\mathsf{GG}^{j',j}$ computes $k_{i,E} = \mathsf{E}_{k_i}(p_B)$; $2ii)$ $c_i = \mathsf{E}_{k_{i,E}}(m_i)$; $2iii)$ $k_{i+1} = \mathsf{E}_{k_i}(p_A)$; and $2iv)$ $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. After that, $3i)$ $\mathsf{GG}^{j',j}$ computes $k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$; and $3ii)$ $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ $\mathsf{GG}^{j',j}$ computes $h = \mathsf{H}_s(c_0\|\ldots\|c_{l+1},a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h,k_0)$, keeping in memory (h,k_0,c_{l+2}) for f. So, she can answer A $c = (c_0\|\ldots\|c_{l+2})$. Moreover, $\mathsf{GG}^{j',j}$ adds (a,c) to the list $\mathcal{S}$.

For this, $\mathsf{GG}^{j',j}$ needs time $t_{\mathsf{H}} + (4(l-j)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L-j)+2)t_{\mathsf{E}}$ except for the j^{th} when she needs time $t_{\mathsf{H}} + (4(l-j))t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L-j))t_{\mathsf{E}}$ and one oracle query.

When A does an encryption query on input (a,m), $\mathsf{GG}^{j',j}$ if $j = l+1$ simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1,\ldots,m_l)$, computes $c_0,\ldots,c_{l+1}$ as follows: $\mathsf{GG}^{j',j}$ picks $c_0,\ldots,c_{j-1}$, $k_0,\ldots,k_j,k_{0,E},\ldots,k_{j-1,E}$ and $k_{0,A}$ uniformly at random. Moreover, for all encryption queries, $\mathsf{GG}^{j',j}$ computes for $i = 1,\ldots,j$ $k_{j,A} = \mathsf{E}_{k_{j-1,A}}(m_j)$. Then, a) for the first $j'-1$ encryption queries, she picks c_{l+1} uniformly at random; b) for the j'^{th} encryption query $\mathsf{GG}^{j',j}$ queries her oracle on input $k_{l,A}$, obtaining c_{l+1}; c) for the last $q_E - j$ encryption queries, $\mathsf{GG}^{j',j}$ picks uniformly at random $k_{l+1,E}$ and computes $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Finally, $4i)$ $\mathsf{GG}^{j',j}$ computes $h = \mathsf{H}_s(c_0\|\ldots\|c_{l+1},a)$; and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h,k_0)$, keeping in memory (h,k_0,c_{l+2}) for f. So, she can answer A $c = (c_0\|\ldots\|c_{l+2})$. Moreover, $\mathsf{GG}^{j',j}$ adds (a,c) to the list $\mathcal{S}$.

For this, $\mathsf{GG}^{j',j}$ needs time $t_{\mathsf{H}} + (4(l-j)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L-j)+2)t_{\mathsf{E}}$ except for the j^{th} when she needs time $t_{\mathsf{H}} + (4(l-j))t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L-j))t_{\mathsf{E}}$ and one oracle query.

When A does a decryption query on input (a,c), $\mathsf{GG}^{j',j}$ simply answers $\perp$. This takes no time.

After all encryption and decryption queries, $\mathsf{GG}^{j',j}$ outputs whatever A does.

Thus, in total $\mathsf{GG}^{j',j}$ runs in time bounded by $t + q_E[t_{\mathsf{H}} + (4(L-j)+2)t_{\mathsf{E}}] - 2t_{\mathsf{E}} \leq t_2$ and does one oracle query.

GG^{j} correctly simulates Game $\bar{1}0^{j',j}$ for A if the oracle she has access is implemented with $\mathsf{E}_{k_{j,E}}$; otherwise Game $\bar{1}0^{j'+1,j}$.

Thus,

$$|\Pr[E_{\bar{1}0^{j',j}}] - \Pr[E_{\bar{1}0^{j'+1,j}}]| = |\Pr[1 \leftarrow \mathsf{GG}^{j',j\mathsf{E}_{k_{j,E}}(\cdot)}] - \Pr[1 \leftarrow \mathsf{GG}^{j',j,\$(\cdot)}]| \leq \epsilon_{\mathsf{PRF}},$$

since $\mathsf{GG}^{j,0}$ is a $(2,t_2)$-adversary and E is a $(2,t_2,\epsilon_{\mathsf{PRF}})$-PRF.

Bounding $|\Pr[E_{10}] - \Pr[E_{11}]|$**.** First, we observe that

$$|\Pr[E_{\bar{1}\bar{0}^{0},j}] - \Pr[E_{\bar{1}\bar{0}^{q_E},j}]| \leq \sum_{j'=0}^{q_E-1} |\Pr[E_{\bar{1}\bar{0}^{j'},j}] - \Pr[E_{\bar{1}\bar{0}^{j'+1},j}]| \leq \sum_{j'=0}^{q_E-1} \epsilon_{\mathsf{PRF}} = q_E\epsilon_{\mathsf{PRF}}.$$

Thus, for all $j = 0, \ldots, L$,

$$\Pr[E_{10}^{j}] - \Pr[E_{10}^{j+1}] \leq \frac{q_E(q_E-1)(3L+4+2j)}{2^{n+1}} + 2q_E\epsilon_{\mathsf{PRF}}.$$

Now, we can easily bound

$$\begin{aligned}
|\Pr[E_{10}] - \Pr[E_{11}]| \leq & \sum_{j=0}^{L+1} |\Pr[E_{10^j}] - \Pr[E_{10^{j+1}}]| \leq \sum_{j=0}^{L+1} \frac{q_E(q_E-1)(3L+4+2j)}{2^{n+1}} + 2q_E\epsilon_{\mathsf{PRF}} \\
&= 2q_E(L+1)\epsilon_{\mathsf{PRF}} + q_E(q_E-1)(L+1)\frac{(3L+4)+(L+2)}{2^{n+1}} = \\
&= 2q_E(L+1)\epsilon_{\mathsf{PRF}} + q_E(q_E-1)(L+1)\frac{4L+6}{2^{n+1}}.
\end{aligned}$$

Concluding the proof. Now, putting all our bounds together we can conclude the proof, because in Game 11 the answers to all decryption queries is $\perp$ (if they are not the forward of a previous encryption query) and of all encryption queries is a sequence of $l+3$ random blocks:

$$\begin{aligned}
&|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{AEnc}_k(\cdot,\cdot),\mathsf{ADec}_k(\cdot,\cdot)}] - \Pr[1 \leftarrow \mathsf{A}^{\$(\cdot,\cdot),\perp(\cdot,\cdot)}]| = |\Pr[E_0] - \Pr[E_{11}]| \leq \\
&|\Pr[E_{11}]| + \sum_{i=0}^{10} |\Pr[E_i] - \Pr[E_{i+1}]| = \sum_{i=0}^{10} |\Pr[E_i] - \Pr[E_{i+1}]| \\
\leq & \epsilon_{\mathsf{sTPRP}} + \frac{q(q-1)}{2^{n+1}} + \epsilon_{\mathsf{CR}} + q_D[(3L+4)(q_E + \frac{q_D-1}{2})]2^{-n} + q_D\epsilon_{\mathsf{PRF}} \\
& + q_D\epsilon_{\mathsf{PRF}} + q_D 2^{-n} + \frac{q_E(q_E-1)(3L+4)}{2^{n+1}} + q_E\epsilon_{\mathsf{PRF}} + q_E\epsilon_{\mathsf{PRF}} \\
& + 2q_E(L+1)\epsilon_{\mathsf{PRF}} + q_E(q_E-1)(L+1)\frac{4L+6}{2^{n+1}} \\
\leq & \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + (2q_D + 2q_E + 2q_E(L+1))\epsilon_{\mathsf{PRF}} + \\
& + \frac{q_D[(3L+4)(q_E+q_D-1)/2] + q_D}{2^n} \\
& + \frac{q_E(q_E-1)(3L+4) + q_E(q_E-1)(L+1)(4L+6) + q(q-1)}{2^{n+1}} \\
\leq & \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + 2(q + q_E(L+1))\epsilon_{\mathsf{PRF}} + \frac{q_D[1 + (3L+4)(q_E + (q_D-1)/2]}{2^n} \\
& + \frac{q_E(q_E-1)(3L+4+(L+1)(4L+6)) + q(q-1)}{2^{n+1}} = \epsilon.
\end{aligned}$$

□

5.2 Division of CONCRETE2 into atoms

Before moving on to proving the security in the presence of faults and leakage, we first need to divide CONCRETE2 into atoms.

Encryption. First, we observe that the associated data, a, is used only once by CONCRETE2, thus, according to the atomic model (Sec. 2.8.1), it cannot be faulted.

For the query on input (a, m), we define $(x_1, \ldots, x_l) = (m_1, \ldots, m_l)$, the parsing of m in n-bit long blocks, $x_{l+1} = a$. Similar to what was done in [10], we assume that the master key is hard-coded into the implementation; thus, we consider $\mathsf{F}_k^{\cdot}(\cdot)$ as an atom; moreover, we consider the production of randomness as an additional atom. Therefore, as atoms, we consider $\mathsf{F}_k^{\cdot}(\cdot)$, $\mathsf{E}_{\cdot}(p_A)$, $\mathsf{E}_{\cdot}(p_B)$, $\mathsf{E}_{\cdot}(\cdot)$, $\mathsf{H}_s(\cdot,\cdot)$, and the random picking $\cdot \xleftarrow{\$} \{0,1\}^n$.

We define $f_{0.1} = k_0 \xleftarrow{\$} \{0,1\}^n$, $f_{0.2} := k_{0,E} = \mathsf{E}_{k_0}(p_B)$, $f_{0.3} := k_{0,A} = \mathsf{E}_{k_0}(p_A)$, $f_{0.4} := c_0 = \mathsf{E}_{k_{0,E}}(p_B)$, $f_{0.5} := k_1 = \mathsf{E}_{k_{0,E}}(p_A)$; Then we iterate these four blocks for $i = 1, ..., l$: $f_{i.1} := k_{i,E} = \mathsf{E}_{k_i}(p_B)$, $f_{i.2} := c_i = \mathsf{E}_{k_{i,E}}(m_i)$, $f_{i.3} := k_{i+1} = \mathsf{E}_{k_i}(p_A)$, and $f_{i.4} := k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$; afterwards, $f_{l+1.1} := k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$, $f_{l+1.2} := c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$, $f_{l+2.1} := h = \mathsf{H}(c_0 \| \ldots \| c_{l+1}, a)$, and $f_{l+2.2} := c_{l+2} = \mathsf{F}_k^h(k_0)$.

We observe that $f_{0.1}$ uses no input, m_i is used only in the $f_{i.2}$ and $f_{i.4}$ atoms. The output is $(z_{0.4}, z_{1.2}, ..., z_{l.2}, z_{l+1.2}, z_{l+2.2}) = (c_0, c_1, ..., c_l, c_{l+1}, c_{l+2})$, that is, $\mathsf{Select}(z_{0.1}, \ldots, z_{l+2.2}) = (z_{0.4}, z_{1.2}, ..., z_{l.2}, z_{l+1.2}, z_{l+2.2})$.

We now present the dependency matrix of the encryption of CONCRETE2:

$$
\begin{pmatrix}
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & z_{0.1} & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & z_{0.1} & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & z_{0.2} & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & z_{0.3} & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\vdots & & & & & & & \vdots & \ddots & \ddots & & \ddots & & \ddots & \vdots & & \vdots & \vdots \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & z_{i-1.3} & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & x_i & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & z_{i.1} & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & z_{i-1.3} & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\cdots & x_i & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & z_{i-1.4} & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon \\
\vdots & & & & & \vdots & & \vdots & \ddots & & \ddots & & & \ddots & \vdots & & \vdots & \vdots \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & z_{l.3} & \epsilon & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & z_{l+1.1} & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & z_{l.4} & z_{l+1.1} & \epsilon \\
\cdots & \epsilon & \cdots & x_{l+1} & \epsilon & \epsilon & \epsilon & z_{0.4} & \cdots & \epsilon & \epsilon & \epsilon & z_{i.2} & \cdots & \epsilon & \epsilon & z_{l+1.1} & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & z_{0.1} & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & z_{l+2.1}
\end{pmatrix}
\tag{5}
$$

where we have represented the lines corresponding to the computations 0.1, 0.2, 0.3, 0.4, 0.5, $i.1$, $i.2$, $i.3$, $i.4$, $l + 1.1$, $l + 1.2$, $l + 2.1$ and $l + 2.2$ (the columns correspond to the variables m_i, a, k_0, $k_{0,E}$, $k_{0,A}$, c_0, k_i, $k_{i-1,A}$, $k_{i,E}$, c_i, k_{l+1}, $k_{l,A}$, c_{l+1}, and h).

Decryption. We proceed similarly for decryption. For a query on input (a, c), we define $(x_0, x_1, \ldots, x_{l+2}) = (c_0, \ldots, c_{l+1}, c_{l+2})$, the parsing of c into n-bit long blocks, $x_{l+3} = a$. Moreover, as atoms, we consider $\mathsf{F}_k^{-1,\cdot}(\cdot)$, $\mathsf{E}_{\cdot}(p_A)$, $\mathsf{E}_{\cdot}(p_B)$, $\mathsf{E}_{\cdot}^{-1}(\cdot)$, $\mathsf{E}_{\cdot}(\cdot)$, and $\mathsf{H}_s(\cdot,\cdot)$. Additionally, as in [10], we do not treat the comparisons $c_0 \stackrel{?}{=} \tilde{c}_0$, $k_{l,A} \stackrel{?}{=} \tilde{k}_{l,A}$ as an atomic operation, since we assume that both values are known to the adversary, either through leakage, or because one is part of the input.

We define $f_{-1.1} := h = \mathsf{H}(c_0 \| ... \| c_{l+1}, a)$, $f_{-1.2} := k_0 = \mathsf{F}_k^{-1,h}(c_{l+2})$, $f_{0.1} := k_{0,E} = \mathsf{E}_{k_0}(p_A)$, $f_{0.2} := k_{0,A} = \mathsf{E}_{k_0}(p_B)$, $f_{0.3} := \tilde{c}_0 = \mathsf{E}_{k_{0,E}}(p_B)$, $f_{0.4} := k_1 = \mathsf{E}_{k_{0,E}}(p_A)$. Then we iterate these four blocks for $i = 1, ..., l$: $f_{i.1} := k_{i,E} = \mathsf{E}_{k_i}(p_B)$, $f_{i.2} := m_i = \mathsf{E}_{k_{i,E}}^{-1}(c_i)$, $f_{i.3} := k_{i+1} = \mathsf{E}_{k_i}(p_A)$, $f_{i.4} := k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$. Finally, we compute $f_{l+1.1} := k_{l+1,E} = \mathsf{E}_{k_{l+1}}(p_B)$, and $f_{l+1.2} := \tilde{k}_{l,A} = \mathsf{E}_{k_{l+1,E}}^{-1}(c_{l+1})$.

The output is $(z_{1.2}, ..., z_{l.2}) = (m_1, ..., m_l)$, if the query is valid, and $\perp$ otherwise.

$$\begin{pmatrix}
\cdots & x_i & \cdots & x_{l+1} & \epsilon & x_{l+3} & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & x_{l+2} & \epsilon & z_{-1.1} & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & z_{-1.2} & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & z_{-1.2} & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & z_{0.1} & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & z_{0.1} & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\vdots & & \vdots & & & & & & & \vdots & \ddots & \ddots & \ddots & & \vdots & & \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & z_{i-1.4} & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & x_i & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & z_{i.1} & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & z_{i-1.4} & \epsilon & \epsilon & z_{i.3} & \cdots & \epsilon & \epsilon \\
\vdots & & \vdots & & & & & & & \vdots & \ddots & \ddots & \ddots & \ddots & \vdots & & \\
\cdots & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & z_{l.4} & \epsilon \\
\cdots & \epsilon & \cdots & x_{l+1} & \epsilon & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & \epsilon & \epsilon & \epsilon & \cdots & \epsilon & z_{l+1.1}
\end{pmatrix} \tag{6}$$

where we have represented the lines corresponding to the computations -1.1, -1.2, 0.1, 0.2, 0.3, 0.4, $i.1$, $i.2$, $i.3$, $i.4$, $l + 1.1$, $l + 1.2$ (the columns correspond to the variables c_i, c_{l+1}, c_{l+2}, a, k_0, $k_{0,E}$, $k_{i-1,A}$, k_i, $k_{i,E}$, m_i, k_{l+1}, and $k_{l,A}$).

5.3 Impossibility Results for the CIL2F2-Security of CONCRETE2

We start by presenting impossibility results concering the number of values that can be set via faults (or per atom). Then, we establish the CIL2F2 security of CONCRETE2. Combining these results, we obtain a tight characterization of security in terms of the number of faults that an adversary can set.

5.3.1 Setting 2 Values

An adversary can easily forge if allowed to set 2 values per encryption query. She can proceed as follows: she chooses k'_0, m', and a', then she computes $c'_0, ..., c'_{l+1}$, and $h' = \mathsf{H}_s(c'_0 \| \ldots \| c'_{l+1}, a')$ as in an encryption query where the key picked is k'_0, the message is m' and the associated data is a' (this can be done since in a real encryption, the only unknown value to the adversary is k_0; here, however, she chooses it herself). Then, she does an encryption query on input (a, m). When c_{l+2} is computed, $c_{l+2} = \mathsf{F}_k^h(k_0)$, she simply uses faults to replace h with h' and k_0 with k'_0. Since no message or associated data is touched via faults, the faults chosen by the adversary are admissible. The encryption oracle outputs $(c_0, \ldots, c_{l+2})$. The adversary outputs as a forgery $c^* = (c'_0, \ldots, c'_{l+1}, c_{l+2})$.

Clearly, c^* is fresh. Moreover, c^* is valid since hashing $c'_0, \ldots, c'_{l+1}$ and a, yields h', and $\mathsf{F}_k^{-1,h'}(c_{l+2}) = k'_0$ and $c'_0, \ldots, c'_{l+1}$ corresponds to the correct encryption of m' (that is, c'_0 is the correct committing for k'_0 and c'_{l+1} is consistent with k'_0 and m').

Thus, CONCRETE2 cannot be CIL2F2 secure when the adversary can set two values via a fault. We discuss next the theoretical reason behind this result and an additional attack where the adversary can set one value per atom.

5.3.2 General Results on Setting Two Values.

The attack represented in Sec. 5.3 is related to a theoretical result which we informally state here:

Theorem 14. *In the unbuonded leakage model, if the adversary can set all the values of an atom in the direct queries (i.e.,* Enc *or* Mac*), then no* CIL2F2 *(or* SUF−FL2*) security is achievable.*

Proof. (*Idea:*) We consider only the atom in which the key of the scheme is used (or part of the key). Using a fault, an adversary can choose the inputs of this atom. Then, via leakage, she obtains the output. Thus, she effectively has oracle access to all atoms that use the key. With this oracle access, it is easy to compute a forgery, because she can compute all other atoms having this oracle access (we have only to check that these faults are admissible). □

This result cannot (luckily for us) be extended when the adversary can inject any fault in the atom of inverse queries (that is, Dec and Vrfy) because, as Berti et al. do for LR−MACr, in these queries it is possible to avoid using the atoms (using the master key) which are used in the direct queries.

For simplicity, we have not used the dependency matrix or the faulty matrix, since they can be derived straightforwardly.

5.3.3 Setting one Value per Atom.

In the previous attack, the adversary can forge simply by setting the two inputs of the atom $\mathsf{F}_k^{\cdot}(\cdot)$, so it is natural to see if it is possible to attack CONCRETE2 if the adversary can set a single value per atom (thus, preventing the previous attack which gives the adversary oracle access to $\mathsf{F}_k^{\cdot}(\cdot)$).

Unfortunately, an attack is possible. The adversary can proceed as follows: she chooses k'_0, then computes the ephemeral keys $k'_{0,A}, k'_1$ and the ciphertext block c'_0 exactly as in an encryption query where the key picked is k'_0. Then, she does an encryption query on input (a, m), where she injects these faults: in the computation of $k_{1,A}$, she replaces $k_{0,A}$ with $k'_{0,A}$, every time k_1 is used, she replaces it with k'_1, in the computation of h we replace c_0 with c'_0. The output of this faulted query is $c = (c_0, \ldots, c_{l+1}, c_{l+2})$. The adversary outputs (a, c^*) with $c^* = (c'_0, c_1, \ldots, c_{l+2})$.

Clearly, c^* is fresh. Moreover, c^* is valid since hashing $c'_0, c_1, \ldots, c_{l+1}$ and a, we obtain the same h used in the encryption query, and $\mathsf{F}_k^{-1,h}(c_{l+2}) = k'_0$. c'_0 is the correct committing for k'_0, and $c_1, \ldots, c_l$ is the correct encryption for m using k'_0 since the faults force the encryption to use k'_0, and c_{l+1} is the correct committing for m and k'_0 (because the computation of c_{l+1} is the product of a chain starting from $k'_{0,A}$ and using the correct m).

Thus, CONCRETE2 cannot be CIL2F2 secure when the adversary can set a value per atom via faults.

Note that in this attack we set 4 values (once c_0, twice k_1, and once $k_{0,A}$).

5.4 Pre-image Resistance for a Chosen Image Offset of a Random Value

Now, we need to introduce a new hash security definition: it is hard to find a pre-image for $h \oplus \Delta$ with $h = \mathsf{H}_s(x)$ where x is sufficiently "random".

Definition 24. Let $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \{0,1\}^n$ be a hash function. We say that H is (t, α, ϵ)-*pre-image resistant for a chosen offset of the image of a random value* (PR−coirv) if, for any t-adversary, $\mathsf{A} = (\mathsf{A}_1, \mathsf{A}_2)$

$$\Pr[x \leftarrow \mathsf{A}_2(\mathsf{st}, x, h') \text{ s.t. } \mathsf{H}_s(x) = h' \mid h' = h \oplus \Delta,$$
$$h = \mathsf{H}_s(x),\ x \xleftarrow{\mathcal{D}} \{0,1\}^*,\ (\mathsf{st}, \mathcal{D}, \Delta) \leftarrow \mathsf{A}_1(s),\ s \xleftarrow{\$} \mathcal{HK}] \leq \epsilon,$$

where $\Delta \in \{0,1\}^n \setminus \{0^n\}$, and $\mathcal{D}$ is any distribution over $\{0,1\}^*$ s.t.

$$\max_{x \in \{0,1\}^*} \Pr[x \xleftarrow{\mathcal{D}} \{0,1\}^*] \leq \alpha.$$

To be meaningful α should be close to 2^{-n}, the uniform on the target space (looking ahead, we cannot use $\alpha = 2^{-n}$ for technical reasons [31]). In essence, the adversary first chooses a distribution $\mathcal{D}$ over $\{0,1\}^n$, subject to the condition that no value is sampled with probability greater than α, and also selects an offset Δ. Then, after receiving $h = \mathsf{H}_s(x)$ with $x \xleftarrow{\mathcal{D}} \{0,1\}^*$, she must compute a pre-image for $h \oplus \Delta$.

We require that $\Delta \neq 0^n$ because, otherwise x is a pre-image, thus, an adversary trivially wins (and, looking ahead, we have no interest in this case in our proof). Implicitly, we require that $\mathcal{D}$ is efficiently samplable and describable. This definition does not hold for arbitrary distributions; for example, a distribution that outputs only pre-images of a fixed value x.

This definition is in the standard model. It is trivial to see that this property holds if we model H as a random oracle.

5.4.1 PR–coirv and Range-Oriented Pre-Image Resistance.

For a good hash function, we expect that h behaves as a random value, since $\mathcal{D}$ is a reasonably good random distribution. Therefore, PR–coirv should be seen as closely related to the range-oriented pre-image resistance that we introduce now. The adversary has to find a pre-image for a value sampled uniformly at random [50].

Definition 25. A hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \{0,1\}^N$ is (t,ϵ)-*range oriented pre-image resistant* (PR) if, $\forall$ t-adversaries A,

$$\Pr[m \leftarrow \mathsf{A}(s,y) \text{ s.t. } \mathsf{H}_s(m) = y \mid s \xleftarrow{\$} \mathcal{HK}, y \xleftarrow{\$} \{0,1\}^N] \leq \epsilon.$$

Now, we compare the PR–coirv definition, Def. 24. In the PR–coirv security definition, the target for the hash function is $\mathsf{H}_s(x) \oplus \Delta$ where Δ is chosen by the adversary and x sufficiently enough. For a reasonable hash function we expect that $\mathsf{H}_s(x)$ is "random" given x "random" enough, so we expect that PR–coirv is equivalent to PR.

In the random oracle model, with probability $\leq q2^{-n}$, the value sampled by $\mathcal{D}$ has not been previously evaluated by H (q is the number of queries made by the adversary). Therefore, except with this negligible probability, the target h' is uniformly random. We expect that a similar situation will happen for reasonably good hash functions. Thus, if the target obtained in the PR–coirv experiment is indistinguishable from a random one, the probability that a PR–coirv adversary wins is the same as the probability that a PR adversary wins.

5.5 The CIL2F2-security of CONCRETE2

This section states and proves the integrity in the presence of leakage and faults of CONCRETE2. We start by giving the leakage functions and the faulty matrices for CONCRETE2.

Leakage function and faulty matrix. For the leakage function, we use the same leakage functions as in CONCRETE: $\mathsf{L}_{\mathsf{AEnc}}(a,m,k_0;k) := k_0$, and $\mathsf{L}_{\mathsf{ADec}}(a,c;k) := k_0$, because from k_0 the adversary can compute correctly all ephemeral values. Regarding faults, the adversary is allowed to set any fault she wants between the atoms in decryption, without

[31] In our proof, we will use a distribution that is very close to 2^{-n}.

any restriction; in encryption, she can use any differential fault she wants, moreover, she is allowed to set a single value. That is, all entries of $\mathcal{I}_{\mathsf{AEnc}}$ are either ϵ (when the corresponding value in the dependency matrix is ϵ) or $\perp$ /$\oplus$ with the exception of a single value. We recall that due to the atomic model, see Sec. 2.8, the adversary cannot set the random value every time it is used. $\mathcal{I}_{\mathsf{ADec}}$ is empty. In other words, we allow the adversary to choose any non-ϵ entry of the matrix defined in Eq. 6 provided that it complies with Sec. 2.8.1. $\mathcal{F}_E$ and $\mathcal{F}_D$ are defined accordingly. We defer the formal division into atoms to Sec. 5.2, which closely resembles the one given for CONCRETE.

Theorem 15. *Let* F *be a strongly protected* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-sTPRP*, let* E *be an ideal block cipher, let* A *be allowed* q_I *queries to* E*, let* H *be a hash function that is* $(t_1, \epsilon_{\mathsf{CR}})$*-collision resistant,* $(t_2, \alpha, \epsilon_{\mathsf{PR\text{-}coirv}})$ *pre-image resistant for a chosen offset of the image of a random value, and* $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$ *pre-image resistant with chosen target and random* n*-bit prefix input. Then, the* $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* CONCRETE2 $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encrypting messages at most* Ln *bits long, with leakage function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and admissible fault sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ *has* $(q_I, q_E, q_D, t, \epsilon)$ ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-then-leakage attacks in encryption and decryption $(\mathsf{CIL2F2} - (\mathsf{de}))$ *with*

$$\begin{aligned}\epsilon \leq & \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + q\epsilon_{\mathsf{PR\text{-}corpi}} + 2q_E\epsilon_{\mathsf{PR\text{-}coirv}} + O(q^3 + q^2 q_I)2^{-n} + O(q^4)2^{-2n},\\ & \text{and } \alpha = 2^{-n} + (L+3)\left[\frac{3}{2^n} + \frac{Q(Q-1)(Q-2)}{6(2^{2n})}\right].\end{aligned}$$

Note that the term $O(q^3 + q^2 q_I)2^{-n}$ arises from the probability of collisions between ephemeral keys (when faults do not set them). $O(q^4)2^{-2n}$ is due to the probability that an adversary, by injecting faults in an encryption query, obtains $c = (c_0, \ldots, c_{l+1}, c_{l+2})$, where $c_0, \ldots, c_{l+1}$ ddiffer from those obtained without faults, but coincide with those of another encryption query without faults (with a different k_0 and message).

Idea of the security argument: First, we observe that CONCRETE2 is CIL2F2-de. An adversary A cannot forge by using faults in decryption, for the same reason as in CONCRETE. Regarding differential faults in encryption, let (h, k_0, c_{l+2}) be the triple associated to F in this encryption query (i.e., the inputs and the output of the F atom). If $h = \mathsf{H}(c_0 \| \ldots \| c_{l+1}, a)$, to forge using this triple, A has to find another pre-image for h (but H is collision-resistant) or to use it in the forgery. Instead, if this is not the case, or if $(c_0, \ldots, c_{l+1})$ is not a valid encryption, A has to find a pre-image for h with a random n-bit prefix (c_0). But, $(c_0, \ldots, c_{l+1})$ can be a valid encryption[32] using the key k_0 for any message m only when no fault has been injected. Thus, the adversary cannot use $(c_0, \ldots, c_{l+1}, c_{l+2})$ to produce a forgery.

Now, we prove the claim: Let m be the message the adversary aims to encrypt, let $c^\dagger = c_0^\dagger, \ldots, c_{l+1}^\dagger$ denote the correct (that is, without any faults injected) encryption of m using the ephemeral key k_0, and let $k_1^\dagger, \ldots, k_{l+1}^\dagger$, $k_{0,E}^\dagger, \ldots, k_{l+1,E}^\dagger$, $k_{0,A}^\dagger, \ldots, k_{l+1,A}^\dagger$ be the correct ephemeral keys. All these values are random (since k_0 is random), and E is an ideal cipher, it is unlikely that an adversary correctly guesses any of them. Let us suppose $c_i \neq c_i^\dagger$. Since $k_{i,E}$ is random, the message block m_i' corresponding to c_i, is random $(m_i' = \mathsf{E}_{k_{i,E}}^{-1}(c_i))$, thus, the correct $k_{i,A}' = \mathsf{E}_{k_{i-1,A}}(m_i')$ and all subsequent $k_{j,A}'$ for $j > i$ are random and consequently c_{l+1}' too. Now, the probability that the adversary can correct this chain with a fault is negligible since the adversary would need to guess the right value (or the right offset).
So, we only need to consider what the adversary can do when she sets a value in encryption.

[32]This means that if in Enc, we encrypt m using k_0, we obtain exactly those values. This is the source of term $O(q^4)2^{-2n}$.

We classify encryption queries in which the adversary sets a value during the computation $\mathsf{AEnc}(a, m)$ (where the key k_0 is picked) as follows: I) *In the computation of* F*:*

1) *Changing* k_0 *to* k_0'*:* This implies that the adversary knows a valid triple for F_k: (h, k_0', c_{l+2}). She can use this information to forge if she can find $c_0', \ldots, c_l'$ that correctly encrypts m' starting from k_0', and a' s.t. $\mathsf{H}_s(c_0' \| \ldots \| c_{l+1}', a') = h$. We have two cases regarding the H atom in the encryption query:
   a) *h is the actual output of the* H *atom*: Again, we have two possibilities:
      i) *the inputs of the* H *atom are* $(c_0' \| \ldots \| c_{l+1}', a')$: The probability that this happens is negligible. Indeed, c_0 is random because k_0 is random and the adversary can only use differential faults, while E is an ideal cipher. Moreover, the probability that the adversary arrives at c_0' using differential faults is at most 2^{-n}.
      ii) *the inputs of the* H *atom are not* $(c_0' \| \ldots \| c_{l+1}', a')$: In this case, we find a collision for H because both the actual input of the hash during the encryption query and $(c_0' \| \ldots \| c_{l+1}', a')$ produce the same output.
   b) *the actual output of the hash function is* $h' \neq h$*:* Here, the previous argument does not apply, since a pre-image for h' may not have been computed. However, the ciphertext blocks $c_0, \ldots, c_{l+1}$ remain unpredictable because k_0 is random, E is ideal, and only differential faults are applied. The adversary only knows the pre-image for $h \oplus \Delta = h'$, where Δ is the offset added to h in F due to the fault Flt. Consequently, the adversary must break the $\mathsf{PR\text{-}coirv}$-security of H.[33]
2) *changing* h *to* h'*:* In this case, the adversary knows a valid triple for F_k: (h', k_0, c_{l+2}). She can attempt to forge if she can find $c_0', \ldots, c_l'$ that correctly encrypt m' starting from k_0', and a' such that $\mathsf{H}_s(c_0' \| \ldots \| c_{l+1}', a') = h'$. Now, k_0 is random (since it originates from a random value affected only by differential faults), so $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$, with $k_{0,E} = \mathsf{E}_{k_0}(p_A)$, is also random. Therefore, to forge, the adversary must break the $\mathsf{PR\text{-}corpi}$-resistance of H.

II)*Not in the computation of* F*:* It implies that the adversary knows a valid triple for F_k (h, k_0, c_{l+2}), with k_0 random (because it is randomly picked and then the adversary can only add differential faults). In particular, let $c_0' = \mathsf{E}_{\mathsf{E}_{k_0}(p_A)}(p_B)$, it is random due to the fact that k_0 is random. We have two possibilities for h:

1) *h is the actual output of the* H *atom:* Now we consider the H atom:
   a) *In the computation of* H *the* c_0 *input is not* c_0' *(obtained with faults).* Therefore, she needs to find a pre-image for h with prefix $c_0' = \mathsf{E}_{\mathsf{E}_{k_0}(p_A)}(p_B)$. This produces a collision, but H is collision-resistant.
   b) *In the computation of* H *the* c_0 *input is* c_0' *(obtained with faults).* Let $((c_0, \ldots, c_{l+1}), a)$ be the inputs of H atom[34]. We have two cases:
      i) $(c_0, \ldots, c_{l+1})$ *is a valid* [35] *encryption using the key* k_0 *for any message* m: As we have already discussed before, this cannot happen (even if she can set one value). Therefore, since the adversary has injected no fault in this encryption query, $c_0, \ldots, c_{l+1}$ is the actual encryption of m with k_0. Consequently, she has to find another valid encryption whose hash is equal h (thus, finding a collision for the hash function).
      ii) $(c_0, \ldots, c_{l+1})$ *is not a valid encryption using the key* k_0 *for any message* m: Therefore, the adversary has to find a valid encryption $c_0', \ldots, c_{l+1}'$ using the key k_0 for any message m. Moreover, we need that she finds a' s.t. $\mathsf{H}_s((c_0' \| \ldots \| c_{l+1}'), a') = h$, thus, we have found a collision for H.
2) *h is not the actual output of the* H *atom*: The treatment is equivalent to case I2).

[33]Note that if the adversary could set k_0 and additional values, she could invalidate the randomness assumption of h' by setting it directly or by controlling other inputs to H, as in the attack described in Sec. 5.3.

[34]Note that the adversary cannot fault a since here it is the only time it is used

[35]It means that if in Enc encryption we use k_0 and m, we obtain exactly those values.

Thus, to forge, an adversary has to break the PR−corpi-resistance of H.

The full details of the theorem with the complete time bounds, Thm. 16, and the complete (very long and complex) proof can be found in the following subsubsection.

Faults inside the atoms. As in CONCRETE, there should not be any problem if the adversary can inject faults in any atom in decryption, except for F_k^{-1}. On the other hand, for encryption queries, if we allow the adversary to inject a fault inside an atom, except for F_k, we have two challenges: First, we must ensure that no fixed value is set. This is prevented using only differential faults. Second, we must ensure that the outputs of E remain random. This is however a theoretical problem, but it is needed in the security proof: if the adversary can insert a differential fault in the computation of $\mathsf{E}_{k_0}(p_B)$, we do not know how to be sure that the real c_0 is random. In the PR−corpi security definition, the hash must be output before the random prefix is obtained; this requires performing the faulted computation of $\mathsf{E}_{k_0}(p_B)$. Thus, we need to be sure that $\mathsf{E}_{k_0}(p_B)$ is random given the faulted $\mathsf{E}_{k_0}(p_B)$, or an alternative assumption must be made (for instance, this could potentially be proved in the random oracle model). We conjecture that an adversary cannot exploit this case with an attack, but we leave this as an open discussion and future challenge. The hash computation should not be a problem, and we believe that, in practice, we need to protect a single TBC call only against faults and leakage.

Take-away for implementers. For decryption queries, the recommendations are the same as for Thm. 7. For encryption queries, the same recommendations as the wCIL2F2 of CONCRETE (Sec. 4.5), that is: implementers should focus on the random source, and in the protection against faults of each primitive, and to be sure that only differential faults can be inserted between the atoms (at most a single value can be set). Regarding leakage, only the single call to F should be protected. In particular, the adversary should not be able to extract the key of F via leakage.

5.5.1 CIL2F2-security of CONCRETE2

Here we provide the full details of Thm. 15 and its complete proofs. Readers not interested in the technical details can skip to Sec. 5.6.

First, we give the statement of Thm. 15 with the complete quantitative bounds:

Theorem 16. *Let* F *be a strongly protected* $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-sTPRP*, let* E *be an ideal block cipher, and let* A *be allowed* q_I *queries to* E*, let* H *be a hash function that is* $(t_1, \epsilon_{\mathsf{CR}})$*-collision resistant,* $(t_2, \alpha, \epsilon_{\mathsf{PR-coirv}})$ *pre-image resistant for a chosen offset of the image of a random value, and* $(t_3, \epsilon_{\mathsf{PR-corpi}})$ *pre-image resistant with chosen target and random* n*-bit prefix input. Then, the* $(\mathcal{I}_{\mathsf{AEnc}}, \mathcal{I}_{\mathsf{ADec}})$*-protected implementation of the* AE*-scheme* CONCRETE2 $\Pi = (\mathsf{Gen}, \mathsf{AEnc}, \mathsf{ADec})$*, encrypting messages at most* Ln *bits long, with leaking function pair* $\mathsf{L} = (\mathsf{L}_{\mathsf{AEnc}}, \mathsf{L}_{\mathsf{ADec}})$ *and fault admissible sets* $\mathcal{F} = (\mathcal{F}_E, \mathcal{F}_D)$ *has* $(q_I, q_E, q_D, t, \epsilon)$ ciphertext integrity in the presence of leakage and faults against $\mathcal{F}$-fault-

then-leakage attacks in encryption and decryption ($\mathsf{CIL2F2}-(\mathsf{de})$) *with*

$$\epsilon \leq \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + q\epsilon_{\mathsf{PR\text{-}corpi}} + 2q_E\epsilon_{\mathsf{PR\text{-}coirv}}$$
$$+ \{2q_E + q(q+1) + 1 + q_D + q_E\frac{q_E-1}{2}q_I(2+2q_D+5q_E)$$
$$+ (3L+4)[6q_Eq_D + 2q_D + 2q_D\frac{q_D+1}{2} + 2q_E + 5q_E\frac{q_E-1}{2}]\}2^{-n} + q_E(L+3)$$
$$\cdot\left[\frac{(q_I+4(L+6)[q_E+q_D])(q_I+4(L+6)[q_E+q_D]-1)(q_I+4(L+6)[q_E+q_D]-2)}{6(2^{2n})}\right],$$
$$\text{and } \alpha = 2^{-n} + (L+3)\left[\frac{3}{2^n} + \frac{Q(Q-1)(Q-2)}{6(2^{2n})}\right],$$

where $q = q_E + q_D + 1$, $t_1 = t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}]$, $t_2 = t + qt_{\mathsf{H}} + [(q-1)4(L+4)]t_{\mathsf{E}}$, $t_3 = t + qt_{\mathsf{H}} + [(q-1)(4L+6) + 2q_D]t_{\mathsf{E}}$, t_{E} *the time needed to compute once* E, t_{H} *to compute once* H *(we assume that picking a value uniformly at random does not take time), and* $Q = q_I + 4(L+6)[q_E+q_D]$.

In the proof, to simplify it, we use two lemmata: The first lemma proves that even if the adversary queries many times an ideal cipher E, if we pick randomly k, the adversary cannot predict $E_k(x)$, even if x is public. The second lemma proves that in an encryption query, if the adversary does not set k_0 in both the computation of $k_{0,A}$ and $k_{0,E}$, then, the adversary cannot predict the full $(c_0, \ldots, c_{l+1})$.

Lemma 1. *Let* $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ *be an ideal cipher, and consider a* q*-adversary* A*. Then, for any* $x \in \{0,1\}^n$*,*

$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x)] \leq q'2^{-n} + \mathsf{Coll}(q', q, n),$$

where given $q' \in \mathbb{N}$, $q' \geq 2$, $\mathsf{Coll}(q', q, n)$ *is the probability that among* q *values chosen uniformly at random in* $\{0,1\}^n$*, there exists a* q'*-collision is found (that is, there are* q' *random samples which result in the same value).*

It is possible to prove that

$$|\Pr[1 \leftarrow \mathsf{A}^{\mathsf{E}}(x,y) \mid y \leftarrow \mathsf{E}_k(x), k \xleftarrow{\$} \{0,1\}^n] - \Pr[1 \leftarrow \mathsf{A}^{\mathsf{E}}(x,y) \mid y \xleftarrow{\$} \{0,1\}^n]| \leq q2^{-n}.$$

This happens because the probability that A has never queried E using the key k picked to compute y is at least $(1-q2^{-n})$. Moreover, if A has never queried E using the key k, $\mathsf{E}_k(x)$ is indistinguishable from a random value. On the other hand, we do not need y to be fully random; it suffices that it is unpredictable. Therefore, we use a more complex proof which allows us to provide a better bound.

Proof. Let $\mathcal{K}$ be the set of keys $\mathsf{A}(x)$ has queried $\mathsf{E}.(x)$. By definition $|\mathcal{K}| \leq q$. Moreover, for all $y \in \{0,1\}^n$, we define $\mathcal{K}_y$ to be the set of keys k s.t. A has queried $\mathsf{E}_k(x)$, obtaining y. By definition, for any $q' \in \mathbb{N}$, $\Pr[\exists y \in \{0,1\}^n \mid |\mathcal{K}_y| \geq q'] = \mathsf{Coll}(q', q, n)$. Note that $\mathsf{Coll}(1, q, n) = 1$ if $q \geq 1$, thus we are only interested for $q' \geq 2$.

$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x)] =$$
$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \notin \mathcal{K}] \Pr[k \notin \mathcal{K} \mid k \xleftarrow{\$} \{0,1\}^n] +$$
$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \in \mathcal{K}] \Pr[k \in \mathcal{K} \mid k \xleftarrow{\$} \{0,1\}^n] \leq$$
$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \notin \mathcal{K}] \Pr[k \notin \mathcal{K} \mid k \xleftarrow{\$} \{0,1\}^n] + \frac{|\mathcal{K}_y|}{|\mathcal{K}|}\frac{|\mathcal{K}|}{2^n} \leq$$
$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \notin \mathcal{K}] \Pr[k \notin \mathcal{K} \mid k \xleftarrow{\$} \{0,1\}^n] +$$
$$(q'-1)2^{-n} + \mathsf{Coll}(q', q, n).$$

The second to last inequality is due to the fact that if $k \in \mathcal{K}$, the adversary can only win if $k \in \mathcal{K}_y$, and the probability that $k \xleftarrow{\$} \{0,1\}^n$ is in $\mathcal{K}$ is $|\mathcal{K}|2^{-n}$. Note that we bound $|\mathcal{K}_y|2^{-n}$ with $(q'-1)2^{-n} + \mathsf{Coll}(q', q, n)$ because with probability at most $\mathsf{Coll}(q', q, n)$ the q queries to $\mathsf{E}(\cdot, x)$ give a collision for any output value y, and, if such a q'-multi collision does not happen, $|\mathcal{K}_y| \leq q' - 1$. Unfortunately, this does not conclude the proof because it is not true that $\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \notin \mathcal{K}] = 2^{-n}$: because it may happen that A has queried $\mathsf{E}_k(x')$ on some input $x' \neq x$. Even for an ideal cipher, we then have that

$$\Pr[y = \mathsf{E}_k(x) \mid k \xleftarrow{\$} \{0,1\}^n,\ \mathsf{E}_k(x') \neq y] = (2^n - 1)^{-1} \neq 2^{-n}.$$

We believe that it may be possible to prove that for an adversary, it is not the optimal strategy to query $\mathsf{E}.(\cdot)$ on input different from (k', x) for any k', but this seems very complex. Thus, we want to simplify the proof giving an additional capability to the adversary: for any key k she queries the ideal cipher E with, we will additionally provide her also $\mathsf{E}_k(x)$ as a gift, before answering her query[36]. Now, when we condition on the fact that $k \xleftarrow{\$} \{0,1\}^n$ and $k \notin \mathcal{K}$,

$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \notin \mathcal{K}] = 2^{-n},$$

which allows us to conclude the proof:

$$\begin{aligned}
&\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) \mid k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x), k \notin \mathcal{K}] \Pr[k \notin \mathcal{K} \mid k \xleftarrow{\$} \{0,1\}^n] + \\
&(q'-1)2^{-n} + \mathsf{Coll}(q', q, n) \leq \\
&2^{-n}\frac{2^n - |\mathcal{K}|}{2^n} + \frac{q'-1}{2^n} + \mathsf{Coll}(q', q, n) \leq \frac{1 + q' - 1}{2^n} + \mathsf{Coll}(q', q, n) = q'2^{-n} + \mathsf{Coll}(q', q, n).
\end{aligned}$$

Clearly, the previous bound is interesting only for $q' \geq 2$ (for $q' = 1$, there are always 1-collisions: it is enough to query E_k on input (x) to have a key that on input x outputs $y = \mathsf{E}_k(x)$). □

To have a better bound, we want to use $q' > 2$. We use two results from Suzuki et al. [58]:

$$\mathsf{Coll}(q', q, n) = \frac{1}{2^{n(q'-1)}} \sum_{i=q'}^{q} \binom{i-1}{q'-1} \left(1 - \frac{1}{2^n}\right)^{i-q'} (1 - \mathsf{Coll}(q', i - q', 2^n - 1)), \quad (7)$$

$$\text{and their bound: } \mathsf{Coll}(q', q, n) = \frac{1}{2^{n(q'-1)}} \binom{q}{q'}. \quad (8)$$

Thus, we have the following corollary:

Corollary 1. *Let* $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ *be an ideal cipher, and consider a* q*-adversary* A*. Then, given a public* $x \in \{0,1\}^n$*,*

$$\Pr[y \leftarrow \mathsf{A}^{\mathsf{E}}(x) | k \xleftarrow{\$} \{0,1\}^n, y = \mathsf{E}_k(x)] \leq \frac{3}{2^n} + \frac{q(q-1)(q-2)}{6(2^{2n})}.$$

[36] With the argument that we are going to develop, we can prove our bounds for any adversary who does q queries to $\mathsf{E}.(x)$ and as many queries as she wants to $\mathsf{E}.(\cdot)$, as long as the key used in the latter queries has already been used for $\mathsf{E}.(x)$ queries. Unfortunately, this is not a proof, that the optimal strategy for a q adversary is to pick q different keys $k_1, \ldots, k_q$ and queries $\mathsf{E}_{k_i}(x)$, because we have to prove that for an adversary it is better to increase the probability that a collision happens, than to increase the probability of guessing when $k \notin \mathcal{K}$. On the other hand, our argument is a very strong hint that only asking queries of type $\mathsf{E}.(x)$ is the optimal adversarial strategy.

Proof. The corollary follows by applying Lemma 1, with $q' = 3$, and using Eq. 8 to bound $\mathsf{Coll}(3, q, n)$ where $\binom{q}{3} = \frac{q(q-1)(q-2)}{3!}$. □

Note that the previous result is beyond birthday secure, being secure up to $q = O(2^{2n/3})$. Using larger values of q' could potentially yield tighter bounds, but $q' = 3$ suffices for our purposes.

Lemma 2. *Let* E *be an ideal cipher, and let* A *be a* (q_I, q_E, q_D, t)*-adversary. Let* (a, m, Flt) *be an encryption query to* CONCRETE2*. Let us suppose that* A *does not set* k_0 *in both the computation of* $k_{0,E}$ *and* $k_{0,A}$ *and she sets at most a single value (and use any differential fault she wants). Then, for any possible adversarial choice of* a, m *and fault* Flt

$$\Pr[(c_0, \ldots, c_{l+1}) \leftarrow \mathsf{A}^{\mathsf{E}} \mid (c_0, \ldots, c_{l+1}) \leftarrow \mathsf{CONCRETE2}'_{k_0}(a, m, \mathsf{Flt})\ k_0 \xleftarrow{\$} \{0,1\}^n,]$$
$$\leq 2^{-n} + (L+3)\left[\frac{3}{2^n} + \frac{Q(Q-1)(Q-2)}{6(2^{2n})}\right],$$

where $\mathsf{CONCRETE2}'_{k_0}$ *is* CONCRETE2 *(Alg. 7) with the exception of the random picking of* k_0*, and the computation of* c_{l+2}*;* $Q = q_I + 4(L+6)[q_E + q_D]$*, and* L *is the maximal number of blocks a message can have.*

Proof. To simplify the proof let $\alpha = \frac{3}{2^n} + \frac{Q(Q-1)(Q-2)}{6(2^{2n})}$. This is the bound obtained in the previous Corollary 1.

It is enough to prove that $\forall(a, c, \mathsf{Flt})$ with $c = (c_0, \ldots, c_{l+1})$ there exists i s.t. c_i is unpredictable, that is, for any $x \in \{0,1\}^n$,

$$\Pr[c_i = x \mid (c_0, \ldots, c_i, \ldots, c_{l+1}) \leftarrow \mathsf{AEnc}_k(a, m, \mathsf{Flt}, k_0),\ k_0 \xleftarrow{\$} \{0,1\}^n] \leq \beta,$$

where $\beta = 2^{-n} + (L+3)\alpha$, and with $(c_0, \ldots, c_i, \ldots, c_{k+1} \leftarrow \mathsf{AEnc}_k(a, m, \mathsf{Flt}, k_0)$ we denote that we compute $\mathsf{AEnc}_k(a, m)$ picking a random k_0 and we inject the faults specified by Flt.

We start by assuming that there is no *set* fault injected in the computation of $k_{0,E} = \mathsf{E}_{k_0}(p_A)$. Thus, k'_0, the key used in the previous computation is uniformly random because $k'_0 = k_0 \oplus \Delta_{0,1}$ with $\Delta_{0.1}$ a differential fault chosen by the adversary before k_0 is picked uniformly at random. Thus, since E is an ideal cipher and k'_0 is uniformly random, the adversary cannot predict the output of the computation $k_{0,E} = \mathsf{E}_{k'_0}(p_A)$ with probability bigger than α (because the adversary had at most $Q = q_I + 4(L+6)[q_E + q_D]$ queries), due to Corollary 1.

Now let us assume that no set fault is injected in the computation $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$, thus, in the computation we use $k'_{0,E} = k_{0,E} \oplus \Delta_{0.3}$ a differential fault chosen by the adversary before k_0 is picked uniformly at random. Thus, to predict c_0, the adversary can either predict $k_{0,E}$, and compute c_0 accordingly or directly c_0: in the first case the adversary can win with probability α, in the second case, she can win with probability α due to the argument used in the proof of Lemma 1 (that is, if the adversary has not queried $\mathsf{E}.(\cdot)$ with a certain key, she has no information about it except from the bound of Lemma 1). Thus, to win this Game, it is not enough to guess the key, but it is necessary to do a call on $\mathsf{E}.(\cdot)$ with that key. Thus, the probability that the adversary guesses correctly c_0 in this case is bounded by 2α (the adversary has two ways: either guessing $k_{0,E}$ or directly c_0).

Now, we have to consider the missing cases. Suppose that the adversary set k_0 to k'_0 in the computation of $k_{0,E} = \mathsf{E}_{k_0}(p_A)$ (or she sets $k_{0,E}$ in the computation of $c_0 = \mathsf{E}_{k_{0,E}}(p_B)$), thus in the computation of $k_{0,A} = \mathsf{E}_{k_0}(p_B)$ we use $k''_0 = k_0 \oplus \Delta_{0.2}$ with $\Delta_{0.2}$ a differential fault chosen by the adversary before k_0 is picked uniformly at random (the adversary has already used her only set fault). Again, the adversary can either try to guess k''_0 or $k_{0,A}$. The first can happen with probability 2^{-n} since k_0 is picked uniformly at random, and no set fault is used. The second can happen with probability at most α due to Lemma 1.

Then, we observe that for all $i = 1, \ldots, l$, in the computation of $k_{i,A} = \mathsf{E}_{k_{i-1,A}}(m_i)$ we use $k'_{i-1,A} = k_{i,A} \oplus \Delta_{i.4,0}$ and $m'_i = m_i \oplus \Delta_{i.4,1}$ with both differential faults chosen before k_0 is picked. By induction, assume that $k_{i-1,A}$ has not been guessed. Then, the probability of correctly guessing $k'_{i-1,A}$ is bounded by α (Lemma 1). We point out that we have already proved that $k_{0,A}$ has not been guessed.

Iterating we arrive to prove that the adversary cannot guess $k_{l+1,A}$. Now, we consider the computation of $c_{l+1} = \mathsf{E}_{k_{l+1,E}}(k_{l,A})$. Again, since the adversary has already used her set fault, if the adversary has not guessed $k_{l,A}$, thus she cannot guess $k'_{l,A} = k_{l,A} \oplus \Delta_{l+1.2,1}$, thus the probability that she guesses c_{l+1} is bounded by α due to Lemma 1.

Combining the above, the probability that an adversary guesses correctly $(c_0, \ldots, c_{l+1})$ is at most $2\alpha + 2^{-n} + (L+1)\alpha \leq 2^{-n} + (L+3)\alpha$.

□

Now, we can finally tackle the proof:

Proof. As usual, we use a sequence of games. Let E_i be the event that the adversary wins Game i, that is, she outputs a fresh and valid forgery.

Game 0. The standard $\mathsf{CIL2F2}$ Game against $\mathsf{CONCRETE2}$.

Game 1. This is Game 0, where we replace F with a tweakable random permutation f.

Transition between Game 0 and Game 1. Since Game 0 and Game 1 are the same except for replacing of F with f, we can build an adversary B to bound the difference.

She plays the sTPRP Game, having to distinguish F_k from f, and simulates A's oracles using its own. At the start of the Game, B picks s from $\mathcal{HK}$ and gives it to A. A has access to an ideal block cipher E, and gives access to it to B. Moreover, she has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a, m, Flt), B simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows [37] : $1i)$ $k_{0,E} = \mathsf{E}_{k'_0}(p_A)$, where k'_0 is how k_0 is modified according to the fault Flt for this computation [38], $1ii)$ $k_{0,A} = \mathsf{E}_{k''_0}(p_B)$, where k''_0 is how k_0 is modified according to the fault Flt for this computation, $1iii)$ $c_0 = \mathsf{E}_{k'_{0,E}}(p_B)$, where $k'_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation, and $1iv)$ $k_1 = \mathsf{E}_{k''_{0,E}}(p_A)$, where $k''_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation. Then, for all $i = 1, \ldots, l$, $2i)$ B computes $k_{i,E} = \mathsf{E}_{k'_i}(p_B)$, where k'_i is how k_i is modified according to the fault Flt for this computation, $2ii)$ $c_i = \mathsf{E}_{k''_{i,E}}(m'_i)$, where $k''_{i,E}$ and m'_i are how $k_{i,E}$ and m_i are modified according to the fault Flt for this computation, $2iii)$ $k_{i+1} = \mathsf{E}_{k''_i}(p_A)$, where k''_i is how k_i is modified according to the fault Flt for this computation, and $2iv)$ $k_{i,A} = \mathsf{E}_{k'_{i-1,A}}(m''_i)$, where $k'_{i-1,A}$ and m''_i are how $k_{i-1,A}$ and m_i are modified according to the fault Flt for this computation. After that, $3i)$ B computes $k_{l+1,E} = \mathsf{E}_{k'_{l+1}}(p_B)$, where k'_{l+1} is how k_{l+1} is modified according to the fault Flt for this computation, and $3ii)$ $c_{l+1} = \mathsf{E}_{k'_{l+1,E}}(k'_{l,A})$, where $k'_{l+1,E}$ and $k'_{l,A}$ are how $k_{l+1,E}$ and $k_{l,A}$ are modified according to the fault Flt for this computation. Finally, $4i)$ B computes $h = \mathsf{H}_s(c'_0 \| \ldots \| c'_{l+1}, a)$, where c'_i is how c_i is modified according to fault Flt, and $4ii)$ she queries her oracle on input (h', k'''_0), where h' and k'''_0 are how h and k_0 are modified according to Flt in this computation, receiving as answer c_{l+2}. So, she can answer $c = (c_0 \| \ldots \| c_{l+2})$ and the leakage k_0 to A. Moreover, B adds (a, c) to the list $\mathcal{S}$.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

[37] Note that each of the steps, into which we divide this computation, corresponds to an atom (see Sec. 2.8).

[38] That is, she calls the ideal cipher E on input (k'_0, p_A), we use *compute* to simplify notation. Moreover, in this transition it is irrelevant the fact that E is an ideal block cipher.

When A does a decryption query on input (a, c, Flt), B parses $c = (c_0, \ldots, c_{l+2})$, and she proceeds as follows: $1i)$ B computes $h = \mathsf{H}_s(c'_0 \| \ldots \| c'_{l+1}, a)$, where c'_i is how c_i is modified according to fault Flt, and $1ii)$ she queries her inverse oracle on input (h', c_{l+2}), where h' is how h is modified according to Flt in this computation [39], receiving as answer k_0. Then, $2i)$ $k_{0,E} = \mathsf{E}_{k'_0}(p_A)$, where k'_0 is how k_0 is modified according to the fault Flt for this computation, $2ii)$ $k_{0,A} = \mathsf{E}_{k''_0}(p_B)$, where k''_0 is how k_0 is modified according to the fault Flt for this computation, $2iii)$ $\tilde{c}_0 = \mathsf{E}_{k'_{0,E}}(p_B)$, where $k'_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation, and she checks if $c_0 \stackrel{?}{=} \tilde{c}_0$, if this is not the case she answers $\perp$ to A and the leakage k_0; otherwise $2iv)$ she computes $k_1 = \mathsf{E}_{k''_{0,E}}(p_A)$, where $k''_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation. After that B computes for all $i = 1, \ldots, l$, $3i)$ B computes $k_{i,E} = \mathsf{E}_{k'_i}(p_B)$, where k'_i is how k_i is modified according to the fault Flt for this computation, $3ii)$ $m_i = \mathsf{E}^{-1}_{k''_{i,E}}(c''_i)$, where $k''_{i,E}$ and c''_i are how $k_{i,E}$ and c_i are modified according to the fault Flt for this computation, $3iii)$ $k_{i+1} = \mathsf{E}_{k''_i}(p_A)$, where k''_i is how k_i is modified according to the fault Flt for this computation, and $3iv)$ $k_{i,A} = \mathsf{E}_{k'_{i-1,A}}(m'_i)$, where $k'_{i-1,A}$ and m'_i are how $k_{i-1,A}$ and m_i are modified according to the fault Flt for this computation. Finally, $4i)$ B computes $k_{l+1,E} = \mathsf{E}_{k'_{l+1}}(p_B)$, where k'_{l+1} is how k_{l+1} is modified according to the fault Flt for this computation, and $4ii)$ $\tilde{c}_{l+1} = \mathsf{E}_{k'_{l+1,E}}(k'_{l,A})$, where $k'_{l+1,E}$ and $k'_{l,A}$ are how $k_{l+1,E}$ and $k_{l,A}$ are modified according to the fault Flt for this computation. $4iii)$ if $\tilde{c}_{l+1}$ is equal to c_{l+1}, B answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A; otherwise $\perp$ and the leakage k_0.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), B proceeds as for a normal decryption query with the following differences: 1) no faults are involved, 2) in step $4iii)$ if $\tilde{c}_{l+1}$ is equal to c_{l+1} and $(a^*, c^*) \notin \mathcal{S}$, B outputs 1, otherwise 0. For this, B does one oracle query and needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

Thus, in total B makes at most $q_E + q_D + 1 = q$ queries to her oracle and runs in time bounded by $t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] = t_1$.

If her oracle is implemented with F, B correctly simulates Game 0 for A, otherwise, she simulates Game 1. Since F is a $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$,

$$|\Pr[E_1] - \Pr[E_0]| \leq \epsilon_{\mathsf{sTPRP}}.$$

Game 2. This is Game 1, where instead of using f and f^{-1}, we use two tweakable random functions, say f and g. During this game, we answer consistently with the previous queries, that is, if we have obtained $c_{l+2} = \mathsf{f}(h, k_0)$ we define $\mathsf{g}(h, c_{l+2}) := k_0$ [and vice-versa].

Transition between Game 1 and Game 2. In Game 2, for decryption queries, we are using a sTPRP, while in Game 3 we use a PRF. By the well-known birthday bound, we have that

$$|\Pr[E_1] - \Pr[E_2]| \leq \frac{2q(q+1)}{2^{n+1}},$$

because in addition to switching two PRPs with two PRFs (but since these functions are correlated, this counts as a single switch), we have to consider the probability that we cannot do the simulation correctly, that is, when there exists a tweak h and two inputs $x,\ x'$ s.t. $\mathsf{f}(h, x) = \mathsf{f}(h, x')$ (in this case, we cannot compute the inverse correctly).

New sampling of f and g. From now on, when we say that an adversary *computes* f and g we mean that we maintain a list $\mathcal{L}$, and every time we have to compute $\mathsf{f}(x, y)$ (resp. $g(x, z)$) we look into $\mathcal{L}$ to see if there is a triple (x, y, z). If it is the case, we output z

[39] We use c_{l+2} since this is the only time, the adversary cannot modify it via a fault.

(resp. y); otherwise, we pick $z \xleftarrow{\$} \{0,1\}^n$ (resp. $y \xleftarrow{\$} \{0,1\}^n$) and we output z (resp. y) and we add (x, y, z) to $\mathcal{L}$.

We have already covered the case where the simulation fails.

Game 3. This is Game 2 where we assume that during the execution of the game, there are no collisions in the hash function.

Transition between Game 2 and Game 3. Since Game 2 and Game 3 are the same except when the following event Bad_1 happens: there is a collision for the hash function.

To bound $\Pr[\mathsf{Bad}_1]$ we build adversary C.

She plays the CR Game against the hash function H, using the adversary A.

At the start of the game, C receives a key s in $\mathcal{HK}$ and gives it to A, and an ideal block cipher E. Moreover, she has two lists $\mathcal{S}$ and $\mathcal{H}$ which are empty.

When A does an encryption query on input (a, m, Flt), C simply picks k_0 uniformly at random from $\{0,1\}^n$ and after having parsed m in $(m_1, \ldots, m_l)$, computes $c_0, \ldots, c_{l+1}$ as follows: $1i)$ $k_{0,E} = \mathsf{E}_{k'_0}(p_A)$, where k'_0 is how k_0 is modified according to the fault Flt for this computation [40], $1ii)$ $k_{0,A} = \mathsf{E}_{k''_0}(p_B)$, where k''_0 is how k_0 is modified according to the fault Flt for this computation, $1iii)$ $c_0 = \mathsf{E}_{k'_{0,E}}(p_B)$, where $k'_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation, and $1iv)$ $k_1 = \mathsf{E}_{k''_{0,E}}(p_A)$, where $k''_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation. Then, for all $i = 1, \ldots, l$, $2i)$ C computes $k_{i,E} = \mathsf{E}_{k'_i}(p_B)$, where k'_i is how k_i is modified according to the fault Flt for this computation, $2ii)$ $c_i = \mathsf{E}_{k''_{i,E}}(m'_i)$, where $k''_{i,E}$ and m'_i are how $k_{i,E}$ and m_i are modified according to the fault Flt for this computation, $2iii)$ $k_{i+1} = \mathsf{E}_{k''_i}(p_A)$, where k''_i is how $k_{i,E}$ and m_i are modified according to the fault Flt for this computation, and $2iv)$ $k_{i,A} = \mathsf{E}_{k'_{i-1,A}}(m''_i)$, where $k'_{i-1,A}$ and m''_i are how $k_{i-1,A}$ and m_i are modified according to the fault Flt for this computation. After that, $3i)$ C computes $k_{l+1,E} = \mathsf{E}_{k'_{l+1}}(p_B)$, where k'_{l+1} is how k_{l+1} is modified according to the fault Flt for this computation, and $3ii)$ $c_{l+1} = \mathsf{E}_{k'_{l+1,E}}(k'_{l,A})$, where $k'_{l+1,E}$ and $k'_{l,A}$ are how $k_{l+1,E}$ and $k_{l,A}$ are modified according to the fault Flt for this computation. Finally, $4i)$ C computes $h = \mathsf{H}_s(c'_0 \| \ldots \| c'_{l+1}, a)$, where c'_i is how c_i is modified according to fault Flt, and she adds $((c'_0 \| \ldots \| c'_{l+1}, a), h)$ to $\mathcal{H}$, and $4ii)$ she computes $c_{l+2} = \mathsf{f}(h', k'''_0)$, where h' and k'''_0 are how h and k_0 are modified according to Flt in this computation, keeping in memory (h', k'''_0, c_{l+2}) for f. So, she can answer A with $c = (c_0 \| \ldots \| c_{l+2})$ and the leakage k_0. Moreover, C adds (a, c) to the list $\mathcal{S}$.

For this, C needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c, Flt), C parses $c = (c_0, \ldots, c_{l+2})$, and she proceeds as follows: $1i)$ C computes $h = \mathsf{H}_s(a, c'_0 \| \ldots \| c'_{l+1})$, where c'_i is how c_i is modified according to fault Flt, and she adds $((c'_0 \| \ldots \| c'_{l+1}, a), h)$ to $\mathcal{H}$, and $1ii)$ she computes $k_0 = \mathsf{g}(h', c_{l+2})$, where h' is how h is modified according to Flt in this computation, keeping in memory (h', k_0, c_{l+2}). Then, $2i)$ $k_{0,E} = \mathsf{E}_{k'_0}(p_A)$, where k'_0 is how k_0 is modified according to the fault Flt for this computation, $2ii)$ $k_{0,A} = \mathsf{E}_{k''_0}(p_B)$, where k''_0 is how k_0 is modified according to the fault Flt for this computation, $2iii)$ $\tilde{c}_0 = \mathsf{E}_{k'_{0,E}}(p_B)$, where $k'_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation, and she checks if $c_0 \stackrel{?}{=} \tilde{c}_0$, if this is not the case she answers $\perp$ to A and the leakage k_0; otherwise $2iv)$ she computes $k_1 = \mathsf{E}_{k''_{0,E}}(p_A)$, where $k''_{0,E}$ is how $k_{0,E}$ is modified according to the fault Flt for this computation. After that C computes for all $i = 1, \ldots, l$, $3i)$ C computes $k_{i,E} = \mathsf{E}_{k'_i}(p_B)$, where k'_i is how k_i is modified according to the fault Flt for this computation, $3ii)$ $m_i = \mathsf{E}^{-1}_{k''_{i,E}}(c''_i)$, where $k''_{i,E}$ and c''_i are how $k_{i,E}$ and c_i are modified according to the fault Flt for this computation, $3iii)$ $k_{i+1} = \mathsf{E}_{k''_i}(p_A)$, where k''_i is how k_i is modified according to the fault Flt for this computation, and $3iv)$ $k_{i,A} = \mathsf{E}_{k'_{i-1,A}}(m'_i)$,

[40] That is, she calls the ideal cipher E on input (k'_0, p_A), we use *compute* to simplify notation. Moreover, in this transition it is irrelevant the fact that E is an ideal block cipher.

where $k'_{i-1,A}$ and m'_i are how $k_{i-1,A}$ and m_i are modified according to the fault Flt for this computation. Finally, $4i$) C computes $k_{l+1,E} = \mathsf{E}_{k'_{l+1}}(p_B)$, where k'_{l+1} is how k_{l+1} is modified according to the fault Flt for this computation, and $4ii$) $\tilde{c}_{l+1} = \mathsf{E}_{k'_{l+1,E}}(k'_{l,A})$, where $k'_{l+1,E}$ and $k'_{l,A}$ are how $k_{l+1,E}$ and $k_{l,A}$ are modified according to the fault Flt for this computation. $4iii$) if $\tilde{c}_{l+1}$ is equal to c_{l+1}, C answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A; otherwise $\perp$ and the leakage k_0.

For this, C needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), C proceeds as for a normal decryption query with the following differences: 1) there are no faults involved, 2) C does not answer A anything 3) after having computed this decryption query, C looks into her list $\mathcal{H}$ to check whether she finds a collision: if it is the case, she outputs it, otherwise $(0^n, 1^n)$.

For this, C needs time $t_{\mathsf{H}} + (4(l+1)+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}$.

Thus, in total C runs in time bounded by $t + q[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] = t_1$.

C correctly simulates Game 2 for A. Since Game 2 and Game 3 are the same, except if event Bad_1 happens, and C wins her CR-Game if event Bad_1 occurs, and C is a t_1-adversary and H is a $(t_1, \epsilon_{\mathsf{CR}})$-collision-resistant hash function,

$$|\Pr[E_2] - \Pr[E_3]| = \Pr[\mathsf{Bad}_1] \leq \epsilon_{\mathsf{CR}}.$$

Game 4. This is Game 3, where in every decryption query (not the challenge one), immediately after computing k_0 (via g), the decryption oracle computes $k_{l+3,E} = \mathsf{E}_{k_0}(p_A)$ and $c_{l+3} = \mathsf{E}_{k_{l+3,E}}(p_B)$ and it is appended to the leakage.

Transition between Game 3 and Game 4. These games are the same except for these two additional calls per decryption query that any adversary could perform. Thus, it is easy to build an adversary A' which when she receives the answer of a decryption query computes $k_{l+3,E} = \mathsf{E}_{k_0}(p_A)$ and $c_{l+3} = \mathsf{E}_{k_{0,E}}(p_B)$, appends them to the leakage and forwards them to the adversary.

Such an adversary needs $2q_D$ calls to the ideal block cipher. Thus,

$$\Pr[E_3] = \Pr[E_4].$$

Game 5. This is Game 4, with the condition that the following event does not happen: during a decryption query, when a new key k_0 is picked by g [41], there exists no query to the ideal cipher on input (k_0, p_A).

Transition between Game 4 and Game 5. The two games are the same except if the following Bad_2 event happens: there exists a decryption query s.t. it requires to pick a new key k_0^i and there exists a previous query to the ideal cipher E on input (k_0^i, p_A). We observe that during an encryption query there are at most $L+2$ queries to $\mathsf{E}(\cdot, p_A)$ with at most $L+2$ different keys, and $2L+1$ queries with $(\cdot, \cdot)$ where the key is either $k_{j,E}$ or $k_{j,A}$ for a certain j and the input is a message m_j or $k_{l,A}$ (which can be equal to p_A [42]), while during a decryption query there are at most $L+2$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_A)$ and $2L+1$ to $\mathsf{E}(\cdot, \cdot)$. In addition, due to the change we have introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_A)$ The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, during the j^{th} decryption query there are at most $q_E(3L+4) + q_I + (j-1)(3L+4)$ different

[41] that is, there is no entry in the list used for f and g containing (h', k_0, c_{l+2}), with h' hash h obtained in step $1i$) and modified according to what fault Flt specifies for the step $1ii$).

[42] If instead of using a block cipher E, we use a tweakable blockcipher, we can avoid this situation and significantly reduce the number of possible calls to E, see Alg. 8.

keys that, if picked, would trigger the Bad_2 event. Thus,

$$\begin{aligned}\Pr[\mathsf{Bad}_2] \leq & \sum_{j=1}^{q_D+1} \frac{q_E(3L+4)+q_I+(j-1)(3L+4)}{2^n} \\ \leq & \frac{q_D[q_E(3L+4)+q_I]+q_D(\frac{q_D+1}{2})(3L+4)}{2^n} \\ = & q_D[(3L+4)(q_E+\frac{q_D+1}{2})+q_I]2^{-n}.\end{aligned}$$

$$\text{Thus, } |\Pr[E_4]-\Pr[E_5]| \leq \Pr[\mathsf{Bad}_2] = q_D[(3L+4)(q_E+\frac{q_D+1}{2})+q_I]2^{-n}.$$

Game 6. This is Game 5, with the condition that the following event does not happen: during a decryption query, when a new key k_0 is picked by g, and the associated $k_{0,E} = \mathsf{E}_{k_0}(p_A)$ is computed, there has been no query to the ideal cipher on input $(k_{0,E}, p_B)$.

Transition between Game 5 and Game 6. The two games are the same except if the following Bad_3 event happens: there exists a decryption query s.t. it requires to pick a new key k_0^i, and s.t., given $k_{0,E}^i = \mathsf{E}_{k_0^i}(p_A)$ its associated key (which is random, since event Bad_2 did not occur), there exists a previous query to the ideal cipher E on input $(k_{0,E}^i, p_B)$. As before, we have to consider $3L+3$ possible problematic ideal cipher queries per encryption query, and $3L+5$ per previous decryption query (considering the additional query due to the change introduced in Game 4) [43], and the q_I adversarial queries to the ideal block cipher. Thus, during the jth decryption query there are at most $q_E(3L+4)+q_I+(j-1)[3(L+1)+2]$ different keys that, if picked, would trigger the Bad_3 event. Thus,

$$\begin{aligned}\Pr[\mathsf{Bad}_3] \leq & \sum_{j=1}^{q_D+1} \frac{q_E(3L+4)+q_I+(j-1)[(3L+4)+1]}{2^n} \\ \leq & \frac{q_D[q_E(3L+4)+q_I]+q_D(\frac{q_D+1}{2})[(3L+4)+1]}{2^n} \\ = & q_D[(3L+4)(q_E+\frac{q_D+1}{2})+q_I+1]2^{-n}.\end{aligned}$$

$$\text{Thus, } |\Pr[E_5]-\Pr[E_6]| \leq \Pr[\mathsf{Bad}_3] = q_D[(3L+4)(q_E+\frac{q_D+1}{2})+q_I+1]2^{-n}.$$

Game 7. This is Game 6, where the following event Bad_4 does not happen: for any tweak h used by g only in decryption queries, there exist two decryption queries, which we call the ith and the jth s.t. $h = \mathsf{H}_s(c_0^j \| \ldots \| c_{l+1}^j, a^j)$ (where $c_0^j, \ldots, c_l^j$ and a^j are modified according to the fault Flt^j for the $1i$) step), s.t. the modified c_0^j is equal to c_{l+3}^i obtained in the ith decryption query, in which $\mathsf{f}^{-1}(h^i, \tau^i)$ is computed (modified according to the Flt^i for step $(1ii)$) and the h^i modified which is equal to h.

Transition between Game 6 and Game 7. Since Game 6 and Game 7 are the same except if event Bad_4 happens, it is enough to bound $\Pr[\mathsf{Bad}_4]$ to bound the difference $|\Pr[E_6]-\Pr[E_7]|$. To bound $\Pr[\mathsf{Bad}_4]$, we divide it in q_D+1 different events: $\mathsf{Bad}_{4.1}, \ldots, \mathsf{Bad}_{4.q_D+1}$, where $\mathsf{Bad}_{4.\iota}$ is event Bad_4 where $\iota = i$. Clearly, $\mathsf{Bad}_4 = \bigcup_{\iota=1}^{q_D+1} \mathsf{Bad}_{4.\iota}$.

To bound $\Pr[\mathsf{Bad}_{4.\iota}]$ we build an adversary D^ι against the n–PR–corpi-security of the hash function H.

[43] Since here we consider queries $\mathsf{E}(\cdot, p_B)$, while in the previous transition we consider queries $\mathsf{E}(\cdot, p_A)$, it is enough to observe that for every query $\mathsf{E}(k_j, p_A)$ (or $\mathsf{E}(k_{0,E}, p_A)$) there exists one query $\mathsf{E}(k_j, p_B)$ (or $\mathsf{E}(k_{0,E}, p_B)$). In addition, there is a final query $\mathsf{E}(k_{l+1}, p_B)$ which does not correspond to any $\mathsf{E}(k_{l+1}, P_A)$.

The n–PR–corpi-adversary D^ι. At the start of the Game, D^ι receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks two random tweakable functions f and g as defined earlier. Finally, she has a list $\mathcal{H}$ which is empty.

When A does an encryption query on input (a, m), D^ι (we use D^ι to identify both D_1^ι and D_2^ι) behaves as C. For this, D^ι needs time $t_{\mathsf{H}} + [4(l+1)+2]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+1)+2]t_{\mathsf{E}}$.

When A does the ith decryption query, $i < \iota$ on input (a, c, Flt), D_1^ι proceeds as C with the following differences: after step $(1ii)$, and before to step $(2i)$, she $1iii)$ computes $k_{l+3,E} = \mathsf{E}_{k_0}(p_A)$ and $1iv)$ $c_{l+3} = \mathsf{E}_{k_{l+3,E}}(p_B)$.

For this, D_1^ι needs time $t_{\mathsf{H}} + 4(l+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + 4(L+2)t_{\mathsf{E}}$.

When A does the ι^{th} decryption query on input (a, c, Flt), D_1^ι proceeds as follows: she performs $1i)$, then she checks whether $\mathcal{L}$ already contains a triple of the form $(\cdot, h^\iota, c_{l+2}^\iota)$. If so, she aborts; otherwise, she outputs h^ι as her challenge, and the state $\mathsf{st} = (h^\iota, c_{l+2}^\iota, \mathcal{H}, s, \mathsf{f}, \mathsf{g})$. D_2^ι receives c_{l+3} which is a random value. D_2^ι parses st in $(h^\iota, c_{l+2}^\iota, \mathcal{H}, s, \mathsf{f}, \mathsf{g})$, she picks $k_{0,A}$ and k_1. From then, she proceeds as C from step $(2i)$. For this D_1^ι needs time t_{H}, while D_2^ι needs time $(4l+2)t_{\mathsf{E}} \leq (4L+2)t_{\mathsf{E}}$.

When A does the ith decryption query, $i > \iota$ on input (a, c, Flt), D_2^ι proceeds as D_1^ι for the ith decryption query $(i > \iota)$, thus she needs time $t_{\mathsf{H}} + 4(l+2)t_{\mathsf{E}} \leq t_{\mathsf{H}} + 4(L+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), D_2^ι proceeds as follows: she computes $h^* = \mathsf{H}_s(c_0^* \| \dots \| c_{l+1}^*, a^*)$ and she adds $((c_0^* \| \dots \| c_{l+1}^*, a^*), h^*)$ to $\mathcal{H}$, and then, she looks through $\mathcal{H}$ to see if there is an entry $(c_0 \| \dots \| c_{l+1}, a), h)$ s.t $h = h^\iota$ and $c_0 = c_{l+3}^\iota$. If it is the case, she outputs it, otherwise 0^n. This takes time t_{H}.

Thus, in total D runs in time bounded by

$$\begin{aligned} &t + q_E[t_{\mathsf{H}} + (4(L+1)+2)t_{\mathsf{E}}] + (\iota - 1)[t_{\mathsf{H}} + (4(L+2))t_{\mathsf{E}}] \\ &+ t_{\mathsf{H}} + (4L+2)t_{\mathsf{E}} + (q_D - \iota)[t_{\mathsf{H}} + (4(L+2))t_{\mathsf{E}}] + t_{\mathsf{H}} \\ &= t + qt_{\mathsf{H}} + [(q-1)(4L+6) + 2q_D]t_{\mathsf{E}} \leq t_3. \end{aligned}$$

Bounding $\Pr[\mathsf{Bad}_{4.\iota} | \neg\mathsf{Bad}_{4.1}, \dots, \mathsf{Bad}_{4.\iota-1}]$. First, we observe that if D_1^ι aborts, it means that there is a previous query for which $h^i = h^\iota$ and $c_{l+3}^i = c_{l+3}^\iota$, thus event $\mathsf{Bad}_{4.\iota}$ is event $\mathsf{Bad}_{4.i}$, thus,

$$\Pr[\mathsf{D}_1^\iota \text{ aborts} \mid \neg\mathsf{Bad}_{4.1}, \dots, \mathsf{Bad}_{4.\iota-1}] = 0.$$

Second, we observe that since k_0^ι, and $k_{l+3,E}^\iota$ are picked randomly (and (k_0^ι, p_A) and $(k_{l+3,E}^\iota, p_B)$ have never been queried before the ι^{th} decryption query to E due to the events Bad_2 and Bad_3 not happening), for the n–PR–corpi Game it is equivalent if either we pick the random prefix uniformly at random or we pick a k_0 uniformly at random, and we compute $k_{l+3,E} = \mathsf{E}_{k_0}(p_A)$ we use as a random prefix $\mathsf{E}(k_{l+3}, p_B)$. We are doing the latter case for D^ι. Thus, if event $\mathsf{Bad}_{4.\iota}\neg\mathsf{Bad}_{4.1}, \dots, \mathsf{Bad}_{4.\iota-1}$ happens, then, D wins the n–PR–corpi Game. Since D is a t_3-adversary and H is $(t_3, \epsilon_{\mathsf{PR\text{–}corpi}})$-$n$-PR–corpi-secure, then,

$$\Pr[\mathsf{Bad}_{4.\iota} \mid \neg\mathsf{Bad}_{4.1}, \dots, \mathsf{Bad}_{4.\iota-1}] \leq \epsilon_{\mathsf{PR\text{–}corpi}}.$$

Bounding $|\Pr[E_6] - \Pr[E_7]|$. We observe that

$$\Pr[\mathsf{Bad}_4] \leq \sum_{\iota=1}^{q_D+1} \Pr[\mathsf{Bad}_{4.\iota} \mid \neg\mathsf{Bad}_{4.1}, \dots, \mathsf{Bad}_{4.\iota-1}] \leq \sum_{\iota=1}^{q_D+1} \epsilon_{\mathsf{PR\text{–}corpi}} \leq (q_D+1)\epsilon_{\mathsf{PR\text{–}corpi}}.$$

$$\text{Thus, } |\Pr[E_6] - \Pr[E_7]| \leq \Pr[\mathsf{Bad}_4] \leq (q_D+1)\epsilon_{\mathsf{PR\text{–}corpi}}.$$

Having proved that no triple (h, k_0, c_{l+2}) associated to a decryption query can be used to forge, we pass to the triples associated to encryption queries.

Game 8. This is Game 7, where the following event Bad_5 does not happen: for every encryption query, if the adversary has not set k_0 in the input of the f computation,

then, there is no previous query to $\mathsf{E}(k'_0, p_A)$, where k'_0 is how k_0 is modified for step $4ii$) according to the fault Flt.

Transition between Game 7 and Game 8. The two games are the same except if the following Bad_5 event happens: there exists an encryption query $(a^j, m^j, \mathsf{Flt}^j)$, the j^{th}, s.t. 1) Flt^j does not set the k_0 input for the computation considered in Step $4ii$) (the call to f), 2) given k_0^j the key picked, and given $k_0^{\dagger,j}$ (that is, k_0^j modified for the computation of Step $4ii$), there is no previous query to the ideal cipher on input $(k_0^{\dagger,j}, p_A)$.

Now, note that since the fault is predetermined, the challenger can compute $k_0^{\dagger,j}$ before having to do any computation for the j^{th} encryption query.

We observe that during an encryption query, there are at most $L+2$ queries to $\mathsf{E}(\cdot, p_A)$ with at most $L+2$ different keys. Moreover, there are $2L+1$ queries on input $(\cdot,\cdot)$ where the key is either $k_{\iota,E}$ or $k_{\iota,A}$ for a certain ι and the input is a message block $m_{\iota,j}$ (which can be equal to p_A). Similarly, during a decryption query there are at most $L+2$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_A)$ and $2L+1$ to $\mathsf{E}(\cdot,\cdot)$. In addition, due to the change we have introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_A)$ The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, during the j^{th} encryption query there are at most $(j-1)(3L+3)+q_D(3L+4)+q_I$ different keys that, if picked, would trigger the Bad_5 event. Therefore,

$$\begin{aligned}
\Pr[\mathsf{Bad}_5] \leq & \sum_{j=1}^{q_E} \frac{(j-1)(3L+3)+q_D(3L+4)+q_I}{2^n} \\
\leq & [q_E(3L+3)(q_E-1)/2 + q_E q_I + q_E q_D(3L+4)]2^{-n} \\
= & q_E[3(q_E-1)(L+1)/2 + q_I + q_D(3L+4)]2^{-n}. \\
\text{Thus, } |\Pr[E_7] - \Pr[E_8]| \leq & \Pr[\mathsf{Bad}_5] \\
= & q_E[3(q_E-1)(L+1)/2 + q_I + q_D(3L+4)]2^{-n}.
\end{aligned}$$

Game 9. This is Game 8, where the following event Bad_6 does not happen: for every encryption query, if the adversary has not set k_0 in the input of the f computation, then, there is no previous query to $\mathsf{E}(k'_{0,E}, p_B)$, where $k'_{0,E} = \mathsf{E}_{k'_0}(p_B)$, and k'_0 is how k_0 is modified for step $4ii$) according to the fault Flt.

Transition between Game 8 and Game 9. The two Games are the same except if the following Bad_6 event happens: there exists an encryption query $(a^j, m^j, \mathsf{Flt}^j)$, the j^{th}, s.t. 1) Flt^j does not set the k_0 input for the computation considered in Step $4ii$) (the call to f), 2) given k_0^j, the key picked, and given $k_0^{\dagger,j}$ (that is, k_0^j modified for the computation of Step $4ii$) via faults, and $k_{0,E}^{\dagger,j}$, there is no previous query to the ideal cipher on input $(k_{0,E}^{\dagger,j}, p_B)$.

Now, note that since the fault is predetermined, a challenger can compute $k_0^{\dagger,j}$ and $k_{0,E}^{\dagger,j}$ (which is random due to the previous transition), before having to do any computation for the j^{th} encryption query.

We observe that during an encryption query there are at most $L+3$ queries to $\mathsf{E}(\cdot, p_B)$ with at most $L+3$ different keys, and $2L+1$ queries with $(\cdot,\cdot)$ where the key is either $k_{\iota,E}$ or $k_{\iota,A}$ for a certain ι and the input is a message m_ι (which can be equal to p_B). Similarly, during a decryption query, there are at most $L+3$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_B)$ and $2L+1$ to $\mathsf{E}(\cdot,\cdot)$. In addition, due to the change introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_B)$ The adversary can do at most q_I queries to the ideal block cipher. Thus, during the j^{th} encryption query there are at most $(j-1)(3L+4)+q_D(3L+4)+q_I$ different keys that, if picked, would

trigger the Bad_6 event. Thus,

$$\begin{aligned}\Pr[\mathsf{Bad}_6] \leq & \sum_{j=1}^{q_E} \frac{(j-1)(3L+4)+q_D(3L+4)+q_I}{2^n} \\ \leq & [q_E(3L+4)(q_E-1)/2 + q_E q_I + q_E q_D(3L+4)]2^{-n} \\ = & q_E[(q_E-1)(3L+4)/2 + q_I + q_D(3L+4)]2^{-n}. \\ \text{Thus, } & |\Pr[E_8] - \Pr[E_9]| \leq \Pr[\mathsf{Bad}_6] \\ = & q_E[(q_E-1)(3L+4)/2 + q_I + q_D(3L+4)]2^{-n}.\end{aligned}$$

Game 10. This is Game 9, where in every encryption query, immediately after Step $4ii$), after $k_0^\dagger$ is computed (where $k_0^\dagger$ is k_0 modified for the computation of Step $4ii$ according to the fault Flt), the encryption oracle computes $k_{l+3,E} = \mathsf{E}_{k_0^\dagger}(p_A)$ and $c_{l+3} = \mathsf{E}_{k_{l+3},E}(p_B)$. These values are appended to the leakage.

Transition between Game 9 and Game 10. These games are the same except for the two additional calls per encryption query that any adversary can do. Thus, it is easy to build an adversary A' which, when she receives the answer of an encryption query, computes $k_{l+3,E} = \mathsf{E}_{k_0^\dagger}(p_A)$ and $c_{l+3} = \mathsf{E}_{k_{0,E}}(p_B)$, appends them to the leakage and forwards them to the adversary.

Such an adversary needs $2q_E$ calls to the ideal block cipher. Thus,

$$\Pr[E_9] = \Pr[E_{10}].$$

Game 11. This is Game 10, which excludes the following event Bad_7: there is an encryption query, the j^{th}, s.t. all these conditions hold: 1) the adversary does not use a set fault for k_0^j in Step $1i$) during the j^{th} encryption query 2) there exists a previous ideal cipher query on input $(k_0^{j,\dagger}, p_A)$ (where $k_0^{j,\dagger}$ is k_0^j modified according to the fault Flt^j for the $(1i)$ step).

Transition between Game 10 and Game 11. The two Games are the same except if event Bad_7 happens. Since A is not setting k_0^j in Step $(1i)$ via a fault, $k_0^{j,\dagger}$ is random, where $k_0^{j,\dagger} = k_0^j \oplus \Delta$ where k_0^j randomly picked and Δ, the offset specified by the fault Flt^j chosen before k_0^j is picked.

As we did before, we observe that during an encryption query there are at most $L+3$ queries to $\mathsf{E}(\cdot, p_A)$ with at most $L+3$ different keys (we have an additional query because of Game 10), and $2L+1$ queries with $(\cdot,\cdot)$ where the key is either $k_{\iota,E}$ or $k_{\iota,A}$ for a certain ι and the input is a message m_ι (which can be equal to p_A), while during a decryption query there are at most $L+2$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_A)$ and $2L+1$ to $\mathsf{E}(\cdot,\cdot)$. In addition, due to the change we have introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_A)$. The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, during the j^{th} encryption query there are at most $(j-1)(3L+4)+q_D(3L+4)+q_I$ different keys that, if picked, would trigger the Bad_7 event. Thus,

$$\begin{aligned}\Pr[\mathsf{Bad}_7] \leq & \sum_{j=1}^{q_E} \frac{(j-1)(3L+4)+q_D(3L+4)+q_I}{2^n} \\ \leq & [q_E(3L+4)(q_E-1)/2 + q_E q_I + q_E q_D(3L+4)]2^{-n} \\ = & q_E[(3L+4)[q_D + (q_E-1)/2] + q_I]2^{-n}. \\ \text{Thus, } & |\Pr[E_{10}] - \Pr[E_{11}]| \leq \Pr[\mathsf{Bad}_7] \\ = & q_E\{(3L+4)[q_D + (q_E-1)/2] + q_I\}2^{-n}.\end{aligned}$$

Game 12. This is Game 11, where the following event Bad_8 does not occur: there is an encryption query, the j^{th}, s.t. all these conditions hold: 1) the adversary does not use a set fault for k_0^j in Step $(1i)$ during the j^{th} encryption query, 2) the adversary does not use a set fault for $k_{0,E}^j$ in Step $(1iii)$ during the j^{th} encryption query 3) there exists a previous ideal cipher query on input $(k_{0,E}^{j,\dagger}, p_A)$ (where $k_{0,E}^{j,\dagger}$ is $k_{0,E}^j$ modified according to the fault Flt^j for the $(1iii)$ step).

Transition between Game 11 and Game 12. The two Games are the same except if event Bad_8 happens. The previous transition has assured that $k_{0,E}^j$ is random, if the condition 1) of event Bad_8 happens. Since A is not setting both k_0^j in Step $1i$) and $k_{0,E}^j$ in Step $1iii$) via a fault, $k_{0,E}^{j,\dagger}$ is random, where $k_{0,E}^{j,\dagger} = k_{0,E}^j \oplus \Delta$ with $k_{0,E}^j$ randomly picked and Δ, the offset specified by the fault Flt^j, chosen before $k_{0,E}^j$ is picked.

As we did before, we observe that during an encryption query there are at most $L+4$ queries to $\mathsf{E}(\cdot, p_B)$ with at most $L+4$ different keys (we have an additional query because of Game 10), and $2L+1$ queries with $(\cdot,\cdot)$ where the key is either $k_{\iota,E}$ or $k_{\iota,A}$ for a certain ι and the input is a message m_ι (which can be equal to p_B), while during a decryption query there are at most $L+3$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_B)$ and $2L+1$ to $\mathsf{E}(\cdot,\cdot)$. In addition, due to the change we have introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_B)$ The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, during the j^{th} encryption query there are at most $(j-1)(3L+5) + q_D(3L+4) + q_I$ different keys that, if picked, would trigger the Bad_8 event. Thus,

$$\begin{aligned}
\Pr[\mathsf{Bad}_8] \leq & \sum_{j=1}^{q_E} \frac{(j-1)(3L+5) + q_D(3L+4) + q_I}{2^n} \\
\leq & [q_E(3L+5)(q_E-1)/2 + q_E q_I + q_E q_D(3L+4)]2^{-n} \\
= & q_E[(3L+4)[q_D + (q_E-1)/2] + (q_E-1)/2 + q_I]2^{-n}. \\
\text{Thus, } & |\Pr[E_{11}] - \Pr[E_{12}]| \leq \Pr[\mathsf{Bad}_8] \\
= & q_E[(3L+4)[q_D + (q_E-1)/2] + (q_E-1)/2 + q_I]2^{-n}.
\end{aligned}$$

Game 13. This is Game 12, where the following event Bad_9 does not occur: the adversary's output $(a^*, (c_0^*, \ldots, c_{l^*+2}^*))$ is fresh and valid (we thus call it a forgery), and let (h^*, k_0^*, c_{l+2}^*) be the triple associated with the forgery (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th} query, for which all these conditions hold: 1) (h^*, k_0^*, c_{l+2}^*) is the triple associated with the call to f done during the j^{th} encryption query, 2) the adversary sets k_0^* in the j^{th} encryption query (that is, she modifies k_0^j in k_0^* with a set fault) for the computation of step $4ii$), 3) h^* is the actual output of the H atom of the j^{th} encryption query, 4) $((c_0^* \ldots, c_{l+1}^*, a^*)$ is the actual input of the H atom the j^{th} encryption query.

Transition between Game 12 and Game 13. Since Game 12 and 13 are the same except if event Bad_9 happens, to bound the difference between these two games, we need to analyze event Bad_9. Let $\mathsf{Bad}_{9.j}$ be the event that Bad_9 happens and the 4 conditions hold for the j^{th} encryption query. Clearly $\mathsf{Bad}_9 = \bigcup_{j=1}^{q_E} \mathsf{Bad}_{9.j}$. Now, we bound $\Pr[\mathsf{Bad}_{9.j}]$. Since the ciphertext is valid, this means that $\mathsf{E}_{k_{0,E}^*}(p_B) = c_0^* = \mathsf{E}_{k_{0,E}^{j,\dagger}}(p_B)$, where $k_{0,E}^* = \mathsf{E}_{k_0^*}(p_A)$, $k_{0.E}^{j,\dagger}$ is $k_{0,E}^j$ modified according to the fault Flt^j for the $(1iii)$ step, and $k_{0,E}^j = \mathsf{E}_{k_0^{j,\dagger}}(p_A)$ (with $k_0^{j,\dagger}$ being k_0^j modified according to the fault Flt^j for the $(1i)$ step). Now, due to the fact that events Bad_7 and Bad_8 do not happen, then, $k_{0,E}^{j,\dagger}$ is random, and so is $c_{0,E}^j$.

Moreover, the actual input of the H atom (step $4i$)) of the j^{th} encryption uses $c^j_{0,E}$ and can only be modified with a fault. Thus, in step $4i$) we use $c^{j,\dagger}_{0,E} = c^j_{0,E} \oplus \Delta$ with Δ the offset specified by the fault Flt^j (which is chosen before all k^j_0, $k^j_{0,E}$, $c^j_{0,E}$ are picked). So, the probability that $c^{j,\dagger}_{0,E} = \mathsf{E}_{k^*_{0,E}}(p_B)$ is 2^{-n} which yields the bound for event Bad_9:

$$\Pr[\mathsf{Bad}_9] \leq \sum_{j=1}^{q_E} \Pr[\mathsf{Bad}_{9.j}] = q_E 2^{-n}.$$

Game 14. This is Game 13, where the following event Bad_{10} does not occur: the adversary's output $(a^*, (c^*_0, \ldots, c^*_{l^*+2}))$ is fresh and valid (thus, we call it a forgery), and let (h^*, k^*_0, c^*_{l+2}) be the triple associated with the forgery (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th} query, for which all these conditions hold: 1) (h^*, k^*_0, c^*_{l+2}) is the triple associated with the call to f done during the j^{th} encryption query, 2) the adversary sets k^j_0 to k^*_0 via the fault Flt^j in the j^{th} encryption query for Step $4ii$), 3) h^* is the actual output of the H atom (Step $4i$)) in the j^{th} encryption query, 4) $((c^*_0 \ldots, c^*_{l+1}, a^*)$ is not the actual input of the H atom (Step $4i$)) in the j^{th} encryption query.

Transition between Game 13 and Game 14. Since Game 13 and 14 are the same except if event Bad_{10} happens, to bound the difference between these two Games, we need analyze Bad_{10}. Let $\mathsf{Bad}_{10.j}$ be the event that Bad_{10} happens and the 4 conditions hold for the j^{th} encryption query. Clearly $\mathsf{Bad}_{10} = \bigcup_{j=1}^{q_E} \mathsf{Bad}_{10.j}$. Now, we bound $\Pr[\mathsf{Bad}_{10.j}]$. We observe that in the j^{th} encryption query, $h^j = \mathsf{H}_s(c^{j,\dagger}_0 \| \ldots \| c^{j,\dagger}_{l^j+1}, a^j)$ is computed (where $c^{j,\dagger}_i$ is c^j_i modified according to fault Flt^j for Step $4i$)), while for the forgery, h^* is computed as $h^* = \mathsf{H}_s(c^*_0 \| \ldots c^*_{l^*+1}, a^*)$. Due to condition 3) $h^j = h^*$, and due to the condition 4) $(c^{j,\dagger}_0 \| \ldots \| c^{j,\dagger}_{l^j+1}, a^j) \neq ((c^*_0 \ldots, c^*_{l+1}, a^*)$, thus, a collision for the hash function has occurred, which is impossible since we have assumed that event Bad_1 (that is, no collision), does not occur, thus $\Pr[\mathsf{Bad}_{10.j}] = 0$ and

$$\Pr[\mathsf{Bad}_{10}] = \sum_{j=1}^{q_E} \Pr[\mathsf{Bad}_{10.j}] = 0. \;\; \text{So,} \;\; \Pr[E_{13}] = \Pr[E_{14}].$$

Game 15. This is Game 14, where the following event Bad_{11} does not occur: a) the adversarial output $(a^*, (c^*_0, \ldots, c^*_{l+2}))$ is fresh and valid (thus, we call it a forgery), and b) given (h^*, k^*_0, c^*_{l+2}), the triple associated with the forgery, (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th} query, for which all these conditions hold: 1) (h^*, k^*_0, c^*_{l+2}) is the triple associated with the call to f done during the j^{th} encryption query, 2) the adversary sets k^j_0 to k^*_0 with fault Flt^j in the j^{th} encryption query for Step $4ii$), 3) h^* is not the actual output of the H atom (Step $4i$)) in the j^{th} encryption query.

Transition between Game 14 and Game 15. Since Games 14 and 15 are the same except if event Bad_{11} happens, to bound the difference between these two Games, we need to analyze Bad_{11}. Let $\mathsf{Bad}_{11.\iota}$ be the event that Bad_{11} happens and the 3 conditions hold for the ι^{th} encryption query. Clearly $\mathsf{Bad}_{11} = \bigcup_{\iota=1}^{q_E} \mathsf{Bad}_{11.\iota}$. Now, we bound $\Pr[\mathsf{Bad}_{11.\iota}]$. For this, we build a t_2-PR-coirv adversary EE^ι.

The PR-coirv-adversary EE^ι. At the start of the Game, EE^ι receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks two random tweakable functions f and g as defined earlier.

When A does the i^{th} encryption query, $i < \iota$, on input $(a^i, m^i, \mathsf{Flt}^i)$, EE_1^ι proceeds as C with the following exceptions: after step $4iii)$ she computes $k_{l+3}^i = \mathsf{E}_{k_0^{i,\dagger}}(p_A)$ where $k_0^{i,\dagger}$ is how k_0^i is modified in step $4ii)$ according to fault Flt^i, and $4iv)$ $c_{l+3} = \mathsf{E}_{k_{l+3}}(p_B)$. For this, EE_1^ι needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the ι^{th} encryption query on input $(a^\iota, c^\iota, \mathsf{Flt}^\iota)$, EE_1^ι proceeds as follows: she outputs $(\mathsf{st}, \mathcal{D}, \Delta)$ where $\mathsf{st} = (s, \mathcal{H}, \mathcal{L}, \mathcal{D})$, Δ is the offset specified by the fault Flt^ι for Step $4ii)$ for the h input (since we are considering event $\mathsf{Bad}_{11.\iota}$ it means that the adversary has already spent her only set fault for the k_0^ι for Step $4ii)$, and she uses a fault here. Thus, it is a differential fault, and we call Δ the difference XORed to h in this computation, and $\mathcal{D}$ is this distribution for the input of the hash function: 1) pick $k_0^\iota \xleftarrow{\$} \{0,1\}^n$ $1i)$ compute $k_{0,E} = \mathsf{E}_{k_0^{\iota,\dagger}}(p_B)$, where $k_0^{\iota,\dagger}$ is how k_0^ι is modified according to the fault Flt^ι, for this computation, $1ii)$ compute $k_{0,A}^\iota = \mathsf{E}_{k_0^{\iota,\ddagger}}(p_B)$, where $k_0^{\iota,\ddagger}$ is how k_0 is modified according to the fault Flt^ι for this computation, $1iii)$ compute $c_0 = \mathsf{E}_{k_{0,E}^{\iota,\dagger}}(p_A)$, where $k_{0,E}^{\iota,\dagger}$ is how $k_{0,E}^\iota$ is modified according to the fault Flt^ι for this computation, and $1iv)$ compute $k_1 = \mathsf{E}_{k_{0,E}^{\iota,\ddagger}}(p_A)$, where $k_{0,E}^{\iota,\ddagger}$ is how $k_{0,E}^\iota$ is modified according to the fault Flt^ι for this computation. Then, for all $i = 1, \ldots, l^\iota$, $2i)$ compute $k_{i,E}^\iota = \mathsf{E}_{k_i^{\iota,\dagger}}(p_B)$, where $k_i^{\iota,\dagger}$ is how k_i^ι is modified according to the fault Flt^ι for this computation, $2ii)$ compute $c_i^\iota = \mathsf{E}_{k_{i,E}^{\iota,\ddagger}}(m_i^{\iota,\dagger})$, where $k_{i,E}^{\iota,\ddagger}$ and $m_i^{\iota,\dagger}$ are how $k_{i,E}^\iota$ and m_i^ι are modified according to the fault Flt^ι for this computation, $2iii)$ compute $k_{i+1}^\iota = \mathsf{E}_{k_i^{\iota,\ddagger}}(p_A)$, where $k_i^{\iota,\ddagger}$ is how k_i^ι is modified according to the fault Flt^ι for this computation, and $2iv)$ compute $k_{i,A}^\iota = \mathsf{E}_{k_{i-1,A}^{\iota,\dagger}}(m_i^{\iota,\ddagger})$, where $k_{i-1,A}^{\iota,\dagger}$ and $m_i^{\iota,\ddagger}$ are how $k_{i-1,A}^\iota$ and m_i^ι are modified according to the fault Flt^ι for this computation. After that, $3i)$ compute $k_{l^\iota+1,E}^\iota = \mathsf{E}_{k_{l^\iota+1}^{\iota,\dagger}}(p_B)$, where $k_{l^\iota+1}^{\iota,\dagger}$ is how $k_{l^\iota+1}$ is modified according to the fault Flt^ι for this computation, and $3ii)$ compute $c_{l^\iota+1}^\iota = \mathsf{E}_{k_{l^\iota+1,E}^{\iota,\dagger}}(k_{l^\iota,A}^{\iota,\dagger})$, where $k_{l^\iota+1,E}^{\iota,\dagger}$ and $k_{l^\iota,A}^{\iota,\dagger}$ are how $k_{l^\iota+1,E}^\iota$ and $k_{l^\iota,A}^\iota$ are modified according to the fault Flt^ι for this computation. The inputs of H are $(c_0^{\iota,\dagger} \| \ldots \| c_{l^{\iota,\dagger}+1}, a^\iota)$, where $c_i^{\iota,\dagger}$ is how c_i^ι is modified according to fault Flt^ι for step $4i)$.

Now, upon receiving st, EE_2^ι samples $(c_0^\iota \| \ldots \| c_{l^\iota+1}, a^\iota)$ according to the distribution $\mathcal{D}$, then she computes $4i)$ $h^\iota = \mathsf{H}_s(c_0^\iota \| \ldots \| c_{l^\iota+1}, a^\iota)$, where $c_i^{\iota,\dagger}$ is how c_i^ι is modified according to fault Flt^ι for this computation, $4ii)$ $c_{l^\iota+2}^\iota = \mathsf{f}(h^{\iota,\dagger}, k_0^{\iota,\dagger\dagger})$, where $h_i^{\iota,\dagger}$ and $k_0^{\iota,\dagger\dagger}$ are how h_i^ι and k_0^ι are modified according to fault Flt^ι for this computation (note that if Event $\mathsf{Bad}_{11.\iota}$ occurs, by the condition 2) the adversary sets k_0^ι to k_0^*). $4iii)$ she computes $k_{l^\iota+3}^\iota = \mathsf{E}_{k_0^{\iota,\dagger\dagger}}(p_A)$ where $k_0^{\iota,\dagger\dagger}$ is how k_0^ι is modified in Step $4ii)$ according to fault Flt^ι, and $4iv)$ $c_{l^\iota+3} = \mathsf{E}_{k_{l+3}^\iota}(p_B)$. For EE_2 $x = (c_0^\iota \| \ldots \| c_{l^\iota+1}, a^\iota)$ and $h' = h_i^\iota = h^\iota \oplus \Delta$.

Thus, EE_2^ι needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the i^{th} encryption query, $i > \iota$, on input $(a^i, m^i, \mathsf{Flt}^i)$, EE_2^ι proceeds as EE_1^ι for the j^{th} encryption query with $j < \iota$. Thus, EE_2^ι needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the ι decryption query on input (a, c, Flt), EE^ι (we use EE^ι) to identify both EE_1^ι and EE_2^ι) behaves as C with the following exceptions: after step $(1ii)$, before doing step $2i$, EE^ι computes $1iii)$ $k_{l^\iota+3}^\iota = \mathsf{E}_{k_0^\iota}(p_A)$, and $1iv)$ $c_{l^\iota+3}^\iota = \mathsf{E}_{k_{l^\iota+3}}(p_B)$.

Thus, EE^ι needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), EE_2^ι proceeds as follows: she computes $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_{l+1}^*, a^*)$ and she adds $((c_0^* \| \ldots \| c_{l+1}^*, a^*), h)$ to $\mathcal{H}$, and then, she looks through $\mathcal{H}$ to see if there is an entry $(c_0 \| \ldots \| c_{l+1}, a), h)$ s.t $h = h'$. If it is the case, she outputs it, otherwise 0^n. This takes time t_{H}.

Thus, in total EE runs in time bounded by

$$\begin{aligned} &t+q_E[t_{\mathsf{H}}+(4(L+2))t_{\mathsf{E}}]+(q_D)[t_{\mathsf{H}}+(4(L+2))t_{\mathsf{E}}]+t_{\mathsf{H}} \\ &=t+qt_{\mathsf{H}}+[(q-1)4(L+4)]t_{\mathsf{E}} \leq t_2. \end{aligned}$$

Bounding $\Pr[\mathsf{Bad}_{11}]$. First, we observe that EE^{ι} simulates perfectly Game 14 for A. As already discussed, to bound $\Pr[\mathsf{Bad}_{11}]$, it is enough to bound $\Pr[\mathsf{Bad}_{11.\iota}]$ $\forall \iota$. First, we observe that EE^{ι} samples correctly according to $\mathcal{D}$ the input for the hash function (in the $\mathsf{PR\text{-}coirv}$ Game (Def. 24) it is irrelevant who samples the input for H, as long as it is sampled according to $\mathcal{D}$). Moreover, due to the fact that if event $\mathsf{Bad}_{11.\iota}$ happens, in the ι^{th} encryption query the adversary A uses her set fault for k_0 in Step $4ii$), we can use Lemma 2 which assures that $\mathcal{D}$ has the property required by the $\mathsf{PR\text{-}coirv}$ definition (for the randomness). Clearly, if event $\mathsf{Bad}_{11.\iota}$ happens, the adversary has chosen an offset for h. Thus, if event $\mathsf{Bad}_{11.\iota}$ happens, then EE^{ι} can find a pre-image for h', where h' is obtained by adding an offset chosen in advance to the hash of a "random enough" input. Since H is a $(t_2, \alpha, \epsilon_{\mathsf{PR\text{-}coirv}})$-$\mathsf{PR\text{-}coirv}$-secure hash function, the maximum probability of the distribution $\mathcal{D}$, denoted α, is bounded by Lemma 2, and EE^{ι} is a t_2-adversary, then

$$\Pr[\mathsf{Bad}_{11.\iota}] \leq \epsilon_{\mathsf{PR\text{-}coirv}}, \text{ and}$$

$$|\Pr[E_{14}] - \Pr[E_{15}]| \leq \Pr[\mathsf{Bad}_{11}] \leq \sum_{\iota=1}^{q_E} \Pr[\mathsf{Bad}_{11.\iota}] = q_E \epsilon_{\mathsf{PR\text{-}coirv}}.$$

Game 16. This is Game 15, where the following event Bad_{12} does not happen: the adversarial output $(a^*, (c_0^*, \ldots, c_{l+2}^*))$ is fresh and valid (thus, we call it a forgery), and given (h^*, k_0^*, c_{l+2}^*), the triple associated with the forgery (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th}, for which these conditions hold: 1) (h^*, k_0^*, c_{l+2}^*) is the triple associated with the call to f done during the j^{th} encryption query, 2) the adversary sets h^j to h^* with fault Flt^j in the j^{th} encryption query for Step $4ii$).

Transition between Game 15 and Game 16. Since Game 15 and 16 are the same except if event Bad_{12} happens, to bound the difference between these two games, we need to analyze Bad_{12}. Let $\mathsf{Bad}_{12.\iota}$ be the event that Bad_{12} happens and the 2 conditions hold for the ι^{th} encryption query. Clearly $\mathsf{Bad}_{12} = \bigcup_{\iota=1}^{q_E} \mathsf{Bad}_{12.\iota}$. Now, we bound $\Pr[\mathsf{Bad}_{12.\iota}]$. For this, we build a t_3-$\mathsf{PR\text{-}corpi}$ adversary GG^{ι}.

The n-$\mathsf{PR\text{-}corpi}$-adversary GG^{ι}. At the start of the game, GG^{ι} receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks two random tweakable functions f and g as discussed earlier.

When A does the i^{th} encryption query, $i < \iota$, on input $(a^i, m^i, \mathsf{Flt}^i)$, GG_1^{ι} proceeds as C with the following exceptions: after step $4iii$) she computes $k_{l^i+3}^i = \mathsf{E}_{k_0^{i,\dagger}}(p_A)$ where $k_0^{i,\dagger}$ is how k_0^i is modified in step $4ii$) according to fault Flt^i, and $4iv$) $c_{l^i+3} = \mathsf{E}_{k_{l^i+3}}(p_B)$. For this, GG_1^{ι} needs time $t_{\mathsf{H}} + [4(l^i+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the ι^{th} encryption query on input $(a^{\iota}, m^{\iota}, \mathsf{Flt}^{\iota})$, GG_1^{ι} proceeds as follows: from Flt^{ι} she checks whether the adversary sets h in Step $4ii$) to $h^{\iota,\dagger}$: if it is the case she outputs $h^{\iota,\dagger}$ as her target; in addition, she outputs $\mathsf{st} = (s, \mathcal{H}, \mathcal{L}, a^{\iota}, m^{\iota}, \mathsf{Flt}^{\iota})$; otherwise, she aborts. GG_2^{ι} receiving st, she proceeds as C for an encryption query on input $(a^{\iota}, m^{\iota}, \mathsf{Flt}^{\iota})$ with the following exceptions: after step $4iii$) she computes $k_{l^i+3}^{\iota} = \mathsf{E}_{k_0^{\iota,\dagger}}(p_A)$ where $k_0^{i,\dagger}$ is how k_0^{ι} is modified in Step $4ii$) according to fault Flt^i, then, she computes $4iv$) $c_{l^{\iota}+3} = \mathsf{E}_{k_{l^{\iota}+3}}(p_B)$. For this, GG_1^{ι} and GG_2^{ι} need in total time $t_{\mathsf{H}} + [4(l^{\iota}+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$. Finally GG_2^{ι} takes $c_{l^{\iota}+3}^{\iota}$ as her random prefix for the pre-image of $h^{\iota,\dagger}$.

When A does the i^{th} encryption query, $i > \iota$, on input $(a^i, m^i, \mathsf{Flt}^i)$, GG_2^ι proceeds as C with the following exceptions: after Step $4iii$) she computes $k^i_{l^i+3} = \mathsf{E}_{k_0^{i,\dagger}}(p_A)$ where $k_0^{i,\dagger}$ is how k_0^i is modified in Step $4ii$) according to fault Flt^i, and $4iv$) $c_{l^i+3} = \mathsf{E}_{k_{l^i+3}}(p_B)$.

For this, GG_2^ι needs time $t_{\mathsf{H}} + [4(l^i+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the ι^{th} decryption query on input (a, c, Flt), GG^ι (we use GG^ι) to identify both GG_1^ι and GG_2^ι) behaves as C with the following exceptions: after step $(1ii)$, before doing step $(2i)$, GG^ι computes $1iii)$ $k^\iota_{l^\iota+3} = \mathsf{E}_{k_0^\iota}(p_A)$, and $1iv)$ $c^\iota_{l^\iota+3} = \mathsf{E}_{k_{l^\iota+3}}(p_B)$.

Thus, GG^ι needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), GG_2^ι proceeds as follows: she computes $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_{l+1}^*, a^*)$ and she adds $((c_0^* \| \ldots \| c_{l+1}^*, a^*), h)$ to $\mathcal{H}$, and then she looks through $\mathcal{H}$ to see if there is an entry $(c_0 \| \ldots \| c_{l+1}, a), h)$ s.t $h = h^{\iota,\dagger}$ and $c_0 = c^\iota_{l^\iota+3}$. If this is the case, she outputs it, otherwise 0^n. This takes time t_{H}.

Thus, in total GG runs in time bounded by

$$t + q_E[t_{\mathsf{H}} + (4(L+2))t_{\mathsf{E}}] + (q_D)[t_{\mathsf{H}} + (4(L+2))t_{\mathsf{E}}] + t_{\mathsf{H}}$$
$$t + qt_{\mathsf{H}} + [(q-1)4(L+4)]t_{\mathsf{E}} \leq t'_3.$$

Bounding $\Pr[\mathsf{Bad}_{12}]$. First, we observe that GG^ι simulates perfectly Game 15 for A. Thus, it is equivalent to pick the random prefix as we do or uniformly at random from $\{0,1\}^*$. As already discussed, to bound $\Pr[\mathsf{Bad}_{12}]$, it is enough to bound $\Pr[\mathsf{Bad}_{12.\iota}]$ $\forall \iota$. First, if event $\mathsf{Bad}_{12.\iota}$ happens, in the ι^{th} encryption query the adversary A uses her set fault for h in Step $4ii$) setting it in $h^{\iota,\dagger}$; moreover, we observe that $c^\iota_{l^\iota+3}$ is random due to the fact that we have assumed that both event Bad_7 and Bad_8 do not happen. Thus, if event $\mathsf{Bad}_{12.\iota}$ happens, then GG^ι can find a pre-image for $h^{\iota,\dagger}$, of her choice whose prefix is random, and it is equal to $c^\iota_{l^\iota+3}$. Since H is a $(t_3, \epsilon_{\mathsf{PR\text{-}corpi}})$-$\mathsf{PR\text{-}corpi}$-secure hash function and GG^ι is a t_3-adversary, then

$$\Pr[\mathsf{Bad}_{12.\iota}] \leq \epsilon_{\mathsf{PR\text{-}corpi}}, \text{ and}$$

$$|\Pr[E_{15}] - \Pr[E_{16}]| \leq \Pr[\mathsf{Bad}_{12}] \leq \sum_{\iota=1}^{q_E} \Pr[\mathsf{Bad}_{12.\iota}] = q_E \epsilon_{\mathsf{PR\text{-}corpi}}.$$

Game 17. This is Game 16, where the following event Bad_{13} does not occur: the adversary's output $(a^*, (c_0^*, \ldots, c_{l+2}^*))$ is fresh and valid (thus called a forgery), and given (h^*, k_0^*, c_{l+2}^*), the triple associated with the forgery (that is, the inputs and output of the call to g used in the decryption) there exists an encryption query, the j^{th}, for which all these conditions hold: 1) (h^*, k_0^*, c_{l+2}^*) is the triple associated with the call to f done during the j^{th} encryption query, 2) in Step $4ii$) of the j^{th} encryption query there is no set fault, 3) h^j the output of Step $4i$) is the h input for step $4ii$), that is, there is no differential fault applied to it. 4) in the input of Step $4i$), $c_0^{j,\dagger} \neq c^j_{l^j+3}$, where $c_0^{j,\dagger}$ denotes c_0 modified according to fault Flt^j for step $4i$) and $c^j_{l^j+3} = \mathsf{E}_{k^*_{0,E}}(p_B)$, with $k^*_{0,E} = \mathsf{E}_{k_0^*}(p_A)$.

Transition between Game 16 and Game 17. Since games 16 and 17 are the same except if event Bad_{13} happens, to bound the difference between these two games, we need to analyze Bad_{13}. Let $\mathsf{Bad}_{13.\iota}$ be the event that Bad_{13} happens and the 2 conditions hold for the ι^{th} encryption query. Clearly $\mathsf{Bad}_{13} = \bigcup_{\iota=1}^{q_E} \mathsf{Bad}_{13.\iota}$. Now, we bound $\Pr[\mathsf{Bad}_{13.\iota}]$.

We observe that if event $\mathsf{Bad}_{13.\iota}$ happens, it means that h^ι has two different pre-images, one computed in the j^{th} encryption query and the other in the forgery. These two pre-images are different because one uses $c_0^{j,\dagger}$, the other is associated with the forgery, therefore $c_0^* = c^\iota_{l^\iota+3}$, since in the forgery we use k_0^* (as specified in condition 1 of event Bad_{13}). By the definition of $c^\iota_{l^\iota+3}$ in Game 10, the previous equality must hold. But, because event

Bad_1 does not happen, it is impossible that there are two pre-images for the same value of the hash function. Thus, $\Pr[\mathsf{Bad}_{13.\iota}] = 0$, and

$$|\Pr[E_{16}] - \Pr[E_{17}]| = \Pr[\mathsf{Bad}_{13}] \leq \sum_{\iota=1}^{q_E} \Pr[\mathsf{Bad}_{13.\iota}] = 0.$$

Game 18. Game 18 is the same as Game 17, except that the following event Bad_{14} does not occur: the adversarial output $(a^*, (c_0^*, \ldots, c_{l+2}^*))$ is fresh and valid (thus, we call it a forgery), and given (h^*, k_0^*, c_{l+2}^*), the triple associated with the forgery, (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th}, for which all these conditions hold: 1) (h^*, k_0^*, c_{l+2}^*) is the triple associated with the call to f made during the j^{th} encryption query, 2) in step $4ii$) of the j^{th} encryption query there is no set fault, 3) the output h^j of Step $4i$) serves as the input h for Step $4ii$), that is, there is no differential fault applied to it. 4) in the input of step $4i$), $c_0^{j,\dagger} = c_{l^j+3}^j$, where $c_0^{j,\dagger}$ is c_0 modified according to fault Flt^j for step $4i$) and $c_{l^j+3}^j = \mathsf{E}_{k_{0,E}^*}(p_B)$, with $k_{0,E}^* = \mathsf{E}_{k_0^*}(p_A)$ (thus $c_0^{j,\dagger} = c_0^*$), 5) the input for Step $4i$) in the j^{th} encryption query, $(c_0^{j,\dagger} \| \ldots \| c_{l^j+1}^{j,\dagger}, a^j)$, where $c_i^{j,\dagger}$ is c_i^j modified according to the fault Flt^j for step $4i$), is a " valid encryption" for k_0^*, that is, there exists a message $m^{j,\ddagger}$ s.t. given k_0^* and $m^{j,\ddagger}$ applying all the encryption steps from $(1i)$ to $(3ii)$ without any fault, the result is $c_0^{j,\dagger}, \ldots, c_{l^j+1}^{j,\dagger}$.

Transition between Game 17 and Game 18. Since Games 17 and 18 are the same except if event Bad_{14} happens, to bound the difference between these two Games, we need to analyze Bad_{14}. Let $\mathsf{Bad}_{14.\iota}$ be the event that Bad_{14} happens and the 5 conditions hold for the ιth encryption query. Clearly $\mathsf{Bad}_{14} = \bigcup_{\iota=1}^{q_E} \mathsf{Bad}_{14.\iota}$. Now, we bound $\Pr[\mathsf{Bad}_{14.\iota}]$. To bound it, we divide the event $\mathsf{Bad}_{14.\iota}$ into two sub-events:

- $\mathsf{Bad}_{14.\iota.1}$: the adversary injects any effective fault (that is, any fault which modifies the output) in the j^{th} encryption query.
- $\mathsf{Bad}_{14.\iota.2}$: the adversary injects no effective fault in the j^{th} encryption query.

Bounding $\Pr[\mathsf{Bad}_{14.\iota.1}]$. We have two possibilities: the adversary has injected a fault (either in Step $4i$) or Step $1ii$)) or not. In the latter case, since the events Bad_6 and Bad_7 do not occur, $c_{l^j+3}^j$ is random, thus, the probability that $c_0^{j,\dagger} = c_{l^j+3}^j$ is $\leq 2^{-n}$ if there is an effective fault in Step $1i$) or $1ii$) or $4ii$) (due to conditions 2 and 3 of event Bad_{14}, in step $4ii$), A can only inject a differential fault the k_0 input). To address the other cases, we use Lemma. 2, obtaining that the probability that this case happens is bounded by

$$2^{-n} + (L+3)\left[\frac{3}{2^n} + \frac{Q(Q-1)(Q-2)}{6(2^{2n})}\right],$$

where $Q = q_I + 4(L+6)[q_E + q_D]$. Thus,

$$\Pr[\mathsf{Bad}_{14.\iota.1}] \leq \frac{3}{2^n} + (L+3) \cdot \left[\frac{(q_I + 4(L+6)[q_E+q_D])(q_I + 4(L+6)[q_E+q_D] - 1)(q_I + 4(L+6)[q_E+q_D] - 2)}{6(2^{2n})}\right].$$

Bounding $\Pr[\mathsf{Bad}_{14.\iota.2}]$. Due to the condition 1) of event Bad_{14}, in the forgery we use the triple (h^*, k_0^*, c_{l+2}^*) associated with the j^{th} encryption query. Moreover, since we consider event $\mathsf{Bad}_{14.\iota.2}$, in the j^{th} encryption query there are no faults. Thus, the adversary must find another pre-image for h^j, which implies that the adversary has found a collision for the hash function. But this is impossible, since we assumed that event Bad_1 does not happen. Thus, $\Pr[\mathsf{Bad}_{14.\iota.2}] = 0$, and $\Pr[\mathsf{Bad}_{14.\iota}] \leq \Pr[\mathsf{Bad}_{14.\iota.1}]$.

Finally, we can bound

$$|\Pr[E_{17}] - \Pr[E_{18}]| = \Pr[\mathsf{Bad}_{14}] \leq \sum_{\iota=1}^{q_E} \Pr[\mathsf{Bad}_{14.\iota}] = q_E 2^{-n} + q_E(L+3)$$
$$\cdot \left[\frac{(q_I + 4(L+6)[q_E+q_D])(q_I + 4(L+6)[q_E+q_D] - 1)(q_I + 4(L+6)[q_E+q_D] - 2)}{6(2^{2n})} \right].$$

Game 19. Game 19 is Game 18, except that the following event Bad_{15} does not occur: the adversary's output $(a^*, (c_0^*, \ldots, c_{l+2}^*))$ is fresh and valid (thus, we call it a forgery), and given (h^*, k_0^*, c_{l+2}^*), the triple associated with the forgery (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th}, for which all these conditions hold: 1) (h^*, k_0^*, c_{l+2}^*) is the triple associated with the call to f made during the j^{th} encryption query, 2) in step $4ii$) of the j^{th} encryption query, there is no set fault, 3) h^j the output of step $4i$) is the h input for step $4ii$), that is, there is no differential fault applied to it. 4) in the input of step $4i$), $c_0^{j,\dagger} = c_{l^j+3}^j$, where $c_0^{j,\dagger}$ is c_0 modified according to fault Flt^j for step $4i$), and $c_{l^j+3}^j = \mathsf{E}_{k_{0,E}^*}(p_B)$, with $k_{0,E}^* = \mathsf{E}_{k_0^*}(p_A)$ (so $c_0^{j,\dagger} = c_0^*$), 5) the input for Step $4i$) in the j^{th} encryption query, $(c_0^{j,\dagger} \| \ldots \| c_{l^j+1}^{j,\dagger}, a^j)$, where $c_i^{j,\dagger}$ denotes c_i^j modified according to the fault Flt^j for step $4i$), is not a "valid encryption" for k_0^*, that is, there exists no message $m^{j,\ddagger}$ s.t. given k_0^* and $m^{j,\ddagger}$ applying all the encryption steps from $(1i)$ to $(3ii)$ without any fault, the result is $c_0^{j,\dagger}, \ldots, c_{l^j+1}^{j,\dagger}$.

Transition between Game 18 and Game 19. Since Game 18 and 19 are the same except when event Bad_{15} happens, to bound the difference between these two Games, we need to analyze Bad_{15}. Let $\mathsf{Bad}_{15.\iota}$ be the event that Bad_{15} happens and the 5 conditions hold for the ι^{th} encryption query. Clearly $\mathsf{Bad}_{15} = \bigcup_{\iota=1}^{q_E} \mathsf{Bad}_{15.\iota}$. Now, we bound $\Pr[\mathsf{Bad}_{15.\iota}]$.

Due to the condition 1) of event Bad_{15}, in the forgery we use the triple (h^*, k_0^*, c_{l+2}^*) associated with the j^{th} encryption query. Moreover, since we consider event Bad_{15}, in the j^{th} encryption query, the input of the hash does not correspond to a "valid" encryption for k_0^*. Thus, the adversary has to find another pre-image for h^j, which means that the adversary has found a collision for the hash function. But this is impossible because we have assumed that event Bad_1 does not happen. Thus, $\Pr[\mathsf{Bad}_{15.\iota}] = 0$ and

$$|\Pr[E_{18}] - \Pr[E_{19}]| = \Pr[\mathsf{Bad}_{15}] \leq \sum_{\iota=1}^{q_E} \Pr[\mathsf{Bad}_{15.\iota}] = 0.$$

Game 20. Game 20 is Game 19, where the following event Bad_{16} does not occur:

- the adversary's output $(a^*, (c_0^*, \ldots, c_{l+2}^*))$ is fresh and valid (thus, we call it a forgery), and
- given (h^*, k_0^*, c_{l+2}^*), the triple associated with the forgery (that is, the inputs and output of the call to g used in the decryption), there exists an encryption query, the j^{th}, for which all these conditions hold:
  - 1) (h^*, k_0^*, c_{l+2}^*) is the triple associated with the call to f made during the j^{th} encryption query,
  - 2) in step $4ii$) of the j^{th} encryption query, there is no set fault,
  - 3) h^j, the output of step $4i$), is not the h input for step $4ii$).

Note that we have ruled out the possibility that A injects a set fault in Step $4ii$) because this case was covered in Game 16, thus, the only possibility is that A injects a differential fault in h in step $4ii$).

Transition between Game 19 and Game 20. Since Game 19 and Game 20 are the same except if event Bad_{16} happens, to bound the difference between these two Games, we

need to analyze Bad_{16}. Let $\mathsf{Bad}_{16.\iota}$ be the event that Bad_{16} happens and the 3 conditions hold for the ι^{th} encryption query. Clearly $\mathsf{Bad}_{16} = \bigcup_{\iota=1}^{q_E} \mathsf{Bad}_{16.\iota}$. Now, we bound $\Pr[\mathsf{Bad}_{16.\iota}]$. For this, we build a t_2-PR–coirv adversary HH^{ι}.

The PR–coirv-adversary HH^{ι}. At the start of the Game, HH^{ι} receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks two random tweakable functions f and g as defined earlier.

When A does the i^{th} encryption query, $i < \iota$, on input $(a^i, m^i, \mathsf{Flt}^i)$, HH_1^{ι} proceeds as C with the following exceptions: after step $4iii$) she computes $k_{l+3}^i = \mathsf{E}_{k_0^{i,\dagger}}(p_A)$ where $k_0^{i,\dagger}$ is how k_0^i is modified in step $4ii$) according to fault Flt^i, and $4iv$) $c_{l+3} = \mathsf{E}_{k_{l+3}}(p_B)$. For this, HH_1^{ι} needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the ι^{th} encryption query on input $(a^{\iota}, c^{\iota}, \mathsf{Flt}^{\iota})$, HH_1^{ι} proceeds as follows: she outputs $(\mathsf{st}, \mathcal{D}, \Delta)$ where

- $\mathsf{st} = (s, \mathcal{H}, \mathcal{L}, \mathcal{D})$,
- Δ is the offset specified by the fault Flt^{ι} for step $4ii$) for the h input (since we are considering event $\mathsf{Bad}_{16.\iota}$ it means that in step $4ii$) the adversary does not use a set fault, but, instead she uses a differential fault for the h input. Let Δ be the difference XORed to h in this computation), and
- $\mathcal{D}$ is this distribution for the input of the hash function:

$\mathsf{st} = (s, \mathcal{H}, \mathcal{L}, \mathcal{D})$, For this, HH_1^{ι} 1) pick $k_0^{\iota} \xleftarrow{\$} \{0,1\}^n$ $1i$) compute $k_{0,E} = \mathsf{E}_{k_0^{\iota,\dagger}}(p_B)$, where $k_0^{\iota,\dagger}$ is how k_0^{ι} is modified according to the fault Flt^{ι}, for this computation, $1ii$) compute $k_{0,A}^{\iota} = \mathsf{E}_{k_0^{\iota,\ddagger}}(p_B)$, where $k_0^{\iota,\ddagger}$ is how k_0 is modified according to the fault Flt^{ι} for this computation, $1iii$) compute $c_0 = \mathsf{E}_{k_{0,E}^{\iota,\dagger}}(p_B)$, where $k_{0,E}^{\iota,\dagger}$ is how $k_{0,E}^{\iota}$ is modified according to the fault Flt^{ι} for this computation, and $1iv$) compute $k_1 = \mathsf{E}_{k_{0,E}^{\iota,\ddagger}}(p_A)$, where $k_{0,E}^{\iota,\ddagger}$ is how $k_{0,E}^{\iota}$ is modified according to the fault Flt^{ι} for this computation. Then, for all $i = 1, \ldots, l^{\iota}$, $2i$) compute $k_{i,E}^{\iota} = \mathsf{E}_{k_i^{\iota,\dagger}}(p_B)$, where $k_i^{\iota,\dagger}$ is how k_i^{ι} is modified according to the fault Flt^{ι} for this computation, $2ii$) compute $c_i^{\iota} = \mathsf{E}_{k_{i,E}^{\iota,\ddagger}}(m_i^{\iota,\dagger})$, where $k_{i,E}^{\iota,\ddagger}$ and $m_i^{\iota,\dagger}$ are how $k_{i,E}^{\iota}$ and m_i^{ι} are modified according to the fault Flt^{ι} for this computation, $2iii$) compute $k_{i+1}^{\iota} = \mathsf{E}_{k_i^{\iota,\ddagger}}(p_A)$, where $k_i^{\iota,\ddagger}$ is how k_i^{ι} is modified according to the fault Flt^{ι} for this computation, and $2iv$) compute $k_{i,A}^{\iota} = \mathsf{E}_{k_{i-1,A}^{\iota,\dagger}}(m_i^{\iota,\ddagger})$, where $k_{i-1,A}^{\iota,\dagger}$ and $m_i^{\iota,\ddagger}$ are how $k_{i-1,A}^{\iota}$ and m_i^{ι} are modified according to the fault Flt^{ι} for this computation. After that, $3i$) compute $k_{l^{\iota}+1,E}^{\iota} = \mathsf{E}_{k_{l^{\iota}+1}^{\iota,\dagger}}(p_B)$, where $k_{l^{\iota}+1}^{\iota,\dagger}$ is how $k_{l^{\iota}+1}$ is modified according to the fault Flt^{ι} for this computation, and $3ii$) compute $c_{l^{\iota}+1}^{\iota} = \mathsf{E}_{k_{l^{\iota}+1,E}^{\iota,\dagger}}(k_{l^{\iota},A}^{\iota,\dagger})$, where $k_{l^{\iota}+1,E}^{\iota,\dagger}$ and $k_{l^{\iota},A}^{\iota,\dagger}$ are how $k_{l^{\iota}+1,E}^{\iota}$ and $k_{l^{\iota},A}^{\iota}$ are modified according to the fault Flt^{ι} for this computation. The inputs of H are $(c_0^{\iota,\dagger} \| \ldots \| c_{l^{\iota,\dagger}+1}, a^{\iota})$, where $c_i^{\iota,\dagger}$ is how c_i^{ι} is modified according to fault Flt^{ι} for step $4i$).

Now, HH_2^{ι} receiving st, samples $(c_0^{\iota} \| \ldots \| c_{l^{\iota}+1}, a^{\iota})$ according to the distribution $\mathcal{D}$, then she computes $4i$) $h^{\iota} = \mathsf{H}_s(c_0^{\iota} \| \ldots \| c_{l^{\iota}+1}, a^{\iota})$, where $c_i^{\iota,\dagger}$ is how c_i^{ι} is modified according to fault Flt^{ι} for this computation, $4ii$) $c_{l^{\iota}+2}^{\iota} = \mathsf{f}(h^{\iota,\dagger}, k_0^{\iota,\dagger\dagger})$, where $h_i^{\iota,\dagger}$ and $k_0^{\iota,\dagger\dagger}$ are how h_i^{ι} and k_0^{ι} are modified according to fault Flt^{ι} for this computation (note that if Event $\mathsf{Bad}.\iota_{11}$ happens, by the condition 2) the adversary sets k_0^{ι} to k_0^*). $4iii$) she computes $k_{l^{\iota}+3}^{\iota} = \mathsf{E}_{k_0^{\iota,\dagger\dagger}}(p_A)$ where $k_0^{\iota,\dagger\dagger}$ is how k_0^{ι} is modified in step $4ii$) according to fault Flt^{ι}, and $4iv$) $c_{l^{\iota}+3} = \mathsf{E}_{k_{l+3}^{\iota}}(p_B)$. For HH_2 $x = (c_0^{\iota} \| \ldots \| c_{l^{\iota}+1}, a^{\iota})$ and $h' = h_i^{\iota} = h^{\iota} \oplus \Delta$.

Thus, HH_2^{ι} needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the i^{th} encryption query, $i > \iota$, on input $(a^i, m^i, \mathsf{Flt}^i)$, HH_2^{ι} proceeds as HH_1^{ι} for the j^{th} encryption query with $j < \iota$. Thus, HH_2^{ι} needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq$

$t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A does the ι decryption query on input (a, c, Flt), HH^{ι} (we use HH^{ι} to identify both HH_1^{ι} and HH_2^{ι}) behaves as C with the following exceptions: after step $(1ii)$, before doing step $(2i)$, HH^{ι} computes $1iii)$ $k_{l^{\iota}+3}^{\iota} = \mathsf{E}_{k_0^{\iota}}(p_A)$, and $1iv)$ $c_{l^{\iota}+3}^{\iota} = \mathsf{E}_{k_{l^{\iota}+3}}(p_B)$.

Thus, HH^{ι} needs time $t_{\mathsf{H}} + [4(l+1)+4]t_{\mathsf{E}} \leq t_{\mathsf{H}} + [4(L+2)]t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), HH_2^{ι} proceeds as follows: she computes $h^* = \mathsf{H}_s(c_0^* \| \ldots \| c_{l+1}^*, a^*)$ and she adds $((c_0^* \| \ldots \| c_{l+1}^*, a^*), h)$ to $\mathcal{H}$, and then, she looks through $\mathcal{H}$ to see if there is an entry $(c_0 \| \ldots \| c_{l+1}, a), h)$ s.t $h = h'$. If this is the case, she outputs it, otherwise 0^n. This takes time t_{H}.

Therefore, in total HH runs in time bounded by

$$t + q_E[t_{\mathsf{H}} + (4(L+2))t_{\mathsf{E}}] + (q_D)[t_{\mathsf{H}} + (4(L+2))t_{\mathsf{E}}] + t_{\mathsf{H}}$$
$$t + qt_{\mathsf{H}} + [(q-1)4(L+4)]t_{\mathsf{E}} \leq t_2.$$

Bounding $\Pr[\mathsf{Bad}_{16}]$. First, we observe that HH^{ι} simulates perfectly Game 19 for A. As already discussed, to bound $\Pr[\mathsf{Bad}_{16}]$, it is enough to bound $\Pr[\mathsf{Bad}_{11.\iota}]$ $\forall \iota$. First, we observe that HH^{ι} samples correctly according to $\mathcal{D}$ the input for the hash function (in the $\mathsf{PR\text{-}coirv}$ Game (Def. 24) it is irrelevant who samples the input for H, as long as, it is sampled according to $\mathcal{D}$). Moreover, due to the fact that if event $\mathsf{Bad}_{16.\iota}$ happens, in the ι^{th} encryption query the adversary A uses her set fault for k_0 in Step $4ii)$, we can use Lemma 2 which assures that $\mathcal{D}$ has the property required by the $\mathsf{PR\text{-}coirv}$ definition (for the unpredictability, see Lemma 2). Clearly, if event $\mathsf{Bad}_{16.\iota}$ happens, the adversary has chosen an offset for h. Thus, if event $\mathsf{Bad}_{16.\iota}$ happens, then HH^{ι} can find a pre-image for h', where h' is obtained by adding an offset chosen in advance to the hash of a "random enough" input. Since H is a $(t_2, \alpha, \epsilon_{\mathsf{PR\text{-}coirv}})$-$\mathsf{PR\text{-}coirv}$-secure hash function, the maximum of the distribution $\mathcal{D}$, α is bounded by Lemma 2, and HH^{ι} is a t_2-adversary, then

$$\Pr[\mathsf{Bad}_{16.\iota}] \leq \epsilon_{\mathsf{PR\text{-}coirv}}, \text{ and}$$

$$|\Pr[E_{19}] - \Pr[E_{20}]| \leq \Pr[\mathsf{Bad}_{16}] \leq \sum_{\iota=1}^{q_E} \Pr[\mathsf{Bad}_{16.\iota}] = q_E \epsilon_{\mathsf{PR\text{-}coirv}}.$$

Studying event E_{20}**.** Since we have covered all cases where the adversary has already seen the triple $(h^*, k_0^*, c_{l^*+2}^*)$ for f and g, the only possibility is that $\mathsf{g}(h^*, c_{l^*+2}^*)$ has never been computed before. So, we define event Bad_{17}: 1) $\mathsf{g}(h^*, c_{l^*+2}^*)$ has never been defined before (thus k_0^* is random), 2) there exists a previous query to $\mathsf{E}(k_0^*, p_A)$. Additionally, we define event Bad_{18}: 1) $\mathsf{g}(h^*, c_{l^*+2}^*)$ has never been defined before, 2) there exists no previous query to $\mathsf{E}(k_0^*, p_A)$ (thus, $k_{0,E}^* = \mathsf{E}_{k_0^*}(p_A)$ is random). 3) there exists a previous query to $\mathsf{E}(k_{0,E}^*, p_B)$.

Bounding $\Pr[\mathsf{Bad}_{18}]$**.** Since $\mathsf{g}(h^*, c_{l^*+2}^*)$ has never been computed before, therefore k_0^* is picked uniformly at random.

Now, we want to compute how many "bad keys" there are, that is, keys for which $\mathsf{E}(\cdot, p_A)$ has been computed before in the execution of the Game. Similar to what we have done before, we observe that during an encryption query there are at most $L+3$ queries to $\mathsf{E}(\cdot, p_A)$ with at most $L+3$ different keys (we have an additional query due to Game 10), and $2L+1$ queries with $(\cdot, \cdot)$ where the key is either $k_{\iota,E}$ or $k_{\iota,A}$ for a certain ι and the input is a message m_{ι} (which can be equal to p_A), while during a decryption query there are at most $L+2$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_A)$ and $2L+1$ to $\mathsf{E}(\cdot, \cdot)$. In addition, due to the modification introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_A)$ The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, there are $(q_E + q_D)(3L+4) + q_I$ different keys that, if picked, would trigger the Bad_{17} event. Thus,

$$\Pr[\mathsf{Bad}_{17}] \leq [(q_E + q_D)(3L+4) + q_I]2^{-n}.$$

Bounding $\Pr[\mathsf{Bad}_{17}]$**.** Since $\mathsf{g}(h^*, c^*_{l^*+2})$ has never been computed before, so k^*_0 is picked uniformly at random.

Now, we want to compute how many "bad keys" there are, that is, keys for which $\mathsf{E}(\cdot, p_B)$ has been computed before in the execution of the Game. Similar to what we have done before, we observe that during an encryption query there are at most $L+3$ queries to $\mathsf{E}(\cdot, p_B)$ with at most $L+3$ different keys (we have an additional query due to Game 11), and $2L+1$ queries with $(\cdot,\cdot)$ where the key is either $k_{\iota,E}$ or $k_{\iota,A}$ for a certain ι and the input is a message m_ι (which can be equal to p_B), while during a decryption query there are at most $L+2$ different queries to the ideal block cipher with input $\mathsf{E}(\cdot, p_B)$ and $2L+1$ to $\mathsf{E}(\cdot,\cdot)$. In addition, due to the change we have introduced in Game 4, there are two additional queries, one of which is to $\mathsf{E}(\cdot, p_B)$ The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, there are $(q_E+q_D)(3L+4)+q_I$ different keys that, if picked, would trigger the Bad_{18} event. Thus,

$$\Pr[\mathsf{Bad}_{18}] \leq [(q_E+q_D)(3L+4)+q_I]2^{-n}.$$

Bounding $\Pr[E_{20}]$**.** If both event Bad_{17} and Bad_{18} do not happen, $\tilde{c}^*_0$ is uniformly at random, thus, the probability that $c^*_0 = \tilde{c}^*_0$ is 2^{-n}. Putting all bounds together, we obtain that

$$\begin{aligned}\Pr[E_{20}] \leq& \Pr[\mathsf{Bad}_{17}] + \Pr[\mathsf{Bad}_{18}] + \Pr[E_{20}|(\neg\mathsf{Bad}_{17}) \wedge (\neg\mathsf{Bad}_{18})] \\ \leq& 2[(q_E+q_D)(3L+4)+q_I]2^{-n} + 2^{-n}.\end{aligned}$$

Completing the proof. Now, putting all our bounds together we can conclude the proof:

$$\begin{aligned}\Pr[E_0] \leq& \Pr[E_{20}] + \sum_{i=0}^{19} |\Pr[E_i] - \Pr[E_{i+1}]| \\ =& 2[(q_E+q_D)(3L+4)+q_I]2^{-n} + 2^{-n} + \epsilon_{\mathsf{sTPRP}} + 2\frac{q(q+1)}{2^{n+1}} \\ &+\epsilon_{\mathsf{CR}} + 0 + q_D[(3L+4)(q_E + \frac{q_D+1}{2}) + q_I]2^{-n} \\ &+q_D[(3L+4)(q_E + \frac{q_D+1}{2}) + q_I + 1]2^{-n} + (q_D+1)\epsilon_{\mathsf{PR\text{-}corpi}} \\ &+q_E[3(q_E-1)(L+1)/2 + q_I + q_D(3L+4)]2^{-n} \\ &+q_E[(q_E-1)(3L+4)/2 + q_I + q_D(3L+4)]2^{-n} \\ &+q_E[(q_E-1)(3L+4)/2 + q_I + q_D(3L+4)]2^{-n} \\ &+0 + q_E[(3L+4)[q_D + (q_E-1)/2] + q_I]2^{-n} \\ &+q_E[(3L+4)[q_D + (q_E-1)/2] + (q_E-1)/2 + q_I]2^{-n} + q_E 2^{-n} + 0 + q_E\epsilon_{\mathsf{PR\text{-}coirv}} \\ &+q_E\epsilon_{\mathsf{PR\text{-}corpi}} + 0 + q_E 2^{-n} + q_E(L+3) \\ &\quad\cdot\left[\frac{(q_I+4(L+6)[q_E+q_D])(q_I+4(L+6)[q_E+q_D]-1)(q_I+4(L+6)[q_E+q_D]-2)}{6(2^{2n})}\right] \\ &\quad+0+q_E\epsilon_{\mathsf{PR\text{-}coirv}} \\ \leq& \epsilon_{\mathsf{sTPRP}} + \epsilon_{\mathsf{CR}} + q\epsilon_{\mathsf{PR\text{-}corpi}} + 2q_E\epsilon_{\mathsf{PR\text{-}coirv}} \\ &+\{2q_E + q(q+1) + 1 + q_D + q_E(\frac{q_E-1}{2})q_I(2+2q_D+5q_E) \\ &+(3L+4)[6q_Eq_D + 2q_D + 2q_D\frac{q_D+1}{2} + 2q_E + 5q_E\frac{q_E-1}{2})]\}2^{-n} + q_E(L+3) \\ &\quad\cdot\left[\frac{(q_I+4(L+6)[q_E+q_D])(q_I+4(L+6)[q_E+q_D]-1)(q_I+4(L+6)[q_E+q_D]-2)}{6(2^{2n})}\right],\end{aligned}$$

□

Algorithm 8 The CONCRETE2-TBC algorithm. The changes from CONCRETE2 (Alg. 7) are highlighted.

It uses a TBC $\mathsf{F} : \mathcal{K} \times \mathcal{TW} \times \{0,1\}^n \to \{0,1\}^n$ with a strongly protected implementation, a TBC $\mathsf{E} : \{0,1\}^n \times \{0,1\}^n \times \{0,1\}^n \to \{0,1\}^n$ with a weakly protected implementation, and a multi-input collision resistant hash function $\mathsf{H} : \mathcal{HK} \times \{0,1\}^* \to \mathcal{TW}$. For simplicity, we assume that all message blocks are *full*, that is, $|m_l| = n$, otherwise, see Sec. 5.1. With $[\![i]\!]$ we denote that we write the number i with $n-2$ bits.

Gen:

- $k \xleftarrow{\$} \mathcal{K}$, $s \xleftarrow{\$} \mathcal{HK}$
- $p_A, p_B \xleftarrow{\$} \{0,1\}^n$

$\mathsf{AEnc}_k(a, m)$:

- Parse m in $m_1, ..., m_l$
- $k_0 \xleftarrow{\$} \{0,1\}^n$
- $k_{0,E} = \mathsf{E}_{k_0}^{[\![0]\!]\|00}(p_A)$
- $k_{0,A} = \mathsf{E}_{k_0}^{[\![0]\!]\|01}(p_B)$
- $c_0 = \mathsf{E}_{k_{0,E}}^{[\![0]\!]\|10}(p_B)$
- $k_1 = \mathsf{E}_{k_{0,E}}^{[\![0]\!]\|11}(p_A)$
- For $i = 1, ..., l$
  - $k_{i,E} = \mathsf{E}_{k_i}^{[\![i]\!]\|00}(p_B)$
  - $c_i = \mathsf{E}_{k_{i,E}}^{[\![i]\!]\|01}(m_i)$
  - $k_{i+1} = \mathsf{E}_{k_i}^{[\![i]\!]\|10}(p_A)$
  - $k_{i,A} = \mathsf{E}_{k_{i-1,A}(m_i)}^{[\![i]\!]\|11}$
- $k_{l+1,E} = \mathsf{E}_{k_{l+1}}^{[\![l+1]\!]\|00}(p_B)$
- $c_{l+1} = \mathsf{E}_{k_{l+1,E}}^{[\![l+1]\!]\|01}(k_{l,A})$
- $h = \mathsf{H}_s(c_0\|...\|c_{l+1}, a)$
- $c_{l+2} = \mathsf{F}_k^h(k_0)$
- Return $c = c_0\|...\|c_l\|c_{l+1}\|c_{l+2}$

$\mathsf{ADec}_k(a, c)$:

- Special parse (l) c in $c_0, ..., c_l, c_{l+1}, c_{l+2}$
- $h = \mathsf{H}_s(c_0\|...\|c_l\|c_{l+1}, a)$
- $k_0 = \mathsf{F}_k^{-1,h}(c_{l+2})$
- $k_{0,E} = \mathsf{E}_{k_0}^{[\![0]\!]\|00}(p_A)$
- $k_{0,A} = \mathsf{E}_{k_0}^{[\![0]\!]\|01}(p_B)$
- $\tilde{c}_0 = \mathsf{E}_{k_{0,E}}^{[\![0]\!]\|10}(p_B)$
- If $c_0 \neq \tilde{c}_0$
  - Return $\perp$
- Else $k_1 = \mathsf{E}_{k_{0,E}}^{[\![i]\!]\|11}(p_A)$
- For $i = 1, ..., l$
  - $k_{i,E} = \mathsf{E}_{k_i}^{[\![i]\!]\|00}(p_B)$
  - $m_i = \mathsf{E}_{k_{i,E}}^{-1,[\![i]\!]\|01}(c_i)$
  - $k_{i+1} = \mathsf{E}_{k_i}^{[\![i]\!]\|10}(p_A)$
  - $k_{i,A} = \mathsf{E}_{k_{i-1,A}}^{[\![i]\!]\|11}(m_i)$
- $k_{l+1,E} = \mathsf{E}_{k_{l+1}}^{[\![l+1]\!]\|00}(p_B)$
- $\tilde{k}_{l,A} = \mathsf{E}_{k_{l+1,E}}^{-1,[\![l+1]\!]\|01}(c_{l+1})$
- If $k_{l,A} \neq \tilde{k}_{l,A}$
  - Return $\perp$
- Return $m = m_1\|...\|m_l$

5.6 Comparison between CONCRETE2 and MEM

With respect to CONCRETE, MEM (Alg. 5, described in SM 2.10.2) is generally a 3-pass scheme, however in terms of performance it is quasi-2-pass, as the pass producing the ciphertext blocks and the computation of the commitment of the message can be performed in parallel. Moreover, for each message block, we use 4 calls to the block cipher E, while CONCRETE uses only two calls. But, similarly to CONCRETE, we still use the master key only once in the leak-free and fault-free component. Since CONCRETE2 processes the message twice, for an adversary utilizing leakage, one may think it is easier to extract information about the message encrypted compared to CONCRETE, but this, in our opinion, is inevitable since we believe that there must be a commitment on the plaintext to avoid attacks that are not covered in the weak fault security model (this result is consistent with [52]).

With respect to MEM [54], *one of the main strengths is that we use the strongly protected component significantly less (once compared to three times), and we provide authenticity even if the adversary can inject any fault in decryption.* On the other hand, CONCRETE2 is a slightly more complex scheme, using 4 queries to E and one hash evaluation, with respect to two queries to E and two hash evaluations. Moreover, CONCRETE2 can be executed faster as the two passes *can be done in parallel (quasi-1-pass), while for* MEM*, the second pass must wait until the first one is completed.*

MEM has been proved to provide privacy in the presence of faults and conjectured to provide privacy in the presence of leakage. We conjecture that CONCRETE2 provides privacy in the presence of leakage (and in the presence of faults in the frAE-framework).

6 Conclusion and Future Work

The complexity of the proofs and the need to introduce new security assumptions show how difficult it is to prove security in such a generous framework for an adversary.

In this paper, we have introduced a model for integrity in the presence of leakage and faults, extending the classical atomic model, and we have highlighted some inherent subtleties in capturing fault-resilience.

We proved that authenticated encryption can still provide integrity against very powerful adversaries: in particular, when only a single call to the tweakable block cipher is assumed to be fault- and leak-free, CONCRETE achieves security against unbounded leakage and fault attacks in decryption. We also showed how to obtain a milder but still meaningful notion of security by restricting the adversary to differential faults in encryption queries. Building on this, we proposed CONCRETE2, a modification of CONCRETE that provides integrity in the presence of leakage and faults in both encryption and decryption, under a generous adversarial model. Note that CONCRETE2 is more efficient than the previously proposed scheme MEM and achieves better security (see Sec. 5.6). The complexity of our proofs and the need for new assumptions underline how challenging it is to obtain rigorous guarantees in such a strong adversarial framework.

Future work. Several research directions remain open:

- Developing suitable and achievable definitions of *privacy* in the presence of faults and leakage, together with concrete constructions.
- Removing the leak-free requirement on the tweakable block cipher, replacing it with weaker assumptions such as strong unpredictability under leakage. And defining an analogous in the presence of leakage and faults.
- Studying the case where the adversary can inject faults *inside* the atomic components, and identifying meaningful security notions in this setting.
- Investigating models where the adversary can set multiple values, but cannot fully determine them.

- Exploring alternative primitives, such as sponges, and whether sponge-based authenticated encryption schemes can provide authenticity (and possibly privacy) against leakage and fault attacks.
- Implementing such schemes according to our security model and assumptions.

Acknowledgments

Francesco Berti and Itamar Levi were founded by the Israel Science Foundation (ISF) grant 2569/21.

References

[1] D. F. Aranha, C. Orlandi, A. Takahashi, and G. Zaverucha. Security of hedged fiat-shamir signatures under fault attacks. In A. Canteaut and Y. Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part I*, volume 12105 of *Lecture Notes in Computer Science*, pages 644–674. Springer, 2020.

[2] H. Bar-El, H. Choukri, D. Naccache, M. Tunstall, and C. Whelan. The sorcerer's apprentice guide to fault attacks. *Proc. IEEE*, 94(2):370–382, 2006.

[3] G. Barwell, D. P. Martin, E. Oswald, and M. Stam. Authenticated encryption in the face of protocol and side channel leakage. In T. Takagi and T. Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 693–723. Springer, 2017.

[4] D. Bellizia, F. Berti, O. Bronchain, G. Cassiers, S. Duval, C. Guo, G. Leander, G. Leurent, I. Levi, C. Momin, O. Pereira, T. Peters, F. Standaert, B. Udvarhelyi, and F. Wiemer. Spook: Sponge-based leakage-resistant authenticated encryption with a masked tweakable block cipher. *IACR Trans. Symmetric Cryptol.*, 2020(S1):295–349, 2020.

[5] D. Bellizia, O. Bronchain, G. Cassiers, V. Grosso, C. Guo, C. Momin, O. Pereira, T. Peters, and F. Standaert. Mode-level vs. implementation-level physical security in symmetric cryptography - A practical guide through the leakage-resistance jungle. In D. Micciancio and T. Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 369–400. Springer, 2020.

[6] S. Berndt, T. Eisenbarth, S. Faust, M. Gourjon, M. Orlt, and O. Seker. Combined fault and leakage resilience: Composability, constructions and compiler. In H. Handschuh and A. Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III*, volume 14083 of *Lecture Notes in Computer Science*, pages 377–409. Springer, 2023.

[7] D. J. Bernstein and T. Lange. Non-uniform cracks in the concrete: The power of free precomputation. In K. Sako and P. Sarkar, editors, *Advances in Cryptology - ASIACRYPT 2013 - 19th International Conference on the Theory and Application*

of Cryptology and Information Security, Bengaluru, India, December 1-5, 2013, Proceedings, Part II, volume 8270 of *Lecture Notes in Computer Science*, pages 321–340. Springer, 2013.

[8] F. Berti, C. Guo, O. Pereira, T. Peters, and F. Standaert. Strong authenticity with leakage under weak and falsifiable physical assumptions. In Z. Liu and M. Yung, editors, *Information Security and Cryptology - 15th International Conference, Inscrypt 2019, Nanjing, China, December 6-8, 2019, Revised Selected Papers*, volume 12020 of *Lecture Notes in Computer Science*, pages 517–532. Springer, 2019.

[9] F. Berti, C. Guo, O. Pereira, T. Peters, and F. Standaert. Tedt, a leakage-resist AEAD mode for high physical security applications. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2020(1):256–320, 2020.

[10] F. Berti, C. Guo, T. Peters, Y. Shen, and F. Standaert. Secure message authentication in the presence of leakage and faults. *IACR Trans. Symmetric Cryptol.*, 2023(1):288–315, 2023.

[11] F. Berti, C. Guo, T. Peters, and F. Standaert. Efficient leakage-resilient macs without idealized assumptions. In M. Tibouchi and H. Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 95–123. Springer, 2021.

[12] F. Berti, C. Hazay, and I. Levi. LR-OT: leakage-resilient oblivious transfer. *IACR Cryptol. ePrint Arch.*, page 1143, 2024.

[13] F. Berti, F. Koeune, O. Pereira, T. Peters, and F. Standaert. Ciphertext integrity with misuse and leakage: Definition and efficient constructions with symmetric primitives. In J. Kim, G. Ahn, S. Kim, Y. Kim, J. López, and T. Kim, editors, *Proceedings of the 2018 on Asia Conference on Computer and Communications Security, AsiaCCS 2018, Incheon, Republic of Korea, June 04-08, 2018*, pages 37–50. ACM, 2018.

[14] F. Berti, O. Pereira, T. Peters, and F. Standaert. On leakage-resilient authenticated encryption with decryption leakages. *IACR Trans. Symmetric Cryptol.*, 2017(3):271–293, 2017.

[15] F. Berti, O. Pereira, and F. Standaert. Reducing the cost of authenticity with leakages: a ciml2-secure AE scheme with one call to a strongly protected tweakable block cipher. *IACR Cryptol. ePrint Arch.*, page 451, 2019.

[16] F. Berti, O. Pereira, and F. Standaert. Reducing the cost of authenticity with leakages: a \mathsf CIML2 -secure \mathsf AE scheme with one call to a strongly protected tweakable block cipher. In J. Buchmann, A. Nitaj, and T. Rachidi, editors, *Progress in Cryptology - AFRICACRYPT 2019 - 11th International Conference on Cryptology in Africa, Rabat, Morocco, July 9-11, 2019, Proceedings*, volume 11627 of *Lecture Notes in Computer Science*, pages 229–249. Springer, 2019.

[17] E. Biham and A. Shamir. Differential fault analysis of secret key cryptosystems. In B. S. K. Jr., editor, *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*, volume 1294 of *Lecture Notes in Computer Science*, pages 513–525. Springer, 1997.

[18] D. Boneh, R. A. DeMillo, and R. J. Lipton. On the importance of checking cryptographic protocols for faults (extended abstract). In W. Fumy, editor, *Advances in Cryptology - EUROCRYPT '97, International Conference on the Theory and Application of Cryptographic Techniques, Konstanz, Germany, May 11-15, 1997, Proceeding*, volume 1233 of *Lecture Notes in Computer Science*, pages 37–51. Springer, 1997.

[19] D. Boneh and V. Shoup. A graduate course in applied cryptography. *Draft 0.5*, page 14, 2020.

[20] G. Cassiers, B. Grégoire, I. Levi, and F. Standaert. Hardware private circuits: From trivial composition to full verification. *IEEE Trans. Computers*, 70(10):1677–1690, 2021.

[21] J. Coron, J. Patarin, and Y. Seurin. The random oracle model and the ideal cipher model are equivalent. In D. A. Wagner, editor, *Advances in Cryptology - CRYPTO 2008, 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2008. Proceedings*, volume 5157 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2008.

[22] I. Damgård. Collision free hash functions and public key signature schemes. In D. Chaum and W. L. Price, editors, *Advances in Cryptology - EUROCRYPT '87, Workshop on the Theory and Application of of Cryptographic Techniques, Amsterdam, The Netherlands, April 13-15, 1987, Proceedings*, volume 304 of *Lecture Notes in Computer Science*, pages 203–216. Springer, 1987.

[23] I. Damgård. A design principle for hash functions. In G. Brassard, editor, *Advances in Cryptology - CRYPTO '89, 9th Annual International Cryptology Conference, Santa Barbara, California, USA, August 20-24, 1989, Proceedings*, volume 435 of *Lecture Notes in Computer Science*, pages 416–427. Springer, 1989.

[24] E. Danieli, M. Goldzweig, M. Avital, and I. Levi. Revealing the secrets of radio embedded systems: Extraction of raw information via rf. *IEEE Transactions on Information Forensics and Security*, 2023.

[25] J. P. Degabriele, C. Janson, and P. Struck. Sponges resist leakage: The case of authenticated encryption. In S. D. Galbraith and S. Moriai, editors, *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part II*, volume 11922 of *Lecture Notes in Computer Science*, pages 209–240. Springer, 2019.

[26] C. Dhar, J. Ethan, R. Jejurikar, M. Khairallah, E. List, and S. Mandal. Context-committing security of leveled leakage-resilient aead. *IACR Transactions on Symmetric Cryptology*, 2024(2):348–370, Jun. 2024.

[27] C. Dobraunig, M. Eichlseder, S. Mangard, F. Mendel, and T. Unterluggauer. ISAP - towards side-channel secure authenticated encryption. *IACR Trans. Symmetric Cryptol.*, 2017(1):80–105, 2017.

[28] C. Dobraunig, S. Mangard, F. Mendel, and R. Primas. Fault attacks on nonce-based authenticated encryption: Application to keyak and ketje. In C. Cid and M. J. J. Jr., editors, *Selected Areas in Cryptography - SAC 2018 - 25th International Conference, Calgary, AB, Canada, August 15-17, 2018, Revised Selected Papers*, volume 11349 of *Lecture Notes in Computer Science*, pages 257–277. Springer, 2018.

[29] C. Dobraunig, B. Mennink, and R. Primas. Leakage and tamper resilient permutation-based cryptography. In H. Yin, A. Stavrou, C. Cremers, and E. Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, pages 859–873. ACM, 2022.

[30] M. Fischlin and F. Günther. Modeling memory faults in signature and authenticated encryption schemes. In S. Jarecki, editor, *Topics in Cryptology - CT-RSA 2020 - The Cryptographers' Track at the RSA Conference 2020, San Francisco, CA, USA, February 24-28, 2020, Proceedings*, volume 12006 of *Lecture Notes in Computer Science*, pages 56–84. Springer, 2020.

[31] D. Goudarzi and M. Rivain. How fast can higher-order masking be in software? In J. Coron and J. B. Nielsen, editors, *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part I*, volume 10210 of *Lecture Notes in Computer Science*, pages 567–597, 2017.

[32] C. Guo, O. Pereira, T. Peters, and F. Standaert. Authenticated encryption with nonce misuse and physical leakage: Definitions, separation results and first construction - (extended abstract). In P. Schwabe and N. Thériault, editors, *Progress in Cryptology - LATINCRYPT 2019 - 6th International Conference on Cryptology and Information Security in Latin America, Santiago de Chile, Chile, October 2-4, 2019, Proceedings*, volume 11774 of *Lecture Notes in Computer Science*, pages 150–172. Springer, 2019.

[33] A. Jana, A. Nath, G. Paul, and D. Saha. Differential fault analysis of NORX using variants of coupon collector problem. *J. Cryptogr. Eng.*, 12(4):433–459, 2022.

[34] A. Journault and F. Standaert. Very high order masking: Efficient implementation and security evaluation. In W. Fischer and N. Homma, editors, *Cryptographic Hardware and Embedded Systems - CHES 2017 - 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*, volume 10529 of *Lecture Notes in Computer Science*, pages 623–643. Springer, 2017.

[35] J. Katz and Y. Lindell. *Introduction to Modern Cryptography, Second Edition.* CRC Press, 2014.

[36] P. C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In N. Koblitz, editor, *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.

[37] P. C. Kocher, J. Jaffe, and B. Jun. Differential power analysis. In M. J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.

[38] Y. Lindell, editor. *Tutorials on the Foundations of Cryptography.* Springer International Publishing, 2017.

[39] M. D. Liskov, R. L. Rivest, and D. A. Wagner. Tweakable block ciphers. In M. Yung, editor, *Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings*, volume 2442 of *Lecture Notes in Computer Science*, pages 31–46. Springer, 2002.

[40] E. List. TEDT2 - highly secure leakage-resilient tbc-based authenticated encryption. In P. Longa and C. Ràfols, editors, *Progress in Cryptology - LATINCRYPT 2021 - 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6-8, 2021, Proceedings*, volume 12912 of *Lecture Notes in Computer Science*, pages 275–295. Springer, 2021.

[41] J. Longo, D. P. Martin, E. Oswald, D. Page, M. Stam, and M. Tunstall. Simulatable leakage: Analysis, pitfalls, and new constructions. In *Advances in Cryptology - ASIACRYPT, Proceedings, Part I*, volume 8873 of *LNCS*, pages 223–242. Springer, 2014.

[42] B. Mennink and B. Preneel. Breaking and fixing cryptophia's short combiner. In D. Gritzalis, A. Kiayias, and I. G. Askoxylakis, editors, *Cryptology and Network Security - 13th International Conference, CANS 2014, Heraklion, Crete, Greece, October 22-24, 2014. Proceedings*, volume 8813 of *Lecture Notes in Computer Science*, pages 50–63. Springer, 2014.

[43] S. Micali and L. Reyzin. Physically observable cryptography (extended abstract). In M. Naor, editor, *Theory of Cryptography, First Theory of Cryptography Conference, TCC 2004, Cambridge, MA, USA, February 19-21, 2004, Proceedings*, volume 2951 of *Lecture Notes in Computer Science*, pages 278–296. Springer, 2004.

[44] A. Mittelbach. Cryptophia's short combiner for collision-resistant hash functions. In M. J. J. Jr., M. E. Locasto, P. Mohassel, and R. Safavi-Naini, editors, *Applied Cryptography and Network Security - 11th International Conference, ACNS 2013, Banff, AB, Canada, June 25-28, 2013. Proceedings*, volume 7954 of *Lecture Notes in Computer Science*, pages 136–153. Springer, 2013.

[45] C. Namprempre, P. Rogaway, and T. Shrimpton. Reconsidering generic composition. In P. Q. Nguyen and E. Oswald, editors, *Advances in Cryptology - EUROCRYPT 2014 - 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings*, volume 8441 of *Lecture Notes in Computer Science*, pages 257–274. Springer, 2014.

[46] O. Pereira, F. Standaert, and S. Vivek. Leakage-resilient authentication and encryption from symmetric cryptographic primitives. In I. Ray, N. Li, and C. Kruegel, editors, *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 96–108. ACM, 2015.

[47] J. Quisquater and D. Samyde. Electromagnetic analysis (EMA): measures and counter-measures for smart cards. In I. Attali and T. P. Jensen, editors, *Smart Card Programming and Security, International Conference on Research in Smart Cards, E-smart 2001, Cannes, France, September 19-21, 2001, Proceedings*, volume 2140 of *Lecture Notes in Computer Science*, pages 200–210. Springer, 2001.

[48] P. Rogaway. Nonce-based symmetric encryption. In B. K. Roy and W. Meier, editors, *Fast Software Encryption, 11th International Workshop, FSE 2004, Delhi, India, February 5-7, 2004, Revised Papers*, volume 3017 of *Lecture Notes in Computer Science*, pages 348–359. Springer, 2004.

[49] P. Rogaway. Formalizing human ignorance: Collision-resistant hashing without the keys. *IACR Cryptol. ePrint Arch.*, page 281, 2006.

[50] P. Rogaway and T. Shrimpton. Cryptographic hash-function basics: Definitions, implications, and separations for preimage resistance, second-preimage resistance, and collision resistance. In B. K. Roy and W. Meier, editors, *Fast Software Encryption,*

11th International Workshop, FSE 2004, Delhi, India, February 5-7, 2004, Revised Papers, volume 3017 of *Lecture Notes in Computer Science*, pages 371–388. Springer, 2004.

[51] P. Rogaway and T. Shrimpton. Deterministic authenticated-encryption: A provable-security treatment of the key-wrap problem. *IACR Cryptol. ePrint Arch.*, page 221, 2006.

[52] S. Saha, M. Alam, A. Bag, D. Mukhopadhyay, and P. Dasgupta. Learn from your faults: Leakage assessment in fault attacks using deep learning. *J. Cryptol.*, 36(3):19, 2023.

[53] S. Saha, A. Bag, D. Jap, D. Mukhopadhyay, and S. Bhasin. Divided we stand, united we fall: Security analysis of some SCA+SIFA countermeasures against sca-enhanced fault template attacks. In M. Tibouchi and H. Wang, editors, *Advances in Cryptology - ASIACRYPT 2021 - 27th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 6-10, 2021, Proceedings, Part II*, volume 13091 of *Lecture Notes in Computer Science*, pages 62–94. Springer, 2021.

[54] S. Saha, M. Khairallah, and T. Peyrin. Exploring integrity of aeads with faults: Definitions and constructions. *IACR Trans. Symmetric Cryptol.*, 2022(4):291–324, 2022.

[55] M. I. Salam, H. Q. A. Mahri, L. Simpson, H. Bartlett, E. Dawson, and K. K. Wong. Fault attacks on tiaoxin-346. In D. Abramson, editor, *Proceedings of the Australasian Computer Science Week Multiconference, ACSW 2018, Brisbane, QLD, Australia, January 29 - February 02, 2018*, pages 5:1–5:9. ACM, 2018.

[56] M. I. Salam, T. H. Ooi, L. Xue, W. Yau, J. Pieprzyk, and R. C. Phan. Random differential fault attacks on the lightweight authenticated encryption stream cipher grain-128aead. *IEEE Access*, 9:72568–72586, 2021.

[57] D. Salomon and I. Levi. Masksimd-lib: on the performance gap of a generic C optimized assembly and wide vector extensions for masked software with an ascon-p test case. *J. Cryptogr. Eng.*, 13(3):325–342, 2023.

[58] K. Suzuki, D. Tonien, K. Kurosawa, and K. Toyota. Birthday paradox for multi-collisions. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 91-A(1):39–45, 2008.

Supplementary Material

A Additional Proofs

A.1 Full Proof of Thm. 10

Proof. We use a sequence of games for the proofs. For simplicity, we consider the decryption of the challenge associated data and ciphertext as the $q_D + 1$th decryption query.

Game 0. It is the standard $\mathsf{wCIL2F2}_{\mathsf{CONCRETE},\mathcal{F},\mathsf{L},\mathsf{A}^{\mathsf{L},\mathsf{Flt}}}$ game. Let E_0 be the event that A wins this game.

Game 1. It is Game 0, where we simply put (a', c', a, m, r) in $\mathcal{S}$ where a' is a modified according to the $l + 1.1$ row of Flt and $c' = (c'_0, \ldots, c'_l, c_{l+1})$ with c'_i $(i = 0, \ldots, l)$ is c_i modified according to the $l+1.1$ row of Flt, and c_{l+1} is left unchanged (That is, we take the actual input of H). Let E_1 be the event that A wins this game. Clearly, $\Pr[E_0] \leq \Pr[E_1]$.

Game 2. It is Game 1 except that we have replaced F with a random tweakable permutation f. Let E_2 be the event that A wins this game.

Transition between Game 1 and Game 2. Since Game 1 and Game 2 are the same except for the replacement of F with f, we can build an adversary B to bound the difference. She plays the sTPRP game, having to distinguish F_k from f, and simulates A's oracles using its own.

At the start of the game, B picks s in $\mathcal{HK}$ and gives it to A. Moreover, she has a list $\mathcal{S}$ which is empty.

When A does an encryption query on input (a, m), B simply picks k_0 uniformly at random from $\{0,1\}^n$. Moreover, she has $L + 1$ lists $\mathcal{S}_0, \mathcal{S}_1, \ldots, \mathcal{S}_L$ which are empty. After having parsed m in $(m_1, \ldots, m_l)$, she proceeds as follows[44]: 1) she computes $c_0 = \mathsf{E}_{k_0}(p_B)$, where k_0 is modified according to the 0.2 row of Flt, and she adds c_0 to $\mathcal{S}_0$; 2) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $c_i = y_i \oplus m_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus m_l$) with y_i and c_i modified according to the $i.3$ line of Flt, and adds c_i to $\mathcal{S}_i$; 3) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where $c_0, \ldots, c_l$, and a are modified according to the $l + 1.1$ row of Flt; moreover, she adds c'_i to $\mathcal{C}_i$, for $i = 0, \ldots, l$, where c'_i is c_i modified according to the element of the $l + 1.1$th row corresponding to c_i. 4) she queries her oracle on input (h, k_0) which are modified according to $l+1.2$ row of Flt, obtaining c_{l+1} as answer; 5) finally, B answers $c = (c_0, \ldots, c_l)$ and the leakage k_0 to A. Moreover, if there is no fault inserted and in the 0.2 row and in the $l + 1.2$ row of fault (or the same fault for k_0 in both rows), she adds $c' = (c'_0, \ldots, c'_l, c_{l+1}, a')$ to $\mathcal{S}$ (where a' is a modified according to the $l + 1.1$th row of Flt), otherwise nothing.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + (2l + 1)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L + 1)t_{\mathsf{E}}$.

When A does a decryption query on input (a, c, Flt), B proceeds as follows: 1) she computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_l)$ where a and $c_0, \ldots, c_l$ are modified according to the 0.0 row of Flt, 2) she queries her inverse oracle on input (h, c_{l+1}) which are modified according to 0.1 row of Flt, obtaining k_0 as answer, 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 4) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 5) finally, B answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, B does one oracle query and time $t_{\mathsf{H}} + (2l + 1)t_{\mathsf{E}} \leq t_{\mathsf{H}} + (2L + 1)t_{\mathsf{E}}$.

[44] Note that each of the steps, into which we divide this computation, corresponds to an atom (see Sec. 2.8).

When A outputs her output (a^*, c^*), B proceeds as follows: 1) she computes $h^* = \mathsf{H}_s(a^*, c_0^* \| \ldots \| c_l^*)$, 2) she queries her inverse oracle on input (h^*, c_{l+1}^*), obtaining k_0^* as answer, 3) she computes $\tilde{c}_0^* = \mathsf{E}_{k_0^*}(p_A)$, if $\tilde{c*}_0 = c*_0$ and $(a^*, c^*) \notin \mathcal{S}$, she outputs 1; otherwise 0.

For this, B does one oracle query and needs time $t_{\mathsf{H}} + t_{\mathsf{E}}$.

Thus, in total B does at most $q_E + q_D + 1 = q$ queries to her oracle and runs in time bounded by $t + q(t_{\mathsf{H}}) + (q(2L+1)+1)t_{\mathsf{E}} = t_1$.

If her oracle is implemented with F, B correctly simulates Game 1 for A, otherwise Game 2. Since F is a $(q, t_1, \epsilon_{\mathsf{sTPRP}})$-$\mathsf{sTPRP}$,

$$|\Pr[E_2] - \Pr[E_1]| \leq \epsilon_{\mathsf{sTPRP}}.$$

Game 3. It is Game 2, where in decryption instead of using f^{-1}, where f is a random tweakable permutation, we pick its answers uniformly at random (we answer consistently using a table, i.e., if the same query repeats, we return the same answer). That is, we replace f^{-1} with a random function (when the inputs are fresh). Let E_3 be the event that A wins this game.

Transition between Game 2 and Game 3. In Game 3, for decryption queries, we are using a random tweakable permutation, while in Game 3 a random function. Due to the well-known birthday bound, we have that

$$|\Pr[E_3] - \Pr[E_2]| \leq \frac{(q_D+2)(q_D+1)}{2^{n+1}}.$$

Game 4. It is Game 3, where during decryption queries (not for the challenge one), immediately after k_0 is computed, the decryption algorithm computes $c_{l+2} := \mathsf{E}_{k_0}(p_B)$, without any faults inserted, and appends c_{l+2} to the decryption algorithm's normal answer.. Let E_4 be the event that the adversary wins this game.

Transition between Game 3 and Game 4. We build an adversary C. C behaves as A for encryption queries. For decryption queries, C proceeds as follows: when A does a decryption query on input (a, c), C simply forwards this query to her oracle. After having received the answer and the leakage k_0, C simply takes k_0, calls the ideal cipher E on input (k_0, p_B), obtains c_{l+2}, and answer A with the oracle's answer appended with c_{l+2}. Therefore, C needs time $t + q_D t_{\mathsf{E}}$ and does $q_I + q_D$ queries to the ideal cipher E, i.e.

$$\Pr[E_3] = \Pr[E_4].$$

Game 5. It is Game 4, with the condition that the following event does not happen: during a decryption query, when a new key k_0 is picked, there does not exist a query to the ideal cipher on input (k_0, p_B). Let E_5 be the event that the adversary wins this game.

Transition between Game 4 and Game 5. The two games are identical except if the following Bad_1 event happens: there exists a decryption query s.t. it requires to pick a new key k_0^i and there exists a previous query to the ideal cipher E on input (k_0^i, p_B). We observe that during an encryption query there are at most $L+1$ queries to (k_i, p_B) with at most $L+1$ different keys, while during a decryption query there are at most $L+2$ different query to the ideal block cipher with input (k_i, p_B). The adversary can do at most q_I queries of its choice to the ideal block cipher. Thus, during the jth decryption query there are at most $q_E(L+1) + q_I + (j-1)(L+2)$ different keys that, if picked, would

trigger the Bad_1 event. Thus,

$$\begin{aligned}\Pr[\mathsf{Bad}_1] \leq & \sum_{j=1}^{q_D+1} \frac{q_E(L+1)+q_I+(j-1)(L+2)}{2^n} \\ \leq & \frac{(q_D+1)(q_E(L+1))+(q_D+1)q_I+(L+2)(q_D+1)q_D/2}{2^n} \\ \leq & \frac{(q_D+1)[q_E(L+1)+q_I+q_D(L+2)/2]}{2^n}\end{aligned}$$

Therefore,

$$|\Pr[E_5]-\Pr[E_4]| \leq (q_D+1)[q_E(L+1)+q_I+q_D(L+2)/2]2^{-n}.$$

Game 6. It is Game 5, where the following event Bad_2 does not happen: the hash queries induced during encryption and decryption queries induce a collision for the hash function. Let E_6 be the event that the adversary wins this game.

Transition between Game 5 and Game 6. Since Game 5 and Game 6 are the same except if event Bad_2 happens, it is enough to bound $\Pr[\mathsf{Bad}_2]$ to bound the difference $|\Pr[E_5]-\Pr[E_6]|$. To bound $\Pr[\mathsf{Bad}_2]$, we build an adversary D who wants to find a collision for the hash function. At the start of the game, D receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks a random tweakable permutation f. Finally, he has a list $\mathcal{H}$ which is empty.

When A does an encryption query on input (a,m), D simply picks k_0 uniformly at random from $\{0,1\}^n$. Moreover, she has $L+1$ lists $\mathcal{S}_0,\mathcal{S}_1,\ldots,\mathcal{S}_L$ which are empty. After having parsed m in $(m_1,\ldots,m_l)$, she proceeds as follows: 1) she computes $c_0=\mathsf{E}_{k_0}(p_B)$, where k_0 is modified according to the 0.2 row of Flt, and she adds c_0 to $\mathcal{S}_0$; 2) for $i=1,\ldots,l$, a) computes $k_i=\mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i=\mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $c_i=y_i\oplus c_i$ (if $i=l$ $\pi_{|c_l|}(y_l)\oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, and adds c_i to $\mathcal{S}_i$; 3) she computes $h=\mathsf{H}_s(c_0\|\ldots\|c_l,a)$ where $c_0,\ldots,c_l$, and a are modified according to the $l+1.1$ row of Flt; moreover, she adds c'_i to $\mathcal{C}_i$, for $i=0,\ldots,l$, where c'_i is c_i modified according to the element of the $l+1.1$th row corresponding to c_i, and she adds $((c'_0\|\ldots\|c'_l,a),h)$ to $\mathcal{H}$. 4) she queries her oracle on input (h,k_0) which are modified according to $l+1.2$ row of Flt, obtaining c_{l+1} as answer; 5) finally, D answers $c=(c_0,\ldots,c_l)$ and the leakage k_0 to A. Moreover, if there is no fault inserted and in the 0.2 row and in the $l+1.2$ row of fault (or the same fault for k_0 in both rows), she adds $c'=(c'_0,\ldots,c'_l,c_{l+1},a')$ to $\mathcal{S}$ (where a' is a modified according to the $l+1.1$th row of Flt), otherwise nothing. Moreover, D adds (a',c') to the list $\mathcal{S}$, where a' is a modified according to the $l+1.1$ row of Flt and $c'=(c'_0,\ldots,c'_l,c_{l+1})$ with c'_i $(i=0,\ldots,l)$ is c_i modified according to the $l+1.1$ row of Flt.

For this, D needs time $t_{\mathsf{H}}+(2l+1)t_{\mathsf{E}}+t_{\mathsf{F}}\leq t_{\mathsf{F}}+t_{\mathsf{H}}+(2L+1)t_{\mathsf{E}}$.

When A does a decryption query on input (a,c,Flt), D proceeds as follows: 1) she computes $h=\mathsf{H}_s(a,c_0\|\ldots\|c_l)$ where a and $c_0,\ldots,c_l$ are modified according to the 0.0 row of Flt, and she adds $((a,c_0\|\ldots\|c_l),h)$ to $\mathcal{H}$, 2) she computes $k_0=\mathsf{f}^{-1}(h,c_{l+1})$ where the inputs are modified according to 0.1 row of Flt, obtaining k_0 as answer, 3) she computes $\tilde{c}_0=\mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt and $c_{l+2}=\mathsf{E}_{k_0}(p_B)$, if $\tilde{c}_0\neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 4) for $i=1,\ldots,l$, a) computes $k_i=\mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i=\mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i=y_i\oplus c_i$ (if $i=l$ $\pi_{|c_l|}(y_l)\oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 5) finally, D answers $m=(m_1,\ldots,m_l)$ and the leakage k_0 to A.

For this, D needs time $t_{\mathsf{F}}+t_{\mathsf{H}}+(2l+2)t_{\mathsf{E}}\leq t_{\mathsf{F}}+t_{\mathsf{H}}+(2L+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), D proceeds as follows: 1) she computes $h^* = \mathsf{H}_s(a^*, c_0^* \| \dots \| c_l^*)$ and she adds $((a^*, c_0^* \| \dots \| c_l^*), h)$ to $\mathcal{H}$, 2) she looks through $\mathcal{H}$ to see if there is a collision. If it is the case, she outputs it, otherwise $0^n, 1^n$ (a dummy pair). For this, D needs time t_{H}.

Thus, in total, D runs in time bounded by $t + q(t_{\mathsf{H}}) + (q_E + q_D)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] = t_2$.

D correctly simulates Game 6 for A. D finds a collision for the hash function H if event Bad_2 happens. Thus, since H is a $(t_2, \epsilon_{\mathsf{CR}})$-collision resistant hash function,

$$|\Pr[E_6] - \Pr[E_5]| = \Pr[\mathsf{Bad}_2] \leq \epsilon_{\mathsf{CR}}.$$

Game 7. It is Game 6, except we rule out the following event Bad_3: for any tweak h for f used only in decryption queries, (that is, used only in f^{-1}) there exist two decryption queries, which we call the ith and the jth s.t.

- $h' = \tilde{h}^i$
- $c_0^j = c_{l+2}^i$, where
  - $h' = \mathsf{H}_s(c_0^j \| \dots \| c_l^j, a^j)$ (where a^j and $c_0^j, \dots, c_l^j$ are modified according to the 0.0 line of Flt^j),
  - c_{l+2}^i obtained in the ith decryption query, in which $\mathsf{f}^{-1}(h^i, \tau^i)$ is computed
  - $\tilde{h}^i$ is modified according to the 0.1 row of Flt^i.

Let E_7 be the event that the adversary wins this game.

Transition between Game 6 and Game 7. Since Game 6 and Game 7 are the same except if event Bad_3 happens, it is enough to bound $\Pr[\mathsf{Bad}_3]$ to bound the difference $|\Pr[E_6] - \Pr[E_7]|$. To bound $\Pr[\mathsf{Bad}_3]$, we divide it into $q_D + 1$ different events:$\mathsf{Bad}_{3.1}, \dots, \mathsf{Bad}_{3.q_D+1}$, where $\mathsf{Bad}_{3.\iota}$ is event Bad_3 happening with $\iota = i$. Clearly, $\mathsf{Bad}_3 = \bigcup_{\iota=1}^{q_D+1} \mathsf{Bad}_{3.\iota}$. To bound $\Pr[\mathsf{Bad}_{3.\iota}]$ we build an adversary EE^ι against the n–PR–corpi-security of the hash function H.

The n–PR–corpi-adversary EE^ι. At the start of the game, EE^ι receives a key for the hash function s in $\mathcal{HK}$ and gives it to A. Moreover, she picks a random tweakable permutation f. Finally, she has two lists $\mathcal{H}$ and $\mathcal{CH}$ which are empty.

When A does an encryption query on input (a, m), EE^ι (we use EE^ι) to identify both EE_1^ι and EE_2^ι) simply picks k_0 uniformly at random from $\{0,1\}^n$. Moreover, she has $L+1$ lists $\mathcal{S}_0, \mathcal{S}_1, \dots, \mathcal{S}_L$ which are empty. After having parsed m in $(m_1, \dots, m_l)$, she proceeds as follows: 1) she computes $c_0 = \mathsf{E}_{k_0}(p_B)$, where k_0 is modified according to the 0.2 row of Flt, and she adds c_0 to $\mathcal{S}_0$; 2) for $i = 1, \dots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $c_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, and adds c_i to $\mathcal{S}_i$; 3) she computes $h = \mathsf{H}_s(c_0 \| \dots \| c_l, a)$ where $c_0, \dots, c_l$, and a are modified according to the $l+1.1$ row of Flt; moreover, she adds c_i' to $\mathcal{C}_i$, for $i = 0, \dots, l$, where c_i' is c_i modified according to the element of the $l+1.1$th row corresponding to c_i, and she adds $((c_0' \| \dots \| c_l', a), h)$ to $\mathcal{H}$. 4) she queries her oracle on input (h, k_0) which are modified according to $l+1.2$ row of Flt, obtaining c_{l+1} as answer; 5) finally, EE^ι answers $c = (c_0, \dots, c_l)$ and the leakage k_0 to A. Moreover, if there is no fault inserted and in the 0.2 row and in the $l+1.2$ row of fault (or the same fault for k_0 in both rows), she adds $c' = (c_0', \dots, c_l', c_{l+1}, a')$ to $\mathcal{S}$ (where a' is a modified according to the $l+1.1$th row of Flt), otherwise nothing. Moreover, EE^ι adds (a', c') to the list $\mathcal{S}$, where a' is a modified according to the $l+1.1$ row of Flt and $c' = (c_0', \dots, c_l', c_{l+1})$ with c_i' $(i = 0, \dots, l)$ is c_i modified according to the $l+1.1$ row of Flt.

For this, EE^ι needs time $t_{\mathsf{H}} + (2l+1)t_{\mathsf{E}} + t_{\mathsf{F}} \leq t_{\mathsf{F}} + t_{\mathsf{H}} + (2L+1)t_{\mathsf{E}}$.

When A does the ith decryption query, $i < \iota$ on input (a, c, Flt), EE_1^ι proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \dots \| c_l, a)$ where $c_0, \dots, c_l$ and a are modified according to the 0.0 row of Flt, and she adds $((c_0, \dots, c_l, a), h)$ to $\mathcal{H}$ (with $c_0, \dots, c_l$ and a modified

according to the 0.0 row of Flt), 2) she computes $k_0 = \mathsf{f}^{-1}(h, c_{l+1})$ where the inputs are modified according to 0.1 row of Flt, obtaining k_0, as answer; she sets $c^i_{l+1} := c_{l+1}$, $h^i = h$ (where c_{l+1} and h are modified according to the 0.1 row of Flt), 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, and she adds $(h^i, c^i_{l+1}, c_{l+2})$ to $\mathcal{CH}$, 4) she sees if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 6) finally, EE^ι_1 answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, EE^ι_1 needs time $t_{\mathsf{F}} + t_{\mathsf{H}} + (2l+2)t_{\mathsf{E}} \leq t_{\mathsf{F}} + t_{\mathsf{H}} + (2L+2)t_{\mathsf{E}}$.

When A does the ι decryption query on input (a, c, Flt), EE^ι_1 proceeds as follows: 1) she computes $h = \mathsf{H}_s(a, c_0 \| \ldots \| c_l)$ where a and $c_0, \ldots, c_l$ are modified according to the 0.0 row of Flt, and she adds $((c_0, \ldots, c_l, a), h)$ to $\mathcal{H}$ (with $c_0, \ldots, c_l$ and a modified according to the 0.0 row of Flt), 2) she sets h^ι and c^ι_{l+1}, which are h and c_{l+1} modified according to the 0.1 row of Flt and she checks if there is an entry in $\mathcal{CH}$ (h^ι, c^ι_{l+1}). If this is the case EE^ι_1 aborts. Otherwise, EE^ι_1 outputs h^ι as her challenge and $\mathsf{st} = (h^\iota, c^\iota_{l+1}, \mathcal{CH}, \mathcal{H}, s, \mathsf{f})$. Then, $\mathsf{EE}^\iota_2(\mathsf{st})$, parses st in $h^\iota, c^\iota_{l+1}, \mathcal{CH}, \mathcal{H}, s, \mathsf{f})$, and she computes $k_0 = \mathsf{f}^{-1}(h^\iota, c^\iota_{l+1})$. 3) after that, EE^ι_2 computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, and she adds $(h^i, c^i_{l+1}, c_{l+2})$ to $\mathcal{CH}$, 4) she sees if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 6) finally, EE^ι_2 answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A. (If $\iota = q_D + 1$ the decryption query associated to A output, EE proceeds as as normal decryption query, simply assuming that there is no fault injected).

For this, EE^ι_1 needs time t_{H}, while EE^ι_2 needs time $t_{\mathsf{F}} + (2l+2)t_{\mathsf{E}} \leq t_{\mathsf{F}} + (2L+2)t_{\mathsf{E}}$. To be polynomial, we assume that f is simulated via lazy sampling and EE_1 puts in st the samples she has already done.

When A does the ith decryption query, $i > \iota$ on input (a, c, Flt), EE^ι_2 proceeds as follows: 1) she computes $h = \mathsf{H}_s(c_0 \| \ldots \| c_l, a)$ where $c_0, \ldots, c_l$ and a are modified according to the 0.0 row of Flt, and she adds $((c_0, \ldots, c_l, a), h)$ to $\mathcal{H}$ (with $c_0, \ldots, c_l$ and a modified according to the 0.0 row of Flt), 2) she computes $k_0 = \mathsf{f}^{-1}(h, c_{l+1})$ where the inputs are modified according to 0.1 row of Flt, obtaining k_0, as answer; she sets $c^i_{l+1} := c_{l+1}$, $h^i = h$ (where c_{l+1} and h are modified according to the 0.1 row of Flt), 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, 4) she sees if $\tilde{c}_0 \neq c_0$, she answers $\perp$ and the leakage k_0 to A; otherwise 5) for $i = 1, \ldots, l$, a) computes $k_i = \mathsf{E}_{k_{i-1}}(p_A)$ with k_{i-1} modified according to the $i.1$ line of Flt, b) computes $y_i = \mathsf{E}_{k_i}(p_B)$ with k_i modified according to the $i.2$ line of Flt c) computes $m_i = y_i \oplus c_i$ (if $i = l$ $\pi_{|c_l|}(y_l) \oplus c_l$) with y_i and c_i modified according to the $i.3$ line of Flt, 6) finally, EE^ι_2 answers $m = (m_1, \ldots, m_l)$ and the leakage k_0 to A.

For this, EE^ι_2 needs time $t_{\mathsf{F}} + t_{\mathsf{H}} + (2l+2)t_{\mathsf{E}} \leq t_{\mathsf{F}} + t_{\mathsf{H}} + (2L+2)t_{\mathsf{E}}$.

When A outputs her output (a^*, c^*), EE^ι_2 proceeds as follows: 1) she computes $h^* = \mathsf{H}_s(c^*_0 \| \ldots \| c^*_l, a^*)$ and she adds $((c^*_0 \| \ldots \| c^*_l, a^*), h)$ to $\mathcal{H}$, 2) she computes $k^=_0 \mathsf{f}^{-1}(h, c_{l+1})$ 3) she computes $\tilde{c}_0 = \mathsf{E}_{k_0}(p_B)$ with k_0 modified according the 0.2 line of Flt, and $c_{l+2} = \mathsf{E}_{k_0}(p_B)$, 4) she sees if $\tilde{c^*}_0 \neq c^*_0$, she answers $\perp$; otherwise 5) for $i = 1, \ldots, l$, a) computes $k^*_i = \mathsf{E}_{k^*_{i-1}}(p_A)$, b) computes $y^*_i = \mathsf{E}_{k^*_i}(p_B)$, c) computes $m^*_i = y^*_i \oplus c^*_i$ (if $i = l$ $\pi_{|c^*_l|}(y^*_l) \oplus c^*_l$, 6) finally, EE^ι_2 observe that A has given a correct decryption query, and she looks through $\mathcal{H}$ to see if there is an entry $(c_0 \| \ldots \| c_l, a), h)$ s.t $h = h^\iota$ and $c_0 = c^\iota_{l+2}$. If it is the case, she outputs it, otherwise 0^n.

Thus, in total EE runs in time bounded by $t + q(t_{\mathsf{H}}) + (q_E + q_D + 1)[(2L+3)t_{\mathsf{E}} + t_{\mathsf{F}}] = t'_3$.

EE correctly simulates Game 5 for A, except for the computation of Flt.

Bounding $\Pr[\mathsf{Bad}_{3.\iota} | \neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}]$**.** First, we observe that if EE^ι_1 aborts, it means

that there is a previous query for which $h^i = h^\iota$ and $c^i_{l+2} = c^\iota_{l+2}$, thus event $\mathsf{Bad}_{3.\iota}$ is event 3.i, thus,

$$\Pr[\mathsf{Bad}_{3.\iota}|\neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] = 0.$$

Second, we observe that since k^ι_0 is picked randomly (and (k^ι_0, p_B) has never been queried before the ι decryption query to E), to pick the random prefix uniformly at random or to pick a k_0 uniformly at random and as a random prefix $\mathsf{E}(k_0, p_B)$ is indistinguishable for the n-PR-corpi game. We are doing the latter case for EE^ι. Thus, if event $\mathsf{Bad}_{3.\iota}\neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}$ happens, then, EE wins the n-PR-corpi game. Since EE is a t'_3-adversary and H is $(t_3, \epsilon_{\mathsf{PR}\text{-}\mathsf{corpi}})$-$n$-$\mathsf{PR}$-$\mathsf{corpi}$-secure (because we need to subtract the time needed to compute the random prefix), then,

$$\Pr[\mathsf{Bad}_{3.\iota}|\neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] \leq \epsilon_{\mathsf{PR}\text{-}\mathsf{corpi}}.$$

Note that as it happened for D the fact that she does not compute correctly Flt it is not a problem, because the correct computation of Flt is only needed to compute the output of the game and the output of EE^ι is produced before the effect of Flt enters into play.

Concluding the proof First, we observe that

$$\Pr[\mathsf{Bad}_3] \leq \sum_{\iota=1}^{q_D+1} \Pr[\mathsf{Bad}_{3.\iota}|\neg\mathsf{Bad}_{3.1}, \ldots, \mathsf{Bad}_{3.\iota-1}] \leq \epsilon_{\mathsf{PR}\text{-}\mathsf{corpi}} \leq (q_D+1)\epsilon_{\mathsf{PR}\text{-}\mathsf{corpi}}.$$

Therefore, $|\Pr[E_5] - \Pr[E_6]| \leq (q_D+1)\epsilon_{\mathsf{PR}\text{-}\mathsf{corpi}}$. Second, we observe that $\Pr[E_6] = 0$. Indeed, in the final verification query, the adversary cannot use an h^* used in an encryption query, since this would require a collision for H (ruled out by Bad_2). Similarly, she cannot is an h^* use in a decryption query, since otherwise Bad_3 would be triggered. Therefore, no adversary can win Game 6. Putting everything together, we obtain,

$$\begin{aligned}\Pr[E_0] \leq & \Pr[E_6] \leq \sum_{i=1}^{6} |\Pr[E_{i+1}] - \Pr[E_i]| \\ \leq & \epsilon_{\mathsf{CR}} + (q_D+1)\epsilon_{\mathsf{PR}\text{-}\mathsf{corpi}} + \epsilon_{\mathsf{sTPRP}} \\ & + \frac{(q_D+2)(q_D+1)}{2^{n+1}} + \frac{(q_D+1)[q_E(L+1) + q_I + q_D(L+2)/2]}{2^n}.\end{aligned}$$

□